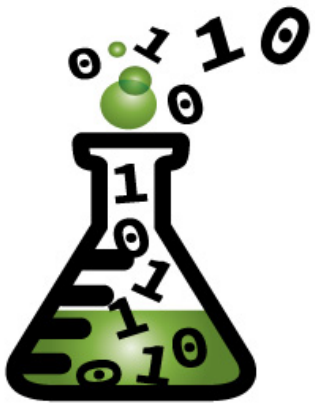
noble desktop

Noble Desktop
JavaScript Bootcamp





Lesson & Lab Workbook

Lesson 00.01: Intro and Output

JavaScript's role in Web Development

What is JavaScript (JS)?

- JavaScript (JS) is the #1 programming language for a web developer to learn.
- JS is used by millions of websites and is required for nearly all developer jobs.
- On the Frontend, JS "sees" web pages as a hierarchical collection of objects: the **Document Object Model (DOM)**.
- On the Backend, **Node.js** and **React.js** work with a **server** object and databases.

What can Javascript do for Websites?

- **Add interactivity**

JavaScript enables interactivity, which empowers the user. When you click a button and something happens, that's probably JavaScript in action.

Manipulate the DOM

JS can get any html element and do stuff, like:

- change an element's content or modify an element's CSS properties:
- retrieve values from form objects
- perform calculations and output the result
- When fresh data appears without the whole page reloading, that's JS.

JavaScript is written...

- between a pair of tags, in either the head or body of an **html** file.
- in a **.js** file, which is imported into either the head or body of an html page.

Per Wikipedia:

```
JavaScript (JS), is a programming language that is one of the core
technologies of the World Wide Web, alongside HTML and CSS. Over 97% of
websites use JavaScript on the client side for web page behavior, often
incorporating third-party libraries. All major web browsers have a
dedicated JavaScript engine to execute the code on users' devices...

JavaScript engines were originally used only in web browsers, but are now
core components of some servers and a variety of applications. The most
popular runtime system for this usage is Node.js ...

Although Java and JavaScript are similar in name, syntax, and respective
```

```
standard libraries, the two languages are distinct and differ greatly in
design.
```

1. Launch your code editor (Visual Studio Code, Sublime Text, etc.). If you are in a Noble Desktop class, launch Visual Studio Code.
2. Navigate to the JavaScript-Fundamentals folder.
3. Open the 00-Introduction folder and from in there, open the file **00.01-Introduction-to-JavaScript.html**.

script tag

When written directly in an html page, JS is wrapped in a script tag, which goes right before the closing body tag. Putting the JS code at the end lets the web page fully load before any JS "goes looking" for any particular element to manipulate.

4. Add a pair of script tags right before the closing body tag. Separate the script tags and skip a line:

```
    <script>
    </script>
</body>
```

output

Programs produce output – be it a single-line greeting or a complete, interactive application. Output is also necessary at each step in the coding process, as it provides feedback that tells if the code written so far is working—or if it's buggy (has errors).

alert()

alert() is a JS method. Think of methods as the verbs of programming languages. They perform actions. In this case, the action is to pop-up a dialog box that displays the string (text) which was passed to the parentheses.

```
alert('Hello World');
```

string

In JS and other programming languages, plain text is known as a string. Strings go in quotes, either single or double.

5. Inside the script tag, add an alert with the string: "Hello World"

```
<script>
    alert('Hello World');
</script>
```

6. Save the file and open it in Chrome. You should see a dialog that says *Hello World*.
7. Close the alert, but leave the html page open in the browser. We'll be reloading the page often as we proceed.

console.log()

Running Javascript in the Browser Console • You do not need a DOM, that is to say, a web page, in order to write and run Javascript. • You can play with JS directly in the Console, as many developers do—no html page required. • To output content to the Console, use console.log(). In the Chrome browser, type Command-Option-I to open Developer Tools. Add this command:

Now, we'll switch to the console.log() method, which is what we'll be using from now on. console.log() also outputs whatever is in its parentheses, but its output goes to the Console.

8. Use the console.log() method to output another message. Strings (text) can go in either double or single quotes, so switch to double quotes:

```
alert('Hello World');
console.log("Hello from the Console");
```

9. Save the file and reload the browser. You get the alert again, but there's more:
10. Right-click the page and choose Inspect; then click the Console tab. It should say: Hello from the Console

Global Window Object

The *global window object* is the top-level object in JavaScript, corresponding to the browser window itself.

- *alert()* is a method of the window
- *console* is an child object of the *window* object.
- *log()* is a method of the *console* object. Since the window object is top-level, it is assumed, and therefore can be omitted from the syntax. But we can also add them:

11. Put *window* in front of the alert and console keywords and run it again. It still works:

```
window.alert('Hello World');
window.console.log("Hello from the Console");
```

semi-colon

Notice the semi-colons (😉 which mark the end of each line of code. Semi-colons are optional, but recommended. We'll be using them.

comment

Comments explain what the code is doing. Comments are for us humans and, as such, are ignored by JS when the program is run. Comments have no effect on how a program runs, but they do make a program easier to read and understand. We recommend you comment your code.

single-line comment

A double slash at the start of a line turns the whole line into a comment.

12. Add a couple of one-line comments to start the script:

```
// alert() and console.log() output what's in parentheses
// window. is optional, since it is understood
window.alert('Hello World');
window.console.log("Hello from the Console");
```

in-line comment

If a comment is short enough, it can go on the same line as the code to which it refers.

13. Add an in-line comment:

```
// alert() and console.log() output what's in parentheses
window.alert('Hello World');
window.console.log("Hello from the Console");
```

commenting-out code

Comments are used to deactivate code without having to delete it. A double slash at the start of a line comments it out.

14. We don't want to keep getting that pop-up, so comment-out the alert.

```
// window.alert('Hello World');
window.console.log("Hello from the Console");
```

We could have just deleted the alert, but by commenting it out instead, we keep an example for study and review purposes.

multi-line comments

If a comment runs more than one line, you can wrap all in /* ... */.

15. Replace the single-line comment at the top of the script with a multi-line comment:

```
/*
2 ways to output JS:
```

```
1.) alert() for a pop-up in the browser
2.) console.log() for output to the console
*/

// window.alert('Hello World');
window.console.log("Hello from the Console");
```



---

anonymous function

A function without a name is called an *anonymous function*. These occur when the function code is written in-line or set equal to an event, as opposed to as a separate location that must be referenced by name. Example of anonymous function:

```
myButton.onclick = function() {
    alert('You clicked the button!');
};

// or:
myButton.addEventListener('click', function() {
    alert('You clicked the button!');
});
```

Also see **function**

argument / parameter

- Data that goes between the parentheses of a method or a function is called an **argument**.
- When the function is defined, the data in the parentheses, known as **parameter(s)** is defined as a variable.
- When the function is called, an argument is passed into the parentheses. In this way, the argument sets the value of the parameter,

```
function greetUser(name) {
    console.log(`Hello, ${name}!`);
}
greetUser("Brian"); // Hello, Brian!
```

array

An array is a type of **object** that store data as numbered list items. The syntax is to put the values, which can be of any datatype, inside of square brackets. Items are referenced by their numeric position, known as the **index**, which start numbering at 0.

```
let fruits = ['kiwi', 'pear', 'plum'];
console.log(fruits[0]); // kiwi
```

Also see **index**, **object**.

assignment operator

The equals sign **=** is known as the *assignment operator*, because it assigns or sets a value: **x = 5** assigns **5** as the value of **x**.

```
let x = 8;
x = 9;
```

asynchronous

The term **asynchronous** (async, for short) refers to code being executed at different times, particularly due to delays in data being made available as in a server request. One function makes a request for data from a server, and another function must asynchronously wait for that data to load and be made available. Callback functions are used to handle asynchronous requests.

block scope

Block refers to variables that are avaialable only within the particular code block in which they are declared, as defined by a pair of curly braces { }. Also see *scope*, *function scope* and *global scope*.
The rules for **variable scope**:
- variables not declared inside any {} are **global**, which means they are available everywhere in the script.
- **let** and **const** declared inside {} are **block scoped**, which means they are available only inside that code block.
- **var** declared inside {} are **global** (unless the {} is a function)
- Any variable declared inside a function is available only to that function. These are known as **local variables** or **function-scoped variables**.

boolean

A variable or value that can be only *true* or *false*. Conditional logic resolves to a boolean:

```
let isOnline = true;
isOnline = false;
```

condition

A condition is something that is evaluated and resolves to *true* or *false*. Conditions are found in loops and in if-else statements (conditional logic). The condition of a loop must be true for the loop's code block to execute. In this example, the condition is **x < 10**. If the condition is true, the code inside the {} will run:

```
let x = 9;
if(x < 10) {
    x++;
}
```

console

The *console* is a browser debugging environment where you can output code via the command. The following would produce output of `25` in the console, which may be accessed via Dev Tools or Inspect > Console: `let x = 5; console.log(x * x); // 25`.

const

The `const` keyword for declaring primitive variables (string, number, boolean) prevents them from being changed. Objects declared with `const` can have their properties and items changed by cannot have their data type changed.

counter variable

A variable that exists to keep count of something. Loops rely on counter variables to be incremented or decremneted with each iteration of the loop so that the condition driving the loop can ultimately become false and the loop can therefore stop.

document

The webpage, a child object of the Window. JS "sees" the document and all of its elements as a hierarchy of so many nested objects, which JS can manipulate in a myriad of ways.

dot syntax

The use of dots to access or "drill down into" objects is known as dot syntax. In this example, **triStates.ny.capitol** returns the value of the **capitol** property, "Albany", which is a child of the **ny** property, itself an object, which is a child of the **triStates** object:

```
    const triStates = {
        ct: {
            nickname: "Constitutino State",
            capitol: "Hartford"
        },
        nj: {
            nickname: "Garden State",
            capitol: "Trenton"
        },
        ny: {
            motnicknameto: "Empire State",
            capitol: "Albany"
        }
    }

    console.log(triStates.ny.capitol); //  Albany
    console.log(triStates.ny); // {nickname: "Empire State", capitol:
"Albany"}
    console.log(triStates.nj.motto); //  Garden State
    console.log(triStates.nj); // {nickname: "Garden State", capitol:
"Trenton"}
```

else

Conditional logic is often expressed by if-else statements: if a condition is true, do something; **else** do something else:

```
let x = 2;
if (x > 3) {
    console.log('The condition is true!');
} else {
    console.log('The condition is false!');
}
```

Also see *if-else logic*.

event

An **event** is something that happens during the running of an application. Events are often used as triggers to call functions. Common events include **click** and **change**:

```
myButton.addEventListener('click', doSomething);
mySelectMenu.addEventListener('change', doSomethingElse);
```

for loop

A for loop executes a block of code again and again, as long as a condition is true. The condition must become false eventually, or else the loop never ends and is called an *infinite loop*. A for loop considers three pieces of information between its parentheses in order to decide if it will run the code inside its curly braces: a **counter**, a **condition** and an **incrementer**:

```
for(let i = 0; i < 10; i++) {
    console.log(i*i); // 1, 4, 9, 16, 25, 36, 49, 64, 81, 100
}
```

function

A function is a block of code that runs only when it is invoked (called). The function can be called in the code or by an event on the web page, such as a button click. A function usually (but not always) has a name to call it by:

```
function greetUser(name) {
    console.log(`Hello, ${name}!`);
}

greetUser("Brian"); // Hello, Brian!
myButton.addEventListener('click', greetUser);
```

function scope / local scope

Function scope / local scope refers to variables that are available only inside the function in which they are defined. Also see *scope*, *global scope* and *local scope*.

global/ window object

The highest level object in JS is the *global window object*, the direct children of which include the *document*, *console*, *navigation* and *history* objects. The window object is assumed and need not be referenced:

```
alert("Hello!");
// or:
window.alert("Hello again!");
```

global sscope, global memory

Global scope / memory refers to variables that are declared at the level of the *global object* and as such are avaialable throughout a script, as opposed to being confined to the scope of a particular code block. Also see *scope*, *function scope* and *local scope*. In this example **x** is global because it is declared outside of any code block, but **i** is scoped to the for loop, and so is *not* available outside the code block. Attempts to reference it result in **ERROR not defined**.

```
let x = 5; // global variable

for(let i = 0; i < 10; i++) {
    console.log(i);
}

console.log(x); // 5
console.log(i); // ERROR i is not defined
```

history object

A direct child of the window, the *history object* has methods pertaining to the browser history. Although the *window* part is optional, it is customary to include window when referencing the history object:

```
// reload the last visited browser page
window.history.back();
```

if-else logic, if-statement

The logic of decision making in programming, conditional logic is often expressed by if-else statements: if a condition is true, do something; else do something else:

```
let x = 10;
let y = 3;

if(x > y) {
    console.log('x is greater than y!');
} else {
    console.log('x is equal to or less than y!');
}
```

Also see *conditional logic*.

index

The position number of an item in an array, with the first item at index 0:

```
let fruits = ['kiwi', 'pear', 'plum'];
console.log(fruits[0]); // kiwi
```

key

The name of a property is called the *key*. This object has keys of **name**, **age** and **retired**.

```
let guy = {
    name: "Bob",
    age: 40,
    retired: false
}
```

length

Called on an array, the *length* property returns the number of items in the array:

```
let fruits = ['kiwi', 'pear', 'plum'];
console.log(fruits.length); // 3
```

let

The **let** keyword is for declaring block-scoped variables, meaning that the variables only exist inside the curly braces in which they are declared. In this example, **power** is declared in the **global scope** and so is available everywhere in the script. But **i** and **cube** are declared inside the for loop, which makes them **block scoped** variables, so NOT available in the global scope:

```
let power = 3;
```

```
for(let i = 0; i < 10; i++) {
    let cube = i ** power;
    console.log(cube);
}

console.log(power); // 3
console.log(i); // ERROR cube is not defined
console.log(cube); // ERROR cube is not defined
```

loop

A loop is a programming routine that executes a block of code again and again, as long as a condition is true. There are a few kinds of loops in JS, including "for loops" and "while loops":

```
for(let i = 0; i < 10; i++) {
    console.log(i ** 4); // 1, 16. 81, 625 ... 10000
}
```

methods

Methods are the verbs of programming languages-—they do stuff; they perform actions. Just as verbs go together with nouns, methods are attached to objects.

- Methods perform a myriad of tasks: output content to html elements, load data from the server, play sounds, create new html elements, and so on.
- Dot-syntax is how we access the methods of a JS object. First comes the object, followed by a dot. Then comes the method. So it goes object-dot-method:

```
let dateTime = new Date();
// the Date object's getHours() method:
let hour = dateTime.getHours();
// the console's log method:
console.log(hour); // integer in 0–23 range
```

modulo

The *modulo operator*, symbol `%` returns the remainder when one number is divided by another:

```
console.log(15 % 4); // 3
```

nesting

Nesting refers to something inside of another similar thing, such as a loop inside of a loop, a block of conditional logic (if-statement inside) conditional logic, or an array inside of an array:

```
// nested for loop: a loop inside a loop
for(let i = 0; i < 5; i++) {
    for(let j = 0; j < 5; j++) {
        console.log(i**j); // i to the j power
    }
}

// nested if-else:
let weather = "cloudy";
let windy = true;

if(weather == "cloudy") {
    console.log("Go to the park");
    if(windy) {
        console.log("Fly a kite");
    } else {
        console.log("Have a picnic");
    }
}
```

objects

Objects are variables that can store more than one value at a time. They are divided broadly into Objects, per se, and a type of object called an Array. Objects proper store data inside curly braces as *properties*, consisting of name-value (key-value) pairs: name:

```
let cat = {
    name: "Fluffy",
    age: 4,
    cute: true
};
```

property

A property is a name-value (key-value pair) that belongs to -- is scoped to -- an object. As such, a property is essentially a local variable. The properties of an object can be nested. Properties are accessed by means od **dot syntax**:

```
let cat = {
    name: "Fluffy",
    age: 4,
    cute: true,
    owner: {
        name: "Bob",
        age: 40,
        job: "Programmer"
    }
};
```

```
console.log(cat.name); // Fluffy
console.log(cat.owner.name); // Bob
console.log(cat.age); // 4
console.log(cat.owner.age); // 40
console.log(cat.cute); // true
console.log(cat.owner.cute); // undefined
console.log(cat.job); // undefined
console.log(cat.owner.job); // Programmer
```

return value

Think of a function as an input-output machine, with the arguments being the input and the **return** value being the output. A **return** value "exports" the function result for use elsewhere. In order to "capture" the return value, set the function call equal to a variable:

```
function addNums(a, b) {
    return a + b;
}

let sum1 = addNums(5, 6);
console.log(sum1); // 11

let sum2 = addNums(8, 9);
console.log(sum2); // 17
```

string

Pure text in quotes is a string. Setting a variable equal to a string gives the variable a data type of **string**:

```
let firstName = "Brian";
console.log(firstName, typeof(firstName)); // Brian string
```

ternary

A ternary expression is a concise, one-line alternative to an else-if statement. It uses **?** and **:** instead of **if()** and **else**, while also dispensing with the curly braces:

```
let x = 5;

// if x is less than 10, add 1; else set x equal to 0:
if(x < 10) {
    x++;
} else {
    x = 0;
}
```

```
    // ternary version of the above:
    x < 10 ? x++ : x = 0;
```

this

The this keyword refers to various objects, depending on where it is encountered:

- in the global scope, *this* is the Global Window Object
- inside a function, *this* is the object (button, menu, etc) whose event ('click', 'change', etc) called the function.
- in a function defined inside an object (a method), *this* refers to the object itself.

var

The `var` keyword for declaring variables has largely been supplanted by `let` due to the greater control over variable scope provided by the latter. A `var` declared inside the curly braces of a loop or if-statement are in the global scope, whereas `let` variables declared inside the curly braces of a loop or if-statement are scoped to that code block:

```
    // j exists outside the loop, because it is declared with var:
    for(var j = 0; j < 5; j++) {
        console.log(j**2); // j squared
    }
    console.log(j); // 5

    // i exists ONLY inside the loop, because it is declared with let:
    for(let i = 0; i < 5; i++) {
        console.log(i**2); // i squared
    }
    console.log(i); // ERROR: i is not undefined
```

while

A **while loop** is a kind of loop. Unlike a **for loop**, which has the counter, condition and incrementer inside parentheses, a while loop has these elements scattered about, with the counter declared above the loop, the conditon evaluated inside parentheses, and the incrementer inside the curly braces. In general, use a for loop if you know exactly how many times you want the loop to run, and use a while loop if the number of iterations is not pre-determined:

```
    let frutes = ['apple', 'banana', 'grape', 'kiwi', 'orange', 'pear'];

    // using a for loop, because the number of iterations is known:
    for(let i = 0; i < frutes.length; i++) {
        console.log(frutes[i] + " has " + frutes[i].length + " letters!");
    }

    /* using a while loop because the number of iterations is not known;
the loop ends with the first 4-letter item, but its index may not be
```

```
known. */

    let n = 0;
    while (frutes[n].length != 4) {
        console.log(frutes[n] + " is " + frutes[n].length + " letters
long!");
        n++;
    }

    /* the for loop will run 6 times as it fully traverses the array, but
the while loop will only run 3 times, because it will quit when the
condition is false, which occurs when the current item is 'kiwi'.
    apple has 5 letters!
    banana has 6 letters!
    grape has 5 letters!
    kiwi has 4 letters!
    orange has 6 letters!
    pear has 4 letters!
    apple is 5 letters long!
    banana is 6 letters long!
    grape is 5 letters long!
    */
```

window

- The Javascript Window object is the top-level "thing" in html: it is above the document (web page) itself.



Workbook Instructions

How to Proceed with each Lesson and Lab in this course:

---

html file

- Open the current lesson **html** file in your Code Editor.
- Scroll down to confirm that the **script** tag is importing **FINAL.js**.
- Preview the **html** file in the Chrome browser.
- Right-click **Inspect** to check the Console for the final output.
- Back in the html file, switch to importing the **PROG.js** file, which is a blank copy of **START.js**.

js files

- Each lesson has 3 JS files: **START.js**, **PROG.js** and **FINAL.js**.
- Write your code for each lesson in **PROG.js**, where starter code and instructional comments are provided.
- Each numbered step in a lesson has code for you to type.
- Each step explains what you are to type next, and why.
- Check the Console with each step to make sure your code is working.
- It is recommended that you do not type your code in **START.js**, but rather keep it as a blank and so a Save As to get a **PROG.js**, as needed. That way, you will always have a blank starter page in case you wish to review a lesson by typing the code over again.

labs

- Most lessons are accompanied by a **Lab** for you to do after you have completed that lesson.
- Labs are challenges, in that the code steps are not provided. Instead, you are given prompts and must figure out what code to type.
- Unlike lesson html files, lab html files do NOT have separate JS file to import. In order to streamline the files system, all Lab JS code is typed in the Lab html page, in the script tag.
- All Lab hmtl files have a companion Solution html file. This is where you check the answers--but do so only after having given the lab your best effort.

sumbitting lab work as HW

- Labs are homework. When you are done, check the Solution file, but also submit your work through the class Slack Channel, or by whatever means your instructor provides.
- The Solution JS code is not necessarily the only way to do something. By submitting your code -- which may differ from the "official solution", your instructor can give you valuable coding tips and

feedback on best practices.

markdown files

- Each lesson has a markdown file with extension **.md**.
- The markdown file contains the step-by-step instructions for the lesson or lab.
- This file can be rendered in your VSCode Editor by typing **Command-Shift-V**.
- You may want to use a vertical split screen view, with your coding page (PROG.js) in one pane, and the readme file for that lesson in the other pane.
- To enable vertical split screen in VSCode, choose **View > Editor Layout > Split Up**.
- Both lessons and labs have companion markdown files.

pdf files

- Your big **PDF** file, **Noble-Desktop-JavaScript-Bootcamp-Workbook.pdf**, is a collection of all the many markdown files for each lesson and lab. It is assembled all in one place for your convenience.
- Workflow option: you may prefer to keep your whole screen for coding and to not to split your screen between the JS file and the markdown file. In that case, you may want to set up a second computer / monitor and keep the the **PDF** file open on that other monitor.
- PDF files can be viewed in Adobe Acrobat Reader, in Preview on Mac and in the Chrome browsser.



UNIT 01

LESSON 01.01



---

var vs. let

string variables

number variables

typeof() method

boolean variables

undefined variables

---

variables

A variable is a container that stores a value. Some variables (vars, for short) can hold many values at a time. Other vars can only hold one value at a time. Vars that only hold one value at a time are sometimes referred to as **primitive tyes**. These "primitives" are the topic of this lesson.

variable data types

There are two major categories of variables, **objects** and **primitives**:

- **object** is a broad **data type** encompassing variables that can store multiple values. These include:
    - **object** -- a data structure with properties as name-value pairs and, optionally, with methods (functions scoped to the object)
    - **array** -- ordered lists of items, with each item stored by its numeric position, called the index.
    - **DOM object** -- JS "versions" of html elements (div, button, etc.)
    - **function** -- code blocks which only run when invoked (called)
    - **null** -- an empty object, used typically as a placeholder value
    -
- **primitives** are variables that are capable of storing only *one* value at a time. Primitives come in various **data types**:
    - **string** -- with value as text in quotes (e.g. "Hello World")
    - **number** -- with value as integer and decimal (e.g. 3, 3.5)
    - **boolean** -- with value of only **true** or **false**
    - **undefined** -- with no value assigned, typically used with the understanding that a value will be assigned later
    -

In this lesson we will focus on primitives.

declaring variables with let

To begin using a variable, we need to *declare* it. This is done by writing a *keyword* (**var** or **let**), followed by the name of your variable, which can be pretty much anything you like, although naming rules and conventions do apply:

variable naming rules and restrictions

- No spaces allowed in variable names
- No special characters allowed, except **$** and **_**
- Name cannot start with a number (**1day** bad; **day1** good)
- No reserved words allowed (**alert** bad; **myAlert** good)

variable naming conventions (best practices)

- Use **camelCase** (it's **highScore**, not **high_score**)
- Don't use all UPPERCASE, unless declaring a constant (value will never change)
- Choose concise names (**tel**, not **telephoneNumber**)
- Choose precise names (**salesTax** not **additionalCharge**)

A variable is declared with **var** or **let**, altnough **let** is the more modern syntax and should be used instead of **var** for reasons that we will expound when we start talking about **variable scope**.

string variables

Strings are text enclosed in either single or double quotes:

1. Declare a variable with **var** and assign it a string in double quotes:

```
var pet = "cat";
console.log(pet); // cat
```

2. Change the value to another string, this time in single quotes:

```
pet = 'dog';
console.log(pet); // dog
```

One difference between **var** and **let** is that a **var** can be redeclared.

3. Redeclare **pet**:

```
var pet = 'bunny';
console.log(pet); // bunny
```

A variable declared with **let** cannot be redeclared--attempting to do so throws an error:

4. Declare a variable with **let**, and then try to redeclare it:

```
let petSound = "Woof!";
console.log(petSound); // Woof!
let petSound = "Grrr!";
Error: Identifier 'petSound' has already been declared
```

To change the value of an existing variable, don't redeclare it. Just set it equal to something else.

5. Comment out **let petSound = "Grrr!"**, and then set **petSound** to "Grrr!" without redeclaring the variable:

```
// let petSound = "Grrr!";
petSound = "Grrr!";
console.log(petSound); // Grrr!
```

Multiple values can be outputted in the same console.log:

6. Output both variables in one console.log:

```
console.log(pet, petSound); // dog Grrr!
```

7. For added clarity, you can include 'labels' in the console output:

```
console.log('pet:', pet, ' petSound:', petSound);
// pet: dog  petSound: Grrr!
```

number variables

A number can be an integer (int, for short) or a float (decimal). There are no commas in numbers. Whether a number is an integer or a float, its data type is number.

8. Declare three number variables, setting them equal to integer, float, and four-digit int:

```
let price1 = 35; // integer
let price2 = 3.5; // float
let price3 = 3500; // no comma
console.log(price1, price2, price3); // 35 3.5 3500
```

Naturally, number variables can be used to do math. The result of a mathematical calculations is often saved to a variable, but directly console logging the answer also works:

9. Take these basic math operators for a spin: **+, -, *, /**:

```
    let sum = price1 + price2 + price3;
    console.log(sum); // 3538.5
    console.log(price1 - price2); // 31.5
    console.log(price1 * price2); // 122.5
    console.log(price1 / price2); // 10
```

boolean variables

A boolean is a variable that can only be **true** or **false**. The names of booleans often begin with *is* to emphasize the either-or concept and to let you know at a glance that this is a boolean.

10. Declare a boolean variable with a value of *true*:

```
    let online = true;
    let isOnline = true; // prefix 'is' helps us know that this is a
boolean
    console.log(online, isOnline); // true true
```

Changing the value of a boolean is known as *toggling* or *flipping* the boolean. This can be done by direct assignment, or by setting the variable equal to itself with **!** in front of it.

11. Flip the boolean from true to false, and then flip it back by putting **!** in front of it:

```
    isOnline = false;
    console.log('isOnline', isOnline); // isOnline false
    isOnline = !isOnline;
    console.log('isOnline', isOnline); // isOnline true
```

undefined

A variable can be declared without a value being assigned. The assumption is that a value will be assigned later. Until then, both the value and datatype are **undefined**.

12. Declare a variable, but don't assign it a value:

```
    let player1;
    console.log('player1', player1);
    // player1 undefined
```

typeof() method

The **typeof()** method takes a variable as its argument and returns the datatype.

13. Declare variables of each of the four major "primitive" types: string, number, boolean and undefined. Then log the variable name, value and data type:

```
let ketchup = "Heinz";
console.log('ketchup', ketchup, typeof(ketchup));
// ketchup Heinz string

let varieties = 57;
console.log('varieties', varieties, typeof(varieties));
// varieties 57 number

let fresh = true;
console.log(fresh); // true
console.log('fresh', fresh, typeof(fresh));

let total;
console.log('total', total, typeof(total));
// total undefined undefined
```

Multiple variables can be delared in one line with a single keyword. The individual declarations are separated by commas.

14. Declare three variables with just one instance of the **let** keyword:

```
let x = 1, y = 2, z = 3;
console.log(x + y * z); // 7
```

Such "one-liner" variable declarations are more common when the vars that are not to be assiged an initial value.

15. Declare four variables with just one instance of the **let** keyword, leaving them all without an initial value, so *undefined*:

```
let day, date, month, year;
console.log(day, date, month, year);
// undefined undefined undefined undefined
```

Undefined variables are not errors, but some programmers prefer to avoid them, anyway. As a workaround, you can assign a starter value and then just change it later. The conventional starters are **""** (empty string) for strings and **0** for numbers:

16. Declare a string and a number, assigning them starter values of "" and 0, respectively:

```
let grade = "";
let score = 0;
```

declaring a variable with value equal to another variable

If you declare a variable and set its value equal to an existing variable, the copy becomes its own independent entity, which means that the copy is in no way "linked" to the original. If you change the value of the copy, the original is not aaffected.

17. Declare a variable, and set it equal to score from the previous step:

```
let myScore = score;
myScore = 98;
console.log(myScore); // 98
console.log(score); // 0
```

- **END Lesson 01.01**
- **NEXT: Lab 01.01**
- **Lesson 01.02**



Instructions:

- Open **01.01-Lab.html** in your Code Editor.
- Write your code in the **script** tags, where starter code is provided.
- Debug the following variables by making them into legal names with value of indicated data type.
- Rreplace all instances of **VAR** in the **console.log** with your variable.

Example:

Question 0:

0. **type: string**

```
// let first-name = Bob;
// console.log('0.', 'VAR', VAR, typeof(VAR));
```

Solution:

0. **type: string**

- Remove the hyphen from the variable name.
- Put the value of the string in quotes: "Bob" or 'Bob'.

```
let firstName = 'Bob';
console.log('0.', 'firstName', firstName, typeof(firstName));
// 0. firstName Bob string
```

On your own:

1. **type: number**

```
let admission Fee = $10;
console.log('1.', 'VAR', VAR, typeof(VAR));
```

2. **type: string**

```
let #1sportsCar = "Porsche";
console.log('2.', 'VAR', VAR, typeof(VAR));
```

3. **type: boolean**

```
let is Online = FALSE;
console.log('3.', 'VAR', VAR, typeof(VAR));
```

4. **type: number**

```
let %done = 22.5%;
console.log('4.', 'VAR', VAR, typeof(VAR));
```

5. **type: undefined**

```
let password "&123b45d";
console.log('5.', 'VAR', VAR, typeof(VAR));
```

6. **type: string**

```
let 26miles = marathon;
console.log('6.', 'VAR', VAR, typeof(VAR));
```

7. **type: string**

```
let $100000Bar = "candy bar";
console.log('7.', 'VAR', VAR, typeof(VAR));
```

8. **type: number**

```
let firstPrize = 7,500;
console.log('8.', 'VAR', VAR, typeof(VAR));
```

9. **type: boolean**

```
let i_Won! = True;
console.log('9.', 'VAR', VAR, typeof(VAR));
```

10. **type: undefined**

```
    let greeting = Hola;
    console.log('10.', 'VAR', VAR, typeof(VAR));
```

- **END Lab 01.01**
- **SEE Lab 01.01 Solution**

01.01 Lab Solution

---

1. **type: number**

- Variable names cannot have spaces.
- Value must be a number, so it cannot be $10, which, is a string requiring quotes: '$10'.

```
// let admission Fee = $10;
let admissionFee = 10;
console.log('1:', 'admissionFee', admissionFee, typeof(admissionFee));
```

2. **type: string**

- Variable names cannot have special characters (other than _ and $).

```
// let #1sportsCar = "Porsche";
let sportsCar1 = "Porsche";
console.log('2:', 'sportsCar1', sportsCar1, typeof(sportsCar1));
```

3. **type: boolean**

- Variable names cannot have spaces.
- Boolean values (true, false) are lowercase.

```
// let is Online = FALSE;
let isOnline = false;
console.log('3:', 'isOnline', isOnline, typeof(isOnline));
```

4. **type: number**

- Variable names cannot have special characters (other than _ and $).
- 22.5% percent as a decimal is .225, not 22.5

```
// let %done = 22.5%;
let pctDone = .225;
console.log('4:', 'pctDone', pctDone, typeof(pctDone));
```

5. **type: undefined**

- A variable without a value is undefined, so remove the value.

```
let password = "&123b45d";
let password;
console.log('5:', 'password', password, typeof(password));
```

6. **type: string**

- Variable names cannot start with a number.
- String values go in single or double quotes.

```
let 26miles = marathon;
let miles26 = "marathon";
console.log('5:', 'password', password, typeof(password));
```

7. **type: string**

- The $ is allowed, so the variable name is fine, as is.
- String values go in single or double quotes.

```
// let $100000Bar = candy bar;
let $100000Bar = "candy bar";
console.log('7:', '$100000Bar', $100000Bar, typeof($100000Bar));
```

8. **type: number**

- Numeric values cannot have commas.

```
// let firstPrize = 7,500;
let firstPrize = 7500;
console.log('8:', 'firstPrize', firstPrize, typeof(firstPrize));
```

9. **type: boolean**

- i_Won is legal, although iWon (camel-case) is better
- Variable names cannot include !
- boolean values (true, false) are lowercase

```
// let i_Won! = True;
let iWon = true;
console.log('9:', 'iWon', iWon, typeof(iWon));
```

10. **type: undefined**

- A variable without a value is undefined, so remove the value.

```
    // let greeting = Hola;
    let greeting;
    console.log('10:', 'greeting', greeting, typeof(greeting));
```

END 01.01 LAB SOLUTION

Next: Lesson 01.02



"number-like strings"

NaN (Not a Number)

Number() method

null

not defined

string concatenation

"number-like strings"

A "number-like string" is a number enclosed in quotes. So, whereas 5 is an actual number, "5" is a "number-like string".

1. Declare a variable with a "number-like string" value, and try to do addition with it:

```
let bill = 50;
let tip = '10';
let total = bill + tip;
console.log(total); // 5010
```

Addition with number-like strings fails, because the plus-sign defaults to concatenation. But for other operations with number-like strings -- subtraction, multiplication, division -- the math works, because there is no plus-sign to confuse things.

2. Declare a number-like string and do division with it:

```
let pizzas = "4";
let people = 8;
let pizzasPP = pizzas / people;
console.log(pizzasPP); // 0.5
```

NaN

NaN (Not a Number) results from trying to do math with something that is neither a number nor a number-like string.

3. Try to do math with a price that includes a dollar sign:

```
let fullPrice = '$80';
let halfPrice = fullPrice * 0.5;
console.log('halfPrice', halfPrice); // NaN number
```

The string '$80' is in no way understood as the number 80, so attemptinh to do math with '$80' fails.

Number() method

The **Number()** method takes a variable as its argument, and where possible, returns a number. It is ideal for converting "number-like strings" into actual numbers.

4. A number in quotes such as "4"` is a "number-like string". Convert it to a number:

```
console.log('pizzas', pizzas, typeof(pizzas));
// pizzas 4 string
pizzas = Number(pizzas);
console.log('pizzas', pizzas, typeof(pizzas));
// pizzas 4 number
```

If the string passed to it cannot be converted to a number, the **Number()** method returns **NaN**.

5. Try to convert "banana" to a number:

```
let fruit = "banana";
let baNaNa = Number(fruit);
console.log('baNaNa', baNaNa, typeof(baNaNa));
// baNaNa NaN number
```

Despite its name, **NaN** has a data type of number.

"Number-like strings" can't be used for addition, but you can convert them with the Number() method.

6. Convert "15" to an actual number, so that it can be used for addition:

```
bill = 70;
tip = '15';
total = bill + tip;
console.log(total); // 7015

total = bill + Number(tip);
console.log(total); // 85
```

not defined vs undefined

not defined means the variable does not exist. It is an error that usually arises from typos. **undefined** means that the variable exists, but has no value.

7. To see the difference between *undefined* and *not defined*, declare a variable with no value, and then misspell it:

```
    let island;
    console.log(island, typeof(island)); // undefined undefined
    island = "Bali";
    console.log(island, typeof(island)); // Bali string
    // console.log(ixland); // error: ixland is not defined
```

null

null and **undefined** are both *falsey* (return false in a boolean context), but null is an actual value assigned to a variable. It has a data type of object, but it's just that null is an *empty* object.

8. Declare a variable, set it to null, and log it:

```
    let user = null;
    console.log('user', user, typeof(user));
    // user null object
```

string concatenation

Variables and substrings can be joined together with plus-signs (**+**) to make one bigger string. The procedure is known as **string concatenation**.

9. Concatenate the **topic** variable with substrings:

```
    let topic = "JavaScript";
    let intro = "Let's learn " + topic + "!";
    console.log(intro); // Let's learn JavaScript!
```

10. Concatenate with two variables:

```
    let firstName = 'Brian';
    let lastName = 'McClain';
    let greeting = 'Hello, class! My name is ' + fullName + ' ' + lastName
+ '.';
    console.log(greeting);
    // Hello, class! My name is Brian McClain.
```

Concatenation is often used for making multiple versions of similar strings, such as a set of image paths that differ only slightly by file name.

11. Concatenate an image file path consisting of two variables and three substrings:

```
let kind = 'Jack';
let suit = 'Hearts';
let imgPath = 'images/' + kind + '-of-' + suit + '.jpg';
console.log(imgPath); // images/Jack-of-Hearts.jpg
```

12. Change the variable values to get new image paths:

```
kind = 'Queen';
suit = 'Diamonds';
imgPath = 'images/' + kind + '-of-' + suit + '.jpg';
console.log(imgPath); // images/Queen-of-Diamonds.jpg

kind = 'King';
suit = 'Clubs';
imgPath = 'images/' + kind + '-of-' + suit + '.jpg';
console.log(imgPath); // images/King-of-Clubs.jpg

kind = 'Ace';
suit = 'Spades';
imgPath = 'images/' + kind + '-of-' + suit + '.jpg';
console.log(imgPath); // images/Ace-of-Spades.jpg
```

A plus-sign performs concatenation rather than addition if the expression includes a string.

13. Try adding numbers with a dollar-sign present. It reverts to string concatenation:

```
let food = 25;
let bev = 15;
let tip = 8;
let tot = '$' + food + bev + tip;
console.log(tot, typeof(tot)); // $25158 string
```

14. Remove the dollar sign, and run it again; this time the math works. *After* the math is done, put back the **$**:

```
tot = food + bev + tip;
console.log(tot); // 48
console.log(tot, typeof(tot)); // 48 number
tot = '$' + tot;
console.log(tot, typeof(tot)); // $48 string
```

changing a variables's data type
In the above example, the number **tot** is changed into a string. Changing the datatype of a variable *is* permitted, but it should be done sparingly.

16. Change a three digit number into a four-digit PIN by concatenating a leading zero. This result is a string, but the reason for changing datatypes does make sense:

```
let num = 582;
console.log(num, typeof(num)); // 582 number
let pin = "0" + num;
console.log(pin, typeof(pin)); // 0582 string
```

- **END Lesson 01.02**
- **NEXT: Lab 01.02**
- **Lesson 01.03**



Instructions:

- Open **01.02-Lab.html** in your Editor. Write the Lab code in the **script** tags.

Assign values to the provided variables and use **typeof()** to log output that matches the comment:

```
// example:
var flavor;
console.log('?'); // vanilla string

// solution:
var flavor = 'vanilla';
console.log(flavor, typeof(flavor));

// on your own:
var msg;
console.log('?'); // Hello World! string

var total;
console.log('?'); // 314 number

var isFresh;
console.log('?'); // true boolean

var price;
console.log('?'); // price: $ 29.95 number

var player1;
console.log('?'); // player1: undefined 'undefined'

var restaurantBill = 80;
var groupon = '$10';
var pleasePay;
console.log(pleasePay); // NaN 'number'
```

Use the Number() method, as needed, to get the expected values:

```
// example:
var x = '55';
var y = 6;
```

```
var z;
console.log(z); // expected: 330

// solution:
var x = '55';
var y = 6;
var z = Number(x) * y;
console.log(z); // 330

// on your own:
var x = '55';
var y = '33';
var z;
console.log(z); // expected: 22

var x = 55;
var y = '33';
var z;
console.log(z); // expected: 88

var x = '55';
var y = 5;
var z;
console.log(z); // expected: 11

var x = 55;
var y = '44';
var z;
console.log(z); // expected: 5544
```

string concatenation

Assign values to the variables and then use the variables in string concatenation to reproduce the ***messages shown***:

1. message 1: ***Sally lives in New York City.***

```
let msg1;
let name;
let city;
console.log(msg1);
```

2. message 2: ***Freddy went to France for 6 months.***

```
let msg2;
let fName;
let country;
let months;
console.log(msg2);
```

3. message 3: ***My Bunny had a baby carrot for breakfast.***

```
    let msg3;
    let pet;
    let veg;
    let meal;
    console.log(msg3);
```

- **END Lab 01.02**
- **SEE Lab 01.02 Solution**

01.02 Lab Solution

Assign values to the provided variables and use typeof() to log output that matches the comment:

On your own:

```
    var msg = 'Hello World';
    console.log(msg, typeof(msg)); // Hello World! string

    var total = 314;
    console.log(total, typeof(total)); // 314 number

    var isFresh = true;
    console.log(isFresh, typeof(isFresh)); // true boolean

    var price = 29.95;;
    console.log('price: $', price, typeof(price)); // price: $ 29.95
number

    var player1;
    console.log(player1, typeof(player1)); // undefined 'undefined'

    var restaurntBill = 80;
    var groupon= '$10';
    var pleasePay = restaurantBill - groupon;
    console.log(pleasePay, typeof(pleasePay)); // NaN 'number'
```

Use the Number() method so that the console.log output changes from NaN to matching the comment.

On your own:

```
    var x = '55';
    var y = '33';
    var z = Number(x) - Number(y);
    console.log(z); // 22

    var x = '55';
    var y = '33';
    var z = Number(x) + Number(y);
    console.log(z); // 88

    var x = '55';
    var y = 5;
    var z = Number(x) / y;
    console.log(z); // 11

    var x = 55;
    var y = '44';
```

```
var z = x + y;
console.log(z); // 5544
```

string concatenation

Assign values to the variables and then use the variables in string concatenation to reproduce the ***messages shown***:

```
// message 1: Sally lives in New York City.
let msg1;
let name = "Sally";
let city = "New York City";
msg1 = name + " lives in " + city + "."
console.log(msg1);

// message 2: Freddy went to France for 6 months.
let msg2;
let fName = "Freddy";
let country = "France";
let months = 6;
msg2 = fName + " went to " + country + " for " + months + " months.";
console.log(msg2);

// message 3: My Bunny had a baby carrot for breakfast.
let msg3;
let pet = "Bunny";
let veg = "baby carrot";
let meal = "breakfast";
msg3 = "My " + pet + " had a " + veg + " for " + meal + ".";
console.log(msg3);
```

- **END Lab Solution 01.02**



---

resoloving nested quotes

string interpolation

const

---

resolving conflicting nested quotes

Strings can go in either double or single quotes, but nested quotes can cause problems.

1. Declare a string in single quotes with an internal single quote (apostrophe). Check the console. There's an error:

```
let song = 'Won't Get Fooled Again';
// Unexpected identifier 't'
```

unexpected identified

An **unexpected identifier**, also known as an **unexpected token**, is a character that doesn't belong. The problem is, the apostrophe was mistaken for the end of the string, which makes 'Won' the entire string--and everything else 'unexpected'.

mixing-and-matching quotes

If you have an inner single quote (apostrophe), avoid conflicts by wrapping the string in double quotes.

2. Resolve the nested quote conflict by switching to double quotes:

```
let song = "Won't Get Fooled Again";
console.log(song); // Won't Get Fooled Again
```

escaping quotes

Instead of mixing-and-matching quotes, you can "escape quotes" with a backslash, which causes the punctuation to be ignored.

3. Declare another variable with single quotes (apostrophes) nested inside single quotes, and then fix the error by escaping the apostrophes:

```
    // let song2 = 'Don't Stop Believin''; // error
    let song2 = 'Don\'t Stop Believin\'';
    console.log(song2);
```

string interpolation

Long string concatenation can get hard to read, with all its quote and plus-signs. A cleaner alternative is **string interpolation**, which doesn't have any quotes or plus signs. To convert concatenation to interpolation, follow these steps:

- Delete all quotes and plus signs.
- Wrap everything in a pair of backticks ** **.
- Put variables in curly braces, preceded by a dollar sign: **${variable}**

4. Do a string concatenation and then switch to string iterpolation:

```
    // string concatenation
    let firstName = 'Bob';
    let greeting = 'Hi, ' + firstName + '! How are you?';
    console.log(greeting); // Hi, Bob! How are you?

    // string iterpolation
    greeting = Hi, ${firstName}! How are you?;
    console.log(greeting); // Hi, Bob! How are you?
```

The advantage of string interpolation is more apparent when there are multiple variables and substrings.

5. Declare a few more variables and make a longer, more complex concatenation:

```
    let petName = "Fluffy";
    let age = 3;
    let food = "tuna";

    let catGreeting = "Meow! My name is " + petName + "! I am " + age + "
years old. My favorite food is " + food + ".";

    console.log(catGreeting);
    // Meow! My name is Fluffy! I am 3 years old. My favorite food is
tuna.
```

This concatenation has four pairs of quotes and six plus signs, making it hard to read and write.

6. Refactor the above as string interpolation:

```
    catGreeting = `Meow! My name is ${petName}. I am ${age} years old. My
favorite food is ${food}`;
```

```
    console.log(catGreeting);
    // Meow! My name is Fluffy! I am 3 years old. My favorite food is
tuna.
```

line breaks and tabs

With string concatenation, hitting enter to get a line break is ignored, as are tabs.

7. Do a string concatenation with line breaks and tabs:

```
    let item1 = 'eggs';
    let item2 = 'bread';
    let item3 = 'milk';
    let shoppingList = 'Shopping List:' +
        item1 +
        item2 +
        item3;
    console.log(shoppingList);
    // Shopping List:eggsbreadmilk
```

8. Try it with string interpolation. Now, hitting enter gives you a line break in the output. Tab works too:

```
    shoppingList = `Shopping List:
        ${item1}
        ${item2}
        ${item3}`;
    console.log(shoppingList);
    /*
    Shopping List:
        eggs
        bread
        milk
    */
```

The line breaks and tabs can be done with regular concatenation:

- **\n** in string concatenation gives a line break.
- **\t** in string concatenation gives a tabs.

9. Try the shopping list with regular string concatenation, including the line breaks and tabs. It can be done, but is a pain to write, so feel free to copy-paste this one:

```
    shoppingList = "Shopping List: \n\t" + item1 + "\n\t" + item2 + "\n\t"
+ item3;
    console.log(shoppingList);
    /*
```

```
    Shopping List:
        eggs
        bread
        milk
    */
```

const means constant

- A **variable** is a named value that can *vary* (change).
- A **constant** is a named value that *cannot* be changed.

Constants, by convention, have UPPERCASE names, but an all-caps name alone does not make for a constant. Below, **let MOON_DIAMETER** is uppercase, but you can still change the value.

10. Declare an uppercase variable with **let**:

```
    let MOON_DIAMETER = 2159.1;
    console.log(MOON_DIAMETER); // 2159.1
    MOON_DIAMETER *= 10; // multiply by 10
    console.log(MOON_DIAMETER); // 21591
```

To declare a true constant, use the **const** keyword. As its name implies, **const** makes true constants. If you try to change it, you get an error.

11. Declare a true constant with **const**:

```
    const DOZEN = 12;
    DOZEN = 13; // Error: Assignment to constant variable
```

12. You cannot change a const, so it gives you an error. Comment it out:

```
    // DOZEN = 13;
```

13. Try redeclaring DOZEN as a **var**:

```
    var DOZEN = 13; // SyntaxError: Identifier DOZEN has already been
declared.
```

14. It gives an error, so comment it out.

```
    // var DOZEN = 13;
```

- **End Lesson 01.03**
- **Next: 01.03 Lab**
- **Lesson 01.04**



Instructions:

- Open **01.03-Lab.html** in your Editor. Write the Lab code in the **script** tags.

1. Debug these strings by mixing-and-matching quotes.

```
var cereal = 'Cap'n Crunch';
console.log(cereal);  // Cap'n Crunch

var questi = ""Who's there?" she asked.";
console.log(questi); // "Who's there?" she asked.
```

2. Debug these strings by escaping quotes with backslash:

```
var phrase = 'Isn't that amazing!';
console.log(phrase); // 'Isn't that amazing!

var quote = ""Show me the money!" is a famous movie line.";
console.log(quote); // "Show me the money!" is a famous movie line.
```

3. Use string interpolation to produce the messages in the comments:

```
var pet = 'dog';
var name = 'Fido';
var toy = 'frisbee';
var msg;
console.log(msg); // My dog Fido's favorite toy is a frisbee.

var animal;
var continent;
var msg;
console.log(msg); // The kangaroo is native to Australia.
```

4. Produce the restaurant bill output as shown in the multi-line comment, below. This one requires you to combine math operations, which you practiced earlier, with string interpolation:

```
var joint = "BOB'S DINER";
var food = 30;
```

```
var bev = 20;
var subTotal = food + bev;
var taxRate = 0.08; // 8%
var tipPct = 0.2; // 20%;
var tax;
var tip;
var grandTotal;
var guestCheck;
console.log(guestCheck);
/*
BOB'S DINER
GUEST CHECK:
Food: $30
Beverage: $20
Sub-Total: $50
Tax: $4
Tip: $10
TOTAL: $64
THANK YOU!
*/
```

- **END Lab 01.03**
- **SEE Lab 01.03 Solution**

01.03 Lab Solution

Instructions:

- Open **01.03-Lab.html** in your Editor. Write the Lab code in the **script** tags.

1. Debug these strings by mixing-and-matching quotes.

```
var cereal = "Cap'n Crunch";
console.log(cereal); // Cap'n Crunch

var questi = '"Who\'s there?" she asked.';
console.log(questi); // "Who's there?" she asked.
```

2. Debug these strings by escaping quotes with backslash:

```
var phrase = 'Isn\'t that amazing!';
console.log(phrase); // Isn't that amazing!

var quote = "\"Show me the money!\" is a famous movie line.";
console.log(quote); // "Show me the money!" is a famous movie line.
```

3. Use string interpolation to produce the messages in the comments:

```
var pet = 'dog';
var name = 'Fido';
var toy = 'frisbee';
var msg = `My ${pet} ${name}'s favorite toy is a ${toy}.`;
console.log(msg); // My dog Fido’s favorite toy is a frisbee.

var animal;
var continent;
var msg = `The ${animal} is native to ${continent}.`;
console.log(msg); // The kangaroo is native to Australia.
```

4. Produce the restaurant bill output as shown in the multi-line comment, below. This one requires you to combine math operations, which you practiced earlier, with string interpolation:

```
var joint = "BOB'S DINNER";
var food = 30;
var bev = 20;
var subTotal = food + bev;
var taxRate = 0.08; // 8%
```

```
var tipPct = 0.2; // 20%;
var tax = subTotal * taxRate;
var tip = subTotal * tipPct;
var grandTotal = subTotal + tax + tip;
var guestCheck = `*** ${joint} ***
*** GUEST CHECK ***
    Food: $${food}
    Beverage: $${bev}
    Sub-Total: ${subTotal}
    Tax: ${tax}
    Tip: ${tip}
    TOTAL: $${grandTotal}
*** THANK YOU! ***`;
console.log(guestCheck);

*** BOB'S DINER ***
*** GUEST CHECK ***
Food: $30
Beverage: $20
Sub-Total: $50
Tax: $4
Tip: $10
TOTAL: $64
*** THANK YOU! ***
```

- **END Lab 01.03**
- **SEE Lab 01.03 Solution**



UNIT 01

LESSON 01.04



Math Object

math operators: +, -, *=, /, **, %

Math.random() .round() .floor() .ceil()

.max() .min() .abs() .pow() .PI

.toFixed()

shorthand operators: +=, -=, *=, /=

Naturally, we can do math with numeric variables.

Mathematical operators include:

```
+   add
-   subtract
*   multiply
/   divide
**  exponent (power of)
%   modulo (remainder)
```

1. Declare two number variables, and use them for some simple calculations:

```
let n1 = 10;
let n2 = 3;

console.log(n1 + n2); // 13
console.log(n1 - n2); // 7
console.log(n1 * n2); // 30
console.log(n1 / n2); // 3.333333333
console.log(n1 ** n2); // 1000
console.log(n1 % n2); // 1
```

Results of mathematical calculations can be stored in variables.

2. Change the values of **n1** and **n2**, and do some more calculations, saving the results to variables:

```
n1 = 6;
n2 = 2;

let sum = n1 + n2;
console.log(sum); // 8

let diff = n1 - n2;
console.log(diff); // 4

let prod = n1 * n2;
console.log(prod); // 12

let quot = n1 / n2;
console.log(quot); // 3

let expo = n1 ** n2;
console.log(expo); // 36

let modul = n1 % n2;
console.log(modul); // 0
```

Order of Operations of Mathematical Expressions

- Do * and / before + and -
- Do * and / from left to right
- Do + and - from left to right
- Operations inside () are done first

3. Declare a third number variable, and do some math that shows how parentheses can affect the result:

```
n1 = 4;
n2 = 5;
let n3 = 8;

let tot = n1 + n2 * n3; // 4 + 40
console.log(tot); // 44

tot = (n1 + n2) * n3; // 9 * 8
console.log(tot); // 72
```

Math Object

JS has a built-in Math Object, which comes with many useful methods:

Math.random() generates a random float from 0-1, to 16 decimal points:

4. Generate a random number and log it:

```
let r = Math.random();
console.log(r); // 0.7492906781140873
```

Math.round() rounds off its argument:

5. Round off a number:

```
let x = Math.round(2.5);
console.log(x); // 3
```

Math.floor() rounds *down* its argument:

6. Round down a number:

```
let y = Math.floor(2.999);
console.log(y); // 2
```

Math.ceil() rounds *up* its argument:

7. Round up a number:

```
let z = Math.ceil(2.001);
console.log(z); // 3
```

getting a random number greater than 1

Math.random() generates a random float from 0-1, so to get a larger number, just multiply by some value.

8. Generate a random number and multiply it by 100:

```
let rando = Math.random() * 100;
console.log(rando)  // some number between 0-100
```

To get an integer, round, floor or ceil the random value.

9. Round down a random number multiplied by 100:

```
let randInt = Math.floor(Math.random() * 100);
console.log(randInt); // some integer between 0-99
```

getting a random integer in a range

To get a random integer in a range, multiply by the range span and then add the starting value.

10. Round down a random number multiplied by 50 and then add 50 to get a number in the 50-100 range:

```
let rand = Math.ceil(Math.random() * 50 + 50);
console.log(rand); // some value between 50-100
```

Math.max() returns the greatest of the multiple values passed to it:

11. Find the maximum of a set of numbers:

```
let maxi = Math.max(3, 6, 8, 2, 12, 4, 10);
console.log(maxi); // 12
```

Math.min() returns the smallest of the multiple values passed to it:

12. Find the minimum of a set of numbers:

```
let mini = Math.min(3, 6, 8, 2, 12, 4, 10);
console.log(mini); // 2
```

Math.pow() takes two arguments: a number and a power to raise it to:

13. Raise a number to a power using the **Math.pow()** method:

```
let pwr = Math.pow(5, 4);
console.log(pwr); // 625 (5x5x5x5)
```

However, as we have seen, the ** operator does the same thing as Math.pow():

14. Raise a number to a power using the ** operator:

```
let powr = 5 ** 4;
console.log(powr); // 625 (5x5x5x5)
```

Math.abs() returns the absolute value of its argument, meaning it just makes it positive:

15. Use **Math.abs()** to get the absolute value:

```
let absolut = Math.abs(-7);
console.log(absolut); // 7
```

Math.sqrt() returns the square root of its argument:

16. Use **Math.sqrt()** to find a square root:

```
let sqRt = Math.sqrt(81);
console.log(sqRt); // 9
```

Math.PI returns the famous constant. If you save it, it should be to a **const**, uppercase:

17. Get **PI** to 16 digits:

```
const PI = Math.PI;
console.log(PI); // 3.141592653589793
// PI = 'apple'; // ERROR
```

toFixed() is a method called on a float. It returns a float with the number of decimal places in the argument:

18. Round PI to 2 digits. The rounded value is actually a string:

```
let pi2 = PI.toFixed(2);
console.log(pi2, typeof(pi2)); // 3.14 string
```

19. Try doing addition with **pi2**. The plus sign does concatenation, because it's working with a string:

```
console.log(pi2 + pi2); // 3.143.14
```

**math shorthand operators: += -= = /=*

Math shorthand operators make math more concise by eliminating the need to repeat a variable:

20. Try these math shorthand operators:

```
x = 20;

x = x + 35; // add
console.log(x); // 55

x += 15; // add
console.log(x); // 70

x = x * 3; // multiply
console.log(x); // 210
```

```
x *= 2; // multiply
console.log(x); // 420

x = x - 80; // subtract
console.log(x); // 340

x -= 100; // subtract
console.log(x); // 240

x = x / 4; // divide
console.log(x); // 60

x /= 3; // divide
console.log(x); // 20
```

- **END Lesson 01.04**
- **NEXT: Lab 01.04**
- **NEXT: Lesson 02.01**

Lab 01.04
Math Object

See script tag in this html file for lab assignment.

As you work, check the console for output and red error messages.

Please check Lab Solution file only after giving the Lab your best effort!

01.04 Lab Solution

Instructions:

- Open **01.04-Lab-Solution.html** in your Editor. Check your Lab code against the solution found in the **script** tags.

1. Set the **y** value to produce console output that matches the comment:

```
let x = 10;
let y = 0;

y = -20;
console.log(x + y); // -10

y = 20;
console.log(x - y); // -10

y = .25;
console.log(x * y); // 2.5

y = 4;
console.log(x / y); // 2.5

y = 5;
console.log(x ** y); // 100000

y = 3;
console.log(x % y); // 1
```

2. Calculate the total cost, as shown in the comment:

```
let unitCost = 50;
let numUnits = 12;
let shipping = 25;
let totalCost = unitCost * numUnits + shipping;
console.log(totalCost); // 625
```

3. Calculate the values of z in terms of w, x and y to get the value shown in the comment:

```
let w = 10;
x = 12;
y = 15;
let z;

z = x * y + w;
console.log(z); // 190
```

```
z = w * x - y;
console.log(z); // 105

z = x * y / w;
console.log(z); // 18

z = w + x - y;
console.log(z); // 8

z = w * x / y;
console.log(z); // 7
```

4. Use the Math Object to generate a random integer between 0-19;

```
let r = Math.floor(Math.random() * 20);
console.log(r);
```

5. Use the Math Object to generate a random integer between 26-50;

```
let ran = Math.ceil(Math.random() * 25 + 25);
console.log(ran);
```

6. Use the Math Object to get the maximum value from these numbers: 3, 5, 21, 7, 1, 3, 12, 6

```
let m = Math.max(3, 5, 21, 7, 1, 3, 12, 6);
console.log(m); // 21
```

7. Use the Math Object to get the square root of 81.

```
let sqR = Math.sqrt(81);
console.log(sqR); // 9
```

8. Use a Math Object method that takes 9.99 as its argument and returns 9.

```
let fl = Math.floor(9.99);
console.log(fl); // 9
```

9. Supply values for x, y and expon so that the console.log output matches the comment next to it:

```

```

```
x = 14;
y = 2;
let expon = x ** y;
console.log(expon); // 196
```

10. The area of a circle equals PI times the radius squared: A = π r². Given a circle of radius 4, use the Math Object to find the area of the circle.

```
// area = π r²
let area = Math.PI * Math.pow(4, 2);
console.log(area); // 50.26548245743669

// OR use the ** operator:
area = Math.PI * (4 ** 2);
console.log(area); // 50.26548245743669
```

11. The hypotenuse (c) of a right triangle is obtained by the formula: a²+b²=c², where a and b are the other two sides.

```
// Using the Math Object, find the hypotenuse of a triangle, where a=5
and b=12.

// a²+b²=c²
let a = 5;
let b = 12;
// let c2 represent the square of the hypotenuse
let c2 = (5 ** 2) + (12 ** 2);
let c = Math.sqrt(c2);
console.log(c);
```

12. Generate two random floats, r1 and r2, in the 0-10 range. Round each of them off to 5 decimal places. Add them together.

```
let r1 = Math.random() * 10;
let r2 =Math.random() * 10;

// round to 3 decimal places
r1 = r1.toFixed(3);
r2 = r2.toFixed(3);
console.log(r1); // 9.202
console.log(r2); // 6.845

// HINTS: You cannot do addition with strings.

let sum = r1 + r2; // 9.2026.845 -- Oops! String concatenation!
```

```
// Try again, first converting r1 and r2 back to numbers:
r1 = Number(r1);
r2 = Number(r2);

sum = r1 + r2;
console.log(sum); // 16.047
```

13. Given this baseball player and his statistics:

    Player: Vladimir Guerrero Jr. Team: Toronto Blue Jays Year: 2021

    Stats: PA AB R H 2B 3B HR RBI 698 604 123 188 29 1 48 111

    Calculate Guerrero's Slugging Percentage (SLG), which equals total bases (TB) divided by at bats (AB): SLG = TB / AB. It is customary for SLG to be rounded to 3 decimal places.

    Total bases is the sum of a player's hits (H), plus their doubles (2B), plus twice their triples (3B), plus three times their home runs (HR): TB = H + 2B + (3B * 2) + (HR * 3)

```
// We don't need all the stats, but make vars of the ones we do need:
let AB = 604;
let H = 188;
let _2B = 29;
let _3B = 1;
let HR = 48;

// Calculate Total Bases (TB):
let TB = H + _2B + (_3B * 2) + (HR * 3);
console.log(TB); // 363

// Calculate Slugging Percentage (SLG):
let SLG = TB / AB;
console.log(SLG); // 0.6009933774834437

// Round to 3 decimal places
SLG = SLG.toFixed(3);
console.log(SLG); // 0.601
```

EXAMPLE / HINT (but only if you need it)

A player has 100 hits, including 17 2B, 4 3B and 19 HR, they have 182 TB: 100 + 17 + (4 * 2) + (19 * 3) = 100 + 17 + 8 + 57 = 182

It took the player 350 AB to amass these 182 TB. Therefore, they have a slugging percentage of 0.520: 100 / 350 = 0.52

- **END Lab Solution 01.04**
- **NEXT Lesson 02.01**



UNIT 02

LESSON 02.01



---

logical operators

conditional logic: if-else

if - else if - else

variable scope: let vs var

A comparison operator *compares* two values and resolves to a boolean (**true** or **false**). The various operators are:

- **==** equal to in value
- **===** equal to in both value and data type
- **!=** not equal to in value
- **!==** not equal to in value or data type or both
- **>** greater than in value
- **<** less than in value
- **>=** greater than or equal to in value
- **<=** less than or equal to in value
- **!** not prefix (opposite)

single vs. double vs. triple equal signs

- **=** assigns a value:
  - **x = 5**. Whatever **x** used to be, now it's 5.
- **==** *compares* two values; it does *not* assign a value.
  - returns **true** if the two values are equal
  - returns **false** if the two values are not equal.
  - does not consider datatype. **5 == "5"** is **true**.
- **===** compares two values; it does *not* assign a value.
  - returns **true** if they are equal in both value *and* datatype.
  - returns **false** if they are not equal in either value *or* datatype.
  - considers datatype: **5 === "5"** is **false**.
- **!=** checks for inequality of value, but not of datatype.
- **!==** checks for inequality of both value and data type.

1. Open the console and type these comparisons, hitting enter each time:

```
8 == 8; // true
8 == "8"; // true
8 === "8"; // false
8 >= "8"; // true
```

2. Make some comparisons using the inequality operators:

```
7 != 8; // true
7 != 7; // false
7 != '7'; // false
7 !== '7'; // true
```

3. Make some comparisons using the greater than less than equal to operators:

```
7 >= 8; // false
7 >= '7'; // true
7 <= 8; // true
7 <= '7'; // true
```

conditional logic with if else

An **if-statement** makes a comparison between two values, which resolves to a **boolen**. If the condition is **true**, the code inside the curly braces of the if statement will run. If the condition is **false**, the code will not run. There may be an "else part" that runs if the condition is false.

How to write an **if-statement()**:

- write *if* followed by a pair of parentheses: **if()**
- inside that put the condition to evaluate: **if(5 > 3)**
- after that, put a pair of curly braces:
  - **if(5 > 3) { }**
- inside the curly braces, write the code that you want to run if the condition is true:
  - **if(5 > 3) { console.log("It's true!"); }**

4. Do some comparisons inside if-statements:

```
if(7 == "7") {
    console.log('close enough');
}

if(7 === "7") {
    console.log('close but no cigar');
}
```

A boolean is already true or false, so we don't need to explicitly compare it true or false.

5. Do two versions of an if-statement with a boolean, with and without parentheses:

```
let raining = true;

if(raining == true) {
    console.log("It's raining, so bring an umbrella!"); // runs
}

if(raining) { // also works
    console.log("It's raining, so bring an umbrella!"); // runs
}
```

if else

What if we want go to the beach if it *not* raining. For that, we need an **else** part":

6. Make raining *false* and add an *else*:

```
raining = false;

if(raining) {
    console.log("It's raining, so bring an umbrella!");
} else {
    console.log("It's not raining! Let's go to the beach!"); // runs
}
```

Strings can also be evaluated with if-else logic.

7. Check if the weather is "sunny":

```
let weather = "sunny";

if(weather == "sunny") { // runs
    console.log("It's sunny! Don't forget your sunglasses!");
} else {
    console.log("It isn't sunny! Leave the sunglasses at home!");
}
```

With "number-like strings", the digits are evaluated like individual letters. "50" is greater than "100", because 5 is greater than 1.

8. Compare two "number-like strings":

```
let full = "100";
let half = "50";
```

```
if(full > half) {
    console.log('"100" > "50"');
} else {
    console.log('because 5 > 1'); // runs
}
```

9. Convert the "number-like strings" to actual numbers and try again:

```
full = Number("100");
half = Number("50");

if(full > half) {
    console.log('100 > 50'); // runs
} else {
    console.log('50 > 100');
}
```

if-else if-else logic

else if adds another condition to evaluate if **if** returns false:

10. Try this **else if**:

```
let highScore = 15000;
let myScore = 10000;

if (highScore > myScore) {
    console.log('You did not beat the high score!');
} else if (highScore < myScore) {
    console.log('You beat the high score!');
} else {
    console.log('You tied the high score!');
}
```

11. Try different values of **myScore** to get all three outcomes.

12. Check the weather with **else if** logic:

```
weather = "cloudy";

if (weather == "rainy") {
    console.log('Go to the museum!');
} else if (weather == "sunny") {
    console.log('Go to the beach!');
} else {
    console.log('Go to the park!');
}
```

13. Try different values of **weather** to get all three outcomes.

There can be any number of **else if** blocks between the opening **if** and the closing **else**.

14. Run this code with its three **else if** clauses. The best approach is to go higher to lower by ranges (90+, 80-89, 70-79, 65-69, 64 and below):

```
let score = 77;
let grade = "";

if(score >= 90) {
    grade = "A";
} else if(score >= 80) {
    grade = "B";
} else if(score >= 70) {
    grade = "C";
} else if(score >= 65) {
    grade = "D";
} else {
    grade = "F";
}
let reportCard = **Your score: ${score}.
Your grade: ${grade}.**
console.log(reportCard);
```

block scope

As you recall from a previous lesson:

let is **block-scoped**, meaning it is available only inside the the curly braces in which it was declared. Block-scoping prevents variables from "leaking out" into parts of the application where they don't belong.

The curly braces of if-statements enclose code blocks, so **let** variables declared therein are confined to that code block. This is not the case with **var**, which is global in scope even when it is declared within curly braces.

15. Write an "if-statement" with a **let** variable declared inside its curly braces. Declare the variable inside and outside the code block:

```
let meal = "lunch";

if (meal == "lunch") {
    let special = "Burritos";
}

console.log(meal); // lunch
console.log(special); // ERROR: special is not defined
```

We get **not defined**, because **special** is block-scoped to its if-statement; **special** does not exist in the global scope where we are attempting to access it.

16. Run the same test, but with **var**:

```
    if (meal == "lunch") {
        var special = "Burritos";
    }
    console.log(special); // Burritos
```

The "var variable" still exists after the if-statement, because, **var** is in the **global scope**, even though it was declared inside an "if-statement".

let instead of var for better scope control

This is why **let** is preferred over **var**. With **let**, you have better control over variable scope. With **var**, variables "leak out" from their blocks into the rest of the application.

global variables with let

With **let**, you can still have **global variables**. Just declare them outside of any curly braces, and they will be available throughout the script.

- **END Lesson 02.01**
- **NEXT Lab 02.02**
- **Lesson 02.02**



Instructions:

- Open **02.01-Lab.html** in your Editor. Write the Lab code in the **script** tags.

1. Declare a variable called lucyIsOnline and set its value to false. Write an if-else statement:

- if lucyIsOnline is true, console.log "Lucy is online",
- if lucyIsOnline is false, console.log "Lucy is not online"

2. Declare a variable price, and set it equal to 88. Write an if-else statement:

- if price is greater than or equal to 100, console.log 'Expensive'
- if price is less than 100, console.log 'Cheap'

3. Add an "else if" clause to the statement:

- if price is greater than 100, console.log 'Expensive'
- if price is between 50-99, console.log 'Reasonable'
- if price is less than 50, console.log 'Cheap'

4. Declare two variables: stars and review. Set stars equal to 4 and review equal to an empty string. Write an if else-else if-else statemetn:

- if 5 stars, review is "Great"
- if 4 stars, review is "Good"
- if 3 stars, review is "Meh"
- if 2 stars, review is "Bad"
- if 1 star, review is "Awful"
- console.log review below the whole thing

5. Debug the following:

```
let animal = 'cow';
let sound = '';

if (animal = 'dog') {
    sound = Woof;
} elseif (animal = 'cat') {
sound = Meow;
} elseif (animal = 'cow') {
    sound = Moo;
} else (Animal and sound both unknown) {
console.log('sound + !');
```

```
// desired output: Moo!
```

6. Given these variables, write an if-else with three "else if" blocks to evaluate multiple temperature ranges:

```
let fahrenheit = 95;
let weather = "";
```

- above 90 is hot
- 70-89 is warm
- 50-69 is cool
- 32-49 is cold
- below 32 is freezing
- **END Lab 02.01**
- **SEE Lab 02.01 Solution**

02.01 Lab Solution

Instructions:

- Open **02.01-Lab.html** in your Editor. Write the Lab code in the **script** tags.

1. Declare a variable called lucyIsOnline and set its value to false. Write an if-else statement:

```
let lucyIsOnline = false;

if (lucyIsOnline) {
    console.log("Lucy is online");
} else {
    console.log("Lucy is not online");
}
```

2. Declare a variable price, and set it equal to 88. Write an if-else statement:

- if price is greater than or equal to 100, console.log 'Expensive'
- if price is less than 100, console.log 'Cheap'

```
let price = 88;

if(price > 100) {
    console.log('Expensive');
} else {
    console.log('Cheap');
}
```

3. Add an "else if" clause to the statement:

- if price is greater than 100, console.log 'Expensive'
- if price is between 50-99, console.log 'Reasonable'
- if price is less than 50, console.log 'Cheap'

```
if(price > 100) {
    console.log('Expensive');
} else if (price > 50) {
    console.log('Reasonable');
} else {
    console.log('Cheap');
}
```

4. Declare two variables: stars and review. Set stars equal to 4 and review equal to an empty string. Write an if else-else if-else statement:

- if 5 stars, review is "Great"
- if 4 stars, review is "Good"
- if 3 stars, review is "Meh"
- if 2 stars, review is "Bad"
- if 1 star, review is "Awful"
- console.log review below the whole thing

```
let stars = 4;
let review = "";

if (stars == 5) {
    review = "Great";
} else if (stars == 4) {
    review = "Good";
} else if (stars == 3) {
    review = "Meh";
} else if (stars == 2) {
    review = "Bad";
} else {
    review = "Awful";
}
```

5. Debug the following:

```
let animal = 'cow';
let sound = '';

if (animal == 'dog') {
    sound = 'Woof';
} else if (animal == 'cat') {
    sound = 'Meow';
} else if (animal == 'cow') {
    sound = 'Moo';
} else {
    console.log('Animal and sound both unknown');
}
console.log(sound + '!'); // expected: Moo!
```

6. Given these variables, write an if-else with three "else if" blocks to evaluate multiple temperature ranges:

```
let fahrenheit = 95;
let weather = "";
```

- above 90 is hot
- 70-89 is warm
- 50-69 is cool
- 32-49 is cold
- below 32 is freezing

```
    if(fahrenheit > 90) {
        weather = "hot";
    } else if (fahrenheit > 70) {
        weather = "warm";
    } else if (fahrenheit > 50) {
        weather = "cool";
    } else if (fahrenheit > 32) {
        weather = "cold";
    } else { // 32 and below
        weather = "freezing";
    }
    console.log('The weather is ' + weather);
```

END Lab 02.01 SEE Lab 02.01 Solution



UNIT 02

LESSON 02.02



---

nested if-else logic

truthy-falsey values

&& (AND) operator

|| (AND) operator

---

nested if-else logic

In the previous lesson we saw if-else logic involving a planned activity:

- if it's rainy, go to the museum.
- else if it's sunny, go to the beach
- else (it's cloudy) so go to the park.

But what if there are also wind conditions that will determine our activities at the beach and park:

```
- If it is sunny, go to the beach.
    - If it is windy, go windsurfing.
    - If it is not windy, play frisbee.
- If it is cloudy, go to the park.
    - If it is windy, fly a kite.
    - If it is not windy, have a picnic.
- If it is rainy, go to the museum.
```

This is nested decision making, and it requires nested if-else logic.

1. Set up the basic, "non-nested" if-else logic, and add a new variable, windy, that we can use for "nested logic":

```
    let weather = "cloudy";
    let windy = true;

    if (weather == "rainy") {
        console.log('Go to the museum!');
    } else if (weather == "sunny") {
        console.log('Go to the beach!');
    } else {
```

```
    console.log('Go to the park!');
}
```

2. Wind conditions do not apply if we are at the museum. So, move on to the "else if", and add this nested logic that specifies what to do at the beach:

```
if (weather == "rainy") {
    console.log('Go to the museum!');
} else if (weather == "sunny") {
    console.log('Go to the beach!');
    if (windy == true) {
        console.log('Go windsurfing!');
    } else {
        console.log('Play frisbee!');
    }
} else {
    console.log('Go to the park!');
}
```

3. Moving on to the last part, specify what to do at the park:

```
if (weather == "rainy") {
    console.log('Go to the museum!');
} else if (weather == "sunny") {
    console.log('Go to the beach!');
    if (windy == true) {
        console.log('Go windsurfing!');
    } else {
        console.log('Play frisbee!');
    }
} else {
    console.log('Go to the park!');
    if (windy) {
        console.log('Fly a kite!');
    } else {
        console.log('Have a picnic!');
    }
}
```

4. Change weather and windy to get different output.

truthy and falsey values

Truthy and falsey values are not literally true or false, but they return true or false in a boolean context, i.e. in an "if-statement". Here are the falsey values:

falsey values

```
- NaN
- undefined
- null
- 0
- "", ''
```

All other values, including all strings and non-zero numbers, both positive and negative, are truthy.

A falsey resolves to false in an if-statement:

5. Do a test to confirm that **NaN** is falsey:

```
    let baNaNa = Number('banana');
    console.log('baNaNa', baNaNa, typeof(baNaNa));
    // baNaNa NaN number

    if(baNaNa) {
        console.log('NaN is truthy');
    } else {
        console.log('NaN is falsey'); // runs
    }
```

6. Do a test to confirm that **undefined** is falsey:

```
    let player1;
    console.log('player1', player1, typeof(player1));
    // player1 undefined undefined

    if(player1) {
        console.log('undefined is truthy');
    } else {
        console.log('undefined is falsey'); // runs
    }
```

7. Do a test to confirm that **null** is falsey:

```
    let score = null;
    console.log('score', score, typeof(score));
    // score null object

    if(score) {
        console.log('null is truthy');
    } else {
        console.log('null is falsey'); // runs
    }
```

8. Do a test to confirm that **0** is falsey:

```
let x = 0;
console.log('x', x, typeof(x));
// x 0 number

if(x) {
    console.log('0 is truthy');
} else {
    console.log('0 is falsey'); // runs
}
```

9. Do a test to confirm that empty strings are falsey:

```
let zip = "";
console.log('zip', zip, typeof(zip));
// zip  string

if(zip) {
    console.log('empty strings are truthy');
} else {
    console.log('empty strings are is falsey'); // runs
}
```

- **END Lesson 02.02**
- **NEXT Lab 02.02**
- **Lesson 02.03**



Instructions:

1. Write a nested if-else block, that satisfies the following conditions:
    - If the student score is greater than or equal to 90, if the student is in high school, they get a grade of "A", but if they are in college they get a grade of 4.
    - If the student score is greater than or equal to 80 and less than 90, if the student is in high school, they get a grade of "B", but if they are in college they get a grade of 3.
    - If the student score is greater than or equal to 70 and less than 80, if the student is in high school, they get a grade of "C", but if they are in college they get a grade of 2.
2. Write a nested if-else block, that satisfies the following conditions:
    - If the pet is a cat, increase the pet price by 20%, unless it is a baby cat, in which case decrease the price by 10%.
    - If the pet is a dog, increase the pet price by 30%, unless it is a baby dog, in which case decrease the price by 15%.
    - If the pet is anything besides a dog or a cat, double the price, unless it is a baby version of the pet, in which case cut the price in half.

- **END Lab 02.02**
- **SEE Lab 02.02 Solution**

02.02 Lab Solution

1. Write a nested if-else block, that satisfies the following conditions:

    - If the student score is greater than or equal to 90, if the student is in high school, they get a grade of "A", but if they are in college they get a grade of 4.
    - If the student score is greater than or equal to 80 and less than 90, if the student is in high school, they get a grade of "B", but if they are in college they get a grade of 3.
    - If the student score is greater than or equal to 70 and less than 80, if the student is in high school, they get a grade of "C", but if they are in college they get a grade of 2.

```
let studentScore = 94;
let isHighSchool = true;
let grade;

if(student_score >= 90) {
    if(isHighSchool) {
        grade = "A";
    } else {
        grade = 4;
    }
} else if(student_score >= 80) {
    if(isHighSchool) {
        grade = "B";
    } else {
        grade = 3;
    }
} else if(student_score >= 70) {
    if(isHighSchool) {
        grade = "C";
    } else {
        grade = 2;
    }
} else {
    grade = "FAIL";
}

console.log(grade);
```

2. Write a nested if-else block, that satisfies the following conditions:

    - If the pet is a cat, increase the pet price by 20%, unless it is a baby cat, in which case decrease the price by 10%.
    - If the pet is a dog, increase the pet price by 30%, unless it is a baby dog, in which case decrease the price by 15%.
    - If the pet is anything besides a dog or a cat, double the price, unless it is a baby version of the pet, in which case cut the price in half.

```
let pet = "cat";
let price = 100;
let baby = false;

if(pet == "cat") {
    if(baby) {
        // to decrease a number by 10%, multiply it by 0.9
        price *= 0.9;
    } else {
        // to increase a number by 20%, multiply it by 1.2
        price *= 1.2;
    }
} else if(pet == "dog") {
    if(baby) {
        // to decrease a number by 15%, multiply it by 0.85
        price *= 0.85;
    } else {
        // to increase a number by 30%, multiply it by 1.3
        price *= 1.3;
    }
} else {
    if(baby) {
        // to decrease a number by half (50%), multiply it by 0.5
        price *= 0.5;
    } else {
        // to double a number, multiply it by 2
        price *= 2;
    }
}

console.log(price);
```

- **END Lab 02.02**
- **SEE Lab 02.02 Solution**



UNIT 02

LESSON 02.03



---

ternary expressions

&& (AND) operator

|| (OR) operator

ternary expression

A ternary expression is a concise alternative to an if-else statement. What takes an if-else five lines of code to accomplish, a ternary expression gets done in one.

Let's start with an if-else statement.

1. Declare three number variables and do some conditional math. If x is less than y, add them; othewise (else) multiply them:

```
let x = 5;
let y = 8;
let z = 0;

if(x < y) {
    z = x * y
} else {
    z = x + y;
}

console.log(z);
```

console.log(z);

Now to convert the if-else to a ternary:

2. Add a question mark right inside the if-block:

```
if(x < y) {
    ? z = x * y;
} else {
    z = x + y;
}
```

3. Add a colon right inside the else-block:

```
if(x < y) {
    ? z = x * y;
} else {
    : z = x + y;
}
```

4. Delete the if() and else, including parentheses:

```
x < y {
    ? z = x * y;
} {
    : z = x + y;
}
```

5. Delete all the curly braces:

```
x < y
  ? z = x * y;
  : z = x + y;
```

6. This is all one line now, so get rid of the first semi-colon and back everything up onto the same line:

```
x < y ? z = x * y : z = x + y;

console.log(z);
```

7. You can even get rid of the second "z=" since it is assumed by the first one:

```
x < y ? z = x * y : x + y;

console.log(z);
```

CHALLENGE:

8. Convert this if-else into a ternary:

```
let n = 5;

if(n == 7) {
    n = 0;
} else {
    n++;
```

```
}

console.log(n);
```

9. Convert this if-else into a ternary:

```
let num = 20;

if(num == 20) {
  num++;
} else {
  num--;
}

console.log(num);
```

multiple conditions

Multiple conditions can be evaluated with the && (AND) and || (OR) operators:

The **&&** (AND) operator requires at least two conditions to be true.

10. Do an if statement with && where two conditions must be true.

```
    let city = 'Texarkana';
    let state = 'Texas';
    let msg = '';

    // && (AND) operator
    if(city == 'Texarkana' && state == 'Texas') {
        msg = `Welcome to ${city}, ${state}!`;
    } else {
        msg = 'This is not Texarkana, Texas--but it could be Texarkana,
Arkansas or Houston, Texas';
    }

    console.log(msg); // Welcome to Texarkana, Texas
```

11. Do an if statement with || where only one condition must be true.

```
    // || (OR) opertor
    if (city == 'Texarkana' || city == 'Houston') {
        msg = `Welcome to ${city}`; // runs
    } else {
        msg = 'This is neither Texarkana nor Houston';
    }
```

```
    console.log(msg); // Welcome to Texarkana
```

12. Change city to 'Dallas'`, and run it again. The else part will run both times.

There can be more than two **&&** conditions.

13. Try one with three conditons; if *any* of them are false, the **else** part runs:

```
    let R = 123;
    let G = 155;
    let B = 202;

    if(R > 100 && G > 100 && B > 100) { // true
        msg = 'All RBG values are greater than 100';
    } else {
        msg = 'At least one RBG value is 100 or less';
    }

    console.log(msg); // All RBG values greater than 100
```

14. Set any R, G, B values to below 100, and run it again. Now you get the else part.

15. Try using more than two || operators:

```
    let car = 'blue';

    if (car == 'black' || car == 'silver' || car == 'blue') {
        msg = 'car is black, sliver or blue.';
    } else {
        msg = 'The car is not black, sliver, or blue. It is actually ' +
car;
    }

    console.log(msg);
```

16. Change the car color to red so that the else part runs.

switch case break

An alternative to if-else if-else logic is a **switch-case-break** statement. Unlike a ternary, a switch-case-break is used for evaluating multiple conditions, which would otherwise require "else if" logic.

Convert if-else if-else to switch-case-break by following these guidelines:

- the evaluated variable is written just once
- there is only one set of curly braces
- there is only one set of parentheses

- there are no equality operators (==, ===)

Other differences: switch, case and break replace if-else keywords

- switch instead of if to start
- case instead of else if
- default instead of else

17. Write out this if-else if-else:

```
let moneySymbol = "JPY";
let currency = "";

if (moneySymbol === "USD") {
    currency = "US Dollar";
} else if (moneySymbol === "JPY") {
    currency = "Japanese Yen";
} else if (moneySymbol === "GBP") {
    currency = "British Pound";
} else {
    currency = "Unknown";
}
```

18. Convert the if-else if-else to a switch-case-break:

```
switch (moneySymbol) {
    case "USD":
    currency = "US dollar";
    break;
    case "JPY":
    currency = "Japanese Yen";
    break;
    case "GBP":
    currency = "British Pounds";
    break;
    default:
    currency = "Unknown";
    break;
}
```

- **END Lesson 02.03**
- **NEXT Lab 02.03**
- **NEXT Lesson 02.04**



1. Convert the following if-else into a ternary:

```
let msg = "";

if ('plums' < 'pears') {
    msg = 'plums b4 pears';
} else {
    msg = 'pears b4 plums';
}

console.log(msg);
```

2. Convert the following if-else into a ternary:

```
let x = 12;
let y = 6;
let z = 0;

if (x <= y) {
    z = x / y;
} else {
    z = y / x;
}

console.log(z);
```

3. Convert the following ternary into if-else:

```
let n = 8;

n == 8 ? console.log("equal") : console.log("not equal");
```

4. Convert the following if-else to switch-case-break:

```
let country = "Ghana";
let continent = "";

if (country === "Canada") {
```

```
        continent = "North America";
    } else if (country === "China") {
        continent = "Asia";
    } else if (country === "Ghana") {
        continent = "Africa";
    } else if (country === "Bolivia") {
        continent = "South America";
    } else if (country === "France") {
        continent = "Europe";
    } else {
        continent = "Australia";
    }
```

- **END Lab 02.03**
- **SEE Lab 02.03 Solution**

02.03 Lab Solution

1. Convert the following if-else into a ternary:

```
let msg = "";

if ('plums' < 'pears') {
    msg = 'plums b4 pears';
} else {
    msg = 'pears b4 plums';
}
console.log('if else:', msg);

'plums' < 'pears' ? msg = 'plums b4 pears' : msg = 'pears b4 plums';
console.log('ternary:', msg);
```

2. Convert the following if-else into a ternary:

```
let x = 12;
let y = 6;
let z = 0;

if (x <= y) {
    z = x / y;
} else {
    z = y / x;
}

console.log(z);

x <= y ? z = x / y : z = y / x;
console.log(z);
```

3. Convert the following ternary into if-else:

```
let n = 8;

n == 8 ? console.log("equal") : console.log("not equal");

if(n == 8) {
    console.log("equal")
} else {
    console.log("not equal");
}
```

switch case break

4. Convert the following if-else to switch-case-break:

```
let country = "Ghana";
let continent = "";

if (country === "Canada") {
    continent = "North America";
} else if (country === "China") {
    continent = "Asia";
} else if (country === "Ghana") {
    continent = "Africa";
} else if (country === "Bolivia") {
    continent = "South America";
} else if (country === "France") {
    continent = "Europe";
} else {
    continent = "Australia";
}

switch (country) {
    case "Canada":
    continent = "North America";
    break;
    case "China":
    continent = "Asia";
    break;
    case "Ghana":
    continent = "Africa";
    break;
    case "Bolivia":
    continent = "South America";
    break;
    case "France":
    continent = "Europe";
    break;
    default:
    continent = "Australia";
    break;
}
```

- **END Lab 02.03 Solution**



Date Object

Outputting date and time to the web page

The Date Object returns the full date and time from the user's computer. It is instantiated (declared) using the **new** keyword.

1. Instantiate an instance of the Date object.

```
let dateTime = new Date();
console.log(dateTime);
```

The individual time units are available by calling the Date object's "get methods".

2. Get the current hour, minute and second:

```
let hour = dateTime.getHours();
console.log(hour); // 0-23

let minute = dateTime.getMinutes();
console.log(minute); // 0-59

let second = dateTime.getSeconds();
console.log(second); // 0-59
```

3. Express the time in 00:00:00 format:

```
let timeIs = `${hour}:${minute}:${second}`;
console.log(timeIs);
```

leading zeroes for minute and second

If minute or second is less than 10, you get wonky output, such as 1:2:3, instead of 1:02:03. To fix this, add leading zeros to minute and second, as needed. This is done with conditional logic.

4. Add a leading 0 if minute or second is less than 10:

```
if(minute < 10) {
  minute = '0' + minute;
}

if(second < 10) {
  second = '0' + second;
}

timeIs = `${hour}:${minute}:${second}`;
console.log(timeIs);
```

If an if-statement has only one line of code inside its curly braces, you can omit the curly braces altogether and put everything on the same line.

5. Make these short if-statements even more concise by eliminating the curly braces:

```
if(minute < 10) minute = '0' + minute;
if(second < 10) second = '0' + second;

timeIs = `${hour}:${minute}:${second}`;

console.log('timeIs w leading 0', timeIs); // 00:00:00
```

converting military time to AM/PM

The hour is from 0-23 ("military time"), so 3pm is 15:00 and 10pm is 22:00.

To convert to AM/PM time, we need:

- a variable to store the string "AM" or "PM".
- two if-statements, done in this order:
  - if hour > 11, use "PM".
  - if hour > than 12, subtract 12.

6. Declare a variable **amOrPm** with an initial value of 'AM', and follow that with the if-statements:

```
let amOrPm = 'AM';

if(hour > 11) {
    amOrPm = 'PM';
}

if(hour > 12) {
    hour -= 12;
}
```

```
    timeIs = `${hour}:${minute}:${second} ${amOrPm}`;
    console.log('time is: ', timeIs);
```

timely greeting

Now let's make a "timely greeting" that is appropriate for the current hour:

- **if** the hour is less than 12 (noon), say "Good morning!".
- **else if** the hour is less than 18 (6:00pm), say "Good afternoon!"
- **else**, say "Good Evening!"

Start the greeting with "Good" and then use += to concatenate the "timely" part:

7. Get a fresh hour, since our original hour may have already had 12 subtracted from it:

```
    let hr = dateTime.getHours();
```

8. Declare greeting with an initial value of 'Good':

```
    let greeting = "Good ";
```

9. Do the logic for **hr < 12** (noon):

```
    if(hr < 12) {
        greeting = "morning";
    }
```

10. Add an **else if** for **hr < 18** (6pm). Follow that with an **else** that runs when **hr** is 18 and up:

```
    if (hr < 12) {
      greeting += "Morning!";
    } else if (hr < 18) {
      greeting += "Afternoon!";
    } else {
      greeting += "Evening!";
    }

    console.log(greeting);
```

"Timely Greeting"

11. Pair the greeting with the time of day in AM-PM format:

```
```

```
let timelyGreeting = `${greeting} The time is: ${timeAMPM}`;
console.log(timelyGreeting);
```

12. Output the "timely greeting" to the web page. Start by getting the tag that will display the greeting:

```
let greetingTag = document.getElementById('greeting');
```

13. Set the **textContent** property of the tag object to **timelyGreeting**:

```
greetingTag.textContent = timelyGreeting;
```

The Date object's other time units can be used to concatenate and output today's date:

14. Get today's date:

```
let date = dateTime.getDate();
console.log('date', date);
```

15. Get the month, which is returned as a number from 0-11 (Jan = 0, Dec = 11):

```
let month = dateTime.getMonth();
console.log('month', month);
```

16. Get the month as a string (January, February, etc). This gives us the flexibility to use the month as either a number or a day:

```
let fullMonth = dateTime.toLocaleString('default', {month:'long'})
console.log('fullMonth', fullMonth);
```

17. Get the day of the week, which is a number, with Sunday=0 and Saturday=6:

```
let day = dateTime.getDay();
console.log('day', day);
```

18. Make an array of the days of the week.

```
let daysArr = ["Sunday", "Monday", "Tuesday", "Wednesday", "Thursday",
"Friday", "Saturday"];
```

We'll explore arrays thoroughly in upcoming lessons, but basically, an array is a variable that stores multiple values as a list, inside square brackets.

19. Look up the first item in the array (Sunday) by its index (0):

```
let Sun = daysArr[0];
console.log('Sun', Sun); // Sun Sunday
```

20. Get the day of the week by looking it up index (number) in the array:

```
let dayOfWeek = daysArr[day];
console.log('dayOfWeek', dayOfWeek); // string
```

21. Get the full 4-digit year:

```
let fullYear = dateTime.getFullYear();
console.log(fullYear);
```

22. Concatenate today's date as **Day, Month Date, Year**, using the non-numeric day and month, e.g. **Tuesday, May 17, 2022**:

```
let todaysDate = `${dayOfWeek}, ${fullMonth} ${date}, ${fullYear}`;
console.log('todaysDate', todaysDate); // string
```

23. Output today's date to its place on the web page:

```
let todaysDateTag = document.getElementById('todays-date');

todaysDateTag.innerHTML = todaysDate;
```

updating the time every second

You may be wondering: Why doesn't the time on the web page updating every 1 second? To auto-update every second would require more code. In a later lesson, we'll get into using **setInterval** to call a function X times per second to do just this sort of thing.

END Lesson 02.04

NO LAB EXERCISES

PROCEED DIRECTLY TO LESSON 03.01



1. Continuing where we left off with the "timely greeting", add to the logic so that:

- if the hour is between midnight and 2 AM, the greeting is "Good Evening"
- if the hour is between 2-4 AM, greeting is "Hey, Night Owl!"
- all other greeting times remain the same

```
let dateTime = new Date();
let hr = dateTime.getHours();
let greeting = 'Good ';

if(hr < 12) {
    greeting += "Morning!";
} else if(hr < 18) {
    greeting += " Afternoon!";
} else {
    greeting += 'Evening!';
}

console.log(greeting);
```

2. Test the output by hard-coding **hr**. Sample hr values and their expected output:

- When **hr = 1**, output should be "Good Evening!";
- When **hr = 3**, output should be "Hey, Night Owl!";
- When **hr = 6**, output should be "Good Morning!";
- When **hr = 15**, output should be "Good Afternoon!";
- When **hr = 20**, output should be "Good Evening!";
- **END Lab 02.04**
- **SEE Lab 02.04 Solution**

02.04 Lab Solution

1. Continuing where we left off with the "timely greeting", add to the logic so that:

- if the hour is between midnight and 2 AM, the greeting is "Good Evening"
- if the hour is between 2-4 AM, greeting is "Hey, Night Owl!"
- all other greeting times remain the same

```
let dateTime = new Date();
let hr = dateTime.getHours();
let greeting = 'Good ';

if(hr < 2) {
    greeting += "Evening!";
} else if(hr < 4) {
    greeting = "Hey, Night Owl!";
} else if(hr < 12) {
    greeting += " Morning!";
} else if(hr < 18) {
    greeting += " Afternoon!";
} else {
    greeting += 'Evening!';
}

console.log(greeting);
```

2. Test the output by hard-coding **hr**. Sample hr values and their expected output:

- When **hr = 1**, output should be "Good Evening!";
- When **hr = 3**, output should be "Hey, Night Owl!";
- When **hr = 6**, output should be "Good Morning!";
- When **hr = 15**, output should be "Good Afternoon!";
- When **hr = 20**, output should be "Good Evening!";

```
let dateTime = new Date();
let hr = dateTime.getHours();
let greeting = 'Good ';

// HARD-CODED EXAMPLE:
hr = 2;

if(hr < 2) {
    greeting += "Evening!";
} else if(hr < 4) {
    greeting = "Hey, Night Owl!";
} else if(hr < 12) {
    greeting += " Morning!";
} else if(hr < 18) {
```

```
        greeting += " Afternoon!";
    } else {
        greeting += 'Evening!';
    }

    console.log(greeting); // Hey, Night Owl!
```

- **END Lab 02.04 Solution**

Lesson 03.01

functions

parameters and arguments

return values

function-scope vs global scope

default parameter values

A function is a block of code that runs only when it is invoked (called). The function can be called in the code or by an event on the web page, such as a button click. A function typically (but certainly not always) has a name to call it by.

Mozilla mdn web docs defines functions in this way:

> Functions are one of the fundamental building blocks in JavaScript. A function in JavaScript is similar to a procedure—a set of statements that performs a task or calculates a value, but for a procedure to qualify as a function, it should take some input and return an output where there is some obvious relationship between the input and the output. To use a function, you must define it somewhere in the scope from which you wish to call it.

A function *should take some input and return an output*, so it is useful to think of a function as an input-output machine. Think of real world analogies:

- coffee grinder as real world function (input-output machine):
  - input: you put coffee beans in
  - procedure: blades turn inside the machine
  - output: out comes ground coffee

All that said, it is possible to have a function without inputs (parameters). This is where we will begin:

how to define (declare) a function

Follow these steps:

1. Begin by writing the keyword: `function`
2. Give the function a name. Variable naming rules apply. Since a function *does* something, it is customary for the name to begin with a verb: `function sayHello`
3. Next put a pair of parentheses: `function sayHello()`

The parentheses are for passing data -- *inputs* -- to the function, but some functions don't need inputs, and so the parentheses are left empty.

4. Next add a pair of curly braces: `function sayHello() {}`

5. Inside the curly braces, put the code that will run when the function is called:

```
    function sayHello() {
        console.log("Hello!");
    }
```

6. Run this example and check the Console. Nothing happens. Where's "Hello"?

Remember, a function must be *called*, but all we have done is *define* it.

7. We do not have a button to click to call the function, so's call it directly in the code:

```
    sayHello();
```

8. Run the code in the Console again; this time you get the "Hello!".

variable scope

global variables are available everywhere in a script: - **var** declared outside of a function is global. - **let** declared outside of a code block is global. - **const** (constants) follow the same scoping rule as let.

function scoped (local) variables - **var**, **let** declared inside a function is local to that function. - **let** declared inside a code block is available only to that code block.

// 9. Declare a local variables inside the function:

```
    function welcomePlayer() {
        var username = "Pikachubaca";
        let highScore = '6234';
        console.log(`Welcome back, ${username}! Your high score is
${highScore}`);
    }

    welcomePlayer();
```

10. Try to access the function variables in the global scope, outside the function:

```
    // console.log('username', username); // Error: username is not
defined
    // console.log('highScore', highScore); // Error: username is not
defined
```

parameters-arguments

Parameters (params for short) are inputs of a function that go in the parentheses. Inside the function, parameters are local variables. When a function is called, the params are assigned values as arguments passed into the function call parentheses.

11. Write a function with two parameters:

```
    function greetPlayer(username, highScore) {
        console.log(`Welcome, ${username}! Your high score is
${highScore}!`);
    }
```

12. Call the function twice with different arguments each time:

```
    greetPlayer("Brian88", 12345);
    // Welcome, Brian88! Your high score is 12345!

    greetPlayer("Bob", 78967);
    // Welcome, Bob! Your high score is 78967!
```

If a function expects an argument, but none is provided, the missing value will be 'undefined'.

13. Call the function again, but this time omit the second argument:

```
    greetPlayer('Sally123');
    // Welcome, Sally123 undefined
```

default parameter values

A parameter can be assigned a value when the function is declared. That way, if no argument is supplied for it, it uses the default.

14. Write a new function, greetPlayerScore(). It is the same as greetPlayer, but with a default high score of 1000:

```
    function greetPlayerScore(username, highScore=1000) {
        console.log(`Welcome, ${username}! Your high score is
${highScore}`);
    }
```

15. Call the function twice, but the second time, only pass in the username. The high score will default to 1000:

```
greetPlayerScore("X12", 13240);
// Welcome, X12! Your high score is 13240!
greetPlayerScore("yMe");
// Welcome, yMe! Your high score is undefined!
```

Global variables are available everywhere, including to all functions:

16. Declare some global vars and use them in a function;

```
let num1 = 10;
let num2 = 14;
let sum = 0;

function addNums() {
    sum = num1 + num2;
    console.log("sum", sum);
}

addNums(); // sum 24
```

A more self-contained approach would be to num1 and num2 to the funcition as parameters (unless num1 and num2 are needed elsewhere in the script).

refactoring Refactoring code means changing it for the better.

17. Refactor addNums() as addNumbers() and give it parameters. The values of num1 and num2 will be passed in as the arguments when the function is called:

```
function addNumbers(num1, num2) {
    let sum = num1 + num2;
    console.log(sum, sum);
}

addNumbers(12, 18); // sum 30
addNumbers(123, 184); // sum 307
```

return value

So far, our output has consisted only of console.log. This is useful for tesing and debugging, but once the output is logged, that data is gone. The answer is for the output to be a "return value" which we can save in the script.

return value

To return a value from a function, put the keyword 'return' in front of a value.

Save the return value by setting the function call equal to a variable.

A return not only returns a value, it exits the function.

18. Define a new function, that is similar to the previous one, but with one big exception: this one has a return value:

```
let numeroUno = 7;
let numeroDos = 8;

function multiplyNumbers(n1, n2) {
    return n1 * n2;
}
```

19. Call the function, setting the function call itself equal to a variable to save the return value:

```
let prod1 = addNums3(12, 18);
console.log('prod1', prod1);

let prod2 = addNums3(3412, 5618);
console.log('prod2', prod2); // 9030
```

20. Refactor the greet function to return a value. Use a new name: getGreeting.

- "get" in function names means it returns a value
- replace the `console.log` with `return`
- set the function calls equal to variables
- log the variables to output the return values

```
function getGreeting(username, highScore) {
    return `Welcome, ${username}! Your high score is ${highScore}`;
}

let welcome1 = getGreeting("Viper", 73489);
console.log('welcome1:', welcome1);
// Welcome, Viper! Your high score is 73489!

let welcome2 = getGreeting("Ktrav26", 12345);
console.log('welcome2:', welcome2);
// Welcome, Ktrav26! Your high score is 12345!
```

21. Declare a function **calcRestaurantBill()**.

- Give it three parameters: **subTotal**, **tipPct** and **taxRate**, the latter two with default values: **tipPct=15** and **taxRate=4**. This assigns a default 15% tip and default tax rate of 4% if no value(s) are provided for these.

```
function calcRestaurantBill(subTotal, tipPct=15, taxRate=4) {
```

```
}
```

The function calculates the tax and tip and adds those amounts to the subtotal

- calculate tip:
  - **let tip = subTotal * tipPct / 100;**
- calculate tax:
  - **let tax = subTotal * taxRate / 100;**
- add tax and tip to subtotal to get total:
  - **let total = subTotal + tax + tip;**

22. Write the calculations. The tax and tip percents are divided by 100, because 15% is not 15, but rather 0.15:

function calcRestaurantBill(subTotal, tipPct=15, taxRate=4) { let tax = subTotal * taxRate / 100; let tip = subTotal * tipPct / 100; let total = subTotal + tax + tip; }

23. Concatenate the bill and return it:

```
function calcRestaurantBill(subTotal, tipPct=15, taxRate=4) {
    let tax = subTotal * taxRate / 100;
    let tip = subTotal * tipPct / 100;
    let total = subTotal + tax + tip;
    let bill = `Sub-total:  $ ${subTotal}
    Tax Rate:   ${taxRate} %
    Tip Pct:    ${tipPct} %
    Tip:        ${tip}%
    Tax:        $ ${tax}
    TOTAL:      $ ${total}
    Thank you!`;
    return bill;
}
```

24. Add toFixed(2) to round the cents to 2 decimal places:

```
Tip:        ${tip.toFixed(2)} %
Tax:        $ ${tax.toFixed(2)}
TOTAL:      $ ${total.toFixed(2)}
```

25. Call the function, and supply all three parameters. Make the tip 20 percent and the tax 6 percent. This overrides the default tipPct and taxRate.

```
// $80 subtotal, 20% tip, 6% tax
let bill1 = calcRestaurantBill(80, 20, 6);
console.log(bill1);
```

26. Call the function again, but this time omit the third argument (tax). The tax rate will therefore default to 4%:

```
// $90 subtotal, 18% tip, 4% tax
let bill2 = calcRestaurantBill(90, 18);
console.log(bill2);
```

27. Call the function, passing in only one parameter, understood to be subTotal. tipPct and taxRate will use the defaults:

```
// $65 subtotal,  15% default tip,  4% default tax
let bill3 = calcRestaurantBill(65);
console.log(bill3);
```

End Lesson 03.01

Next: Lab 03.01



1. Complete this function to convert the inputted **feet** value to meters and **return** the result. Conversion units: 39.37 inches = 1 meter;

```
function convertDistance(feet) {
    // convert feet to meters
}
```

2. Improve the previous function so that it can convert in both directions: feet-to-meters and meters-to-feet. You will need to provide some way for the function to know which conversion to do.

3. Make a function that:

    - takes one number as its input (argument)
    - squares that number (multiplies it by itself)
    - logs the answer to the Console. So, if you input 4, it logs 16. Run the function three times with different inputs.

4. Make another version of the previous function, so that:

    - instead of logging the answer, it returns the answer.
    - When you call the function, save the return value to a variable.
    - Log the variable to see if the function works. So, if you input 5, it returns 25. Run the function three times with different inputs.

5. Make a function that:

    - takes one number as its input (argument)
    - cubes the input number
    - returns the answer So, if you input 4, you get back 64. Run the function three times with different values.

6. Make another version of the previous function that:

    - takes one number as its input (argument)
    - if the number is even, it squares the number
    - but if the number is odd, it cubes the number
    - returns the answer. So, if you input 3, you get back 27. So, if you input 4, you get back 16. Run the function three times with different values.

7. The weight of anything on the moon is one-sixth its weight on earth. Make a function called **calcMoonWeight** that takes in an earth weight number as its argument, calculates the the moon

weight equivalent, and returns that number. So if the input is 180, the output is 30.

8. Declare a function called **introducePet**, that:

    - has four parameters: **pet**, **name**, **age** and **sound**, each of which gets set when the function is called.
    - uses string interploation to make a message that includes all four arguments.
    - returns a message, such that if the arguments are **cat**, **Fluffy**, **3** and **Meow**, the returned message is: **Meow! My name is Fluffy! I am a 3-year-old cat!**. Run the function three times, with different pet inputs each time.

9. Declare a function **introduceSelf** with two parameters: **name** and **city**. The function does not have a return value, but instead logs a message, such as: **Hi! My name is Jill. I am from Miami!**

10. Declare a function with two parameters, **num1** and **num2**. The function call passes in two arguments, both numbers.
    The function does the following math:

    - If the **num1** is greater than **num2**, subtract **num2** from **num1**
    - If **num1** is less than or equal to **num2**, add the numbers together.
      Return the answer. Run the function twice, once with the numbers being subtracted, the other with the numbers being added.

11. Given: two sides of a right triangle as global variables **sideA** and **sideB**

- Write a function with parameters **a** and **b**
    - Function uses the Pythagorean Theorem (**$a^2 + b^2 = c^2$**) to find the hypotenuse, **c**, of **a** and **b**.
    - Function returns **c**, the hyotenuse.
    - Call the function, passing in **sideA** and **sideB** as its two arguments.

12. Write a function that:

- takes in numbers of pennies, nickels, dimes and quarters
- calculates the total value of all coins
- returns the total as dollars and cents, to two decimal places and with dollar-sign

END Lab 03.01
SEE Lab 03.01 Solution

Lab 03.01 - Solution:

1. Complete this function to convert the inputted **feet** value to meters and **return** the result. Conversion units: 39.37 inches = 1 meter;

```
function convertDistance(feet) {
    // convert feet to meters
}
```

2. Improve the previous function so that it can convert in both directions: feet-to-meters and meters-to-feet. You will need to provide some way for the function to know which conversion to do.

3. Make a function that:

    - takes one number as its input (argument)
    - squares that number (multiplies it by itself)
    - logs the answer to the Console. So, if you input 4, it logs 16. Run the function three times with different inputs.

```
function squareNumber(x) {
    console.log(x ** 2); // or: x * x;
}
squareNumber(4); // 16
squareNumber(5); // 25
squareNumber(6); // 36
```

4. Make another version of the previous function, so that:
    - instead of logging the answer, it returns the answer.
    - When you call the function, save the return value to a variable.
    - Log the variable to see if the function works. So, if you input 5, it returns 25. Run the function three times with different inputs.

```
function squareNum(x) {
    return x * x; // or: x ** 2;
}

let sq4 = squareNum(4);
console.log(sq4); // 16

let sq9 = squareNum(9);
console.log(sq9); // 81

let sq12 = squareNum(12);
console.log(sq12); // 144
```

5. Make a function that:
    - takes one number as its input (argument)
    - cubes the input number
    - returns the answer So, if you input 4, you get back 64. Run the function three times with different values.

```
function cubeIt(x) {
    let cube = x ** 3;
    return cube;
}

let cube4 = cubeIt(4);
console.log(cube4); // 64

let cube5 = cubeIt(5);
console.log(cube5); // 125

let cube6 = cubeIt(6);
console.log(cube6); // 216
```

6. Make another version of the previous function that:
    - takes one number as its input (argument)
    - if the number is even, it squares the number
    - but if the number is odd, it cubes the number
    - returns the answer. So, if you input 3, you get back 27. So, if you input 4, you get back 16. Run the function three times with different values.

```
function squareOrCubeIt(x) {
    let answer = 0;
    // if x is even, dividing by 2 yields remainder of 0
    if(x % 2 == 0) { // if x is even, square it
        answer = x * x;
    } else { // x is odd, so cube it
        answer = x ** 3;
    }
    return answer;
}

let a = squareOrCubeIt(5); // cube it
console.log(a); // 125

let b = squareOrCubeIt(6); // square it
console.log(b); // 36

let c = squareOrCubeIt(7); // cube it
console.log(c); // 343
```

7. The weight of anything on the moon is one-sixth its weight on earth. Make a function called **calcMoonWeight** that takes in an earth weight number as its argument, calculates the the moon weight equivalent, and returns that number. So if the input is 180, the output is 30.

```
    function calcMoonWeight(earthWt) {
        // convert earth wt on moon wt
        var  moonWt = earthWt / 6;
        return moonWt;
    }

    let myMoonWt = calcMoonWeight(192); // 32

    console.log(myMoonWt);
```

8. Declare a function called **introducePet**, that:
    - has four parameters: **pet**, **name**, **age** and **sound**, each of which gets set when the function is called.
    - uses string interploation to make a message that includes all four arguments.
    - returns a message, such that if the arguments are **cat**, **Fluffy**, **3** and **Meow**, the returned message is: **Meow! My name is Fluffy! I am a 3-year-old cat!**. Run the function three times, with different pet inputs each time.

```
    function introducePet(pet, name, age, sound) {
        let intro = `${sound}! My name is ${name}! I am a ${age}-year-old
${pet}`;
        return intro;
    }

    let petIntro1 = introducePet('cat', 'Fluffy', 3, 'Meow');
    console.log(petIntro1);
    // Meow! My name is Fluffy! I am a 3-year-old cat!

    let petIntro2 = introducePet('dog', 'King', 2, 'Woof');
    console.log(petIntro2);
    // Woof! My name is King! I am a 2-year-old cat!

    let petIntro3 = introducePet('canary', 'Tweety', 4, 'Chirp');
    console.log(petIntro3);
    // Chirp! My name is Tweety! I am a 4-year-old canary!
```

9. Declare a function **introduceSelf** with two parameters: **name** and **city**. The function does not have a return value, but instead logs a message, such as: **Hi! My name is Jill. I am from Miami!**

```
    function introduceSelf(name, city) {
        console.log(`Hi! My name is ${name}! I am from ${food}?`);
    }
```

```
// function expects two arguments
greetByNameAndFood('Sally', 'grapes');
```

10. Declare a function with two parameters, **num1** and **num2**. The function call passes in two arguments, both numbers.
    The function does the following math:
    - If the **num1** is greater than **num2**, subtract **num2** from **num1**
    - If **num1** is less than or equal to **num2**, add the numbers together.
      Return the answer. Run the function twice, once with the numbers being subtracted, the other with the numbers being added.

```
function addOrSubtractNums(num1, num2) {
    let answer = 0;
    if(num1 > num2) {
        answer = num1 - num2;
    } else {
        answer = num1 + num2;
    }
    return answer;
}

let answr1 = addOrSubtractNums(30, 20); // 10
let answr2 = addOrSubtractNums(20, 30); // 50
```

11. Given: two sides of a right triangle as global variables **sideA** and **sideB**

- Write a function with parameters **a** and **b**
  - Function uses the Pythagorean Theorem (**$a^2 + b^2 = c^2$**) to find the hypotenuse, **c**, of **a** and **b**.
  - Function returns **c**, the hyotenuse.
  - Call the function, passing in **sideA** and **sideB** as its two arguments.

```
let sideA = 3;
let sideB = 4;

function findHypotenuse(a, b) {
    let cSquared = a**2 + b**2;
    let c = Math.sqrt(cSquared);
    return c;
}

let c1 = findHypotenuse(sideA, sideB);
console.log(c1); // 5

let c2 = findHypotenuse(6, 8);
console.log(c2); // 10

let c3 = findHypotenuse(9, 12);
console.log(c3); // 15
```

12. Write a function that:

- takes in numbers of pennies, nickels, dimes and quarters
- calculates the total value of all coins
- returns the total as dollars and cents, to two decimal places and with dollar-sign

```
    function countCoins(pennies, nickels, dimes, quarters)  {
        let cents = (pennies) + (nickels * 5) + (dimes * 10) + (quarters *
25);
        return '$' + ((cents / 100).toFixed(2));
    }
    let money1 = countCoins(250, 50, 25, 10); // 2.50 + 2.50 + 2.50 + 2.50
= $10.00
    console.log(money1);
    let money2 = countCoins(100, 100, 100, 100); // 1 + 5 + 10 + 25 =
$41.00
    console.log(money2);
```

END Lab 03.01
SEE Lab 03.01 Solution



DOM (Document Object Model) functions

events calling functions (click, change)

Document Object Model (DOM)

JS "sees" the web page as a hierarchical collection of objects, known as the DOM (Document Object Model).

DOM manipultion

JS can make stuff happen and things change on the web page. This is known as "DOM manipultion".

DOM event

A DOM event is something that happens on the web page that can call a function. Functions that manipulate the DOM are typically triggered by a button click, menu change, or other **DOM event**.

HTML elements as JS Objects

Any HTML element can be brought into JS where it becomes an object. The commands for getting elements are:

- **document.getElementById()** gets the element of specified id and stores it in an object
- **document.querySelector()** gets the first element of specified class or tag name and stores it in an object
- **document.querySelectorAll()** gets all elements of specified class or tag name and stores them in an array called a Node List

button.addEventListener()

To enable a button or other element to call a function, attach the **addEventListener** method to the object. The method requires two arguments: an event to 'listen' for and a function to call when the event takes place.

DOM function exploration page

This lesson has a page already set up with the HTML and CSS for several buttons, some input text fields and a select menu.

Open the HTML file in the browser and have a look: lots of buttons and things, but nothing works yet, because it is our job to write the code for that.

return value vs. no return value

Before we get started, it is important to note that these functions do not **return** a value. The function output is being displayed on the web page, rather than being stored in a variable in the script. That said, it is possible to combine return values with DOM output, as we will see.

function #1: greetWorld()

We want the **greet world** button to call a **greetWorld()** function. The button is in the html as: `greet world`

1. In the JS START file, add this function, and call it from the script (no button yet):

```
function greetWorld() {
  alert('Hello World');
}
greetWorld();
```

2. Comment out the function call since a button click is taking over.
3. Get the element `greet world` by id so that we can tell it to call the function. Save it to a variable so that we can tell it to call a function:

```
const btn1 = document.getElementById('btn-1');
```

4. Also get the

   output

   tag to hold the output. Save it to a variable so that we can output our function results there:

```
const output = document.getElementById('output');
```

const for objects

As we learned in a previous lesson, **const** for primitive data types (string, number, boolean) cannot be changed in any way.

But **const** works differently for objects:

- **const** objects cannot be *mutated*, which means the *data type* cannot be changed.
- **const** properties, however, *can** be changed.
- **const** protects objects from being accidentally changed to, say, a string, but otherwise you can change/add/remove properties at will.

For these reasons, we typically use **const** to save DOM objects that we bring into JS.

element attributes become object properties

Once in JS, the elements' attributes are the object's properties.

5. Log **btn1** and **object** as well as and some properties:

```
  console.log(btn1);
  console.log(btn1.id);
  console.log(btn1.textContent);
  console.log(output);
  console.log(output.id);
  console.log(output.textContent);
```

addEventListener()

The **addEventListener()** is called on an object and takes two arguments: an event to "listen" for and a function to call when the event takes place.

6. Tell the **btn1** object to "listen" for a click upon itself and to call **greetWorld** when the click occurs.

```
  btn1.addEventListener('click', greetWorld);
```

Notice that it is **greetWorld** *not* **greetWorld()**. This is because the parentheses would instantly call the function. We want the funtion to wait for the click.

7. Run the page and click the button. You should get the Hello World alert.

outputting to the DOM

The function works, so next let's have the output appear on the web page, rather than in a pop up.

8. In the function, comment out the alert and switch to displaying the message as the **textContent** property of the **output** tag:

```
  function greetWorld() {
    // alert('Hello World');
    output.textContent = 'Hello World';
  }
```

.textContent

The **textContent** property is everything between the tags, but cannot include any html. It is therefore best for simple strings, such as 'Hello World'.

.innerHTML

innerHTML refers to everything between the tags, including html, so tags within tags.

9. Change the output to be a link, which necessitates a switch from *textContent* to *innerHTML*:

```
function greetWorld() {
  // alert('Hello World');
  output.innerHTML = '<a href="#">Hello World</a>';
}
```

function #2: greetByName()

This next function also runs on button click. It gets the user-inputted names from the input fields, concatenates a personalized greeting, and outputs it to the page.

The html is already written:

```
<div>
  <input type="text" id="firstName" placeholder="First Name">
  <input type="text" id="lastName" placeholder="Last Name">
  <button id="btn-2">greet by name</button>
</div>
```

10. Get the button `greet by name`:

```
const btn2 = document.getElementById('btn-2');
```

11. Have the button "listen" for a click and call the function when the click occurs:

```
btn2.addEventListener('click', greetByName);
```

12. Write the function. Get the **values** of the text input fields, which have id's of **firstName** and **lastName**. The values are whatever the user typed:

```
function greetByName() {
    let fName = document.getElementByIc('firstName').value;
    let lName = document.getElementByIc('lastName').value;
    output.textContent = **Hey, ${fname} $lName}**;
}
```

13. Run the page and enter a first and last name.

14. Click the **greet by name** button. The personalized output should appear.

function #3: pickFruit()

This next function runs when the user chooses a fruit from the menu. The html is already there:

```
<select id="fruit-menu">
  <option value="choose">Pick a Fruit:</option>
  <option value="apple">Apple</option>
  <option value="banana">Banana</option>
  <option value="cherry">Cherry</option>
  // etc. more options
</select>
```

The way it works is:

- user chooses a fruit from the **select** menu.
- the choice is a **change** event, which calls the function, which:
    - gets the menu **value**, which equals the value of the selected **option**
    - concatenates and outputs a message to the **output** tag.

15. Get the select menu:

```
let fruitMenu = document.getElementById('fruit-menu');
```

16. Tell it to listen for a **change** event. When it hears the change (to itself), it calls a **pickFruit** function:

```
fruitMenu.addEventListener('change', pickFruit);
```

17. Write the function:

- Get the value of **fruitMenu** and save it to a local variable, **fruit**.
- Append **toUpperCase()** to make the fruit UPPERCASE.
- Concatenate and output a message about the chosen fruit.

```
function pickFruit() {
    let fruit = fruitMenu.value.toUpperCase();
    output.textContent = **Your chosen fruit is: ${fruit}!**;
}
```

function #4: addNumbers()

Next, we will do a function that adds numbers inputted by the user and outputs the sum.

We already have the html for this:

```
<div>
  <input type="number" id="num1-box" placeholder="Enter a Number">
  <input type="number" id="num2-box" placeholder="Enter a Number">
```

```
  <button id="btn-3">Add Numbers</button>
</div>
```

The **input** elements are set to **type="number"** to prevent non-numeric characters from being entered.

19. Get the **add numbers** button and tell it to call a function when clicked:

```
let btn3 = document.getElementById('btn-3');
btn3.addEventListener('click', addNumbers);
```

20. Write the function. Get the values from the input boxes. Then add the numbers and concatenate the output:

```
function addNumbers() {
  let num1 = document.getElementById('num1-box').value;
  let num2 = document.getElementById('num2-box').value;
  let sum = num1 + num2;
  output.textContent = sum;
}
```

21. Run the page. Enter two numbers and click the **add numbers** button.

There is a bug. The sum is no sum at all, but rather a concatenation of the two numbers. This happened because even though the input boxes are of type number, the "numbers" still came into JS as strings.

22. Convert the numeric input to actual numbers so that the + adds them:

```
  function addNumbers() {
    let num1 = document.getElementById('num1-box').value;
    let num2 = document.getElementById('num2-box').value;
    num1 = Number(num1);
    num2 = Number(num2);
    let sum = num1 + num2;
    output.textContent = sum;
}
```

23. Run the page again. The math should work now.

data- attribute

You can attach additional data to any tag by adding a "data-name" attribute. The data is available in JS as: object.dataset.name

24. Comment out the select menu and "uncomment out" the other select menu--the one where each option has a data-food attribute:

```
    <select id="fruit-menu">
        <option value="choose">Pick a Fruit:</option>
        <option value="apple" data-food="apple sauce">Apple</option>
        <option value="banana" data-food="banana bread">Banana</option>
        - ETC -
    </select>
```

options array

The select object has an options property, which is an array. We will get into arrays in Unit 4, but for now, just know that an array is a variable that holds more than one item at a time. The items are stored by number, called index. We can look up an array item by index. When a menu choice is made, the object selectedIndex property is set with a number that corresponds to the option chosen. We can use that number to look up the selected option from the options array. Once we have that option, can access any **data-** attribute via the **dataset** property. We have an attribute data-food, the value of which is **dataset.food**

Refactor the pickFruit function to include the food:

25. In the function, save the we numeric index of the selected item to a variable. This simplifies the next line.

```
    let indx = fruitMenu.selectedIndex;
```

26. Get the selected option by its index in the options array. Save that to a variable **optn**:

```
    let optn = menu.options[indx];
```

27. Get the value of the **data-food** attribute for the option:

```
    let food = optn.dataset.food;
```

28. Add the food to the output:

```
      output.textContent = `Your chosen fruit is: ${fruit}! Do you like
  ${food}?`;

    } // end function pickFruit()
```

END LESSON 03.02

NEXT LESSON 03.03

Lesson 03.03

Login Form

event object

event.preventDefault()

hiding DOM elements with display='none'

_____-

We have a Login form that we will make work in an emulation scenario. The html and css for the form are all ready.

Open the html page in the browser to have a look at the form. It doesn't work yet--that's our job to write the code to make it work.

How the log in works:

- The user will enter a username and password
- The user clicks the LOGIN button to call the login() function
- The function gets the values inputted in the username and password fields
- The entered values are compared to the "real" username and password
- If they both match, we get a welcome message and the form elements disappear
- If either the username or password are wrong. we get a 'login failed' message

There are several DOM elements that we need in our login function:

1. Get the loging button and save it as an object:

```
    let loginBtn = document.getElementById('login-btn');
```

2. Make the button clickable to call the login function:

```
    loginBtn.addEventListener('click', login);
```

3. Get the 'Please Log In' heading, since we will hide it on login success:

```
    let plsLogin = document.getElementById('pls-login');
```

4. Get the username and password input boxes:

```

```

```
let userBox = document.getElementById('username');
let pswdBox = document.getElementById('password');
```

This login form doesn't have a database connection to look up a real username and password, so we will hard-code our username and password:

5. Hard-code the "correct" username and password:

```
let correctUser = "joe";
let correctPswd = "abc";
```

6. Get the response element, which is where the output goes:

```
let response = document.getElementById('response');
```

event.preventDefault() When a button inside a form is clicked, it reloads the page by default. This is to export the form variables to a server-side processing script. But since we don't have server connectivity, are just hard-coding the username and password. We don't want to reload, because doing so will clear the input boxes. So to prevent that default behavior, we call the `event.preventDefault()`.

event object

The `event` object is default parameter that is there even if you don't add it as a parameter. The default name is `event`, but you can rename the event object anything you like; `evt` and `e` are the most commonly used aliases.

7. Write the function, passing in the **event** argument and calling the **event.preventDefault()** method:

```
function login(event) {
    event.preventDefault();
}
```

8. Get the inputted `username` and `password`. These are the `value` properties of their respective input objects. We just need the values--not the whole objects.

```
function login(event) {
    event.preventDefault();
    let inputtedUser = userBox.value;
    let inputtedPswd = pswdBox.value;
}
```

9. Compare the entered username and password to the "correct" values. Since there are two conditions to evaluate, use the `&&` operator:

```
    function login(event) {
        event.preventDefault();
        let inputUser = userBox.value;
        let inputPswd = pswdBox.value;

        if(inputUser === correctUser && inputPswd === correctPswd) {
        }

    } // end login function
```

10. If login was successful, output the welcome message and hide the form elements by setting their `display` property to `none`:

```
    function login(event) {

        event.preventDefault();

        let inputUser = userBox.value;
        let inputPswd = pswdBox.value;

        if(inputUser === correctUser && inputPswd === correctPswd) {
            response.style.fontSize = '1.5em';
            response.textContent = 'Welcome, ' + correctUser;
            userBox.style.display = "none"; // hide username field
            pswdBox.style.display = "none"; // hide password field
            loginBtn.style.display = "none"; // hide Log In button
            plsLogin.style.display = "none"; // hide Please Log In
        }

    } // end login function
```

11. Add an `else` part to run if either the username or password are incorrect:

```
        } else {
            response.textContent = "Log in failed. Try again.";
        }

    } // end login function
```

leaving out event.preventDefault()

If you omit `event.preventDefault()`, the form won't work because the page will reload, which will reset the username and password fields.

12. Comment out the `event.preventDefault()`, reload the page and log in again. You will see the page flash as it reloads and the form fields will be reset.

```
    function login(event) {

        // event.preventDefault();
```

END Lesson 03.03

NEXT: Lesson 03.04



UNIT 03

LESSON 03.04



RESTAURANT BILL TAX AND TIP CALCULATOR

The restaurant bill calculator consists of a web form:

- inputs for Food and Beverage subtotals
- select menus for Tip and Tax percent
- a Calculate button to call the `calcBill` function
- an h2 to display the itemized bill made by the function

1. Get the form objects:

```
let food = document.getElementById('food');
let bev = document.getElementById('bev');
let taxMenu = document.getElementById('tax-menu');
let tipMenu = document.getElementById('tip-menu');
let calcBtn = document.getElementById('calc-btn');
let response = document.getElementById('response');
```

2. Have the button listen for a click and call the calcBill function when it is clicked:

```
calcBtn.addEventListener('click', calcBill);
```

3. Define the function. First thing it does is prevent the form from doing a default page reload:

```
function calcBill(event) {
    event.preventDefault();
}
```

4. Next, get the values from the form.

- Use the Number() method to convert the "number-like strings" to real numbers:

```
function calcBill(event) {
    event.preventDefault();

    let foodTot = Number(food.value);
    let bevTot = Number(bev.value);
    let tipPct = Number(tipMenu.value);
```

```
    let taxPct = Number(taxMenu.value);
    // calc a sub total by adding food and bev
    let subTot = foodTot + bevTot;
}
```

5. Calculate the tip and tax from their percents. To do this, multiply `subTot` by the tip and tax rates, respectively. Divide by 100, because 15% has a numeric value of 0.15.

```
function calcBill(event) {
    event.preventDefault();

    let foodTot = Number(food.value);
    let bevTot = Number(bev.value);
    let tipPct = Number(tipMenu.value);
    let taxPct = Number(taxMenu.value);

    // get a sub total by adding food and bev
    let subTot = foodTot + bevTot;

    // calc the tax and tip from their pcts
    let taxTot = subTot * taxPct / 100;
    let tipTot = subTot * tipPct / 100;
}
```

6. Add the tax and tip to the subtotal and round off `tipTot`, `taxTot` and `total` to two decimal places with `toFixed(2)`;

```
function calcBill(event) {
    event.preventDefault();

    let foodTot = Number(food.value);
    let bevTot = Number(bev.value);
    let tipPct = Number(tipMenu.value);
    let taxPct = Number(taxMenu.value);

    // get a sub total by adding food and bev
    let subTot = foodTot + bevTot;

    // calc the tax and tip from their pcts
    let taxTot = subTot * taxPct / 100;
    let tipTot = subTot * tipPct / 100;

    // add up the total:
    let total = subTot + taxTot + tipTot;

    // round off tip and total to 2 decimal places
    // do this AFTER all math is done, because toFixed()
    // converts the number to a string
    tipTot = tipTot.toFixed(2); // becomes a string
```

```
        taxTot = taxTot.toFixed(2); // becomes a string
        total = total.toFixed(2);
    }
```

7. Concatenate a bill message and output it. The output includes `<br>` tags, so use `innerHTML` instead of `textContent`:

```
        // round off tip and total to 2 decimal places
        // do this AFTER all math is done, because toFixed()
        // converts the number to a string
        tipTot = tipTot.toFixed(2); // becomes a string
        taxTot = taxTot.toFixed(2); // becomes a string
        total = total.toFixed(2);

        // concat bill
        let bill = `
        ***JS BISTRO***
        <br>***GUEST CHECK***
        <br>Food: $${foodTot}
        <br>Beverage: $${bevTot}
        <br>Sub-Total: $${subTot}
        <br>Tax %: ${taxPct}
        <br>Tax: $${taxTot}
        <br>Tip Pct %: ${tipPct}
        <br>Tip: $${tipTot}
        <br>***PLEASE PAY:***
        <br>TOTAL: $${total}
        <br>***THANK YOU!***`;

        // output the bill:
        response.innerHTML = bill;
    }
```

END Lesson 03.04 NEXT: Lesson 03.05



Form Submission

evaluating checkboxes

Open 03.05-Get-Free-eBook-Form.html in the browser. As we see, the form's html and css are all done, but the form itself does not work. This is because we will be writing the JS for the form in this lesson.

The form works as follows:

- The user enters their first name and email into text fields.
- Optionally, the user chooses from the select menu.
- The user can check or uncheck the checkbox to subscribe to the newsletter.
- The user clicks the 'Sock It To Me' button to submit the form.
- The button click calls a function which:
    - gets the values from the name and email from the text input fields.
    - checks to see if the checkbox is checked.
    - ignores the select menu
    - concatenates and outputs an appropriate message.
    - to make room for the message, the submt ubtton is hidden

1. Get the form input elements and have the button listen for a function:

```
let firstName = document.getElementById('firstName');
let email = document.getElementById('email');
let subscribeCheckbox = document.getElementById('subscribe');
let feedback = document.getElementById('feedback');

let btn = document.querySelector('button');
btn.addEventListener('click', processEbookForm);
```

2. Write the function, starting with the prevent default:

```
function processEbookForm(event) {
    event.preventDefault();
}
```

3. Get the inputted text values and concatenate the first part of the response message. Rather than pass the values to local variables, we can just concatenate directly:

```
    function processEbookForm(event) {
        event.preventDefault();

        let msg = `Thank you ${firstName.value}. Check ${email.value}
for your Free eBook.`;

    }
```

4. Evaluate the checkbox by seeing if its checked property returns true. If it is checked, thank the user for subscribing. This gets added to the existing message with += :

```
    function processEbookForm(event) {
        event.preventDefault();

        let msg = `Thank you ${firstName.value}. Check ${email.value}
for your Free eBook.`;

        if(subscribeCheckbox.checked) {
            msg += ' The latest issue of the newsletter will accompany
the email--thanks for subscribing!';
        }
    }
```

5. Hide the btn to make room for the output, and output the message:

```
     function processEbookForm(event) {
         event.preventDefault();

         let msg = `Thank you ${firstName.value}. Check ${email.value}
for your Free eBook.`;

         if(subscribeCheckbox.checked) {
             msg += ' The latest issue of the newsletter will
accompany the email--thanks for subscribing!';
         }

         btn.style.display = 'none';
         feedback.textContent = msg;
     }
```

END Lesson 03.05 NEXT: Lesson 03.06



Hoisting

Function Expressions

Anonymous Functions

hoisting

Hoisting refers to functions being lifted to the top of their scope, so that they are available everywhere in their scope. Since they are hoisted, functions can be called before their declaration appears in the code.

1. Declare a function and call it before and after:

```
console.log(1, sayHello('Joe')); // Hello, Joe!

function sayHello(name) {
    return **Hello, ${name}!**;
}

console.log(2, sayHello('Joe')); // Hello, Joe!
```

Hoisting does not work for variables, however. To access a variable, it must have already been declared:

2. Try to use a variable before it is declared. You get an error:

```
console.log(`Do you like ${froot}?`);
// ReferenceError: froot is not defined
let froot = 'apples';
console.log(`Do you like ${froot}?`);
// Do you like apples?
```

function experession

A function expression is a variable with a function for a value. As a variable, it doesn't get hoisted. A function expression must be defined *before* it can be called.

3. Write a function expression:

```
const welcomeUser = function(user) {
    return `Welcome ${user}!`;
}
```

4. Call the function above itself. We get an error:

```
console.log(welcomeUser('Jane1'));
// ReferenceError: welcomeUser is not defined

const welcomeUser = function(user) {
    return `*Welcome ${user}!`;
}

console.log(welcomeUser('Jane1'));
```

5. Comment out the error line and call the function down below. It works:

```
// console.log(welcomeUser('Jane123'));
// ReferenceError: welcomeUser is not defined

const welcomeUser = function(user) {
    return **Welcome ${user}**;
}

console.log(welcomeUser('Janet456'));
// Welcome Janet456
```

anonymous functions

An anonymous function is a function that does not have a name. Anytime you see the word function followed by parentheses with no name in between, it's an anonymous function:

```
// named function:
function addNumbers(x, y) {
    return x y;
}

// anonymous function:
function(x, y) {
    return x, y;
}
```

The reason functions have names at all is so they can be called. It stands to reason then that anonymous functions do not need to be called. But if that's true, how do they run?

function set equal to an event property

An anonymous function may be set equal to the event property of an object. When that event happens, the function runs. First let's just look at a function that is called by a listener:

6. Get the BTN 1 button and have it call a function when clicked:

```
    const btn1 = document.getElementById('btn-1');
    btn1.addEventListener('click', doSomething);
```

7. Define the function:

```
    function doSomething() {
        output.textContent = 'You clicked the button! That did
something!';
    }
```

8. Get the element where the function output goes:

```
    const output = document.getElementById('output');
```

9. Reload the page and click the button. We should get the message: *You clicked the button! That did something!*

This review of addEventListener is just to set up the contrast between calling function with a name and running an anonymous function on click.

10. Get BTN 2:

```
    const btn2 = document.getElementById('btn-2');
```

onclick property

Events that can be called on objects co-exist as properties. The 'click' event has its counterpart in the **onclick** property, which is called on the button and set equal to a function. When the click takes place, the function runs:

11. Access the button's onclick property and set it equal to an anonymous function:

```
    btn2.onclick = function() {
        output.textContent = 'Hello from the anonymous function!';
    }
```

For that matter, an anonymous function can be used in a listener, rather than callng a function.

inline anonymous functions

Anonymous functions do not have to come to the right of an equal sign. An anonymous function can be written inline, such as inside the addEventListener method.

13. Get BTN 3 and have it listen for a click. Instead of calling a function by name, as we usually do, just run an inline anonymous function, right there on the spot:

```
    const btn3 = document.getElementById('btn-3');

    btn3.addEventListener('click', function() {
        output.textContent = 'Hello from inline anonymous function!';
    })
```

Let's try one more example of a listener that calls a function by name vs. a listener that runs an anonymous function, inline:

14. Have the body listen for the 'load' event, which fires when the body, is fully loaded, and to call a function when the loading of the page is complete:

```
    document.body.addEventListener('load', onBodyLoaded);

    function onBodyLoaded() {
        output4.textContent = 'BODY LOADED says onBodyLoaded function';
    }
```

15. Refactor the listener to run an inline anonymous function instead of calling a function by name:

```
    document.body.addEventListener('load', function() {
        output5.textContent = 'BODY LOADED says inline anonymous
function';
    });
```

16. Do away with the listener altogether, and instead use the onload event property to run the anonymous function:

```
    document.body.onload = function() {
        output6.textContent = 'BODY LOADED says document.body.onload =
anonymous function';
    };
```

keyboard object and events

The pressing of any key is also an event. These are typically also used in conjunction with anonymous functions.

The **document** can listen for a keyboard event and call a function when any key is pressed (**keydown**) or released (**keyup**).

The event properties for the keyboard are **event.key** which is the key that was pressed and **event.keyCode** which is the key's numeric code. The function takes **event** as its argument.

17. Have the document listen for a **keydown** event and call a function when any key is pressed. The function reports the **key** and its **keyCode**.

```
    document.addEventListener('keydown', reportKeyAndCode);

    function reportKeyAndCode(event) {
        output4.textContent = `listener keydown calls reportKeyAndCode
function:\nKey: ${event.key} Code: ${event.keyCode}`;
    }
```

18. Refactor the above to use an inline anonymous function:

```
    document.addEventListener('keydown', function(event) {
        output5.textContent = `listener keydown runs anonymous inline
function:\nKey: ${event.key} Code: ${event.keyCode}`;
    });
```

19. And finally, use the onkeydown property to call an anonymous function:

```
    document.onkeyup = function(event) {
        output6.textContent = `onkeyup = anonymous function \nKey:
${event.key} Code: ${event.keyCode}`;
    };
```

- **END: Lesson 03.06**
- **NEXT: Lesson 03.07**



Apartment Rent Estimator

Apartment Rent Estimator Form

In this **form**, we present the user with two select menus and four "amenities checkboxes" from which they configure their desired apartment. Each choice has a price: the more bedrooms, bathrooms and amenities chosen, the higher the rent. The choices are:

- Number of Bedrooms, chosen from a select menu.
- Number of Baths, also chosen from a select menu.
- Doorman Building checkbox
- Parking Space checkbox
- Fitness Center checkbox
- Skyline View checkbox

The html and css are already done, but of course the form doesn't work yet. We will write the JS that powers the form.

The select menus are for choosing numbers of Bedrooms and Baths. The greater the choice, the higher the value (rent).

- The 1 Bath choice has a value of 0, so choosing that will not raise the rent.
- One bedroom is pre-selected by default, even though it is not the first menu choice:

```
<select name="bdrms" id="bdrms">
  <option value="1000">Studio ($1,000)</option>
  <option value="1400" selected>1 Bedroom ($1,400)</option>
  <option value="1800">2 Bedrooms ($1,800)</option>
  <option value="2300">3 Bedrooms ($2,300)</option>
</select>

<select name="baths" id="baths">
  <option value="0">1 Bath</option>
  <option value="450">1.5 Baths (+$450)</option>
  <option value="850">2 Baths (+$850)</option>
  <option value="1250">2.5 Baths (+$1,250)</option>
</select>
```

checkboxes

The fees associated with the checkboxes will only be charged if the checkbox is checked. We will first assess the flat fees and add those to the rent. Then we will calculate the percent surchargess on top of the

rent.

- Parking value = 350, for the $350 parking fee
- Fitness Center (gym) value = 100, for the $100 gym fee
- Skyline View value = 0.25, for the 25% surcharge for a skyline view.
- The Doorman value = 0.1, for the 10% surcharge for a doorman building.

```
<input type="checkbox" id="doorman" value="0.1">
<label for="doorman">Doorman Building (+10%)</label><br>

<input type="checkbox" id="parking" value="350">
<label for="parking">Indoor Parking (+$350)</label><br>

<input type="checkbox" id="riverview" value=".25">
<label for="riverview">Overlooking River (+25%)</label><br>

<input type="checkbox" id="gym" value="350"'>
<label for='gym'>Fitness Center (+$100)</label>
```

The html also includes a `calculate rent()` button and an `h3` for posting the answer.

```
<button>calculate rent</button>
<h3></h3>
```

1. Get the button and have it call the function when clicked:

```
const calcBtn = document.querySelector('button');
calcBtn.addEventListener('click', calculateRent);
```

2. Get the "feedback" tag:

```
const feedback = document.getElementById('feedback');
```

3. Declare the function. Start with the preventDefault() method, so that the button click does not reload the page and reset the form:

```
function calculateRent(event) {
  event.preventDefault();
}
```

4. Get the values of the select menus. We don't need the whole objects--just their values:

```
function calculateRent(event) {
  event.preventDefault();

  let bdrms = document.getElementById('bdrms').value;
  let baths = document.getElementById('baths').value;
}
```

5. The `bdrms` and `baths` values come in as strings, so convert them to numbers. Then add them together, saving them to a var called `rent`:

```
  bdrms = Number(bdrms);
  baths = Number(baths);
  let rent = bdrms + baths;
```

6. Get the checkboxes:

```
  let doorman = document.getElementById('doorman'); // +10%
  let parking = document.getElementById('parking'); // +$350
  let view = document.getElementById('view'); // +25%
  let gym = document.getElementById('gym'); // +$100
```

See if each checkbox has its `checked` property set to true. If it returns true, increase the rent accordingly:

- Charge the flat fees first (parking, gym)
- For percent surcharges (doorman and view), multiply the value by the rent and then add that fee to the rent.
- Round off the final rent and output it to the feedback tag.

7. Write if-statements to see if the checkboxex are checked, and if so, increase the rent, accordingly:

```
  if(parking.checked) {
    rent += Number(parking.value);
  }

  if(gym.checked) {
    rent += Number(gym.value);
  }

  if(doorman.checked) {
    rent += rent * Number(doorman.value);
  }

  if(view.checked) {
    rent += rent * Number(view.value);
  }
```

8. Round off rent to an integer, and output the estimated rent:

```
    rent = Math.round(rent);

    feedback.textContent = 'Your estimated rent is: $' + rent;

  } // calculateRent(event)
```

9. Run the page in the browser.

- Choose the most expensive configuration: 3 bedrooms, 2.5 baths, and check all four boxes.
- Follow that by choosing the cheapest configuration (no checkboxes).
- Try various other configurations.

END: Lesson 03.07

NeXT: Lesson 03.08



keydown and keyup events

event.key and event.keyCode properties

set CSS

random hex color

keyboard events

The pressing of a key is a **keydown** event The release of a key is a **keyup** event

Like any event, a keyboard event can call a function. The event target (thing that fires the event) is the document, so the syntax is: **document.addEventListener('keydown', function)** An if-statement can check the **event.keyCode** to know which key was pressed.

The key **event** has **key** and **keyCode** properties:

The q **event.key** is **q**. The q **event.keyCode** is **81**.

The left arrow **event.key** is **ArrowLeft**. The left arrow **event.keyCode** is **37**.

In this activity, we will set up several keys to do different things, most of which involve setting CSS:

random body background color on 'c' keydown

When the **c** or **Enter** key is pressed, our function will set the body background to a random color.

toggle dark mode on 'd' key press

When the **d** key is pressed, our function will toggle the container between dark mode and light mode.

toggle font on 'f' key press

When the **f** key is pressed, our function will toggle the font between *serif* and *sans-serif*.

generate random 4-digit PIN on 'p' keydown

When the **p** or **n** key is pressed, our function will generate and output a random 4-digit PIN number.

move space ship to the left and right

When the left or right arrow is pressed, our function will move the space ship 10 pixels in that direction. The numeric codes for the left and right arrows are **37** and **39**, respectively.

There are four DOM elements we need:

- **container** div that holds instructions and some output divs:
- **rand-pin-box** div for outputting the **key** and **keyCode** of the pressed key
- **key-box** div for outputting hold a random number
- an image of a space ship

1. Get the DOM elments:

```
const container = document.querySelector('.container');
const keyBox = document.getElementById('key-box');
const spaceShip = document.getElementById('space-ship');
const randPinBox = document.getElementById('rand-pin-box');
```

2. Set the left position of the space ship to be approximately in the middle of the screen. Also set a var for the speed of the ship:

```
let leftPos = window.innerWidth / 2 - 150;
spaceShip.style.left = leftPos + 'px';
let shipSpeed = 20;
```

3. Set booleans to keep track of font (serif or not) and dark mode:

```
let serif = true;
let dark = false;
```

4. Have the document listen for **keyup** and call an inline anonymous function which outputs the **key** and its **keyCode**:

```
document.addEventListener('keyup', function() {
    keyBox.textContent = **Key: ${event.key}
    Code: ${event.keyCode}**;
});
```

5. Have document listen for **keydown** event and call the **respondToKeyPress**. The function checks if the key is **n**, **p**, **d**, **l**, **c**, **Enter** or the left or right arrow. If one of these keys was pressed, the specified action takes place.

```
document.addEventListener('keydown', respondToKeyPress);
```

6. Define the function **respondToKeyPress**, which runs on **keydown** (press). All functions take **event** object as its default argument, but pass in **event** explicitly, so that we can access its properties:

```
function respondToKeyPress(event) {
}
```

7. Inside the function, set up a scaffoling of a big if-else if block to check for each key: c, d, f, p, Enter (13), Left (37) and Right (39) arrows:

```
function respondToKeyPress(event) {

    if(event.key === 'c' || event.keyCode === 13) {

    } else if(event.key === 'd' ) {

    } else if(event.key === 'f') {

    } else if(event.key === 'p' || event.key === 'n') {

    } else if(event.keyCode === 37) { // ArrowLeft

    } else if(event.keyCode === 39) { // ArrowRight

    }

}
```

Next, we will use conditinal logic to handle what to do (if anything) when a key is pressed:

8. If the **c** key is pressed, call the **makeRandHexColor()** function that generates and returns a random hexadecimal color. Save the color to a variable, and use that to set the background color:

```
function respondToKeyPress(event) {

    if(event.key === 'c'  || event.keyCode === 13) {
        let randHex = makeRandHexColor();
        document.body.style.backgroundColor = randHex;
    }

} // end function
```

9. If the **d** key is pressed, toggle between dark and light mode by adding-removing their styles. Toggle the boolean each time, so that the behavior also toggles.

```
    } else if (event.key === 'd') {
        if(!dark) {
            container.classList.remove('light-mode');
            container.classList.add('dark-mode');
            dark = true;
```

```
            } else {
                container.classList.remove('dark-mode');
                container.classList.add('light-mode');
                dark = false;
            }
        }

    } // end function
```

10. If the **f** key is pressed, toggle between serif and sans-serif font. Toggle the boolean each time, so that the behavior also toggles.

```
        } else if (event.key === 'f') {
            if(serif) {
                document.body.style.fontFamily = "sans-serif";
                serif = false;
            } else {
                document.body.style.fontFamily = "serif";
                serif = true;
            }
        }

    } // end function
```

11. If the **p** or **n** key is pressed, generate a random 4-digit number from 0-9999. If the number is less than 4 digits, add leading zero(es) by means of if-statements with ranges; utput the random pin:

```
        } else if (event.key === 'p' || event.key === 'n') {

            let r = Math.floor(Math.random() * 10000);

            // add leading zero(es), as needed
            if(r < 1000 && r > 99) r = '0' + r; // 3 digits
            if(r < 100 && r > 9) r = '00' + r; // 2 digits
            if(r < 10) r = '000' + r; // 1 digit
            if(r === 0) r = '0000';

            randPinBox.textContent = 'PIN:\n' + r;
        }

    } // end function
```

12. If the **ArrowLeft (37)** key is pressed, and if the space ship is not all the way to the left, move the ship **shipSpeed** pixels to the left:

```
        } else if (event.keyCode === 37) {
            if(leftPos > 0) {
```

```
                leftPos -= shipSpeed;
                spaceShip.style.left = leftPos + 'px';
            }
        }

    } // end function
```

13. If the **ArrowRight (39)** key is pressed, and if the space ship is not all the way to the right, move the ship **shipSpeed** pixels to the right:

```
        } else if (event.keyCode === 39) {
            if(leftPos < window.innerWidth - 300) {
                leftPos += shipSpeed;
                spaceShip.style.left = leftPos + 'px';
            }
        }

    } // end function
```

14. If the spaceship doesn't move, try switching from event.keyCode to event.key:

```
        } else if (event.key === 'ArrowLeft') {
            if(leftPos > 0) {
                leftPos -= shipSpeed;
                spaceShip.style.left = leftPos + 'px';
        } else if (event.key === 'ArrowRight') {
            if(leftPos < window.innerWidth - 300) {
                leftPos += shipSpeed;
                spaceShip.style.left = leftPos + 'px';
            }
        }

    } // end function
```

Random Hex Color

It is possible to generate a random hexadecimal color. Here's the concept and procedure:

- A base-10 number consists of the digits 0-9
- A base-16 string uses the characters 0-9, A-F
- A hexadecimal color is a 6-character, base-16 string
- The **toString()** method converts a number to a string
- **toString(16)** converts a base-10 number to a base-16 string
- There exist 16,777216 (256 ** 3) colors in the RGB spectrum
- Calling **toString(16)** on an integer in the RGB range from 0-16777215 returns a hexadecimal color value
- Putting '#' before the hex value comletes the hex color

This function makes and returns a random hex color. It is called from within the **respondToKeyPress** function when the **c** key is pressed.

```
    function makeRandHexColor() {

        // A. Generate a random 16-digit float from 0-1
        let rando = Math.random();

        // B. Multiply the random float by 16,777,216 (RGB range)
        rando = rando * 16777216;

        // C. round down the random number:
        rando = Math.floor(rando);

        // D. convert the random number to a base-16 hexadecimal string
        rando = rando.toString(16);

        // E. add the # to complete the hexadecimal color
        rando = '#' + hex;

        // F. return the hexadecimal color value
        return hex;

    }
```

END: Lesson 03.08

NEXT: Lesson 03.09



Guess Mystery Number

Random Numbers

if-else if-else logic

show-hide buttons

buttons calling functions

Open 03.09-Number-Guessing-Game.html

In this lesson we will write a number guessing program:

- user clicks PLAY button
- button click calls function playGame
- function generates a random integer from 1-100
- input box and GUESS button replace PLAY button
- player enters number from 1-100
- player clicks GUESS button
- button calls function evalGuess
- function gets value from input box and converts it to a real number
- compares user guess number to the random number
- h2 tag of id "feedback" displays all messages
- when user gets number right, a PLAY AGAIN button appears.

First we need a few global variables. They need to be global, because they must be available to more than one function:

Declare randNum to hold the random number; playerGuess to store the player's number; and guessesLeft to count down from 7.

```
let randNum = 0;
let playerGuess = 0;
let guessesLeft = 7;
```

Next, get the DOM elements used for the game. There are two buttons, but the GUESS button is hidden on page load, so all we see is the PLAY button.

The input box for entering the guess number is also hidden on page load. Both hidden elements will appear when the user clicks PLAY button:

```
let playBtn = document.getElementById('play-btn');

// hide on page load, show on PLAY click
let guessBtn = document.getElementById('guess-btn');
```

Have each button listen for its respective function to call when clicked.

```
playBtn.addEventListener('click', playGame);

guessBtn.addEventListener('click', evalGuess);
```

Also get the "guess box" (number input box), which is also hidden on page load. The hidden states are handled in the CSS, with the display:none property.

```
// hide on page load, show on PLAY click
let guessBox = document.querySelector('input');
```

Get the h2 that displays the feedback:

```
let feedback = document.getElementById('feedback');
```

Now for the functions. Declare the playGame function, which runs when the player clicks the PLAY button. The function does the following:

- generates the random number from 1-100
- hides "this", which is itself, the PLAY button
- shows (display = "block") the guess box (number input field) and the GUESS button.
- sets the input box to an initial value of 0, so the user knows to click there and enter a number.
- gives the user some feedback: 'Guess the number!'

```
function playGame() {
    randNum = Math.ceil(Math.random() * 100);
    this.style.display = 'none';
    guessBox.style.display = 'block';
    guessBtn.style.display = 'block';
    guessBox.value = 0;
    feedback.textContent = 'Guess the mystery number from 1-100!';
}
```

The evalGuess function runs when the user clicks the GUESS button. This function gets the value from the input box and converts it to a real number using the `Number()` method.

```
    function evalGuess() {
        playerGuess = Number(guessBox.value);
    }
```

Now for the big if-else if-else block:

- if the guess is too low, user gets feedback (too LOW) and guesses again
- else if the guess is too high, user gets feedback (too HIGH) and guesses again
- else guess matches mystery number, so user is congratulated
- GUESS button is replaced by PLAY AGAIN button
- user gets 7 guesses, which count down and are displayed with each guess
- when the user gets it right, the resetGame() function is called
- there is no enforcement of guess limit. Game just keeps going after 7 guesses.

```
    function evalGuess() {
        playerGuess = Number(guessBox.value);

        if (playerGuess < randNum) {
            guessesLeft--;
            feedback.innerHTML = '<p>Your guess is too low! <br>You have '
+ guessesLeft + ' guesses left!</p>';
        } else if (playerGuess > randNum) {
            guessesLeft--;
            feedback.innerHTML = '<p>Your guess is too high! <br>You have
' + guessesLeft + ' guesses left!</p>';
        } else {
            guessesLeft--;
            feedback.innerHTML = '<p>Congrats!<br>The mystery number is
indeed ' + randNum + '! <br>You got it in ' + (7 - guessesLeft) + '
guesses!</p>';
            resetGame();
        }
    }
```

The resetGame function is called when the user guesses correctly. The function restores all the settings as they are on page load. The only difference is that the PLAY button now says PLAY AGAIN (it is the same button, with different text).

```

```

```
    function resetGame() {
        guessesLeft = 7;
        guessBtn.style.display = 'none';
        guessBox.style.display = 'none';
        playBtn.style.display = 'block';
        playBtn.textContent = 'PLAY AGAIN';
        guessBox.value = 0;
    }
```

END: Lesson 03.09

NEXT: Lesson 04.01



arrays

const

index, array.length

methods (push(), sort(), pop(),)

nested / 2D / matrix arrays

arrays

Arrays are variables that can hold more than one value at a time. Arrays have a datatype of **object**. Here are some key things to know about arrays:

- array items exist as a list, surrounded by square brackets: `[item1, item2, item3]`
- each item is assigned a position number, or **index**, with the first item at index 0
- array items can be of any data type
- items of different data types can be in the same array: `[ 'hola', 38, true, ['apple, 'banana'] ]`

using const to declare an array

As detailed in a previous lesson, primitive variables (strings, numbers, booleans) declared with **const** cannot be modified (changed) in any way.

1. As a recap, declare a constant with `const`, and try to change it. Cannot be done:

```
const LITERS_PER_GALLON = 3.78;
LITERS_PER_GALLON = 4; // ERROR: Assigment to constant variable
```

const for objects

However, arrays declared with `const` *can* be modified. Items can be changed, added or deleted. But the array object itself cannot be *mutated* to a different datatype. So, with *const*, once an array, always an array.

how to declare an array

To declare an array, start with the `let` or `const` keyword, followed by a name. We will start with **let** in order to later prove our point about mutability. The array is wrapped in square brackets, with items separated by a comma.

It is a good practice to pluralize array names so: *fruits* not *fruit*. You can also add *Arr* to make it crystal clear that this is an array: *fruitsArr*.

2. Declare an array called `fruitsArr`, with three items:

```
let fruitsArr = ['apple', 'banana', 'cherry'];
console.log(fruitsArr);
```

3. Check the Console. Expand the arrow to see the numbered items, with 'apple' at index 0.

index

Array items are stored in a numeric order, called **index**.

- The first item in an array is at index 0, so if there are 3 items in an array, the last item is at index 2.
- Use square bracket syntax to access items: `array[0]`

4. Get the first item at index 0 and the last item at index 2:

```
console.log(fruitsArr[0]); // apple
console.log(fruitsArr[2]); // cherry
```

setting array values by index

To set the value of an array item, refer to it by index:

5. Replace the first item, 'apple', with 'peach':

```
fruitsArr[0] = 'peach';
console.log(fruitsArr);
// ['peach', 'banana', 'cherry']
```

One way to add an item at the end of the array is to assign a value after the last item:

6. Add 'mango' to the end of the array, at index 3:

```
fruitsArr[3] = 'mango';
console.log(fruitsArr);
// ['peach', 'banana', 'cherry', 'mango']
```

mutating an array (changing its datatype

One downside of `let` for arrays is that it does not protect the data type. You could inadvertently change the array to a string or any other datatype. Use `const` to prevent such an accident from occurring.

7. Change *fruitsArr* to a string. Just like that, no more array:

```
fruitsArr = "fresh";
console.log(fruitsArr); // fresh
```

8. Declare another fruits array, this time with `const`. We need a new name, as `const` cannot redeclare an existing variable:

```
const fruits = ['apricot', 'pear', 'kiwi', 'grape'];
```

9. Try to mutate this `const` array into a string. You get an error:

```
const fruits = "ripe";
// ERROR: Assignment to constant variable
```

array.length property

The `length` property of an array returns the number of items in the array.

10. Get the length of the array:

```
console.log(fruits.length); // 4
```

array[arrray.length-1]

- The first item in an array is at: **array[0]**.
- The last item is at: **array[array.length-1]**.

11. Use `length-1` to access the last item in the array:

```
let lastFruit = fruits[fruits.length - 1];
console.log(lastFruit); // grape
```

random array items

To get a random item from an array, generate a random integer within the range of the array length, and pass that number to the array square brackets:

12. Get a random fruit from the *fruits* array.

```
let randInt = Math.floor(Math.random() * fruits.length);
let randFru = fruits[randInt];
```

```
console.log(randFru);
```

13. Run it a few times to see that the fruit keeps changing.

array methods

The array object has numerous methods, some of which we will learn about now:

array.push(item)

We used `fruits[3]` to add to the end of the array. But what if the length of the array is unknown? Better is to use the `push()` method, which adds an item to the end without needing to know how many items are in the array.

14. Using the **push()** method, add 'lime' to the end of the array.

```
fruits.push('lime');
console.log(fruits);
// ['apricot', 'pear', 'kiwi', 'grape', 'lime']
```

declaring a new empty array

15. Use the **sort()** method to put the string items of an array in alphabetical order:

```
fruits.sort();
console.log(fruits);
// ['apricot', 'grape', 'kiwi', 'lime', 'pear']
```

array.pop()

The **pop()** method removes the last item from the array and returns it, so the item can be saved by setting `pop()` equal to a variable.

16. Using the **pop()** method, remove the last item and save it to a variable, `popped`:

```
let popped = fruits.pop();
console.log(popped); // pear
console.log(fruits);
// ['apricot', 'grape', 'kiwi', 'lime']
```

An array can be declared with just a pair of empty curly braces--no items.

17. Declare a new, empty array as a pair of square brackets:

```
const veggies = [];
```

18. Use the push() method to populate the array:

```
veggies.push('carrot');
veggies.push('celery');
console.log(veggies);
// ['carrot', 'celery']
```

2D maxtrix arrays

An array can have arrays for its items. The terms to describe such an array include: **matrix, 2D array** and **nested array**

19. Make a 3x3 matrix of three items, each an array of three items.

```
const nestedArr = [[1,2,3], [4,5,6], [7,8,9]];
console.log(nestedArr.length);
console.log(nestedArr);
console.log(nestedArr[0]); // [1,2,3]
```

20. Use double square brackets: the first to get the inner array, the second to get the inner array item:

```
console.log(nestedArr[0][0]); // 1
console.log(nestedArr[1][1]); // 5
console.log(nestedArr[2][2]); // 9
```

21. Represent a tic-tac-toe board, where all 9 squares start out with a value of null:

```
let ticTacToe = [
    [null, null, null],
    [null, null, null],
    [null, null, null]
];
```

22. Then the game starts; "X" chooses the middle square, and "O" chooses the lower left square:

```
ticTacToe[1][1] = "X";
ticTacToe[2][0] = "O";
```

23. Log the result: it's a tic-tac-toe game in progress:

```
console.log(ticTacToe);
/*
(3) [Array(3), Array(3), Array(3)]
0: (3) [null, null, null]
1: (3) [null, 'X', null]
2: (3) ['O', null, null]
*/
```

END: Lesson 04.01

NEXT: 04.01 Lab Lesson 04.02



1. Declare an array called gems and give it three items: 'diamond', 'sapphire' and 'ruby'.
2. Log the entire array and also the first item.
3. Add 'emerald' to the end of the array, using the index of the new item
4. Use an array method to add 'amethyst' to the end of the array.
5. Use an array method to sort the array.
6. Log the length (number of items of the array). Should be 5.
7. Use length-1 to get the last item in the array. Save it to a variable and log it.
8. Use an array method to remove the last item from the array. Save it to a new variable. HINT: the popped item should be 'sapphire'.
9. Given these 12 months: Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec. Make a 2D array called months, containing the 12 items, divided into 4 sub-arrays or 3 items each, so: Jan, Feb, Mar would be the first array. Log the months that start with 'J' and 'M'.
10. Make four arrays called winter, spring, summer and autumn, each of which contains three items, assigned as one item of the months array. Log random month from each array.

SEE 04.01 Lab Solution

04.01 Lab Solutions

1. Declare an array called gems and give it three items: 'diamond', 'sapphire' and 'ruby'.

```
const gems = ['diamond', 'sapphire', 'ruby'];
```

2. Log the entire array and also the first item.

```
console.log(gems);
// ['diamond', 'sapphire', 'ruby']
console.log(gems[0]); // diamond
```

3. Add 'emerald' to the end of the array, using the index of the new item.

```
gems[3] = 'emerald';
console.log(gems);
// ['diamond', 'sapphire', 'ruby', 'emerald']
```

4. Use an array method to add 'amethyst' to the end of the array.

```
gems.push('amethyst');
console.log(gems);
// ['diamond', 'sapphire', 'ruby', 'emerald', 'amethyst']
```

5. Use an array method to sort the array. gems.sort();

```
console.log(gems);
// ['amethyst', 'diamond', 'emerald', 'ruby', 'sapphire']
```

6. Log the length (number of items of the array). Should be 5.

```
console.log(gems.length); // 5
```

7. Use length-1 to get the last item in the array. Save it to a variable and log it.

```
let lastGem = gems[gems.length-1];
console.log(lastGem); // sapphire
```

8. Use an array method to remove the last item from the array. Save it to a new variable. HINT: the popped item should be 'sapphire'.

```
let poppedGem = gems.pop();
console.log(poppedGem); // sapphire
console.log(gems);
// ['amethyst', 'diamond', 'emerald', 'ruby']
console.log(gems.length); // 4
```

9. Given these 12 months: Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec. Make a 2D array called months, containing the 12 items, divided into 4 sub-arrays or 3 items each, so: Jan, Feb, Mar would be the first array. Log the months that start with 'J' and 'M'.

```
const months = [ ['Jan', 'Feb', 'Mar'],
                 ['Apr', 'May', 'Jun'],
                 ['Jul', 'Aug', 'Sep'],
                 ['Oct', 'Nov', 'Dec']
             ];
console.log(months[0][0]); // Jan
console.log(months[0][2]); // Mar
console.log(months[1][1]); // May
console.log(months[1][2]); // Jun
console.log(months[2][0]); // Jun
```

10. Make four arrays called winter, spring, summer and autumn, each of which contains three items, assigned as one item of the months array. Log random month from each array.

```
const winter = months[0];
let randWinterMo = Math.floor(Math.random() * winter.length);
console.log(randWinterMo);

const spring = months[1];
let randSpringMo = Math.floor(Math.random() * spring.length);
console.log(randSpringMo);

const summer = months[2];
let randSummerMo = Math.floor(Math.random() * summer.length);
console.log(randSummerMo);

const autumn = months[3];
let randAutumnMo = Math.floor(Math.random() * autumn.length);
console.log(randAutumnMo);
```

NEXT: Lesson 04.02



Objects

object

An object is a variable that stores a collection of properties bundled in curly braces {}.

properties

- Properties are variables belonging to an object; as such, they have a name and a value.
- Properties are child variables of the parent object variable.

key-value pairs

- Properties are name-value pairs, called **key-value** pairs.
- A property name is called a **key**. Unlike "regular" variable names, keys can have spaces.
- The value can be of any data type: string, number, boolean--even object and function.

method

- An object property whose value is a function is called a **method**.

const for objects

As with arrays, we will usually declare with `const`. This gives us the freedom to modify the properties at will, while keeping us in our swim lane, so to speak, by preventing us from changing the data type.

difference between object and array

Arrays have a data type of object, but array items have no name-value pairs, but are rather numbered by position (index), and are bundled in square brackets.

DOM objects

When a web page (DOM) element is brought into JS, it exists in the script as an **object**. The properties of the object are the attributes of the elements. We have already been working with the DOM by getting elements and setting their properties.

1. Open **04.02-Objects.html** and preview it in the browser. This page is just a springboard for checking our console.log output. Remember to switch the file being used from FINAL .js to START .js.

2. In the JS file, declare a new object variable, and assign it some properties in between its curly braces. The key-values are separated by colons. Each property ends with a comma to separate it from the next property. Optionally, a so-called **trailing comma** may come after the last property.

```
const car = {
    make: 'Ford',
    model: 'Mustang GT',
    year: 1999,
    color: 'red',
    condition: 'excellent',
    miles: 123456,
    onRoad: true,
    forSale: false
};
```

Properties of an object are available only to the object, and are referenced by dot-sytnax: `object.property`.

3. Log the whole object, as well as a few properties. In the console, open the arrow to see the properties. Notice that, in contrast to array items, properties are not numbered:

```
console.log(car);
console.log(car.year, car.make, car.model);
```

Properties are added to an existing object by dot syntax: `object.property = value`. Adding a property amounts to declaring a variable *scoped* to the object.

nested objects: objects as object properties

Object properties can be other objects, including arrays.

4. Add two more properties to the car object, one an array, the other a child object: and then access them using "dot.dot" and square bracket syntax

```
car.mpg = {city: 18, hwy: 25};
car.options = ['sun roof', 'CD player', 'leather seats'];

console.log(car);
```

5. Access the new properties, using "dot.dot" for the child object and square brackets for array items:

```
console.log('MPG City:', car.mpg.city);
console.log('Interior:', car.options[1]);
```

Set (change) a property value with dot-syntax in the same way you would add a property:

6. Update some of the property values of the car object.

```
car.miles = 135790;
car.condition = "very good";
car.mpg.city = 17;
car.mpg.hwy = 24;
car.options[0] = "moon roof";
car.forSale = true;

console.log(car);
```

consolidating related properties into child objects

7. Add three properties having to do with the engine:

```
car.cylinders = 8;
car.liters = 4.6;
car.horsepower = 260;
```

Since `cyl` (cylinders), `hp` (horsepower) and `mpg` (miles per gallon) all have to do with the engine, we can bundle these into an `engine` property.

8. Retool the object by "bundling" `cyl`, `hp` and `mpg` into an `engine` property. This also lets us abbreviate propeterties without obscuring the meaning:

```
car.engine = { cyl: 8, L: 4.5, hp: 270 };
```

9. Log both versions of the duplicate engine properties:

```
console.log(car.horsepower, car.engine.hp); // 260 260
console.log(car.cylinders, car.engine.cyl); // 8 8
console.log(car.liters, car.engine.L); // 4.6 4.6
```

delete keyword

The `delete` keyword is used to remove object properties, as `detete object.propery`.

10. Delete `horsepower`, `cylinders` and `liters` and then log `car` to make sure they've been deleted:

```
delete car.horsepower;
delete car.cylinders;
delete car.liters;

console.log(car);
```

keys can have spaces

While there is generally no upside to having spaces in keys, it is allowed. The key goes in quotes and is accessed with square brackets: `object[key]`.

11. Add two properties with spaces in the keys. Log the `car` object to confirm that they got added:

```
car["consumer reviews"] = 234;
car["star rating"] = 4.7;

console.log(car);
```

toLocaleString()

The `number.toLocaleString()` method is called on a number and returns the number with commas, which converts it to a string.

12. Convert a number with no commas to a "number-like string" with commas:

```
let num1 = 16777216;
let numCommasStr = num1.toLocaleString();
console.log(numCommasStr); // 16,777,216
```

object methods

- When a property value is a function, that property is known as a **method**.
- A method must `return` a value
- A method is called using dot syntax: `object.method()`
- A method can make use of the object's properties, accessing them by referring to the object itself as `this`.

this keyword

The `this` keyword refers to different objects, depending on the context:

- in the global scope, `this` is the Global Window Object
- inside a function, `this` is the object that "owns" the event that calls the function; in the case of a function called by a button click, `this` is the button.
- inside a method, `this` is the object itself.

13. Define a method called `listForSale`. Have it return a "FOR SALE" listing. Refer to various properties using the `this` keyword:

```
car.listForSale = function() {
    return `<strong>FOR SALE!</strong>
    ${this.year} ${this.make} ${this.model},
    only ${car.miles.toLocaleString('en-us')} miles,
    ${this.condition} condition.
```

```
        Loaded with options:<br>${this.options[0]}, ${this.options[1]},
${this.options[2]} and much more.
        MPG: ${this.mpg.hwy} highway, ${this.mpg.city} city.
        Price negotiable.`;
    }
```

13. Call the method in two ways; directly log it and also save it to a variable, and then log the variable:

```
    console.log(car.listForSale());
    let listing = car.listForSale();
    console.log(listing);
```

DOM output of object properties as "Details Page"

In a search application, that page that displays the search results is the "results page". When you click on an individual result, you go to the "details page" where you see more info about that item. Let's make a "details page" for the car, as if it were a search result.

The html and css for this are already done:

```
    <div>
        <img src="images/1999-Ford-Mustang-GT.jpg" id="car-pic">
        <h2 id="car-title">year make model</h2>
        <h3 id="car-mpg">hwy city</h3>
        <h3 id="car-engine">engine L cyl hp</h2>
        <h3 id="car-options">options</h2>
        <hr>
        <h3 id="car-reviews-ratings">reviews ratings</h2>
        <hr>
        <p id="car-listing">listing</p>
    </div>
```

14. Get the elements for displaying the car data:

```
    const carTitle = document.getElementById('car-title');
    const carMPG = document.getElementById('car-mpg');
    const carEngine = document.getElementById('car-engine');
    const carOptions = document.getElementById('car-options');
    const carListing = document.getElementById('car-listing');
    const carReviewsRatings = document.getElementById('car-reviews-
ratings');
    const carPic = document.getElementById('car-pic');
```

There's no button or menu for displaying the car data; it just displays automatically on page load. But still we can have the document call a function when it loads.

15. Have the body `onload` event call an anonymous function when the document is fully loaded:

```
    document.body.onload = function() {
    }
```

16. Output the object properties to the webpage:

```
    document.body.onload = function() {

        carTitle.textContent = `${car.year} ${car.make} ${car.model}`;

        carMPG.textContent = `MPG: ${car.mpg.hwy} hwy - ${car.mpg.city}
city`;

        carEngine.textContent = `Engine: V${car.engine.cyl}
${car.engine.L} L - ${car.engine.hp} horsepower`;

        carOptions.textContent = `Options: ${car.options[0]},
${car.options[1]}, ${car.options[2]}`;

        carReviewsRatings.textContent = `${car['consumer reviews']}
customer reviews - ${car['stars rating']} stars`;

        carListing.innerHTML = car.listForSale();
    }
```

looking up object property by dynamic key

A dynamic key is a variable serving as an object key. Looking up items by dynamic key requires square bracket -- not dot -- syntax.

Next we'll work with an `animals` object with numerous properties, each an individual animal.

17. Open `animals.js` and have a look:

```
    const animals = {

        'American bison': { class: 'mammal', herbivore: true, continent:
'North America' },

        anaconda: { class: 'reptile', herbivore: false, continent: 'South
America' },

        -- ETC. --
    }
```

- It is an object called `animals` with 18 properties, each an individual animal.

- Each `key` is an animal name, with the two-word keys in quotes.
- Each `value` is an object with four properties:
  - class (string)
  - herbivore (boolean)
  - continent (string)
  - legs (number)

We will write a program where the user chooses an animal from the menu to load its info:

- The user chooses an animal from a menu
- The change event calls a function
- The function gets the choice and saves it to a variable, `chosenAnimal`. The choice is the name of an animal (e.g. 'panda', 'lion')
- The function looks up that animal by dynamic key: `animals[chosenAnimal]`
- The animal data is outputted to the page

The html and css for this are already done, with zebra displayed by default. Both JS files: `animals.js` and `04.02-Objects-FINAL.js` are already imported into the html file.

```
<select id="animals-menu">
    <option id="choose">Choose an animal..</option>
    <option id="American bison">American bison</option>
    -- ETC. --
</select>

<div id="animal">
    <img src="images/zebra.jpg" id="animal-pic">
    <div>
        <p id="animal-name">zebra</p>
        <p id="animal-class">class: mammal</p>
        <p id="continent">continent: Africa</p>
        <p id="herbivore">herbivore: yes</p>
        <p>
    </div>
</div>
```

18. Get the elements for displaying the animal data:

```
const animalsMenu = document.getElementById('animals-menu');
const animalName = document.getElementById('animal-name');
const animalClass = document.getElementById('animal-class');
const continent = document.getElementById('continent');
const herbivore = document.getElementById('herbivore');
const animalPic = document.getElementById('animal-pic');
```

19. Have the menu call a function on 'change' (menu choice):

```
animalsMenu.addEventListener('change', displayAnimalInfo);
```

20. Write the function, starting with saving the menu choice to a variable:

```
function displayAnimalInfo() {

    // get the menu choice, which is an animal name
    let chosenAnimal = animalsMenu.value; // e.g. giraffe, panda
}
```

21. Look up the animal in the `animals` object, using the variable as the key. For this dynamic property accessor, use square brackets, not dot-syntax:

```
let animalObj = animals[chosenAnimal];
```

22. Output the animal properties to their respective tags:

```
animalName.textContent = chosenAnimal;
animalClass.textContent = 'Class: ' + animalObj.class;
continent.textContent = 'Continent: ' + animalObj.continent;
herbivore.textContent = 'Herbivore: ' + animalObj.herbivore;
```

23. Set the souce of the animal image. That completes the function:

```
    animalPic.src = `images/${chosenAnimal}.jpg`;

} // end function
```

END: Lesson 04.02 NEXT: 04.02 LAB, Lesson 04.03



Making an Object with Properties

Adding a Method (function) to an Object

Updating / Setting Object Properties

---

1. Make an object variable called product. - In the object variable declaration, include the following properties of indicated key and datatype. - For the values, refer to the method's return value

- product object's properties: key (datatype)
  - prodName (string)
  - patentNum (number)
  - assemblyRequired (boolean)
  - buzzwords (array with 3 items)
  - company (object with 2 properties: name, country
  - promote (method) the returns a string concatenated from ALL the other properties and array items: promote method returns: "Announcing Doo-Dad Incorporated's newest high-tech, labor-saving, water-resistant Wonder Widget! Made in U.S.A. Patent number: 7654321. Some assembly required." HINT: Use if-else logic in the method to concatnate or not based on the boolean

```
const product = {
    // your code
}
```

2. Log the following properties of the object:

   - the whole object
   - the boolean, including with it the label "Assembly Required:"
   - the first item in the buzzwords array
   - the last item in buzzwords, accessed via the array length
   - the company name and country as: "All Doo-Dad, Inc. products are made in U.S.A."
   - the promote method: Get Doo-Dad Inc.'s new high-tech, labor-saving, water-resistant Wonder Widget! Made in U.S.A. Patent number: 7654321. Some assembly required.

```
console.log(product); // {} (contains all properties)
console.log(product); // Assembly Required: true
console.log(product); // high-tech
```

```
    console.log(product); // water-resistant
    console.log(product); // All Doo-Dad, Inc. products are made in U.S.A.
    console.log(product); // Get Doo-Dad Inc.'s new high-tech, labor-
saving, water-resistant Wonder Widget!
    // Made in U.S.A. Patent number: 7654321. Some assembly required.
```

3. Add four new properties outside of the object. In other words, do not add these new properites directly into the object declaration:
    - msrp (number) -- MSRP (Manufacturer's Suggested Retail Price)
    - discount (number) -- as decimal: .2 means 20%
    - salePrice (result of math done with mspr and discount)
    - caution (string)
4. Update the promote method to include the new properties.
    - do not edit the original object method HINT: product.promote = function()
    - The promote method's updated return value should be: "Get Doo-Dad Inc.'s new high-tech, labor-saving, water-resistant Wonder Widget! Made in U.S.A. Made in U.S.A. Patent number: 7654321. Some assembly required. MSRP: $40 * * * ON SALE! 20% OFF * * * NOW ONLY: $32 *** (CAUTION: Water-resistant does NOT mean waterproof. Do not submerge.)
    - log the promote method to see the changes.

```
    product.promote = function() {
    }

    console.log(product.promote());
    // Get Doo-Dad Inc.'s new high-tech, labor-saving, water-resistant
Wonder Widget!
    // Made in U.S.A. Patent number: 7654321. Some assembly required.
    // *** MSRP: $40 *** ON SALE! $20% OFF *** NOW ONLY: $32;
```

5. Update / change properties and log the whole object and the method again:
    - flip the boolean to false
    - increase the MSRP to $75
    - increase the discount to 50%
    - push another buzzword into the array: "shock-resistant"
    - update the promote method to include the new array item
    - log the promote method to see the changes.

```
    product.promote = function() {
    }

    console.log(product.promote());
    // Get Doo-Dad Inc.'s new high-tech, labor-saving, water-resistant,
shock-resistant Wonder Widget!
```

```
    // Made in U.S.A. Patent number: 7654321. Some assembly required.
    // *** MSRP: $75 *** ON SALE! $50% OFF *** NOW ONLY: $37.50;
```

SEE Lab 04.02 Solution

04.02 Lab Solution

Making an Object with Properties

Adding a Method (function) to an Object

Updating / Setting Object Properties

1. Make an object variable called product.

- In the object variable declaration, include the following properties of indicated key and datatype.
- For the values, refer to the method's return value
- product object's properties: key (datatype)
    - prodName (string)
    - patentNum (number)
    - assemblyRequired (boolean)
    - buzzwords (array with 3 items)
    - company (object with 2 properties: name, country
    - promote (method) the returns a string concatenated from ALL the other properties and array items: promote method returns: "Announcing Doo-Dad Incorporated's newest high-tech, labor-saving, water-resistant Wonder Widget! Made in U.S.A. Patent number: 7654321. Some assembly required." HINT: Use if-else logic in the method to concatnate or not based on the boolean

```
        const product = {
            prodName: 'Wonder Widget',
            patentNum: 7654321,
            assemblyRequired: true,
            buzzwords: ['high-tech', 'labor-saving', 'water-resistant'],
            company: {name: "Doo-Dad, Inc.", country: "U.S.A."},
            promote: function() {
                let assembly = "";
                if(this.assemblyRequired) assembly = "Some assembly
required.";
                return `\Get ${this.company.name}'s new
${this.buzzwords[0]}, ${this.buzzwords[1]},
${this.buzzwords[this.buzzwords.length-1]} ${this.prodName}!
                \nMade in ${this.company.country}. Patent number:
${this.patentNum}. ${assembly}`
            }
        }
```

2. Log the following properties of the object:

- the whole object
- the boolean, including with it the label "Assembly Required:"
- the first item in the buzzwords array
- the last item in buzzwords, accessed via the array length
- the company name and country as: "All Doo-Dad, Inc. products are made in U.S.A."
- the promote method: Get Doo-Dad Inc.'s new high-tech, labor-saving, water-resistant Wonder Widget! Made in U.S.A. Patent number: 7654321. Some assembly required.

```
    console.log(product); // {} (click arrow to open and see all
properties)
    console.log("Assembly Required:", product.assemblyRequired); //
Assembly Required: true
    console.log(product.buzzwords[0]); // high-tech
    console.log(product.buzzwords[product.buzzwords.length-1]); // water-
resistant
    console.log(`All ${product.company.name} products are made in
${product.company.country}`);
    // All Doo-Dad, Inc. products are made in U.S.A.
    console.log(product.promote());
    // Get Doo-Dad Inc.'s new high-tech, labor-saving, water-resistant
Wonder Widget!
    // Made in U.S.A. Patent number: 7654321. Some assembly required.
```

3. Add four new properties outside of the object. In other words, do not add these new properites directly into the object declaration:
    - msrp (number) -- MSRP (Manufacturer's Suggested Retail Price)
    - discount (number) -- as decimal: .2 means 20%
    - salePrice (result of math done with mspr and discount)
    - caution (string)

```
    product.msrp = 40;
    product.discount = .2;
    product.salePrice = product.msrp * (1 - product.discount);
    product.caution = "Water-resistant does NOT mean waterproof. Do not
submerge.";
```

4. Update the promote method to include the new properties.
    - do not edit the original object method HINT: product.promote = function()
    - The promote method's updated return value should be: "Get Doo-Dad Inc.'s new high-tech, labor-saving, water-resistant Wonder Widget! Made in U.S.A. Made in U.S.A. Patent number: 7654321. Some assembly required. MSRP: $40 * * * ON SALE! 20% OFF * * * NOW ONLY: $32 *** (CAUTION: Water-resistant does NOT mean waterproof. Do not submerge.)
    - log the promote method to see the changes.

```
    product.promote = function() {
        let assembly = "";
```

```
        if(this.assemblyRequired) assembly = "Some assembly required";
        return `Get ${this.company.name}'s new ${this.buzzwords[0]},
        ${this.buzzwords[1]}, ${this.buzzwords[this.buzzwords.length-1]}
${this.prodName}!
        Made in ${this.company.country}. Patent number: ${this.patentNum}.
${assembly}
        *** MSRP: $${this.msrp} *** ON SALE! ${this.discount * 100}% OFF
*** NOW ONLY: $${this.salePrice}`;
    }

    console.log(product.promote());
    // Get Doo-Dad Inc.'s new high-tech, labor-saving, water-resistant
Wonder Widget!
    // Made in U.S.A. Patent number: 7654321. Some assembly required.
    // *** MSRP: $40 *** ON SALE! $20% OFF *** NOW ONLY: $32;
```

5. Update / change properties and log the whole object and the method again:
    - flip the boolean to false
    - increase the MSRP to $75
    - increase the discount to 50%
    - push another buzzword into the array: "shock-resistant"
    - update the promote method to include the new array item
    - log the promote method to see the changes.

```
    product.assemblyRequired = false;
    product.msrp = 75;
    product.discount = .5;
    product.buzzwords.push('shock-resistant');

    product.promote = function() {
        let assembly = "";
        if(this.assemblyRequired) assembly = "Some assembly required";
        return `Get ${this.company.name}'s new ${this.buzzwords[0]},
${this.buzzwords[1]},
        ${this.buzzwords[2]}, ${this.buzzwords[this.buzzwords.length-1]}
${this.prodName}!
        Made in ${this.company.country}. Patent number: ${this.patentNum}.
${assembly}
        *** MSRP: $${this.msrp} *** ON SALE! ${this.discount * 100}% OFF
*** NOW ONLY: $${this.salePrice}`;
    }

    console.log(product.promote());
    // Get Doo-Dad Inc.'s new high-tech, labor-saving, water-resistant,
shock-resistant Wonder Widget!
    // Made in U.S.A. Patent number: 7654321. Some assembly required.
    // *** MSRP: $75 *** ON SALE! $50% OFF *** NOW ONLY: $37.50;
```

***NEXT: Lesson 04.03 ***



UNIT 04

LESSON 04.03



Array Methods

push(), pop(), sort()

unshift(), shift(), concat()

splice(), slice(), includes()

indexOf(), lastIndexOf()

join(), flat(), reverse()

Array methods are called on arrays and perform operations. We looked at a few array methods in a previous lesson, namely: `push()`, `pop()` and `sort()`. Let's recap those three before moving on:

1. Declare an array of a few items, and use `push()` to add a couple of items at the end:

```
const fruits = ['kiwi', 'cherry', 'banana'];
fruits.push('orange');
fruits.push('grape');

console.log(fruits);
// ['kiwi', 'cherry', 'banana', 'orange', 'grape']
```

2. Remove the last item using `pop()`. The method returns the popped item, so save that to a variable:

```
let poppedItem = fruits.pop();
console.log(poppedItem); // grape

console.log(fruits);
// ['kiwi', 'cherry', 'banana', 'orange']
```

3. Arrange the items in alphabetical order with `sort()`:

```
fruits.sort();

console.log(fruits); // ['banana', 'cherry', 'kiwi', 'orange']
```

Now, for some more array methods:

- **unshift()** -- add item to beginning
- **shift()** -- remove item from beginning
- **concat()** -- combine two or more arrays
- **splice()** -- remove or swap out items
- **slice()** -- make a new array from a range of items
- **includes()** -- looks for item and returns true or false
- **indexOf()** -- returns index of first matching item
- **lastIndexOf()** -- returns index of last matching item
- **join()** -- turn an array into a string
- **flat()** -- turn a matrix into a 1D array
- **reverse()** -- reverse the order of items

shift() and unshift()

- **unshift()** adds an item to the beginning of an array.
- **shift()** removes the first item and returns it.

To help remember which is which, "unshift" is a longer word than "shift", just as "push" is longer than "pop". The longer words make the array longer, while the shorter words make the array shorter.

4. Use **unshift()** to add an item to the beginning of the fruits array:

```
fruits.unshift('apple');

console.log(fruits);
// ['apple', 'banana', 'cherry', 'kiwi', 'orange']
```

5. Use **shift()** to remove and return the first item, saving that to a variable:

```
let shiftedItem = fruits.shift();

console.log(shiftedItem); // apple
console.log(fruits);
// ['banana', 'cherry', 'kiwi', 'orange']
```

concat()

As the name implies, **concat()** concatentates (combines) two or more things, in this case arrays. You call **concat()** on one array, and pass the method the other array(s) as its argument(s).

6. Declare three arrays, and then concat them into one:

```
const tropicalFruits = ['mango', 'kiwi', 'banana', 'pineappple'];

const citrusFruits = ['orange', 'lemon', 'lime', 'tangerine'];
```

```
    const blossomFruits = ['apple', 'peach', 'cherry', 'plum'];

    const fruitCocktailArr = tropicalFruits.concat(citrusFruits,
blossomFruits);

    console.log(fruitCocktailArr);
    // ['mango', 'kiwi', 'banana', 'pineappple', 'orange', 'lemon',
'lime', 'tangerine', 'apple', 'peach', 'cherry', 'plum']
```

7. Sort the fruit cocktail array. The `sort()` method changes the original array; it does not return a new array.

```
    fruitCocktailArr.sort();

    console.log(fruitCocktailArr);
    // ['apple', 'banana', 'cherry', 'kiwi', 'lemon', 'lime', 'mango',
'orange', 'peach', 'pineappple', 'plum', 'tangerine']
```

splice()

The **splice()** method removes or swaps out items at a specified index or range of indices. With **splice()**, you can remove or replace any item, or sequence of items, anywhere in the array.

- `splice()` takes two arguments: the index of the first item to remove, and the number of items to remove.
- `splice()` returns the removed item(s). If more than one item was removed, it returns an array.

8. Remove the item at index 9, which is 'pineapple':

```
    fruitCocktailArr.splice(9,1);

    console.log(fruitCocktailArr);
    // ['apple', 'banana', 'cherry', 'kiwi', 'lemon', 'lime', 'mango',
'orange', 'peach', 'plum', 'tangerine']
```

9. Starting at index 2, splice out four consecutive items, saving the result to a new array:

```
    let splicedItems = fruitCocktailArr.splice(2,4);

    console.log(splicedItems);
    // ['cherry', 'kiwi', 'lemon', 'lime']
    console.log(fruitCocktailArr);
    // ['apple', 'banana', 'mango', 'orange', 'peach', 'plum',
'tangerine']
```

To swap an item with `splice()`, pass in the new item as additional argument(s).

10. Using `splice()`, swap 'peach' for 'papaya':

```
    fruitCocktailArr.splice(4, 1, 'papaya');

    console.log(fruitCocktailArr);
    // ['apple', 'banana', 'mango', 'orange', 'papaya', 'plum',
'tangerine']
```

To swap more than one item with `splice()`, just add more arguments.

11. Replace 'banana' and 'mango' with 'blueberry' and 'strawberry'.

```
    fruitCocktailArr.splice(1, 2, 'blueberry', 'strawberry');

    console.log(fruitCocktailArr);
    // ['apple', 'blueberry', 'strawberry', 'orange', 'papaya', 'plum',
'tangerine']
```

splice for avoiding repeats of random array items

If you keep choosing items at random from an array, you will eventually get repeats. To avoid repeats, splice out each item as you go.

12. Pick two random fruits from `fruitCocktailArr`, and log them both:

```
    let r1 = Math.floor(Math.random() * fruitCocktailArr.length);
    let rFruit1 = fruitCocktailArr[r1];

    let r2 = Math.floor(Math.random() * fruitCocktailArr.length);
    let rFruit2 = fruitCocktailArr[r2];

    console.log(rFruit1, rFruit2);
```

13. Rerun the console.log until you get a repeat fruit. To rerun the console command, hit the Up Arrow and then hit Enter.

To avoid repeats, each time a random fruit is chosen, splice it out of the array with `splice(r, 1)`. The index, `r`, is the random item and the number of items to remove is 1.

14. Choose two random fruits again, this time splicing out the first fruit before picking the second one:

```
    r1 = Math.floor(Math.random() * fruitCocktailArr.length);
    rFruit1 = fruitCocktailArr[r1];
```

```
fruitCocktailArr.splice(r1, 1);

r2 = Math.floor(Math.random() * fruitCocktailArr.length);
rFruit2 = fruitCocktailArr[r2];
```

15. Rerun the console.log command repeatedly to verify that no repeats occur. Soon, the array will be empty and the results `undefined`:

```
console.log(rFruit1, rFruit2);
```

slice()

The **slice()** method is called on an array and takes two arguments: a starting and ending index.

- `slice()` returns a new array without affecting the original array.
- `slice()` end index is exclusive, so *not* included in the new array.

16. Starting at index 2 and ending at index 5 (exclusive), get a new an array of 3 items:

```
let fruitSlices = fruitCocktailArr.slice(2, 5);

console.log(fruitSlices);
// ['strawberry', 'orange', 'papaya']
```

If you omit the second argument, it slices from the start index (first argument) all the way to the end:

17. Starting at index 4, slice all the way to the end:

```
let slicedFruit = fruitCocktailArr.slice(4);

console.log(slicedFruit);
// ['papaya', 'plum', 'tangerine']
```

includes()

The **includes()** method is called on an array and returns true if its argument is found in the array, and false if it is not:

18. Call the `includes()` method to get one true and one false result:

```
console.log(slicedFruit.includes('plum'));
// true
```

```
    console.log(slicedFruit.includes('pear'));
    // false
```

indexOf()

The **indexOf()** method is called on an array and returns the index of the first instance of the argument. If it is not found, it returns -1.

19. Declare an array of reptiles and run the indexOf() method a few times:

```
    const reptiles = ['iguana', 'snake', 'turtle', 'snake', 'gekko',
'snake', 'lizard'];

    console.log(reptiles.indexOf('snake')); // 1
    console.log(reptiles.indexOf('turtle')); // 2
    console.log(reptiles.indexOf('Komodo dragon')); // -1
```

To specify a starting index, pass in a second argument. This skips earlier instances of the word:

20. Starting at index 2, get the position of the first snake after that:

```
    console.log(reptiles.indexOf('snake', 2)); // 3
```

lastIndexOf()

The **lastIndexOf()** method returns the index of the last occurrence of the argument. If it is not found, it returns -1:

```
    console.log(reptiles.lastIndexOf('snake')); // 4
    console.log(reptiles.lastIndexOf('Gila monster')); // -1
```

join()

The **join()** method is called on an array and returns a string of all the items, separated by commas. It does not change the array.

21. Join the reptiles into a long slithering string:

```
    let reptilesStr = reptiles.join();
    console.log('reptilesStr', reptilesStr);
    // iguana,snake,turtle,snake,gekko,snake,lizard
```

The join method can take a *delimiter* argument--character(s) that will appear between the items in the resulting string.

22. Put an asterisk surrounded by spaces between each fruit in the string:

```
    let starryFruits = fruitCocktailArr.join(' * ');
    console.log(starryFruits);
    // apple * blueberry * strawberry * orange * papaya * pineappple *
plum * tangerine
```

flat()

The **flat()** method takes a nested array (2D, matrix) as its argument and returns a flat, one-dimensional array. It does not change the original array.

23. Declare a 3x3 / 2D / nested array:

```
    const ticTacToe = [
        ['X', 'O', null],
        [null, 'X', 'O'],
        ['O', null, 'X']
    ];
```

24. Flatten the array:

```
    const flatArray = ticTacToe.flat();

    console.log(flatArray);
    // console.log('flatArr', flatArr);
    // ['X', 'O', null, null, 'X', 'O', 'O', null, 'X']
```

copying an array with **slice(0)

An array can be copied in a number of ways. One way is `slice(0)`. Starting at index 0 and going all the way to the end by omitting the second argument, it returns a new array which includes all items in the original.

25. Declare an array and make a copy of it:

```
    const animals = ['giraffe', 'leopard', 'ostrich', 'zebra',  'panda',
'moose', 'bison', 'aardvark', 'baboon', 'rhinoceros'];

    const animalsCopy = animals.slice(0);

    console.log(animalsCopy);
    // ['giraffe', 'leopard', 'ostrich' etc.]
```

reverse()

The **reverse()** method reverses the order of the items in an array. It is often combined with **sort()** to flip the sorted order:

26. Sort and then reverse the animals array:

```
    console.log(animals.sort());
    console.log(animals.reverse());
    // ['zebra', 'rhinoceros', 'panda', 'ostrich', 'moose', 'leopard',
'giraffe', 'bison', 'baboon', 'aardvark']
```

methods chaining

Methods can be called one after another, on the same line. This is known as *methods chaining*.

27. In one line of code, sort and then reverse the `animalsCopy` array:

```
    animalsCopy.sort().reverse();

    console.log('animalsCopy', animalsCopy);
```

END Lesson 04.03

NEXT: Lab 04.03 Lesson 04.04



array methods

1. Make a pets array containing 'cat', 'dog' and 'hamster'.

```
// your code
console.log(pets); // ['cat', 'dog', 'hamster']
```

array.push() adds item(s) to end of array

2. Push 'iguana' into the array.
3. Push 'parrot' into the array in one line.
4. Push 'snake' into the array in one line.

```
// your code
console.log(pets);
// ['cat', 'dog', 'hamster', 'iguana', 'parrot', 'snake']
```

array.pop() removes item from end of array

5. Remove (pop) the last item, which is 'snake'

```
// your code
console.log(pets);
// ['cat', 'dog', 'hamster', 'iguana', 'parrot']
```

6. Put (push) back 'snake':

```
// your code
console.log(pets);
// ['cat', 'dog', 'hamster', 'iguana', 'parrot', 'snake']
```

7. Do a pop() again, but this time save the return value:

```

```

```
    // your code
    console.log(pets);
    // ['cat', 'dog', 'hamster', 'iguana', 'parrot']
```

array.unshift() adds item to beginning of array

8. Add 'bunny' to the beginning of pets array:

```
    // your code
    console.log(pets);
    // ['bunny', 'cat', 'dog', 'hamster', 'iguana', 'parrot']
```

***array.shift() method removes first item of array**

9. Remove 'bunny' from the beginning of the pets array:

```
    // your code
    console.log(pets);
    // ['cat', 'dog', 'hamster', 'iguana', 'parrot']
```

array1.concat(array2) combines 2 or more arrays

10. Combine pets and morePets into the petsGalore array:

```
    // given:
    const morePets = ['canary', 'gerbil', 'kitten', 'python', 'turtle'];
    // combine pets and morePets into a new array, petsGalore:
    // your code
    // console.log(petsGalore);
    // ['cat', 'dog', 'hamster', 'iguana', 'parrot', 'canary', 'gerbil',
'kitten', 'python', 'turtle']
```

array.sort() from A-Z

11. Sort the petsGalore array:

```
    // your code
    // ['canary', 'cat', 'dog', 'gerbil', 'hamster', 'iguana', 'kitten',
'parrot', 'python', 'turtle']
```

array.splice(index, count)

12. remove the first two items, saving them to a variable called twoSplicedPets:

```
    // your code
    console.log(twoSplicedPets); // ['canary', 'cat']
    console.log(petsGalore); // ['dog', 'gerbil', 'hamster', 'iguana',
'kitten', 'parrot', 'python', 'turtle']
```

13. starting at the 4th item in petsGalore, splice 3 items in a row; save the items as threeSplicedPets

```
    // your code
    // console.log(threeSplicedPets);
    // ['iguana', 'kiten', 'parrot']
    // console.log(petsGalore);
    // ['dog', 'gerbil', 'hamster', 'python', 'turtle']
```

14. Using concat(), restore petsGalore to its original state of 10 pets and sort it from A-Z:

```
    // your codde
    console.log(petsGalore);
    // ['canary', 'cat', 'dog', 'gerbil', 'hamster', 'iguana', 'kitten',
'parrot', 'python', 'turtle']
```

15. Make a string connected by & of all items // console.log('petsStr:', petsStr); // canary & cat & dog & gerbil .. etc.

END: Lab 04.03
SEE Labo 04.03 Solution

04.04 Lab Exercises - SOLUTION

array methods

1. Make a pets array containing 'cat', 'dog' and 'hamster':

```
const pets = ['cat', 'dog', 'hamster'];
console.log('pets:', pets); // ['cat', 'dog', 'hamster']
```

array.push() adds item to end of array

2. Push 'iguana' into the array:

```
pets.push('iguana');
console.log('pets:', pets); // ['cat', 'dog', 'hamster', 'iguana']
```

3. Push 'parrot' into the array:

```
pets.push('parrot');
console.log('pets:', pets);
// ['cat', 'dog', 'hamster', 'iguana', 'parrot']
```

4. Push 'snake' into the array:

```
pets.push('snake');
console.log('pets:', pets);
// ['cat', 'dog', 'hamster', 'iguana', 'parrot', 'snake']
```

array.pop() removes item from end of array

5. remove (pop) the last item, which is 'snake':

```
pets.pop(); // remove last item, which is the snake
console.log('pets:', pets);
pets.push('snake');
```

6. Put (push) back 'snake':

```
console.log('pets:', pets);
// ['cat', 'dog', 'hamster', 'iguana', 'parrot', 'snake']
let poppedPet = pets.pop();
```

7. Pop 'snake' again, but this time save it as the return value:

```
    let poppedPet = pets.pop();
    console.log('poppedPet:', poppedPet); // snake
    console.log('pets:', pets);
    // ['cat', 'dog', 'hamster', 'iguana', 'parrot']
```

array.unshift() adds item to beginning of array

8. Add 'bunny' to the beginning of pets array

```
    pets.unshift('bunny');
    console.log('pets:', pets);
    // ['bunny', 'cat', 'dog', 'hamster', 'iguana', 'parrot']
```

array.shift() method removes first item of array

9. Remove 'bunny':

```
    pets.shift();
    console.log('pets:', pets); // ['cat', 'dog', 'hamster', 'iguana',
'parrot']
```

array1.concat(array2) combines 2 or more arrays

10. Combine pets and morePets into the petsGalore array:

```
    const morePets = ['canary', 'gerbil', 'kitten', 'python', 'turtle'];
    const petsGalore = pets.concat(morePets);
    // combine pets and morePets into new array, petsGalore
    console.log('pets:', pets);
    console.log('morePets:', morePets);
    console.log('petsGalore:', petsGalore);
    // ['cat', 'dog', 'hamster', 'iguana', 'parrot', 'canary', 'gerbil',
'kitten', 'python', 'turtle']
```

array.sort() from A-Z

11. Sort the petsGalore array:

```
    petsGalore.sort();
    console.log('petsGalore sorted:', petsGalore);
```

```
    // ['canary', 'cat', 'dog', 'gerbil', 'hamster', 'iguana', 'kitten',
'parrot', 'python', 'turtle']
```

array.splice(index, count)

12. get rid of the first two items but save them to a var called twoSplicedPets:

```
    let twoSplicedPets = petsGalore.splice(0, 2);
    console.log('twoSplicedPets:', twoSplicedPets); // ['canary', 'cat']
    console.log('petsGalore spliced:', petsGalore);
    // ['dog', 'gerbil', 'hamster', 'iguana', 'kitten', 'parrot',
'python', 'turtle']
```

13. starting at the 4th item in petsGalore, splice 3 items in a row; save the items as threeSplicedPets:

```
    let threeSplicedPets = petsGalore.splice(3, 3);
    console.log('threeSplicedPets:', threeSplicedPets);
    // ['iguana', 'kiten', 'parrot']
    console.log('petsGalore spliced:', petsGalore);
    // ['dog', 'gerbil', 'hamster', 'python', 'turtle']
```

14. Using concat(), restore petsGalore to its original state of 10 pets and sort it from A-Z:

```
    const petsGaloreRestored = petsGalore.concat(twoSplicedPets,
threeSplicedPets).sort();
    console.log('petsGalore restored:', petsGaloreRestored);
```

15. Make a string connected by & of all items:

```
    let petsStr = petsGalore.join( ' & '); // array.join() is called on an
array and returns a string of all items
    console.log('petsStr:', petsStr);
```

NEXT: Lesson 04.04



UNIT 04

LESSON 04.04



string methods

String methods are actions that are called on strings.

string[index] With arrays, you get items by index. With strings you get *characters* by index.

1. Declare a string and get the first and third characters:

```
let fruit = 'cherry';
// get the first and third characters:
console.log(fruit[0]); // c
console.log(fruit[2]); // e
```

string.length

As with arrays, the length prpoerty of a string returns the number of items, in this case characters.

2. Get the length of a password and then run an if-statement that tells if the password is too short (less than 15 characters):

```
let password = '35khD%ewG@1';
let pswdLen = password.length;
console.log('Password length:', pswdLen);
if(password.length < 15) {
    console.log('Password is too short!');
}
```

string.replace(a, b)

The **replace()** method is called on a string. It takes two arguments:

- what you want to replace
- what you want to replace with

It returns a new string, without changing the original. Change 'cherry' to 'berry':

```
let newStr = fruit.replace('ch', 'b');
console.log(newStr); // berry
```

Change 'n' to 't' in an attempt to change 'banana' to 'batata' (a kind of sweet potato):

```
fruit = 'banana';
let foodItem = fruit.replace('n', 't');
console.log(foodItem); // batana
```

Notice that we only got 'batana', which means that the replace() method only changes the first occurrence of the target character(s).

The **replace()** method can replace a space with a word. Turn 'fresh salad' into 'fresh garden salad':

```
appetizer = appetizer.replace(' ', ' garden '));
console.log(appetizer); // fresh garden salad
```

replaceAll()

To replace all instances of a character, use replaceAll():

```
console.log(fruit); // banana
foodItem = fruit.replaceAll('n', 't');
console.log(foodItem); // batata
```

includes()

The **includes()** method takes a character(s) as its argument and returns true or false, based on whether or not the character(s) are in the string;

```
let bev = 'papaya smoothie';
console.log(bev.includes('smooth')); // true
console.log(bev.includes('mango')); // false
```

indexOf()

The **indexOf()** method takes a character(s) as its argument and returns the index of the first occurrence:

```
console.log(fruit); // banana
console.log(fruit.indexOf('n')); // 2
```

lastIndexOf()

The **lastIndexOf()** method takes a character(s) as its argument and returns the index of the last occurrence:

```
console.log(fruit); // banana
console.log(fruit.lastIndexOf('n')); // 4
```

If the char is not found in the string, indexOf returns -1:

```
console.log(fruit.indexOf('x')); // -1
```

The **indexOf()** method is useful for finding the space between two words, which gives the length of the first word:

```
let appetizer = 'fresh salad';
console.log(appetizer.indexOf(' ')); // 5
```

charAt()

The **charAt()** method is called on a string. It takes an integer as an argument and returns the character at that index:

```
let fruit = 'apple';
console.log(fruit.charAt(0)); // a
console.log('Hello World'.charAt(6)); // W
```

slice()

The **slice()** method returns a substring of the string it is called on, without changing the original string. It takes two arguments: a starting and an ending index. The end index is exclusive, meaning it is not included in the returned string. If the end index is omitted, it slices to the end.

```
let dynamicDuo = 'Batamn and Robin';
let hero1 = dynamicDuo.slice(0, 7);
console.log(hero1);
```

indexOf() and lastIndexOF() with slice()

But what if we didn't know the length of the word. We would have to get the index of the first space and then use that as the end index:

```
let superCouple = 'Superman and Lois Lane';
let indxSpace = superCouple.indexOf(' ');
let superhero = superCouple.slice(0, indxSpace);
console.log(superhero);
```

To get 'Lois Lane', we could get the index of the first 'L' and then just slice to the end of the string:

```
let Lindex = superCouple.indexOf('L');
let supermansGF = superCouple.slice(Lindex);
console.log(supermansGF);
```

To get the last word of a string, get the index of the last space, and then slice from there to the end:

```
let lastSpaceIndex = superCouple.lastIndexOf(' ');
let lastWord = superCouple.slice(lastSpaceIndex);
console.log(lastWord);
```

split()

The **split()** method is called on a string and returns an array.

```
let movieQuote = 'Show me the money';
var movieQuoteArr1 = movieQuote.split();
console.log(movieQuoteArr1); // ['Show me the money']
```

Notice that there is only one item in the resulting array. To have each word become an array item, pass the split() method the space between the words as its **delimiter**:

```
let movieQuoteArr2 = movieQuote1.split(' ');
console.log(movieQuoteArr2); // ['Show', 'me', 'the', 'money']
```

toLowerCase()

The **toLowerCase()** method is called on a string and returns an all-lowercase version of the string, without changing the original:

```
let newsFlash = 'Yankees Win World Series';
console.log(newsFlash.toLowerCase());
console.log(newsFlash); // unchanged
```

string.split() + array.join() + toLowerCase() Split and join can be used to break a string into an array and then put it back as a string, with some transformation being done in the process.

Turn this news headline into a hyphenated file name.

```

```

```
let news = 'Mets Win World Series';
let newsArr = headline.split(' ');
console.log(newsArr);
// ['Mets', 'Win', 'World', 'Series']
let imgFile = newsArr.join('-') + '.jpg';
console.log(imgFile);
// Mets-Win-World-Series.jpg;
let imgFileLC = imgFile.toLowerCase();
 console.log(imgFileLC);
// mets-win-world-series.jpg;
```

toUpperCase()

The **toUpperCase()** method is called on a string and returns an all-uppercase version of the string, without changing the original:

```
let headline = 'Jets Win Superbowl';
let shouting = headline.toUpperCase());
console.log(shouting); // JETS WIN SUPERBOWL
```

toUpperCase() + slice() to capitalize a word

There is no method for capitalizing a word, but we can get the first character and uppercase that. We also slice off the rest of the string, and then reconnect that part to the uppercased first letter.

Get the first letter and uppercase it:

```
let firstName = 'alexandria';
let firstChar = firstName[0];
let firstCharUC = firstChar.toUpperCase();
console.log(firstCharUC);
```

Get the rest of the string:

```
let restOfName = firstName.slice(1);
```

Connect the capitalized first letter with the rest of the string:

```
let capitalizedName = firstCharUC + restOfName;
console.log(capitalizedName);
```

NEXT: 04.05 Lab Exercises



Dynamic Object Properties

Seaching by Object Properties

Given this array of vegetables:

```
const veggies = ['Cucumber', 'Tomato', 'Kale'];
```

1. Use the **replace()** method and array items to log the expected message to the console:

```
let msg1 = 'Have you ever ordered the Sandwich?';
console.log(msg1); // Have you ever ordered the Cucumber Sandwich?
```

2. Use the replace() method and array items to log the expected message to the console:

```
let msg2 = 'The "T" in "BLT" stands for "Terminator."';
console.log(msg2); // The "T" in "BLT" stands for "Tomato."
```

3. Use the replace() method and array items to log the expected message to the console:

```
let msg3 = 'Bunny goes to Yale University.';
console.log(msg3); // Bunny goes to Kale University.
```

Given this array of phrases:

```
const phrases = ["A real customer", "A breeze", "Hangin' with the
kids", "Snoopy is a dog"];
```

4. Using string methods and concatenation, make "cool" the next to the last word, no matter which phrase is chosen. Expected output is one of the following:

- a real cool customer
- a cool breeze
- hangin' with the cool kids
- Snoopy is a cool dog

Since we can't predict what the random phrase will be, we cannot use the **replace()** method on a hard-coded last word of the string. So, where does cool need to go? It needs to go in at the last space location. What string method returns the last space of a phrase: **lastIndexOf(" ")** test this on **phrases[0]**, which is the first item in phrases Save the last index of the the space to a variable:

```
    let lastSpaceIndex;
```

then use the **slice()** method twice: once to get everything up to (but not including) the first space; use the **slice()** method again to get everything from the last space to the end of the string; concatenate the two slices together--with " cool " in between.

```
    let coolPhrase;
    console.log("Cool phrase test:", coolPhrase);
```

test the result with a random number in the range of the array

```
    let r;
    console.log("Random cool phrase:", coolPhrase);
```

Given this array of fruits and vegetables:

```
    const words = ['Avocado', 'Broccoli', 'Cauliflower', 'Dragonfruit',
'Eggplant'];
```

5. Use the includes() method and if-else logic to check if the word starts with a consonant or a vowel

- If the word is "Avocado", log: Avocado starts with "A", which is a vowel.
- If the word is "Broccoli", log: Broccoli starts with "B", which is a consonant.

```
    let word;
    let firstChar;
    console.log('firstChar', firstChar);
    let vowels;
    if(vowels) {
    }
```

REFACTOR: Since the if-else console logs differ only in one word: vowel / consonant, better would be to have a variable "consonant" that the if statements sets to "vowel" and then concat that "consonant" or "vowel" vvariable into the output:
Try it again:

```
    let consonantOrVowel;
    if(vowels) {
    }
```

Given this file name:

```
    let fileName = "Mets-Lead-Off-Game-By-Hitting-Three-Straight-
Homers.html";
```

6. Turn the file name into a news headline, with all words capitalized:

Expected result:
Mets Lead Off Game By Hitting Three Straight Homers

```
    let wordsArr;
    console.log(wordsArr); // ['Mets', 'Lead', 'Off', 'Game', 'By',
'Hitting' .. etc.]
    let headline;
```

Remove the file extension, but don't assume '.html' -- detect the dot and remove everything from the dot to the end of the string

```
    let indexOfDot;
```

Use slice() method to get everything from the beginning of the string to to dot (but not includding the dot)

See: Lab 04.04 Solution

Dynamic Object Properties

Seaching by Object Properties

Given this array of vegetables:

```
    const veggies = ['Cucumber', 'Tomato', 'Kale'];
```

1. Use the **replace()** method and array items to log the expected message to the console:

```
    let msg1 = 'Have you ever ordered the Sandwich?';
    msg1 = msg1.replace('Sandwich', veggies[0] + ' Sandwich');
    console.log(msg1); // Have you ever ordered the Cucumber Sandwich?
```

2. Use the replace() method and array items to log the expected message to the console:

```
    let msg2 = 'The "T" in "BLT" stands for "Terminator."';
    msg2 = msg2.replace('Terminator', veggies[1]);
    console.log(msg2); // The "T" in "BLT" stands for "Tomato."
```

3. Use the replace() method and array items to log the expected message to the console:

```
    let msg3 = 'Bunny goes to Yale University.';
    msg3 = msg3.replace('Yale', veggies[2]);
    console.log(msg3); // Bunny goes to Kale University.
```

Given this array of phrases:

```
    const phrases = ["A real customer", "A breeze", "Hangin' with the
kids", "Snoopy is a dog"];
```

4. Using string methods and concatenation, make "cool" the next to the last word, no matter which phrase is chosen. Expected output is one of the following:

- a real cool customer
- a cool breeze
- hangin' with the cool kids
- Snoopy is a cool dog

Since we can't predict what the random phrase will be, we cannot use the **replace()** method on a hard-coded last word of the string. So, where does cool need to go? It needs to go in at the last space location. What string method returns the last space of a phrase: **lastIndexOf(" ")** test this on **phrases[0]**, which is the first item in phrases Save the last index of the the space to a variable:

```
    let lastSpaceIndex = phrases[0].lastIndexOf(' ');
```

then use the **slice()** method twice: once to get everything up to (but not including) the first space; use the **slice()** method again to get everything from the last space to the end of the string; concatenate the two slices together--with " cool " in between.

```
    let coolPhrase = phrases[0].slice(0, lastSpaceIndex) + " cool " +
phrases[0].slice(lastSpaceIndex+1);
    console.log("Cool phrase test:", coolPhrase);
```

test the result with a random number in the range of the array

```
    let r = Math.floor(Math.random() * phrases.length);
    // get the index of the last space of the new, randomly selected
phrase
    lastSpaceIndex = phrases[r].lastIndexOf(' ');
    // concatenate the two slices together--with " cool " in between.
    coolPhrase = phrases[r].slice(0, lastSpaceIndex) + " cool " +
phrases[r].slice(lastSpaceIndex+1);
    console.log("Random cool phrase:", coolPhrase);
```

Given this array of fruits and vegetables:

```
    const words = ['Avocado', 'Broccoli', 'Cauliflower', 'Dragonfruit',
'Eggplant'];
```

5. Use the includes() method and if-else logic to check if the word starts with a consonant or a vowel

- If the word is "Avocado", log: Avocado starts with "A", which is a vowel.
- If the word is "Broccoli", log: Broccoli starts with "B", which is a consonant.

```
    r = Math.floor(Math.random() * words.length);
    let word = words[r];
    let firstChar = word[0];
    console.log('firstChar', firstChar);
    let vowels = 'aeiouAEIOU';
    if(vowels.includes(firstChar)) {
        console.log(`${word} starts with "${firstChar}", a vowel.`);
```

```
    } else {
        console.log(`${word} starts with "${firstChar}", a consonant.`);
    }
```

REFACTOR: Since the if-else console logs differ only in one word: vowel / consonant, better would be to have a variable "consonant" that the if statements sets to "vowel" and then concat that "consonant" or "vowel" vvariable into the output:
Try it again:

```
    r = Math.floor(Math.random() * words.length);
    let consonantOrVowel = "consonant";
    word = words[r];
    firstChar = word[0];
    if(vowels.includes(firstChar)) {
        consonantOrVowel = "vowel";
    }
    console.log(`${word} starts with "${firstChar}", which is a
${consonantOrVowel}`);
```

Given this file name:

```
    let fileName = "Mets-Lead-Off-Game-By-Hitting-Three-Straight-
Homers.html";
```

6. Turn the file name into a news headline, with all words capitalized:

Expected result:
Mets Lead Off Game By Hitting Three Straight Homers

```
    let wordsArr = fileName.split('-');
    console.log(wordsArr); // ['Mets', 'Lead', 'Off', 'Game', 'By',
'Hitting' .. etc.]
    let headline = wordsArr.join(' ');
```

Remove the file extension, but don't assume '.html' -- detect the dot and remove everything from the dot to the end of the string

```
    let indexOfDot = headline.indexOf('.');
```

Use slice() method to get everything from the beginning of the string to to dot (but not includding the dot)

```

```

```
    headline = headline.slice(0, indexOfDot);
    console.log(headline);
```

END: Lab 04.04

NEXT: Lesson 05.01



For Loops

Interating (Looping) Arrays

A loop repeatedly executes a block of code, as long as a condition remains true. There are a few kinds of loops, including "for loops" and "while loops". In this lesson, we will focus on "for loops".

for loops

A for loop considers three pieces of information in parentheses to decide if it will run the code inside its curly braces: a **counter**, a **condition** and an **incrementer**:

```
for (counter, condition, incrementer) {
    // do stuff
}
```

counter

The counter is a number variable that goes up or down each time the loop is run. The counter is usually `i`, but can be any name. The counter is assigned an initial value, typically 0, although that can be any number.

condition

The condition compares the counter to another value. If the condition is true, the code inside the curly braces will run. If the condition is false, the loop ends.

incrementer

The incrementer (or decrementer), specifies the value by which the counter increases or decreases with each iteration of the loop. Typically, a counter will go up by one each time though the loop, as assigned by the shorthand, `i++`.

1. Write a for loop with a counter, `i`, that starts at 0, goes up by one each time, and stops when `i` gets to 10:

```
for (let i = 0; i < 10; i++) {
    console.log(i); // 0, 1, 2...9
}
```

let variables are block scoped

As discussed in previous lessons, a `let` variable is `block scoped`, meaning that it is available only inside the code block in which it is declared.

2. Try to access `i` outside the loop. We get an error:

```
for (let i = 0; i < 10; i++) {
    console.log(i); // 0, 1, 2...9
}

console.log(i); // ERROR: i is not defined
```

The error indicates that `i` does not exist in the global scope, which is fine because `i` was only needed in the loop. Once the loop ended, `i` was deleted from memory ("garbage collected", as they say).

var variables are not block scoped

A `var` is not block scoped, so if you use `var i` for a loop counter, `i` will be instantiated in the global scope--and will still be there after the loop ends. It is said that `var` counters "leak out" of their loop. We don't want variables persisting in memory with nothing to do, so use `let` instead.

3. Do another loop with `var` instead of `let`, and verify that `i` still exists after the loop has ended:

```
for (var i = 0; i < 10; i++) {
    console.log(i); // 0, 1, 2...9
}

console.log('after loop', i); // after loop 10
```

Notice that the value of `i` after the loop ends is 10, which matches the loop condition. Once the counter reached 10, the condition `i < 10` became false, so the loop ended.

4. Change the condition to `i <= 10` to get to 10 inside the loop and 11 outside the loop.

Let's try counting down. Start `i` at 10 and *decrement* by 1 each time with `i--`. When `i` reaches 0, the condition `i > 0` is false, so the loop ends.

5. Write a loop with a counter that decrements (counts backwards):

```
for (let i = 10; i > 0; i--) {
    console.log(i) // runs 10 times
}

console.log(i) // 11
```

The 11 reminds us that we still have a lingering `var i` which "leaked out" of its loop into the global scope.

6. Change `i` to `j` and see that `j` ceases to exist once the loop ends:

```
for (let j = 10; j > 0; j--) {
    console.log(j) // runs 10 times
}
console.log(j) // ERROR: j is not defined
```

infinite loop

An infinite loop is one that never ends, because the condition is always true. The condition is supposed to eventually flip from true to false, but in an infinite loop that never happens due to a flaw in the logic.

This next loop runs forever because `i` starts at 10 and goes up from there. The condition `i > 0` is therefore always true.

7. Write but do not run this infinite loop, as doing so may freeze your browser. Just study it before commenting it out.

```
/* infinite loop; do not run
for (let i = 10; i > 0; i++) {
    console.log(i);
}
*/
```

the += and -= operators

The counter can be incremented / decremented by any value. To increment by 5, it's `i+=5`. To decrement by 2, it's `i-=2`.

8. Run a loop where the counter starts at 0, goes up by 5 each time until it reaches 100 (inclusive):

```
for (let i = 0; i <= 100; i+=5) {
    console.log(i); // 0, 5, 10...95, 100
}
```

continue

The `continue` keyword skips an iteration of a loop. It is used with conditional logic to specify when to skip:

9. Output all numbers from 1-10, with the exception of 7:

```
for(let i = 0; i < 10; i++) {
    if(i == 7) {
        continue; // skip 7
    }
    console.log(i); // 0,1,2,3,4,5,6,8,9
}
```

iterating (looping) arrays

Loops are commonly used to iterate arrays, which means to go through them, item by time. The counter is used to get the current item by index.

In these next steps, we will loop through an array while also working in some review of array methods:

10. Declare an array:

```
    const fruits = ['apple', 'blueberry', 'cherry', 'kiwi', 'lime',
'orange', 'plum'];
```

11. Add a few more items to the end of the array:

```
    fruits.push('apricot');
    fruits.push('papaya')
    fruits.push('grape');
```

12. Add a few items to the beginning of the array:

```
    fruits.unshift('grapefruit');
    fruits.unshift('watermelon');
    fruits.unshift('tangerine');
```

13. Output the array and its length:

```
    console.log(fruits, fruits.length);
    // ['tangerine', 'watermelon', 'grapefruit', 'apple', 'blueberry',
'cherry', 'kiwi', 'lime', 'orange', 'plum', 'apricot', 'papaya', 'grape']
13
```

14. Starting at index 3, splice out 3 items ('apple', 'blueberry', 'cherry'), and replace them with 'lemon' and 'pear':

```
    fruits.splice(3, 3, 'lemon', 'pear');

    console.log(fruits, fruits.length);
    // ['tangerine', 'watermelon', 'grapefruit', 'lemon', 'pear', 'kiwi',
'lime', 'orange', 'plum', 'apricot', 'papaya', 'grape'] 12
```

15. At index 7, insert 4 items without removing any. The new fruits will go in before 'orange':

```
    fruits.splice(7, 0, 'apple', 'blueberry', 'cherry', 'peach');

    console.log(fruits, fruits.length);
    //  ['tangerine', 'watermelon', 'grapefruit', 'lemon', 'pear', 'kiwi',
'lime', 'apple', 'blueberry', 'cherry', 'peach', 'orange','plum',
'apricot', 'papaya', 'grape'] 16
```

16. Iterate the array of 16 items with a for loop. Each time through the loop, make a fruit jellybean.

```
    for(let i = 0; i < 16; i++) {
        let bean = fruits[i] + ' jellybean';
        console.log(bean); // tangerine jellybean, etc.
    }
```

17. Push in a few more fruits, and then sort the array:

```
    fruits.push('banana');
    fruits.push('pineapple');
    fruits.push('mango');
    fruits.sort();

    console.log(fruits, fruits.length);
    // ['apple', 'apricot', 'banana', 'blueberry', 'cherry', 'grape',
'grapefruit', 'kiwi', 'lemon', 'lime', 'mango', 'orange', 'papaya',
'peach', 'pear', 'pineapple', 'plum', 'tangerine', 'watermelon'] 19
```

18. Run the loop again:

```
    for(let i = 0; i < 16; i++) {
       let bean = fruits[i] + ' jellybean';
       console.log(bean);
       // apple jellybean, apricot jellybean, etc.
    }
```

The last three fruits ('plum', 'tangerine', 'watermelon') are not showing up. The problem is, the loop stops when `i=16`, but we now have 19 items.

array.length as loop condition

For a loop that iterates an array, do not use a hard-coded number in the condition. Instead, use the dynamic condition `i < array.length`. This way, the loop fully iterates the array--and then stops.

19. Make a new loop with the dynamic condition `i < fruits.length`:

```
    for(let i = 0; i < fruits.length; i++) {
        let bean = fruits[i] + ' jellybean';
        console.log(bean);
        // apple jellybean, apricot jellybean, etc.
    }
```

string.length

As we recall, the `length` property applies not only to arrays, but to strings, as well.

conditional logic inside a loop

Loops that iterate arrays often include conditional logic to evaluate the individual items. Let's give this a try.

20. Make jellybeans only if the fruit has no more than 5 letters. Btw, we don't really need the bean variable, so just log the string directly:

```
    for(let i = 0; i < fruits.length; i++) {
        if(fruits[i].length <= 5) {
            console.log(fruits[i] + ' jellybean');
            // apple jellybean, grape jellybean, etc.
        }
    }
```

making a new array inside a loop

The above loops are not saving the jellybeans anywhere; the beans are just being "spilled" out onto the console.

21. Do another loop that saves the jellybeans by pushing them into a new array, `jellybeans`, declared above the loop:

```
    const jellybeans = [];

    for(let i = 0; i < fruits.length; i++) {
        if(fruits[i].length <= 5) {
            jellybeans.push(fruits[i] + ' jellybean');
        }
    }

    console.log(jellybeans);
    // ['apple jellybean', 'grape jellybean', 'kiwi jellybean', 'lemon
jellybean, 'lime jellybean', 'mango jellybean', 'peach jellybean', 'pear
jellybean', 'plum jellybean']
```

Let's try some if-else if-else logic in a loop:

- `if` the fruit has a max of 5 letters, it goes into the `lilFruits` array.

- `else if` the fruit has 6-8 letters, it goes into the `medFruits` array.
- `else`, the fruit of 9 or more letters it goes into the `bigFruits` array

22. Declare the three arrays, all empty:

```
    const lilFruits = [];
    const medFruits = [];
    const bigFruits = [];
```

23. Run the loop with the conditional logic. Each part pushes eligible fruits into its respective array:

```
    for(let i = 0; i < fruits.length; i++) {
        if(fruits[i].length <= 5) {
            lilFruits.push(fruits[i]);
        } else if(fruits[i].length <= 8) {
            medFruits.push(fruits[i]);
        } else { // 9+ chars
            bigFruits.push(fruits[i]);
        }
    }

    console.log('lilFruits', lilFruits);
    // ['apple', 'grape', 'kiwi', 'lemon', 'lime', 'mango', 'peach',
'pear', 'plum']
    console.log('medFruits', medFruits);
    // ['apricot', 'banana', 'cherry', 'orange', 'papaya']
    console.log('bigFruits', bigFruits);
    // ['blueberry', 'grapefruit', ' 'pineapple', 'tangerine',
'watermelon']
```

loop with nested if-else

Let's finish the lesson with some nested logic. Building upon what we have so far, if the "big fruit" is a berry, save it to the `berries` array. For this we need a few more berries.

We will use `splice()` to maintain alphabetical order as we add 'boysenberry', 'raspberry' and 'strawberry'. Here is the current array of 19 items for reference:

```
['apple', 'apricot', 'banana', 'blueberry', 'cherry', 'grape',
'grapefruit', 'kiwi', 'lemon', 'lime', 'mango', 'orange', 'papaya',
'peach', 'pear', 'pineapple', 'plum', 'tangerine', 'watermelon']
```

24. We need to put 'boysenberry' needs between 'blueberry' and 'cherry'. Starting at index 4 ('cherry'), remove zero items and add 'boysenberry':

```
    fruits.splice(4, 0, 'boysenberry');
```

splice() with negative index values

The splice method recognizes negative indexes, with the last item at -1. We want to add 'raspberry' and 'strawberry', consecutively, right before 'tangerine', which, as the next to the last item, occupies index -2.

25. Add 'raspberry' and 'strawberry' before 'tangerine':

```
    fruits.splice(-2, 0, 'raspberry', 'strawberry');

    console.log(fruits, fruits.length);
    // ['apple', 'apricot', 'banana', 'blueberry', 'boysenberry',
'cherry', 'grape', 'grapefruit', 'kiwi', 'lemon', 'lime', 'mango',
'orange', 'papaya', 'peach', 'pear', 'pineapple', 'plum', 'raspberry',
'strawberry', 'tangerine', 'watermelon']
```

26. Declare all new empty arrays for a fresh start:

```
    const smFruits = [];
    const mdFruits = [];
    const lgFruits = [];
    const berries = [];
```

27. Do the for loop with nested if-else. To check if a fruit *includes* 'berry', call `includes('berry')` on the item:

```
    for(let i = 0; i < fruits.length; i++) {
        if(fruits[i].length <= 5) {
            smFruits.push(fruits[i]);
        } else if(fruits[i].length <= 8) {
            mdFruits.push(fruits[i]);
        } else { // 9+ chars
            if(fruits[i].includes('berry')) {
                berries.push(fruits[i]);
            } else {
                lgFruits.push(fruits[i]);
            }
        }
    }

    console.log('smFruits', smFruits);
    // ['apple', 'grape', 'kiwi', 'lemon', 'lime', 'mango', 'peach',
'pear', 'plum' ]
    console.log('mdFruits', mdFruits);
    // ['apricot', 'banana', 'cherry', 'orange', 'papaya']
    console.log('lgFruits', lgFruits);
    // ['grapefruit', ' 'pineapple', 'tangerine', 'watermelon']
```

```
    console.log('berries', berries);
    // ['blueberry', 'boysenberry', 'raspberry', 'strawberry']
```

- **END: Lesson 05.01**
- **NEXT: Lab 05.01, Lesson 05.02**



1. Write a for loop that makes the following array:

```
[3, 5, 7, 9, 11, 13, 15, 17, 19, 21]
```

2. Write a for loop that makes the following array:

```
[100, 80, 60, 40, 20, 0, -20, -40, -60, -80, -100]
```

3. Given an array of numbers, **nums**, use a for loop to add up all the numbers. Save the total to a variable, **sum**.

```
let nums = [154, 236, 314, 467, 532, 689, 703];
let sum = 0;
```

4. Given an array of mixed numbers, 'num-like strings' and fruits, add up the nums and 'num-like strings'. This requires you to ignore the fruits and to convert the 'num-like strings' to actual numbers. Hint: You need the **Number()** method for this:

```
const mixedBag = [93, 67, 'banana', '87', '189', 'cherry', 211, 67,
'kiwi'];
```

05.01 Lab Exercises - SOLUTION

LOOPS & ARRAYS

1. Write a for loop that makes the following array:

[3, 5, 7, 9, 11, 13, 15, 17, 19, 21]

Solution explained:

- New, empty array, holds the output
- Counter variable, i, starts at 3
- Loop continues as long as counter i <= 21
- Values are increasing by 2, so we need incrementer: i += 2.

```
const myNums = [];
for(let i = 3; i <= 21; i += 2) {
    myNums.push(i);
}
console.log(myNums);
```

2. Write a for loop that makes the following array:

[100, 80, 60, 40, 20, 0, -20, -40, -60, -80, -100]

Solution explained:

- New, empty array, holds the output
- Counter variable, `i`, starts at 100
- Loop continues as long as counter `i >= -100`
- Values are decreasing by 20, so we need decrementer: `i -= 20`.

```
const numsArr = [];
for(let i = 100; i >= -100; i -= 20) {
    console.log(i);
}
console.log(numsArr);
```

3. Given an array of numbers, use a for loop to add up all the numbers. Save the total to a variable, sum.

```
let nums = [154, 236, 314, 467, 532, 689, 703];
let sum = 0;
```

Solution expained:

- Loop through the array.

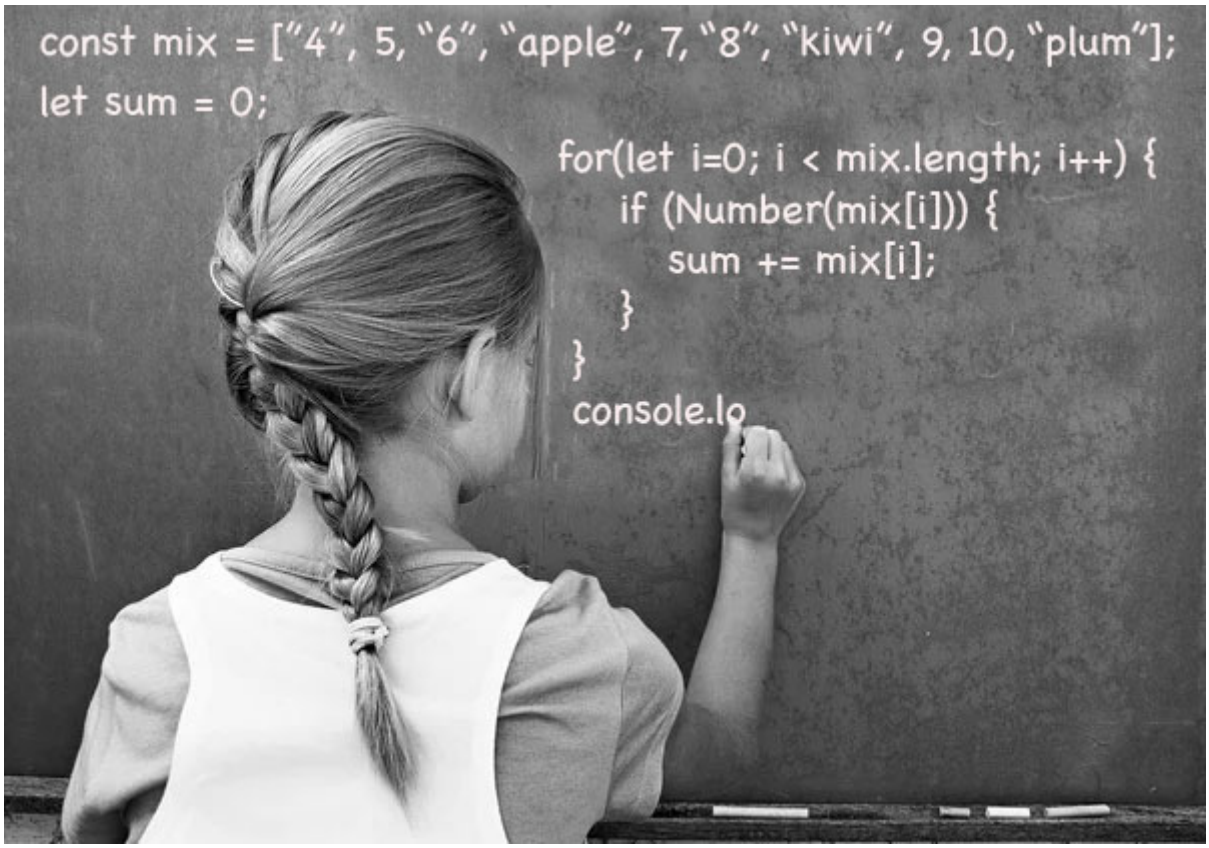
- Each time through, add the current item, nums[i] to sum.

```
for(let i = 0; i < nums.length; i++) {
    sum += nums[i];
}
console.log('nums sum:', sum); // 3095
```

4. Given this array of mixed numbers, 'number-like strings' and fruits, find the sum of the numbers and 'number-like strings'. This requires you to ignore the fruits and to convert the 'number-like strings' to actual numbers.
   Hint: Think Falsey!

```
const mix = [”4”, 5, “6”, “apple”, 7, “8”, “kiwi”, 9, 10, “plum”];
```

Solution explained



- Convert the 'num-like strings' to numbers, so that we can add them.
- Skip the fruits, since they cannot be converted to numbers.
- Add each eligible item as we loop through the array.
- Number('55') returns 55
- Number('apple') returns NaN.
- NaN is falsey, which returns false in a boolean context, e.g. in an if-statement.
- Therefore, pass each item to the Number() method and pass that to an if() statement. Any values that cannot be converted to a number -- these being the fruits -- will return NaN, and so the if-code will

not run.

- Inisde the if, add up values that are NOT NaN.

```
    let tot = 0;

    for(let i = 0; i < mix.length; i++) {
        if(Number(mix[i]) { // NaN is falsey
            tot += mix[i]; // only add truthy values
        }
    }
    console.log('mix tot:', tot);
```

Lesson 05.02

Looping Arrays of Objects

making a video player interface

looping an array of objects

1. Open `05.02-Looping-Arrays-of-Objects.html` and preview it in the browser.

The interface loads a grid of 18 animal images at left and a video player with description underneath, at right. The first animal, 'American bison' appears on page load.

2. Click an animal picture to swap the video and description.

This is the application that we will be building in this lesson. Let's have a look at where all this animal data is located.

3. Open `animals.js` and have a look. The file contains one variable, an array called `animalsArr`.

Each array item is an object of six properties: - `name` ('giraffe', 'ostrich', etc.) - `class` ('mammal', 'bird', 'reptile') - `herbivore` (boolean: true or false) - `continent` ('Asia', 'Africa', etc.) - `youTube` (YouTube video embed code) - `desc` (description of the animal)

4. Switch to the lesson file's JS file, `05.02-Looping-Arrays-of-Objects-START.js`.

To make the application, we will loop through the animals array:

- Each time through the loop, we will concatenate an `img` tag.
- The `img` tag will have an `onclick` event that calls a function called `swapImage(i)`.
- The `i` argument is the index of the current item.
- The argument `i` tells the function which video and description it needs to load.
- The YouTube video is embedded with an `iframe` tag, which is the same for each video, except for an 11-character code specific to that particular video.

5. First, get all the elements that we need for the output:

- `animal-pics` div for the grid of animal pics
- `video-player` div for holding the video
- `h2` for displaying the animal name
- `p` for displaying the description.

```
// get animals pic div to hold images
let animalPicsDiv = document.getElementById('animal-pics');

// get video player div
let videoPlayerDiv = document.getElementById('video-player');

// get h2 for animal name
let h2 = document.querySelector('h2');
h2.textContent = animalsArr[0].name;
```

```
    // get p tag for description
    let p = document.querySelector('p');
    p.textContent = animalsArr[0].desc;
```

6. Output the name and description of the first animal immediately on page load.

- the first array item is `animalsArr[0]`
- each array item is an object, so the name of the first animal is `animalsArr[0].name`

```
    h2.textContent = animalsArr[0].name;
    p.textContent = animalsArr[0].desc;
```

7. Display the video for the first animal.

- the `iframe` for each video is identical, except for an 11-character code
- the code is stored as the object's `youTube` property
- we access the property with `animalsArr[0].youTube` and concatenate that into the `iframe`:

```
    // display video for first animal
    videoPlayerDiv.innerHTML = '<iframe width="560" height="315"
src="https://www.youtube.com/embed/' + animalsArr[0].youTube + '"
title="YouTube video player" frameborder="0" allow="accelerometer;
autoplay clipboard-write; encrypted-media; gyroscope; picture-in-picture"
allowfullscreen></iframe>';
```

8. Make sure the html page is using START.js and reload the page in the browser. We should have the video and description at right, and an empty box at left.

Now, to output the animal images to the `animal-pics` div, the empty box at left. Each image needs to be clickable to call a function to swap the video and description.

9. Set up a loop that iterates `animalsArr`. First thing to do in the loop is to save the current object to a variable:

```
    for (let i = 0; i < animalsArr.length; i++) {
        let animal = animalsArr[i];
    }
```

10. Open the `images` folder. Notice that the two-word file names are hyphenated, whereas the animal names in the data have spaces--not hyphens:

- FILE: american-bison.jpg, andean-bear.jpg
- DATA: 'American bison', 'Andean bear'

We will use object names ('American bison') to concatenate image file paths, so we need to replace the spaces with hyphens.

11. Using backticks, concatenate an entire `img` tag, using `${name}` to add the file name, dynamically:

```
    for (let i = 0; i < animalsArr.length; i++) {
        let animal = animalsArr[i];
        let name = animal.name.replace(' ', '-');
        let tag = `<img src="images/${name}.jpg">`;
    }
```

12. We need each tag to be clickable to call the `swapAnimal()` function, so add that to the tag as an `onclick` event handler:

```
    for (let i = 0; i < animalsArr.length; i++) {
        let animal = animalsArr[i];
        let name = animal.name.replace(' ', '-');
        let tag = `<img src="images/${name}.jpg" onclick="swapAnimal()">`;
    }
```

13. The function needs to know which animal was clicked, so pass in the index, `i`, as the argument.

```
    for (let i = 0; i < animalsArr.length; i++) {
        let animal = animalsArr[i];
        let name = animal.name.replace(' ', '-');
        let tag = `<img src="images/${name}.jpg"
onclick="swapAnimal(${i})">`;
        animalPicsDiv.innerHTML += tag;
    }
```

14. Output the `img` tag. Since this is an html tag, set the `innerHTML` rather than the `textContent`:

```
    for (let i = 0; i < animalsArr.length; i++) {
        let animal = animalsArr[i];
        let name = animal.name.replace(' ', '-');
        let tag = `<img src="images/${name}.jpg"
onclick="swapAnimal(${i})">`;
        animalPicsDiv.innerHTML += tag;
    }
```

15. Save and reload the page; the images should all be there although clicking them doesn't work since we have yet to write the `swapAnimal()` function. That's next.

16. Define the function. It has a parameter `i` which comes in when the function is called as the index of the clicked animal.

```
    function function swapAnimal(i) {
    }
```

16. Using the `i` argument, look up the animal in the array and set the heading and description to that animal's `name` and `desc` properties, respectively:

```
    function function swapAnimal(i) {
        h2.textContent = animalsArr[i].name;
        p.textContent = animalsArr[i].desc;
    }
```

17. Now for the big `iframe` tag for the video. Just slot in the chosen animal's 11-digit YouTube video code, which we access as `animalsArr[i].youTube`:

```
    function swapAnimal(i) {
        h2.textContent = animalsArr[i].name;
        p.textContent = animalsArr[i].desc;

        videoPlayerDiv.innerHTML = '<iframe width="560" height="315"
src="https://www.youtube.com/embed/' +
        animalsArr[i].youTube + '" title="YouTube video player"
frameborder="0" allow="accelerometer; autoplay; clipboard-write;
encrypted-media; gyroscope; picture-in-picture" allowfullscreen>
</iframe>';
    }
```

18. Save and reload the page. Click an animal to load its video and description.

END: Lesson 05.02 NEXT: Lesson 05.03



while loops

do while loops

break

nested for loops

while loop**

A while loop has the essential three ingredients found in a for loop: a counter, a condition and an incrementer. The difference is that in a for loop, these three things are all neatly bundled in parentheses. In a while loop, only the condition is inside the parentheses. The counter is actually declared *outisde* the loop, above it in the global scope; the counter is incremented *inside* the loop's curly braces.

1. Write this while loop:

```
let n = 0;
while (n <= 10) {
    console.log(n);
    n++;
}
console.log(n);
```

do while loop

In a "do while loop", the condition is evaluated only *after* the loop has already run, and so it must run at least once.

2. Write this 'do while' loop:

```
let num = 0;
do {
    console.log(num);
    num++;
} while(num <= 10);
```

Overall, while and do while loops are pretty similar:

- both declare the counter *outside (before)* the loop

- both increment the counter *inside* the loop

3. Write these while and do while loops that concatenate the counter, resulting in output of 1, 12, 123, 1234, etc:

```
let i = 0, str = "";

while (i <= 10) {
    str += i;
    i++;
    console.log(str);
}

str = "";
i = 10;

do {
    str += i;
    i--;
    console.log(str);
} while (i >= 1);
```

break

Recall that **continue** skips an iteration of a loop. The keyword **break** exits the loop.

4. Write a while loop that ends the loop when the counter gets to 7:

```
let x = 0;
while(x <= 10) {
    if(x == 7) {
        console.log('break');
        break;
    }
    console.log('x', x);
    x++;
}
```

`break` is good for ending a loop once we have found something in an array. After all, if we found what we were looking for, there's no need to check the other items.

5. Write a while loop that iterates an array, looking for a number divisible by 7:

- *if* the number is found, save it to `target` and `break`.
- *if* no number divisible by 7 is found, set the index to -1.
- when it's all over, log the `target` and its index.

```
let nums = [53, 37, 123, 88, 112, 136, 155];
let indx = 0, target = 0;
while(indx <= nums.length) {
    if(nums[indx] % 7 == 0) {
        target = nums[indx];
        console.log('break');
        break;
    }
    indx++;
}
if (!target) indx = -1;
console.log('target', target, 'index', indx);
```

Recall that 0 is falsey, meaning that it returns false in an if-statement. So, just to be fancy, we're using `if(!target)` rather than `if(target == 0)` -- and putting it all on one line with no curly braces.

6. Change the array values so that the loop runs with and without finding a number divisible by 7.

nested loops:

A **nested loop** is a loop-inside-a-loop. Each time the outer loop runs once, it iterates over the entire inner loop. So, if each loop is running ten times, the total is 10x10 = 100 iterations.

7. Run a nested loop where we output the values of both inner and outer counter as we go:

```
for (let i = 1; i <= 10; i++) {
    for (let j = 1; j <= 10; j++) {
        console.log('i:', i, 'j:', j);
    }
}
```

This next one involves an array of city abbreviations. Once again, we will use a nested loop. Both loops will iterate over the same array. The inner loop will use the outer and inner counters, **i** and **j**, respectively, to concatenate a pair of cities. The names will be hyphenated, like departures-arrivals: NY-MIA

8. Declare an array of city abbreviations:

```
const cities = ['CHI', 'NY', 'PHI', 'BOS', 'MIA', 'LA', 'SF'];
```

We will iterate the array with a for loop and console log all possible pairings of cities. We need a nested loop for this, since each city needs to be paired with every other city.

9. Pair each city with every other city--including itself:

```
for(let i = 0; i < cities.length; i++) {
    for(let j = 0; j < cities.length; j++) {
```

```
            console.log(cities[i] + '-' + cities[j]);
        }
    }
```

10. Use conditional logic to only pair cities when `i` and `j` are not equal:

```
    for(let i = 0; i < cities.length-1; i++) {
        for(let j = 0; j < cities.length; j++) {
            if(i != j) { // don't pair a city with itself
                console.log(cities[i] + '-' + cities[j]);
            }
        }
    }
```

What if we didn't want any two-word combo to repeat, not even in the opposite direction. So, MIA-NY means no NY-MIA. For flights this may not be appropriate, because planes fly in both directions, but for some pairings, it may be best to only have the one pairing.

The challenge is to "make smoothies" as pairs of fruits. We want all possible combinations, but without any repeats in reverse, so "banana-peach" is good, but "peach-banana" is bad.

11. Declare an array of fruits:

```
    const fruits = ['apple', 'apricot', 'banana', 'blueberry', 'cherry',
'kiwi', 'mango', 'orange', 'peach', 'pear', 'plum', 'raspberry'];
```

Start with a rough version that gives you all 144 possible pairings--the 12x12 max, including self-pairs like "apple-apple", and revserve pairs like "apple-apricot" and "apricot-apple"). Include a counter that we increment with each pair, so that we can know if the total is going down as we proceed:

12. Make all 144 possible 2-fruit combinations, and keep count:

```
    let counter = 1;

    for(let i = 0; i < fruits.length; i++) {
        for(let j = 0; j < fruits.length; j++) {
            console.log(counter, fruits[i] + '-' + fruits[j]);
            counter++;
        }
    }
```

13. Reset the counter, and run a new loop, where you only get 132 pairs, because the self-pairings ("apple-apple") have been eliminated. This is done with conditional logic: we only make the pair if `i` and `j` are *not* equal:

```
counter = 1;

for(let i = 0; i < fruits.length; i++) {
    for(let j = 0; j < fruits.length; j++) {
        if(i != j) { // don't pair a fruit with itself
            console.log(counter, fruits[i] + '-' + fruits[j]);
            counter++;
        }
    }
}
```

Upgrade again, this time to the final version:

- Each time through the loop, set the just-finished item to "done".
- Add another condition to the if-statement requiring that the current fruit has not yet been 'done'.
- The result is pairs that do not repeat in reverse. We get "apple-banana", but not "banana-apple".
- Save the pairs to a smoothies array, rather than just logging them to the console.

14. Write the final version of the "smoothies maker", in accordance with the above guidelines:

```
counter = 1;
const smoothies = [];

for(let i = 0; i < fruits.length; i++) {
    for(let j = 0; j < fruits.length; j++) {
        // don't pair a fruit with itself or reuse a 'done' fruit
        if(i != j && fruits[j] != 'done') {
            smoothies.push(fruits[i] + '-' + fruits[j]);
            counter++;
        }
    }
    fruits[i] = 'done'; // replace the done fruit with 'done'
}
console.log('2-fruit smoothies', smoothies);
```

15. Make it into a function that takes the array as its argument, runs the "smoothie maker" code and, in good input-output fashion, returns the smoothies:

```
function makeSmoothies(arr) {

    counter = 1;
    const smoothesArr = [];

    for(let i = 0; i < arr.length; i++) {
        for(let j = 0; j < arr.length; j++) {
            // don't pair item w itself or reuse 'done'
            if(i != j && arr[j] != 'done') {
                smoothesArr.push(arr[i] + '-' + arr[j]);
```

```
                    counter++;
                }
            }
            arr[i] = 'done'; // replace done item with 'done'
        }

        return smoothesArr;

    }
```

16. Log fruits to see that all its items have been replaced with 'done':

```
    console.log('fruits', fruits, fruits.length);
```

17. Redeclare fruits under a new name:

```
    const froots = ['apple', 'apricot', 'banana', 'blueberry', 'cherry',
'kiwi', 'mango', 'orange', 'peach', 'pear', 'plum', 'raspberry'];
```

18. Declare an array of tropical fruit, so that we can test the function on more than just one array:

```
    const tropicalFruits = ['banana', 'pineapple', 'kiwi', 'mango',
'mangosteen', 'cherimoya', 'acai', 'noni', 'papaya', 'dragon fruit',
'passion fruit', 'guava'];
```

19. Call the function twice, passing in a different array each time. Save the function call to a variable so that you capture the return value (the smoothies):

```
    let smoothies1 = makeSmoothies(froots);
    console.log('froot smoothies', smoothies1);

    let smoothies2 = makeSmoothies(tropicalFruits);
    console.log('tropical smoothies', smoothies2);
```

- **End Lesson 05.03**
- **Next: Lab 05.03, Lesson: 05.04**



Nested Loops to make Deck of Cards

nested loop to make a deck of playing cards

In this lesson we will use a nested loop to make a deck of cards from which we will deal a 5-card hand of poker.

1. Open **05.04-Deck-of-Cards.html** and see that it has a div of five image tags, under which is a DEAL CARDS button:

```
    <div id="card-box">
        <img src="images/10-of-Diamonds.png">
        <img src="images/Jack-of-Diamonds.png">
        <img src="images/Queen-of-Diamonds.png">
        <img src="images/King-of-Diamonds.png">
        <img src="images/Ace-of-Diamonds.png">
    </div>

    <button>DEAL CARDS</button>
```

2. Preview the html file in the browser. It's a 5-card hand of poker--a royal flush, no less.
3. Click the DEAL CARDS button. Five random cards replace the royal flush.
4. Open the **images** folder and click a card to see it. Each card has two identifiers: a **kind** and a **suit** for a file name structure of **kind-of-suit.png**:

- 2-of-Clubs.png - Queen-of-Spades.png

5. Switch the html file to use START .js.
6. Open **05.04-Deck-of-Cards-START.js**. The **kind** and **suit** arrays are given, but we also need a new empty array to store the cards.
7. Declare a new empty array, called **deck**:

```
    const kinds = [2, 3, 4, 5, 6, 7, 8, 9, 10, 'Jack', 'Queen', 'King',
'Ace'];
    const suits = ['Diamonds', 'Hearts', 'Spades', 'Clubs'];
    const deck = [];
```

We will begin by just making a deck of file names. Once we get that to work, we will upgrade the deck so that each card is an object with several properties.

8. Set up a nested loop, where the outer loop iterates over the **kinds** array, and the inner loop iterates over **suits**:

```
    for (let i = 0; i < kinds.length; i++) {
        for (let j = 0; j < suits.length; j++) {
        }
    }
```

The outer loop runs 13 times, and the inner loop runs four times, for a total of 52 iterations, one for each card.

9. In the inner loop, concatenate the card file name, and push the result into the deck:

```
        let cardFileName = `${kinds[i]}-of-${suits[j]}.png`;
        deck.push(cardFileName);
```

10. After the loop ends, log the deck to see what we get.

```
    console.log(deck);
    /*
    ['2-of-Diamonds.png', '2-of-Hearts.png', '2-of-Spades.png',
    '2-of-Clubs.png', '3-of-Diamonds.png', '3-of-Hearts.png',
        -- ETC. --
    'King-of-Spades.png', 'King-of-Clubs.png', 'Ace-of-Diamonds.png',
'Ace-of-Hearts.png', 'Ace-of-Spades.png', 'Ace-of-Clubs.png']
    */
```

deck of cards as an array of objects

Let's run the nested loop again, but this time we will make each card an object, in which the file name is just one of several properties. The five properties will be:

- **name** ("2 of Diamonds", etc.)
- **file** ("2-of-Diamonds.png", etc.)
- **kind** (2, 3, 4..Jack, Queen, King, Ace)
- **suit** (Diamonds', 'Hearts', 'Spades', 'Clubs')
- **valu** (numeric value; face cards = 10, Ace = 11)

valu is for storing the numeric value of each card, from 1-11, using the blackjack scoring system, where aces start with a value of 11, face cards are worth 10 and the numbered cards are their respective numeric values.

11. Declare a new, empty array to hold our 52 card objects.

```
const deckOfCards = [];
```

12. Set up the nested loop;

```
for (let i = 0; i < kinds.length; i++) {
    for (let j = 0; j < suits.length; j++) {
    }
}
```

13. Simplify the current array items by passing them to variables:

```
for (let i = 0; i < kinds.length; i++) {
    for (let j = 0; j < suits.length; j++) {
        let kind = kinds[i];
        let suit = suits[j];
    }
}
```

14. Concatenate the card and image file names. The "Queen of Diamonds" has a file name of "Queen-of-Diamonds.png":

```
        let kind = kinds[i];
        let suit = suits[j];
        let name = `${kind} of ${suit}`;
        let file = `${kind}-of-${suit}.png`;
```

15. Declare a variable, **valu**, with an inital value of 0:

```
        let name = `${kind} of ${suit}`;
        let file = `${kind}-of-${suit}.png`;
        let valu = 0;
```

16. Run conditional logic to set the **valu** property. Only "Jack", "Queen" and "King" have more than three characters, which is what **(kind.length > 3** checks for:

```
        let valu = 0;
        if(kind == 'Ace')  {
            valu = 11;
        } else if(kind.length > 3) {
            valu = 10;
        } else {
```

```
                valu = kind;
            }
```

17. Declare an object called **card**. Its properties equal the variables we have made:

```
            let card = { name: name, file: file, kind: kind,
                         suit: suit, valu: valu };
```

18. Push the card object into the **deckOfCards** array:

```
            deckOfCards.push(card);
```

19. Below the nested loop, output the array of 52 objects:

```
    console.log(deckOfCards);
    /*
    0: {name: '2 of Diamonds', file: '2-of-Diamonds.png', kind: 2, suit:
'Diamonds', valu: 2}
    51: {name: 'Ace of Clubs', file: 'Ace-of-Clubs.png', kind: 'Ace',
suit: 'Clubs', valu: 11}
    */
```

ouputting cards to the DOM

Let's deal a hand of 5-card poker. Clicking the DEAL CARDS button will:

- call a function called **dealCards**
- the function will get all five of the card img tags from the DOM
- the function will run a loop five times, once for each card image
- each iteration of the loop will generate a random int from 0-51
- the random integer will be used to get a card by its array index
- the item's file name will be concatenated into an image path
- the img object will have its src property set to the image path

This will be the simplest implementation possible, meaning that there will only be one player hand dealt and all five cards will appear at once, without the benefit of a realistic-looking time delay between cards.

20. Get the button and tell it to run the function when clicked:

```
    const btn = document.querySelector('button');
    btn.addEventListener('click', dealCards);
```

21. Get the five images. The **querySelectorAll** method will get all img tags and make an array of them called a Node List:

```
const imgArr = document.querySelectorAll('img');
```

22. Now for the function, which loops through the array of images:

```
function dealCards() {
    for (let i = 0; i < imgArr.length; i++) {
    }
}
```

23. Each time the loop runs, generate a random number from 0-51:

```
function dealCard() {
    for (let i = 0; i < imgArr.length; i++) {
        let r = Math.floor(Math.random() * deckOfCards.length);
    }
}
```

24. Get a random card from the deck by index:

```
    for (let i = 0; i < imgArr.length; i++) {
        let r = Math.floor(Math.random() * deckOfCards.length);
        let card = deckOfCards[r];
    }
```

25. Set the src of the image in the imgArray by concatenating the file pate using the file property of the card object:

```
        let r = Math.floor(Math.random() * deckOfCards.length);
        let card = deckOfCards[r];
        imgArr[i].src = "images/" + card.file;
```

26. Reload the page and click the DEAL CARDS button. Five new cards should appear.

If you keep clicking the DEAL CARDS button, you will notice a bug: the same card showing up more than once in the same hand. To fix this, remove cards from the deck as they are dealt.

27. Using the splice() method, remove the dealt card from the deck:

```
            let card = deckOfCards[r];
            imgArr[i].src = "images/" + card.file;
            deckOfCards.splice(r, 1);
```

Now we have a new bug: after 10 hands we have used 50 cards and are down to our last two. To fix this, we can use conditional logic to replenish the deck once the supply runs too low. To do this, we need to make a "backup" -- a copy -- of the original deck.

28. Outside of the function, copy the array using **slice(0)**, which returns a copy from index 0 to the end of the array:

```
    let deckCopy = deckOfCards.slice(0);
```

29. Comment out the dealCards function, and rewrite it. Start with an if statement that makes a fresh copy if there are fewer than 5 cards left:

```
    function dealCard() {

        if(deckCopy.length < 5) {
            deckCopy = deckOfCards.slice(0);
        }

    }
```

30. Switch to using deckCopy throughout; after each hand, log how many cards are left, so you can see when the card supply is replenished:

31. Rerun the page. Now you should be able to deal hands indefinitely, without ever running out of cards.

END: Lesson 05.04 NEXT: Lab 05.04 Lesson 05.05



UNIT 05

LESSON 05.05



Passing Arrays to Functions

In this lesson, we will be writing functions that take arrays as their arguments.

function #1: add article in front of word

The first function will take an array of words and add the article "a" or "an" in front of each word. If the word starts with "a", "e", "i" or "o", it should start with "an". The final product ("an apple", "a banana", etc.) will be saved to a new array.

The idea is that we can pass in any array of strings and the function will produce the same result, so we start by declaring two arrays:

1. Declare two arrays of strings:

```
    const fruits = ['apple', 'apricot', 'banana', 'blueberry',
'blueberry', 'cherry', 'elderberry', 'grape', 'kiwi', 'lemon', 'lime',
'mango', 'orange', 'papaya', 'peach', 'pear', 'pineapple', 'plum',
'raspberry', 'strawberry', 'tangerine', 'watermelon'];

    const cars = ['Acura', 'Alfa Romeo', 'Audi', 'BMW', 'Chevrolet',
'Dodge', 'Edsel', 'Eagle', 'Fiat', 'Ford', 'Honda', 'Kia', 'Infiniti',
'Isuzu', 'Mazda', 'Nissan', 'Opel', 'Porsche', 'Rolls Royce', 'Tesla',
'Toyota', 'Volvo'];
```

2. Declare the function and assign it a parameter, **arr**:

```
    function addArticle(arr) {
    }
```

3. The function needs a few variables:

- an empty array to hold the results
- a variable set equal to the article followed by a space
- a variable for checking if a word starts with a, e, i, o

```

```

```
function addArticle(arr) {
    let newArr = [];
    let article = 'a ';
    let vowels = 'aeio';
}
```

4. The function needs to check each item in the array, so set up a loop that iterates the array:

```
function addArticle(arr) {
    let newArr = [];
    let article = 'a ';
    let vowels = 'aeio';

    for(let i = 0; i < arr.length; i++) {
    }
}
```

5. Each time through the loop, save the current item, as well as the first letter of that item:

```
function addArticle(arr) {
    let newArr = [];
    let article = 'a ';
    let vowels = 'aeio';

    for(let i = 0; i < arr.length; i++) {
        let word = arr[i]; // apple
        let firstChar = word[0]; // a
    }

}
```

6. Add an if-statement that uses the **string.includes()** method to check if the first character, set to lowercase, is "a", "e", "i" or "u". If it is, set the value of **article** to "an":

```
    for(let i = 0; i < arr.length; i++>) {
        let word = arr[0]; // apple
        let firstChar = word[0]; // a
        if(vowels.includes(firstChar)) {
            article = 'an ';
        }
    }
```

7. Just for practice, comment out the if-else and switch to a ternary expression:

```
vowels.includes(firstChar) ? article='an ' : article='a ';
```

8. Concatenate **article** with the original word the original word, **arr[i]**, which may be capitalized. Push it into newArr, and return it:

```
    for(let i = 0; i < arr.length; i++>) {
        let word = arr[0]; // apple
        let firstChar = word[0]; // a
        if(vowels.includes(firstChar)) {
            article = "an ";
        }
        newArr.push(article + word);
    }
    return newArr;
```

Be sure to push *inside* the loop and return *outside* the loop. If the **return** is inside the loop, the loop will only run once, because **return** always ends a function.

9. Call the function twice, passing in a different array of strings each time. Save the return values to variables and log them:

```
let fruitsArr = addArticle(fruits);
console.log(fruitsArr);

let carsArr = addArticle(cars);
console.log(carsArr);
```

function #2: find target number in array

This next function has not just an array parameter, but also a number param. The function loops through the array, looking for the number.

- If the number is found, the function returns the index.
- If the number is NOT found, the function returns -1.

10. Declare an array of numbers, followed by a function:

```
const numsArr = [5, 7, 9, 12, 14, 16, 20, 25, 30, 40, 50];
```

11. Write the function with two parameters.

```
function findNum(arr, num) {
}
```

12. Iterate the array with a for loop:

```
function findNum(arr, num) {
    for(let i = 0; i < arr.length; i++) {
    }
}
```

13. Check if the current array item equals the target number, and if it does, return the index:

```
function findNum(arr, num) {
    for(let i = 0; i < arr.length; i++) {
        if(arr[i] === num) {
            return i;
        }
    }
}
```

14. If the loop ends without finding the target number, return -1:

```
function findNum(arr, num) {
    for(let i = 0; i < arr.length; i++) {
        if(arr[i] === num) {
            return i;
        }
    }
    return -1;
}
```

15. Call the function. The function returns a value, so set the call equal to a variable to "capture" the return value:

```
let find9 = findNum(numsArr, 12);
console.log(find9);  // 3

let find8 = findNum(numsArr, 123);
console.log(find8);  // -1
```

16. Instead of just returning the index, let's return a string, specifying the number being looked for and where it was found. Do this as a new function:

```
function findNumber(arr, num) {
    for(let i = 0; i < arr.length; i++) {
        if(num === arr[i]) {
            return `${num} found at index ${i}`;
```

```
            }
        }
        return `${num} not found`;
    }

    let find25 = findNumber(numsArr, 25);
    console.log(find25);  // 25 found at index 7

    let find250 = findNumber(numsArr, 250);
    console.log(find250);  // 250 not found
```

function #3: check array to see if all numbers are even

Let's try one more function that takes an array for its argument. The function checks each number to see if it is odd or even.

- If all numbers are even, the function returns true.
- If an odd number or float are found, the function returns false.

17. Declare three arrays of numbers:

- The first array contains only even numbers.
- The second array contains an odd number; the rest are even.
- The third array contains a float; the rest are even integers.

```
    let nums1 = [2, 4, 6, 8, 10];
    let nums2 = [2, 4, 7, 8, 10];
    let nums3 = [2, 4, 6.78, 10];
```

18. Write a function that expects an array to be passed to it:

```
    function allEven(arr) {
    }
```

19. Have the function iterate over the array:

```
    function allEven(arr) {
        for(let i = 0; i < arr.length; i++) {
        }
    }
```

20. Each time through the loop, run an if statement that uses the modulo operator **%** to divide the current number by 2. If the reminder is *not equal to* **!=** 0, the number is *not* even, the number is *not* even, so return false:

```
function allEven(arr) {
    for(let i = 0; i < arr.length; i++) {
        if(arr[i] % 2 != 0) {
            return false;
        }
    }
}
```

21. If the loop ends without finding a non-even number, all numbers in the array must be even, so return true:

```
function allEven(arr) {

    for(let i = 0; i < arr.length; i++) {
        if(arr[i] % 2 != 0) { // if num is not even
            return false;
        }
    }
    return true;
}
```

22. Call the function three times, once for each array; save the return value to a variable and log it:

```
let allEven1 = allEven(nums);
console.log('allEven1', allEven1); // true

let allEven2 = allEven(nums);
console.log('allEven2', allEven1); // false

let allEven3 = allEven(nums);
console.log('allEven3', allEven1); // false
```

END Lesson 05.05 NEXT: Lab 05.05 Lesson 05.06



Classic Programming Job Interview Challenges:

A. Fizz-Buzz

B. Find nth Fibonacci Number

C. Check for duplicates in array

A. Fizz-Buzz

Output to the console all integers from 1-100 along with "Fizz" and/or "Buzz" according to the following rules:

- If the number is evenly divisible by both 3 and 5, output the number followed by ' Fizz-Buzz'.
- If the number is evenly divisible by 3 (but not 5), output the number followed by ' Fizz'.
- If the number is evenly divisible by 5 (but not 3), output the number followed by ' Buzz'.
- If the number is divisible by neither 3 nor 5, output just the number.

One aim of the challenge is to see if the candidate figures out that the **modulo** operator is the key to the solution. Before tackling "Fizz-Buzz", let's try an example of modulo (%), which returns the remainder of the first number divided by the second number:

```
let remainder = 17 % 5;
console.log('remainder', remainder); // 2
```

And now for "Fizz-Buzz"...

1. Set up a for loop that goes from 1-100:

```
for(let i = 1; i <= 100; i++) {
}
```

Another aim of the challenge is to see if the candidate can figure out to start by checking if the number is divisible by both and and 5, as opposed to starting by seeing if it's divisible by one or the other. This test requires the **&&** (AND) operator.

2. Use the **&&** operator to see if the current number yields a remainder of 0 when divided by 3 and 5:

```
    for(let i = 1; i <= 100; i++) {
        if(i % 3 == 0 && i % 5 == 0) {
            console.log(i + ' Fizz BUzz');
        }
    }
```

3. If the number is not divisible by both 3 and 5, check if it's divisible by 3 only. If it is, log the number followed by 'Fizz':

```
    for(let i = 1; i <= 100; i++) {
        if(i % 3 == 0 && i % 5 == 0) {
            console.log(i + ' Fizz BUzz');
        } else if(i % 3 == 0) {
            console.log(i + ' Fizz');
        }
    }
```

4. Add another *else if* to see if the number is divisible by just 5. If it is, log the number followed by 'Buzz':

```
for(let i = 1; i <= 100; i++) {
    if(i % 3 == 0 && i % 5 == 0) {
        console.log(i + ' Fizz BUzz');
    } else if(i % 3 == 0) {
        console.log(i + ' Fizz');
    } else if(i % 5 == 0) {
        console.log(i + ' Buzz');
    }
}
```

5. Finally, if the number is divisible by neither 3 nor 5, log just the number:

```
for(let i = 1; i <= 100; i++) {
    if(i % 3 == 0 && i % 5 == 0) {
        console.log(i + ' Fizz BUzz');
    } else if(i % 3 == 0) {
        console.log(i + ' Fizz');
    } else if(i % 5 == 0) {
        console.log(i + ' Buzz');
    } else {
        console.log(i);
    }
}
```

B. Find nth Fibonacci number

Another popular coding challenge is to produce a sequence of Fibonacci numbers. Part of the point is to see if the candidate even knows what a Fibonacci sequence is. The wording can vary, but is typically something like:

"Find the 20th number in the Fibonacci sequence."

In a Fibonacci sequence, each number is the sum of the previous two. A starter array **[0, 1]** is usually given. The sequence continues as: **[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, etc.]**.

In programming challenges, it is often required that solutions be implemented as a function, with any givens provided as arguments. Our function will be called **makeFibos** and will take two arguments: the starter array and **n**, the target, which is this case is 20.

The **makeFibos(arr, n)** function will do the following:

- iterate the array, which to start off is just **[0, 1]**; each iteration adds a new item to keep the loop going
- our loop runs **n-2** (18) times, because we have a 2-item head start
- each time the loop runs, get the current item: **fibos[i]**
- we also need the next item: **fibos[i+1]**, which is also the last item
- add the numbers: **fibos[i] + fibos[i+1]**. The sum is the next number in the sequence.
- **push()** the sum into the array. The array just got longer by 1, so the loop can run one more time. This repeats until the loop has run **n-2** (18) times.
- when the loop is over, log the last item of the 20-item array, which is the item at index **n-1** (19).

6. Start by declaring an array, **fibosArr**, containing the two starter values. Also declare a number variable to store the target, which is 20:

```
const fibosArr = [0, 1];
let n = 20;
```

In the interview challenge, the givens will not likely be provided as variables; you have to declare them, as we just did.

7. Set up the function with parameters **arr** and **n**:

```
function makeFibos(fibos, n) {
}
```

8. Set up a loop that runs **n-2** times:

```
function makeFibos(fibos, n) {
    for(let i = 0; i < n-2; i++) {
    }
}
```

9. Add the current and next item, which are the last two numbers: **fibos[i] + fibos[i+1]**. Push the sum into the array:

```
function makeFibos(fibos, n) {
    for(let i = 0; i < n-2; i++) {
        let nextFibo = fibos[i] + fibos[i+1];
        fibos.push(nextFibo);
    }
}
```

10. Return the answer. Console log the full array inside the function as a test, and return the last item, which is the nth (20th) item:

```
function makeFibos(fibos, n) {
    for(let i = 0; i < n-2; i++) {
        let nextFibo = fibos[i] + fibos[i+1];
        fibos.push(nextFibo);
    }
    console.log(fibos);
    return fibos[fibos.length-1];
}
```

11. Call the function, saving the call to a variable which captures the return value:

```
let fibo20th = makeFibos(fibosArr, n);
```

12. Log the return value, which is the 20th fibo:

```
console.log(fibo20th);
```

One refinement: to make the loop more dynamic, we should not hard-code **n-2** (18). After all, what if the provided starter array is **[0, 1, 1, 2]**? Then we would need to run the loop **n-4** (16) times to get to the 20th item. The answer is to save **n-arr.length** to a variable above the loop.

13. Above the loop, save the target **n** minus the length of the array argument to a variable, **loopTimes**. This lets the starter array can have more than two items:

```
function makeFibos(fibos, n) {
    let loopTimes = n-fibos.length;
    for(let i = 0; i < loopTimes; i++) {
        let nextFibo = fibos[i] + fibos[i+1];
        fibos.push(nextFibo);
    }
```

```
        console.log(fibos);
        return fibos[fibos.length-1];
    }
```

14. Make a new starter array with a few more "fibos" in it:

```
    const fibos = [0, 1];
    let n = 20;
```

15. Call the function, passing in the five-item starter array, saving the return value to a new variable:

```
    let twentiethFibo = makeFibos(fiveFibos, n);
    console.log(twentiethFibo);
```

C. Find duplicate items in an array

"Given an array of numbers or strings, find all duplicates and return it/them as a new array containing one instance of each duplicate. If there are no duplicates, return 'none'"

The solution function takes an array as its argument. An example array may be given, but make sure your function works on any array of numbers or strings.

16. Assuming **nums1** is given, declare three more arrays, to give us more testing scenarios:

```
    const nums1 = [2, 4, 5, 7, 8, 9, 4, 11, 6, 8, 7, 9, 11, 6, 7];
    const nums2 = [2, 4, 5, 7, 1, 9, 11, 3, 6, 8, 10];
    const fruits1 = ['apple', 'banana', 'apple', 'cherry', 'kiwi',
'banana', 'mango', 'pear', 'peach', 'kiwi', 'grape'];
    const fruits2 = ['apple', 'banana', 'cherry', 'mango'];
```

17. Define a function that takes an array as its argument:

```
    function findDuplicates(arr) {
    }
```

18. Iterate the array:

```
    function findDuplicates(arr) {
        for(let i = 0; i < arr.length; i++) {
        }
    }
```

The key to the solution is to keep track of which values have come up so far. We can do this by saving each unique item as a key of an object, which we declare inside the function.

- check if the current array item is already an object key.
- if the current item is already a key, it is a duplicate.
- if the current item is not yet a key, make a key for it. The syntax uses square brackets for both array index and dynamic object property accessor.

19. Define an object above the loop. Inside the loop, check if the current array item is **not !** a key. If it is not, make a key for that item:

```
function findDuplicates(arr) {
    let obj = {};
    for(let i = 0; i < arr.length; i++) {
        if(!obj[arr[i]]) {
            obj[arr[i]] = arr[i];
        }
    }
}
```

20. But if the current array item, **arr[i]** is already a key, we have a duplicate item, so push it into the array of duplicates. Also, declare a new empty array, **dups**, above the loop for storing the duplicates:

```
function findDuplicates(arr) {
    let obj = {};
    let dups = [];
    for(let i = 0; i < arr.length; i++) {
        if(!obj[arr[i]]) {
            obj[arr[i]] = arr[i];
        } else {
            dups.push(arr[i]);
        }
    }
}
```

avoid looking up array items again and again

Here's a refinement that interviewers will be sure to appreciate. The function has to repeatedly look up the value of the current array item (**arr[i]**). More memory-efficient would be to save **arr[i]** to a variable and just refer to the variable. This also makes the code cleaner by avoiding nested square brackets: **obj[arr[i]]**.

21. Above the if-else, save arr[i] to a variable::

```
function findDuplicates(arr) {
    let obj = {};
    let dups = [];
    for(let i = 0; i < arr.length; i++) {
```

```
            let item = arr[i];
            if(!obj[item]) {
                obj[item] = item;
            } else {
                dups.push(item);
            }
        }
    }
```

22. After the loop ends, check if the **dups** array has any items it it. If it doesn't, return the array. Else, return "none":

```
    function findDuplicates(arr) {
        let obj = {};
        let dups = [];
        for(let i = 0; i < arr.length; i++) {
            if(!obj[item]) {
                obj[item] = item;
            } else {
                dups.push(item);
            }
        }
        if(dups.length > 0) {
            return dups;
        } else {
            return "none";
        }
    }
```

23. Call the function four times, once per array. Save the function call to a variable to store the return value, and log the variable:

```
    let n1 = findDuplicates(nums1);
    console.log('duplicates:', n1);

    let n2 = findDuplicates(nums2);
    console.log('duplicates:', n2);

    let f1 = findDuplicates(fruits1);
    console.log('duplicates:', f1);

    let f2 = findDuplicates(fruits2);
    console.log('duplicates:', f2);
```

24. We are getting duplicates of duplicates, such as: ['apple', 'kiwi', 'banana', 'kiwi']). Let's add logic so that the duplicate gets pushed into the array only if it is *not* already there:

```
    } else {
        if(!dups.includes(item)) {
            dups.push(item);
        }
    }
```

- **END: Lesson 05.06**
- **NEXT: Lab 05.06**
- **Lesson 05.07**



Calculator Application

Assigning Click Event Listeners on a Loop

Working with Math Operator Buttons

This Calculator application is a simplified version of a calculator.

Here's what the calculator can do:

- Add, subtract, multiply or divide 2 numbers

Here's what the calculator cannot do:

- Perform any other math operations
- Calculate with more than 2 numbers
- Store answers (all is reset when you click "clear")

The user interface consists of buttons that call functions. The functions are called in order, as follows

I. onNumberClick()

- The user clicks a digit (0-9) or the decimal point to call the **onNumberClick()** function, which:
  - concatenates the digit (or decimal point) as **numStr**
  - updates the display in the ouput box.

II. onOperatorClick()

- The user clicks an operator (+,-,*,/) to call the **onOperatorClick()** function, which:
  - converts **numStr** to **num**, an actual number
  - pushes **num** into the **nums** array
  - saves the operator to a variable, **oper**
  - updates the ouput box to include the operator

III. onNumberClick() again

- The user inputs the second number, which again calls **onNumberClick()**

IV. calculateAnswer()

- After entering the second number, the user clicks the equal sign (**=**) to call the **calculateAnswer()** function, which:
  - pushes the second number into the **nums** array

    - runs a series of if-statements, to determine what the operator is and to perform the correct calculation:
        - If operator is **'+'**, add the two numbers
        - If operator is **'-'**, subtract the second number from the first
        - If operator is **'*'**, multiply the two numbers
        - If operator is **'/'**, divide the first number by the second
    - updates the ouput box to include the answer

V. clearBox()

- Clicking **clr** (clear) calls **clearBox()** function, which:
    - empties the box
    - resets the variables

output box NOT directly editable
To reduce the amount of code required for this application, the output box has been intentionally *not* been made editable. Since the user cannot click inside the box and type, the only way to enter content into the box is by clicking the buttons.

numbers cannot be erased / deteted
Also to reduce the amount of code required for this application, there is no delete button to remove content or change the inputted numbers.

1. Open the file **05.07-Calculator.html**, and take it for a spin in the browser:

- enter a number (as many digits as you like)
- click one of the four operators: *+, -, , /
- enter another number
- click the **=** (equals) sign to get the answer
- click **clear** to reset

2. Have a look at the tags in the html file. Note that the "buttons" are actually divs:

```
    <section>
        <div id="0" class="num-btns">0</div>
        <div id="1" class="num-btns">1</div>
        <div id="2" class="num-btns">2</div>
        <div id="3" class="num-btns">3</div>
        <div id="4" class="num-btns">4</div>
        <div id="5" class="num-btns">5</div>
        <br>
    </section>

    <section>
        <div id="6" class="num-btns">6</div>
        <div id="7" class="num-btns">7</div>
        <div id="8" class="num-btns">8</div>
        <div id="9" class="num-btns">9</div>
        <div id="." class="num-btns">.</div>
        <div id="=" class="equals-btn">=</div>
    </section>
```

```
    <section>
        <div id="+" class="oper-btns">+</div>
        <div id="-" class="oper-btns">-</div>
        <div id="*" class="oper-btns">*</div>
        <div id="/" class="oper-btns">/</div>
        <div id="c" class="clear-btn">clear</div>
    </section>

    <div id="num-box">
        <!-- output appears here -->
    </div>
```

- The **num-btns** class occurs 10 times, once for each digit.
- The **oper-btns** class occurs 4 times, once for each operator.
- The **delete-btn**, **clear-btn** and **equals-btn** class occurs once each.

3. In the script tags, comment out the **FINAL.js** file and reactivate **START.js**.

4. Our task is to recreate **FINAL.js** from scratch, so open **05.07-Calculator-START.js**

We start by bringing in the "buttons". We will use **document.querySelectorAll()** to bring in multiple elements at once. These come in as an array of objects, known as a **Node List**.

5. Get the divs that have a class of **num-btns**. These are the digits from 0-9:

```
    const numBtnsArr = document.querySelectorAll('.num-btns');
```

6. Loop through the **numBtnsArr** array and assign event listeners to the objects:

```
    for(let i = 0; i < numBtnsArr.length; i++) {
        numBtnsArr[i].addEventListener('click', onNumberClick);
    }
```

7. Get the operator divs (+,-,*,/), which have a class of **oper-btns**:

```
    const operBtnsArr = document.querySelectorAll('.oper-btns');
```

8. Loop through the **operBtnsArr** arrays, adding listeners to each item:

```
    for(let i = 0; i < operBtnsArr.length; i++) {
        operBtnsArr[i].addEventListener('click', onOperatorClick);
    }
```

9. Get the other three "buttons" which call functions; these are the **=** (equals) and **clear** buttons:

```
const equalsBtn = document.querySelector('.equals-btn');
equalsBtn.addEventListener('click', calculateAnswer);

const deleteBtn = document.querySelector('.delete-btn');
deleteBtn.addEventListener('click', delete);

const clearBtn = document.querySelector('.clear-btn');
clearBtn.addEventListener('click', clearBox);
```

10. Get the **num-box**, which is the div for the output:

```
const numBox = document.getElementById('num-box');
numBox.textContent = "";
```

11. Next, declare several global variables for use by the functions:

- **numStr**: for storing the inputted digit(s) as a string
- **nums**: an array for storing the two **numStr** values
- **num**: the numeric version of **numStr**, for doing the math
- **oper**: a string for storing the operator symbol
- **answer**: for storing the calculated result

```
let numStr = "";
let num = 0;
let nums = [];
let oper = '';
let answer = 0;
```

12. Write the **onNumberClick()** function, which runs on click of any digit from 0-9 or the decimal point.

The **onNumberClick()** function concatenates the clicked digit onto **numStr**.

- The keyword **this** in a function always refers to the object which called the function, so **this.id** is the digit
- If the user clicks 1, 2, 3 in order, then **numStr** equals "123"
- The value of **numStr** is displayed in the box

```
function onNumberClick() {
    numStr += this.id; // concatenate the clicked digit
    numBox.textContent += this.id; // updates the output box
}
```

13. Write the **onOperatorClick()** function, which runs when an operator "button" is clicked:

- saves **this.id** (the clicked object id) to the **oper** variable
- pushes **numStr**, the second "number-like string", into the **nums** array
- resets **numStr** to make way for the second number
- outputs the operator, surrounded by spaces, to the box

```
function onOperatorClick() {
    oper = this.id;
    nums.push(numStr);
    numStr = '';
    numBox.textContent += ' ' + oper + ' ';
}
```

The user inputs the second of the two numbers. With each digit or decimal click, the **onNumberClick()** function is called, which concatenate the second "number-like string", saves it to **nums** and displays it in the box.

When the user is done inputting the second number, they click the equal sign, which calls the **calculateAnswer()** function.

14. Write the **calculateAnswer()** function, which:

- pushes the second "number-like string" into the **nums** array
- converts the two array items to actual numbers, using the **Number()** method and saves the results as variables, **num1** and **num2**
- runs a series of if-statements to identify the operator and does the correct mathematical calculation based on the operator
- runs an if-else-statement, which passes the **answer** to the **Number.isInteger()** method, which returns true if the answer is an integer. Else, the answer is a float, which gets passed to the **toFixed()** method, which rounds it to 7 decimal places
- The **answer** is outputted, preceded by **' = '**

```
function calculateAnswer() {
    nums.push(numStr);
    let num1 = Number(nums[0]);
    let num2 = Number(nums[1]);
    if(oper == '+') answer = num1 + num2;
    if(oper == '-') answer = num1 - num2;
    if(oper == '*') answer = num1 * num2;
    if(oper == '/') answer = num1 / num2;
    if(Number.isInteger(answer)) {
        numBox.textContent += ' = ' + answer;
    } else { // not an integer
        numBox.textContent += ' = ' + answer.toFixed(7);
    }
}
```

15. Write the **clearBox()** function, which empties the output box and resets the global variables. The calculator is ready for fresh input.

```
function clearBox() {
    numBox.textContent = '';
    nums = [];
    num = 0;
    oper = '';
    total = 0;
    numStr = '';
}
```

END: Lesson 05.07
NEXT: Lesson 06.01



Chinese Zodiac Animals, Part 1

making DOM elements dynamically

createElement(), appendChild()

new Image()

Chinese Zodiac Animals

In this first of two lessons, we will begin to make an educational interface called **Chinese Zodiac Animals**. It is for learning the Chinese names of the 12 animals of the Chinese zodiac. The interface is equally suitable for students of English as a foreign language to learn the English names of these dozen animals.

From a coding standpoint, we will learn how to create DOM elements dynamically, as the interface is *not* hard-coded in the HTML. It is all made dynamically with JS.

1. Open **06.01-Creating-Elements.html**, and have a look at the code. There is a **container** which is practically empty: just an h1 and an empty **section**:

```
<div class="container">
    <h1>Lesson 06.01: Creating Elements – Chinese Zodiac Animals</h1>
    <section></section>
</div>
```

2. Preview the page in the browser. The HTML file is using the next lesson's final JS, **06.02-Chinese-Zodiac-Animals-FINAL.js**. This gives us a sneak preview of the completed interface, which will require this and the next lesson to code from scratch.

The completed interface consists of 12 divs, one for each animal of the Chinese zodiac. Each div contains a myriad of animal info, but in this lesson all we will be making are:

- the animal images (chicken, cow, dog, dragon, etc.)
- the input boxes for spelling the animal in English or piynin

Adding the Chinese characters and years, implementing the audio and displaying animal names will be left to **Lesson 06.02**.

3. Click the sound icon to play the **mp3** for that animal. You should hear the animal name spoken in English and Chinese. We will leave until the next lesson the implementation of the **Audio** object.

4. Try out the input box:

- type **chicken** in the chicken's text box and hit Enter. The box turns green and the English and pinyin appear.
- mess up the spelling and hit Enter again; the box turns red. - type **ji** and hit Enter. The box turns green, because *ji* is the pinyin for *chicken*.

5. Open the **images/animals** folder. The file naming format is **"english.jpg"**, so: chicken.jpg, cow.jpg, dog.jpg, etc.

6. The HTML file that **js/zodiac-animals-data.js** is already being imported. Open the file. It contains an **animals** array of 12 objects:

```
    let animals = [
        {eng: "chicken", also: 'rooster', chi: "ji", pin: "ji&#772;",
year: 2029, tone: 1},
        -- ETC --
        {eng: "tiger", chi: "hu", also: 'laohu', pin: "la&#780;ohu&#780;",
year: 2022, tone: 3}
    ];
```

animals object properties

The 12 objects are ordered alphabetically by their **eng** (English) property, from "chicken" to "tiger". Each object is an animal with six properties. We will be using all of these properties to build our interface, although in this lesson we will only be using the **eng** property.

What's pinyin?

The Chinese language does not use an alphabet. Instead, each word-syllable exists as a *character*. Even so, Chinese pronunciation can be spelled out in a Romanization system called **pinyin**. In our application, the text input can be English or pinyin. So, the English *chicken* and the pinyin *ji* are both acceptable spellings.

7. Open **06.01-Creating-Elements.css** and have a look. There are classes that we will apply to the dynamically generated DOM elements.

8. Switch the html page to using **START .js** and reload the page. Now the **section** element is just a big, empty box.

how to make HTML elements dynamically with JS

There are two ways to make an element:

- **createElement('div')** method creates an object, in this case a div
- **new Image()** creates an object, in this case an image
- the object needs to be saved to a variable, preferably a **const** to prevent it from having its data type mutated:
    - **const divvy = createElement('div')** makes a div object named **divvy**
    - **const pic = new Image()** makes an image object named **pic**
- besides Image, Audio, Video and a few other elements, all elements are made with **createElement()**

- the
- once created, elements can have any attribute applied as a property:
    - **divvy.id = 'div1'** assigns divvy an id of div1.
    - **divvy.className = 'my-div'** assigns divvy a class of my-div.
    - **image.src = 'images/dog.jpg'** sets the image source to dog.jpg.
- **appendClild()** is used to output the dynamically generated object to the DOM. The method is called on an existing DOM element and takes the newly created element as its argument:
    - **section.appendChild(divvy)** outputs **divvy** as a child element of **section**

Now let's start to actually make the Chinese Zodiac Animals interface.

9. Open **06.01-Creating-Elements-START.js** and get the **section** that serves as the parent wrapper of the interface:

```
const section = document.querySelector('section');
```

10. Make a div with **createElement()**:

```
const divvy = document.createElement('div');
```

11. Apply the existing **.divvy** class to the div and Give the div some text content, as a test:

```
divvy.className = 'divvy';
divvy.textContent = 'dynamic div';
```

12. Use **appendChild()** to output the div inside the section:

```
section.appendChild(divvy);
```

13. Reload the page. In the upper left corner of the section there should be one smallish box that says *dynamic div*.

making the set of 12 Chinese Zodiac Animals

Now that we have that working, let's use a loop to make all 12 divs.

14. Wrap the code that made the div in a for loop that runs 12 times. For text output, number the divs with the counter, **i**:

```
for(let i = 0; i < 12; i++) {
    const divvy = document.createElement('div');
    divvy.className = 'divvy';
    divvy.textContent = 'div ' + i;
```

```
        section.appendChild(divvy);
    } // end loop
```

15. Reload the page. All 12 divs should appear, in two rows of six. The divvy class has *display: inline-block* to arrange the divs side-by-side.

16. Still in the loop, make an image for each div, using **new Image()**:

```
        let pic = new Image();
    } // end loop
```

17. Set the source of the image to **cow.jpg**. The result will be 12 cows:

```
        let pic = new Image();
        pic.src = 'images/animals/cow.jpg';
```

18. Apply the **.animal-pic** class to the image:

```
        let pic = new Image();
        pic.src = 'images/animals/cow.jpg';
        pic.className = 'animal-pic';
```

19. Delete the text and output the image inside the div. In terms of nesting, **divvy** goes inside **section** and **pic** goes inside **divvy**:

```
        pic.className = 'animal-pic';
        divvy.textContent = '';
        divvy.appendChild(pic);
    } // end loop
```

20. Reload the page. We should have 12 divs, each with the cow.

21. Change the source to be dynamic, so that we get all 12 animals--not just the cow. Concatenate the **eng** property into the file path. The current animal is available as **animals[i]** and its English name is **animals[i].eng**:

```
        let pic = new Image();
        pic.src = `images/animals/${animals[i].eng}.jpg`;
```

22. Reload the page. We should have 12 divs, each with a different animal.

23. We will be using the **animals** array a lot, so to simplify property access, save the current animal **animals[i]** to a variable, **animal**. Update the concatenation to use the more streamlined **animal.eng**:

```
        let animal = animals[i];
        let pic = new Image();
        pic.src = `images/animals/${animal.eng}.jpg`;
```

24. Still in the loop, make the **input** box:

```
        const inputBox = document.createElement('input');
```

25. Assign the input box a type of **search**. This will give the box a little X to clear it, and it also provides a **search** event, which fires when the user hits Enter:

```
        inputBox.type = 'search';
```

26. Assign the box a **placeholder** to prompt the user to enter the animal name:

```
        inputBox.placeholder = 'name';
```

27. Assign **eng**, **chi** and **also** properties to the input box. These will be used by the **checkSpelling** function to see if the user input matches any of the three accepted spellings:

```
        inputBox.eng = animal.eng;
        inputBox.chi = animal.chi;
        inputBox.also = animal.also;
```

28. Also save the current index as a property of the input box. This will come in handy down the road (next lesson):

```
        inputBox.also = animal.also;
        inputBox.i = i;
```

29. Have the input box call the function when its search event is fired:

```
        inputBox.addEventListener('search', checkSpelling);
```

30. Also the input box call the function on **blur**, and event which fires when the user hits **Tab** to leave an input box. Tab moves the cursor to the next input box, so it's a handy way to navigate from one animal div to the next:

```
        inputBox.addEventListener('blur', checkSpelling);
```

31. Output the input box to the div. It will appear under the animal image:

```
        divvy.appendChild(inputBox);
    } // end loop
```

We need to make the **checkSpelling** function before reloading the page in the browser. If the function referred to in the listener does not exist at that point, we'll get an error.

this keyword

Inside a function, the **this** keyword refers to the object that called the function. In the **checkSpelling** function, **this** is the input box:

- **this.eng** is the English name
- **this.chi** is the pinyin spelling (without tone markings)
- **this.also** is the alternate spelling

With these properties, we compare the user input to the correct spelling:

32. Define the **checkSpelling** function and start by getting the **value** of the input element--whatever the user typed into the box:

```
    function checkSpelling() {
        let input = this.value;
    }
```

33. Compare the user input to the English, pinyin and alternate (also) spelling. The user input needs to match *one* of the three correct spellings:

```
    function checkSpelling() {
        let input = this.value;
        if(input == this.eng || input == this.chi || input == this.also) {
        }
    }
```

34. If the user input is correct, turn the input box green; else turn the box red:

```
    function checkSpelling() {
        let input = this.value;
        if(input == this.eng || input == this.chi || input == this.also) {
            this.style.backgroundColor = '#0B0';
            this.style.color = '#fff';
        } else {
            this.style.backgroundColor = '#921';
            this.style.color = '#fff';
        }
    }
```

35. Reload the page. Each box should have an animal pic and an input box.

36. Enter a spelling in a box and hit Enter.

- if the spelling is correct, the input box turns green.
- else, the spelling is incorret, so the box turns red.
- **END: Lesson 06.01**
- **NEXT: Lesson 06.02**



Chinese Zodiac Animals, Pt. 2

playing Audio

reveal hidden elements

output years series via loop

Chinese Zodiac Animals, Part 2

In this Part 2 of the Chinese Zodiac Animals application, we will continue where we left off. Our START file is the FINAL file from Lesson 06.01.

Up to this point, we have outputted the 12 animal images, each with a text box beneath for entering the English or pinyin of the animal. We got the input part working too: if the spelling is correct, the box turns green; else, it turns red.

1. Preview the page with **FINAL.js** as a reminder of what the final application looks like.

- click a sound icon to play the mp3 file for that animal
- enter an animal name and hit enter to turn the box green

2. Switch to **START.js** and reload the page. Now we have just the animal pic and the input box--right where we left off last lesson. The content to add in this lesson:

- years of tha animal in the 12-year cycle
- a sound icon to click to play an mp3 audio file
- the jpg of the Chinese character for that animal
- three bits of text which are hidden on page load:
    - the English name of the animal
    - the pinyin spelling (Romanization of Chinese pronunciation)
    - the tone number from 1-4
    - the hidden elements will appear if the user spells the animal correctly in English or pinyin

3. Open **js/zodiac-animals-data.js** and have another look at the **animals** array of objects. Let's take a closer look at the properties, since we will be using all of them in this lesson:

```
    let animals = [
        {eng: "chicken", also: 'rooster', chi: "ji", pin: "ji&#772;",
year: 2029, tone: 1},
        -- ETC --
        {eng: "tiger", chi: "hu", also: 'laohu', pin: "la&#780;ohu&#780;",
```

```
year: 2022, tone: 3}
    ];
```

- **eng** is the English name. We use this to concatenate the animal image and audio file names, to output the English name and to check English spelling.
- **chi** is the pinyin name, without tone symbols. We use this to concatenate the character image file name and to check pinyin spelling.
- **pin** is the Chinese Pinyin name, with gibberishy symbols. We use this to output the Pinyin, complete with tone markings.
- **also** is an alternate name: "rooster" is an accepted alternative to "chicken", while "hu" (tiger) and "laohu" ("old tiger") are both good.
- **year** is a recent or upcoming year in the 12-year cycle. We use that to run a loop that backs up 180 years and outputs 15 years at 12 year intervals
- **tone** from 1-4. In the Pinyin spelling, tone markings appear above certain vowels. In our interface, the tone number accompanies the pinyin.

Now that we have reviewed the project, let's forge ahead with new content. First up: get the sound working again.

the html audio element and JS Audio object

HTML has an **audio** tag which is used to play audio files, such as **mp3** files. The **src** attribute is set to the file to play. The JS version of this tag is the **Audio** object.

4. Open the **audio** folder and check the files. The naming format is the **eng** property followed by the **.mp3** file extension: **chicken.mp3, cow.mp3**, etc.
5. In the **START.js** file, inside the loop, make an image and set its source to **sound-icon.png**:

```
    let soundIcon = new Image();
    soundIcon.src = 'images/sound-icon.png';
```

6. Assign the **sound-icon** class to the image. The class uses absolute positioning, which will park the icon in the upper right corner of the div:

```
    soundIcon.className = 'sound-icon';
```

7. Assign the **eng** proprety of the current animal object to be the **eng** property of the image. This way, when the image is clicked, the function can identify the animal as **this.eng**. This is used to concatenate the **mp3** file name:

```
    soundIcon.eng = animal.eng;
```

8. Make the clicking of the sound icon call the **playSound** function:

```
    soundIcon.addEventListener('click', playSound);
```

9. Output the sound icon to the div:

```
    divvy.appendChild(soundIcon);
```

We need to define the **playSound** function before reloading the page in the browser, or else we will get an error. But first, we need an **Audio** object to play the sound.

10. Above the loop, in the global scope, instantiate an instance of the **Audio** object. It's the same syntax as the **Image** object, with the **new** keyword:

```
    const sound = new Audio();
```

11. Below that, also in the global scope, write the function.

- **pause()** stops any sound which may be playing
- **src** sets the source of the **mp3** file
- **play()** plays the source file

```
    function playSound() {
        sound.pause();
        sound.src = `audio/${this.eng}.mp3`;
        sound.play();
    }
```

12. Reload the browser. The sound icons should be back. Click one to play that animal's mp3 file.

13. Open the **chars** folder in **images**. It contains Chinese character jpgs, with the naming format: **"char-pinyin.jpg": char-gou.jpg, char-hou.jpg**, etc.

14. Still inside the loop, make an image and set its source to the character jpg for the animal. The **chi** property has the pinyin (sans tone markings) that we need to concatenate the file name:

```
    let chineseChar = new Image();
    chineseChar.src = `images/chars/char-${animal.chi}.jpg`;
```

15. Assign the **chinese-char** class to the image:

```
    chineseChar.className = 'chinese-char';
```

16. Output the Chinese character to the div:

```
    divvy.appendChild(chineseChar);
```

Notice that we did not assign the Chinese character image any properties. This is because it won't be clickable or have any behavior that would require us to refer to it during runtime.

17. Reload the page. The application is really starting to shape up: we have the animal and character pics, as well as the sound and input box--and everything so far works.

Chinese zodiac year cycle

In the Chinese zodiac, animals repeat every 12 years. The data object has just one **year** for each animal, but that is enough, since we can use a loop to increment by 12 and make as many years as we want. We can fit 15 years down the side of the div, which is what the generous space at left is for.

18. Still in the big loop, make a **p** tag to hold the years and assign it its class. The css sets the width narrow, so that only one year can fit per line.

```
    let year = document.createElement('p');
    year.className = 'zodiac-year';
```

19. We will concatenate the year series as a string. Declare a variable **yearsSeries** and assign it an initial value of the current year, minus 168. This takes us back 14 cycles, to the mid-19th century. Add a space after the year:

```
    let yearsSeries = (animal.year - 168) + ' ';
```

20. Run a for loop that starts at 156, continues until the counter reaches -12 and goes down by 12 each time:

```
    for(let y = 156; y > -12; y-=12) {
    }
```

21. Each time through the loop, **y**, which is decremented by 12. The expression **animal.year - y** gives us the next year in the series, which is concatenated **yearsSeries**:

```
    for(let y = 156; y > -12; y-=12) {
        yearsSeries += (animal.year - y) + ' ';
    }
```

22. Below the loop, output **yearSeries** to the **year** element, and then output that **year** to the div:

```
    for(let y = 156; y > -12; y-=12) {
        yearsSeries += (animal.year - y) + ' ';
    }
    year.textContent = yearsSeries;
    divvy.appendChild(year);
```

23. Reload the browser. Viola--we should have 15 years at 12-year intervals running down the left hand side of each div. The last year is the next future one in the cycle.

showing hidden text

The next and final step is to make elements to display text:

- English name centered at the top of the div
- Pinyin with tone markings next to the Chinese character
- The tone number from 1-4 underneath the pinyin

24. Still in the big loop, make a **span** tag to hold the English name:

```
    let english = document.createElement('span');
```

25. Apply its class, which has **display: none** to hide the tag on page load:

```
    english.className = 'english';
```

26. Set its text to be the English name ("cow", "dog", etc.):

```
    english.innerHTML = animal.eng;
```

27. Assign the tag a dynamic id that uses the current index: **eng0, eng1**, etc. The element needs an id, so that we can get -- and show -- the element in the **checkSpelling** function:

```
    english.id = 'eng' + i;
```

28. Output the span tag to the div:

```
    divvy.appendChild(english);
```

29. Still in the big loop, make a span tag to hold the pinyin and its tone number. The process is the same as for making the span to hold the English name, so we will do this all one step:

```
    let pinyin = document.createElement('span');
    pinyin.innerHTML = animal.pin + '<br>' + animal.tone;
    pinyin.pin = animal.pin;
    pinyin.className = 'pinyin';
    pinyin.id = 'pin' + i;
    divvy.appendChild(pinyin);
```

30. Reload the page. The text does not appear, nor do we have any functionality yet to make it appear.

31. In the CSS, in the **.english** and **.pinyin** classes, comment out the **display: none** properties:

```
    .english {
        top: 5px;
        right: 50%;
        /* display: none; */
        font-size: 1.1rem;
        color: var(--colr);
    }

    .pinyin {
        top: 200px;
        right: 23%;
        font-size: 1rem;
        /* display: none; */
        letter-spacing: 1px;
    }
```

32. Reload the page. Now the text appears.

33. Restore the **display: none** in the CSS so that the text is hidden again.

We want the **checkSpelling** function to show the text when the user spells an animal correctly, either in English or pinyin.

34. In the **checkSpelling** function, in the **if** part, get the English and pinyin elements by id. The first animal's English and pinyin span tags have ids of **eng0** and **pin0**. To get the elements by id, we need to first concatenate the ids using the index:

```
    if(input == this.eng || input == this.chi || input == this.also) {
        this.style.backgroundColor = '#0B0';
        this.style.color = '#fff';
```

```
        let pinID = 'pin' + this.i;
        let pinSpan = document.getElementById(pinID);

        let engID = 'eng' + this.i;
        let engSpan = document.getElementById(engID);
```

```
        let pinID = 'pin' + this.i;
        let pinSpan = document.getElementById(pinID);
        pinSpan.style.display = 'inline';

        let engID = 'eng' + this.i;
        let engSpan = document.getElementById(engID);
        engSpan.style.display = 'inline';
```

36. Reload the page and enter an animal name in the text box. Hit Enter or Tab. The English and pinyin should appear.

- **END: Lesson 06.02**
- **NEXT: Lesson 06.03**



createElement() appendChild()

Math.random()

for loops

functions called by button click events

IIFE (Immediately Invoked Function Expression)

evaluating checkboxes with conditional logic

string concatenation

dynamic text selection

dynamic copying of text to the clipboard

In this lesson, we will make a **Password Generator** application that makes random passwords of specified lengths using character sets chosen by the suer. The interace works like so:

- the user chooses a password length from the menu
- the user unchecks any boxes of character types to exclude from the password
- the user clicks the **generate password** button to call the **generatePassword** function
- the function sees which checkboxes are checked and uses them to make a big string of eligible characters.
- then the function generates random integers in the range of the string length, using the number to get a character by index
- the function charactconcates a password from the random selections.
- the password is outputted to a text box
- the user can click the **copy** button to copy the password to the clipboard

1. Open **06.03-Password-Generator.html** from the **06.03-Password-Generator** folder, and preview the file in the browser.

2. Take it for a spin: choose a password length from the menu and click the **generate password** button. A randomly generated password appears in the text box.

3. Click **copy** to copy the password to the clipboard.

4. To confirm that the password has been copied, empty the input box and paste in the password.

5. Have a look at the html code. There are a few things to note:

- The select menu is empty. This is because we will use a for loop to generate the numbered options from 8 to 25, dynamically.
- The form elements (select, input, label, button) are not wrapped in a form tag. Since we are not sending the data for processing, so we don't need a form.

tooltip

There is a thing called a "tooltip" near the end of the code. This is a cool feature where you get a popup (tooltip) when you mouse over something, in this case the copy button. A tooltip typically looks like a cartoon dialogue balloon, complete with little pointer arrow. The CSS to render a tooltip is fairly tricky, and you would do well to spend some time studying that, although CSS is not the focus of this lesson (or of this course, for that matter).

Now that we have taken the app for a spin and checked out the HTML, let's turn our attention to recreating **FINAL.js** from scratch.

6. Switch from using **FINAL.js** to **START.js** amd open **06.03-Password-Generator-START.js.**.

We start by declaring strings of the characters that go with each checkbox. If a checkbox is checked, its associated characters are eligible for inclusion in the random password. As we will not be editing these strings, they are declared uppercase, with **const**

7. Declare consts for the letters of the alphabet, the digits 0-9 and some special character symbols:

```
const LOWERS = "abcdefghijklmnopqrstuvwxyz";
const UPPERS = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
const NUMBRS = "0123456789";
const SYMBLS = "@#$%^&*_+?!";
```

8. Get the select menu:

```
const selectPswdLen = document.getElementById("select-pswd-len");
```

9. Get the four checkboxes:

```
const lowcaseCB = document.getElementById("lowcase-cb");
const uprcaseCB = document.getElementById("upcase-cb");
const numbersCB = document.getElementById("num-cb");
const symbolsCB = document.getElementById("symbol-cb");
```

10. Get the two buttons:

- one button calls the function that generates the password
- the other button calls the function that copies the password

```
const genPswdBtn = document.getElementById("gen-pswd-btn");
const copyPswdBtn = document.getElementById("copy-btn");
```

11. Have the buttons call their respective functions on click:

```
genPswdBtn.addEventListener('click', generatePassword);
copyPswdBtn..addEventListener('click', copyPassword);
```

12. Get the text input element for displaying the new password:

```
const newPswdOutput = document.getElementById("new-pswd-output");
```

13. Run a loop that makes an **option** tag with each iteration of the loop. The loop counter, **i**, runs from 8 to 25:

```
for (let i = 8; i <= 25; i++) {
}
```

14. Inside the loop, make the option tags:

- Use **createElement()** to make the option.
- Assign **value** and **text** properties equal to the counter.
- Use **appendChild()** to add the option to the menu:

```
for (let i = 8; i <= 25; i++) {
    const option = document.createElement('option');
    option.text = i;
    option.value = i;
    selectPswdLen.appendChild(option);
}
```

15. Set the default choice to 15 using conditional logic: if the value of the counter is 15, add a selected property to the option object:

```
  for (let i = 8; i <= 25; i++) {
    const option = document.createElement('option');
    option.text = i;
    option.value = i;
    if(i == 15) option.selected = 'selected';
    selectPswdLen.appendChild(option);
  }
```

16. Run the page. The menu should be populated with choices from 8-25.

function generatePassword()

Now for the **generate password** function.

17. Define the function and declare two string variables:

- **charSet** stores the eligible characters, per the checkboxes
- **randomPswd** stores the generated password.

```
function generatePassword() {
  let charSet = ""; // the eligible characters
  let randPswd = ""; // the random password
}
```

evaluating the checkboxes

If a checkbox is checked, its associated string is concatenated onto the **charSet**. Since the if-statements are just one line of logic each, the curly braces may be omitted.

18. Concatenate the **charSet** variable:

```
function generatePassword() {
  let charSet = ""; // the eligible characters
  let randPswd = ""; // the randomly generated password
  if(lowcaseCB.checked) charSet += LOWERS;
  if(uprcaseCB.checked) charSet += UPPERS;
  if(numbersCB.checked) charSet += NUMBRS;
  if(symbolsCB.checked) charSet += SYMBLS;
}
```

generate the random password

Now that all eligible characters are in the **charSet** string, we can proceed to make the actual password:

- run a **for loop** as many times as there are characters in the password
- each time through the loop, generate a random integer in the range of the **charSet**
- use the **charAt()** method to select the character that occurs at the random index
- concatenate the randomly chosen **charSet** character onto **randPswd**
- repeat the loop until the password reaches the target length
- output **randPswd** to the "Your new password" field

19. Still in the function, run a for loop that iterates once for each character of the password:

```
  for (let i = 0; i < selectPswdLen.value; i++) {
  }
```

charAt(index)

the **charAt(index)** method is called on a string and takes an index as its argument. It returns that character at that index:

20. Generate a random integer in the range of the charSet string and then get the character at the random index using **charSet()** method:

```
    let r = Math.floor(Math.random() * charSet.length);
```

21. Look up that character at that index using **charAt(index)** and concatenate that character onto the **randPswe** string. Finally, output the password:

```
function generatePassword() {
    let charSet = ""; // the eligible characters
    let randPswd = ""; // the randomly generated password
    if(lowcaseCB.checked) charSet += LOWERS;
    if(uprcaseCB.checked) charSet += UPPERS;
    if(numbersCB.checked) charSet += NUMBRS;
    if(symbolsCB.checked) charSet += SYMBLS;
    for (let i = 0; i < selectPswdLen.value; i++) {
      r = Math.floor(Math.random() * charSet.length);
      randPswd += charSet.charAt(r);
    } // end loop

    newPswdOutput.value = randPswd;

  } // end function generatePassword()
```

Once the password is generated, we want to make it easy for the user to copy it.

copying text to the clipboard

Clicking the **copy** button calls the **copyText()** function, which does the following:

- selects the generated password from the text box
- uses **navigator.clipboard.writeText()** to copy the password
- shows a tooltip with a "Password copied" message

22. Define the **copyPassword** function:

```
function copyPassword() {
}
```

23. Use the **select()** and **navigator.clipboard.writeText()** methods to copy the password from the output field:

- **select()** method selects (highlights) the text in the target element
- **navigator.clipboard.writeText(text)** copies its argument text to the clipboard

```
  function copyPassword() {
    newPswdOutput.select();
    navigator.clipboard.writeText(newPswdOutput.value);
  }
```

24. Provide feedback that the password has been copied:

```
  function copyPassword() {
    newPswdOutput.select();
    navigator.clipboard.writeText(newPswdOutput.value);
    document.querySelector(".tooltiptext").innerHTML = "Password copied: "
+ newPswdOutput.value;
  }
```

25. Save all and run the page. The application should work.

26. Choose a password length from the menu and click the GENERATE PASSWORD button. A random password should appear in the output box.

27. Click the **copy** button, the password in the box is highlighted (selected) and also copied to the clipboaard.

Some nifty css makes the cartoon speech balloon-like tooltip. You can check out the css on your own, although it is outside the scope of this lesson.

- **END: Lesson 06.03**
- **NEXT: Lesson 06.04**



Spell It!

In this lesson, we will build an application that challenges children or students of English as a second language to spell words of various things on the web page.

1. Open **06.04-ABCs-Spell-It.html** and preview it in the browser. The interface should load with a set of lettered buttons above a big empty area.

2. Click the **A** button. It should generate a grid of "A-words".

Below each image is a text *input* box. The *placeholder* text says **inglés 英文**, which means "English" in Spanish and Chinese. This is the user's cue to spell the word in English.

3. The first word is *abacus*. Type *abacus* in the box below the picture, and hit Enter. The box will turn green.

4. Click in any other text box, or hit Tab to advance to the next box.

5. Misspell a word, and hit Enter or Tab. The box will turn red.

6. Fix the spelling, and hit Enter (or Tab). The box will turn green.

7. Click the **B** button to clear away the A-words and load the B-words.

previewing the html file in the code editor

8. Have a look at the HTML code. As we can see, none of the ABC's content appears. That's because, as with the Chinese Zodiac Animals application, the interface is generated dynamically, with JS:

```
<div class="container">
    <header>
        <h1>
            Spell words in English:<br>
            Escribe palabras en inglés:<br>
            拼写以字母开头的单词 :
        </h1>
        <div class="buttons-box"></div>
    </header>
    <main></main>
</div>
```

The **main** element is empty on page load. That is because its content is generated by a function which is called when the user clicks a letter button. The function contains a *for loop* in which the picture boxes for the chosen letter are produced.

the data file

9. Open **abc-data.js**, the dataset for the application. It contains a single variable, **abcs**, which is an array of objects. Each object but the first represents a word:

```
    let abcs = [

['a','b','c','d','e','f','g','h','k','l','m','n','o','p','r','s','t','v'],
        { word: "abacus", level: 3, keywords: ["Chinese", "calculator"],
alive: false },
        -- ETC.  --
        { word: "vinegar", level: 2, keywords: ["food", "liquid",
"cupboard", "meal"], alive: false }
    ];
```

The first item in **abcs** is likewise an array--of letters. If the letter is in the array, there's a button for it.

10. Remove the **'a'** from the array.

11. Save the file and reload the browser. The A-button is gone.

12. Restore the **'a'** and reload. The "A-button" is back.

13. Add **"z"** to the array of letters.

14.Reload the browser. There should be a "Z-button" now. Clicking it doesn't yield any "Z-words", though, since there are none in the dataset.

15. Remove the **"z"** from the array and reload the browser; the "Z-button" is gone.

array objects

All other items in the array are **objects**, arranged alphabetically by their **word** property, the value of which is a string ("alligator", "anvil", etc). These values are used to:

- concatenate the file name of the image to be displayed.
- assign an id to each text input box. If the text entered by the user matches the id of that text box, the spelling is correct.

Each object has three other properties: **level**, **keywords** and **alive**, but these are not used in the current exercise.

16. Switch to using the **START.js** file and reload the browser. The app no longer works, of course. All we see is the static html and css.

17. Get the elements that hold the interface:

- **buttons-box** div holds the lettered buttons

- **main** holds the grid of wrod-picture boxes Neither of these divs has an id, so use **querySelector**:

```
const buttonsBox = document.querySelector('.buttons-box');
const main = document.querySelector('main');
```

Next we'll make the lettered buttons, which exist as an array that is itself the first item in the **abcs** array, and so is accessed by the syntax **abcs[0]**.

18. Save the first item in the **abcs** array as its own array variable:

```
const letters = abcs[0];
```

19. Loop through the array of **letters**:

```
for(let i = 0; i < letters.length; i++) {
}
```

20. Make a button object each time through the loop:

```
for(let i = 0; i < letters.length; i++) {
    let btn = document.createElement('button');
}
```

21. Assign the **letter-btn** class to the button:

```
    btn.className = 'letter-btn';
```

22. Assign an id to the button. This is the letter of the button, so the "A button" is has **id='a'**, and so on. We get the current letter by using **i** to look up that item in the letters array by index.

```
    btn.id = letters[i];
```

23. Label the button by setting its **textContent** property to the current letter, to uppercase:

```
    btn.textContent = letters[i].toUpperCase();
```

24. Have the button call the function **makeWordBoxes**:

```
        btn.addEventListener('click', makeWordBoxes);
```

25. Output the button to the parent element div, **buttonsBox**:

```
        buttonsBox.appendChild(btn);
```

26. Declare the **makeWordBoxes** function. For openers, just have it log the id of the button, as **this.id**. Remember: inside a function **this** is the thing that called the function, in this case the button:

```
    function makeWordBoxes() {
        console.log(this.id);
    }
```

27. Reload the browser, click a letter button, and check the console. The button's id should appear.

28. Now for the actual code. Start by clearing **main** in case any lettered buttons are already there.

```
    function makeWordBoxes() {
        console.log(this.id);
        main.innerHTML = "";
    }
```

29. Loop the **abcs** array. The counter **i** start at 1, since we want to skip the array of letters at index 0. We want the loop to only iterate over the word objects:

```
    function makeWordBoxes() {
        console.log(this.id);
        main.innerHTML = "";
        for(let i = 1; i < abcs.length; i++) {
        }
    }
```

30. In the loop, simplify the code by saving the current word object as **word**.

```
        for(let i = 1; i < abcs.length; i++) {
            let word = abcs[i].word;
        }
```

If the "A-button" was clicked, we want A-words *only* ("abacus", "actor", etc.). Such filtering requires if-logic:

31. Check if the first letter of the word (**word[0]**) matches the id of the clicked button (**this.id**):

```
    for(let i = 1; i < abcs.length; i++) {
        let word = abcs[i].word;
        if(word[0] == this.id) {
        }
    }
```

32. If the condition returns true, make a "word box", and assign it the divvy class:

```
        if(word[0] == this.id) {
            let wordBox = document.createElement('div');
            wordBox.className = "divvy";
        }
```

33. Output the **wordBox** div to **main**:

```
    if(word[0] == this.id) {
        let wordBox = document.createElement('div');
        wordBox.className = "divvy";
        main.appendChild(wordBox);
    }
```

making images to go inside the word box divs

34. Still inside the if statement, make a new image object:

```
        let pic = new Image();
```

35. Concatenate the image's **src** property. The folder names are the letter to uppercase, followed by a lowercase "s": letters **As**, etc.:

```
        pic.src = 'img/ABCs/' + word[0].toUpperCase() + 's/' + word +
 '.png';
```

36. Ouput the image to the word box div:

```
        wordBox.appendChild(pic);
```

37. Still in the if statement, make an **input** object of type **search**:

```
    let inputBox = document.createElement('input');
    inputBox.type = "search";
```

38. Assign the input box a **placeholder** that says *English* in Spanish and Chinese. This prompts the user to spell the word:

```
    inputBox.placeholder = "ingl\u00E9s 英文";
```

The input box call the function **checkSpelling** when the user hits Enter or Tab, which fire the **search** and **blur** events, respectively:

39. Add the two listeners to the input box.

```
        inputBox.addEventListener('search', checkSpelling);
        inputBox.addEventListener('blur', checkSpelling);
```

40. Assign **word** as the id to the input box. With this, we will be able to check spelling by comparing the id of the box to what the user typed into the box:

```
        inputBox.id = word;
```

41. Output the input box to the div:

```
        wordBox.appendChild(inputBox);
```

42. Define the **checkSpelling** function:

```
    function checkSpelling() {
    }
```

43. Compare **this.value** and **this.id**. If they are equal, the spelling is correct, so set the input box background to green and its text to white:

```
    function checkSpelling() {
        if(this.value == this.id) {
            this.style.backgroundColor = '#181';
            this.style.color = '#fff';
        }
    }
```

44. Else, the spelling is wrong, so set the input background to red:

```
        } else {
            this.style.backgroundColor = '#811';
            this.style.color = '#fff';
        }
```

45. Save all and take it for a spin. Buttons should load the image packs for their respective letters. Text boxes should turn green or red when you enter text and hit Enter or Tab.

- **END: Lesson 06.04**
- **NEXT: Lesson 06.05**



UNIT 06

LESSON 06.05



Sorting Arrays of Objects by Key

Randomizing (Shuffling) Arrays

Object.keys(), Object.value()

In this lesson, we will again work with objects and arrays as we learn how to:

- A. sort arrays of strings vs. arrays of numbers
- B. combine multiple arrays into a single array of objects
- C. sort arrays of objects by number key
- D. sort arrays of objects by string key
- E. randomize (shuffle) arrays of objects
- F. make an array from an object's keys
- G. make an array from an object's values

1. Start by declarings 3 arrays, each with 7 items.

```
    const fNames = ['Pat', 'Jose', 'Ping', 'Amy', 'Shujia', 'Jed',
'Betty'];
    const lNames = ['Hanson', 'Diaz', 'Liu', 'Smith', 'Guo', 'Martin',
'Jones'];
    const ids = [7445, 1214, 748, 4545, 962, 53, 6342];
```

A. sort arrays of strings vs. arrays of numbers

2. Let's review sorting arrays of strings, which we first looked at in Unit 4:

```
    lNames.sort();
    console.log(lNames);
    // ['Diaz', 'Guo', 'Hanson', 'Jones', 'Liu', 'Martin', 'Smith']
```

3. What we haven't tried yet is to sort arrays of numbers. Let's try that now (it won't work):

```
    ids.sort();
    console.log(ids);
    // [1214, 4545, 53, 6342, 7445, 748, 962]
```

With no arguments, the **sort()** method only works on strings. It even "stringifies" numbers, seeing the digits as characters. From that point of view, 9 is greater than 1, just as "z" is greater than "a".

To sort arrays of numbers, we pass a **callback** to the **sort()** method:

- a **callback** is a function passed to another function as its argument
- the callback function is **anonymous** and has two parameters: **a** and **b**
- **a** and **b** as arguments are consecutive items in the array
- the function returns **a - b**, which reverses the position of consecutive items as needed to sort them. This happens under the hood on a loop.

4. Call the sort method on the **ids** array; pass in the callback function:

```
ids.sort(function(a, b) {
    return a - b;
});
console.log(ids);
// [53, 748, 962, 1214, 4545, 6342, 7445]
```

B. combine multiple arrays into a single array of objects

The items in the 3 arrays (**fNames, lNames, ids**) correspond to each other, so the first item of each array line up to give us Pat Hanson in Sales, with an id of 7445. This means that we can -- and should -- consolidate the data into one array of objects.

5. Declare a new empty array, **team**, and loop the length of one of the 4 arrays to be combined:

```
const team = [];
for(let i = 0; i < ids.length; i++) {
}
```

6. Declare an object, and give it four properties. The key-value pairs are derived from the corresponding arrays. Push the completed object into the array:

```
const team = [];
for(let i = 0; i < ids.length; i++) {
    let obj = { fName: fNames[i], lName: lNames[i], id: ids[i] };
    team.push(obj);
}
```

7. Below the loop, log the array along with a sampling of its data:

```
console.log(team);
/*
0: {fName: 'Pat', lName: 'Hanson', id: 7445}
```

```
1: {fName: 'Jose', lName: 'Diaz', id: 1214}
   -- ETC. --
6: {fName: 'Betty', lName: 'Jones', id: 6342}
*/
console.log(team[0]);
// {fName: 'Pat', lName: 'Hanson', id: 7445}
console.log(team[1].fName); // Jose
console.log(team[2].lName); // Liu
console.log(team[3].id); // 4545
```

C. sorting arrays of objects by number key

8. Call the **sort()** method on the **team** array. It does nothing, because the array items are not strings or numbers, but objects:

```
team.sort();
console.log(team.sort()); // no change
```

To sort by an object by **key**, we need to pass in the **callback function** with the **a** and **b** arguments. To sort by numeric key return **a.key - b.key**:

9. Calling the **sort()** method on the **team** array again, pass it the callback with arguments **a** and **b**:

```
team.sort(function(a, b) {
});
```

10. Sort by **id** by returning **a.id - b.id**:

```
team.sort(function(a, b) {
    return a.id - b.id;
});
```

11. Log **team** again. The objects are sorted by **id**, ascending:

```
console.log(team);
/* 0: {fName: 'Jed', lName: 'Martin', id: 53}
    -- ETC --
6: {fName: 'Pat', lName: 'Hanson', id: 7445} */
```

12. To sort in descending order call the **reverse()** method:

```
team.reverse();
console.log(team);
```

13. Or sort in descending order by chaining **reverse()** onto **sort()**:

```
team.sort(function(a, b) {
    return a.id - b.id;
}).reverse();
console.log(team);
```

D. sorting arrays of objects by string key

Sorting objects by keys with string values instead of numbers requires different logic. Here **a** and **b** are compared to see which is greater. The return value is 1 or -1, depending on the result of the evaluation.

14. Sort the team array by last name:

```
team.sort(function(a, b) {
    if(a.lName > b.lName) {
        return 1;
    } else {
        return -1;
    }
});
console.log(team);
```

15. Sort by first name, *descending*, by chaining **sort()** into **reverse()**:

```
team.sort(function(a, b) {
    if(a.fName > b.fName) {
        return 1;
    } else {
        return -1;
    }
}).reverse();
console.log(team);
```

E. randomize (shuffle) arrays of objects

Another use of the **sort(callback)** method is to randomize an array. Here the return value is **return 0.5 - Math.random()**, which has a 50-50 chance of being positive or negative.

16. Declare an array of consecutive numbers:

```
let nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15];
```

17. Randomize the numbers:

```
    nums.sort(function(a, b) {
        return 0.5 - Math.random();
    });
    console.log(nums); // [12,6,4,2,9,8,15,1,7,13,5,11,10,14,3]
```

The random sort / shuffle works with strings, too.

18. Declare an array of alphabetized strings and then randomize it:

```
    let froots = ['apple', 'banana', 'cherry', 'grape', 'kiwi', 'mango',
'orange', 'papaya', 'peach', 'pear', 'tangerine'];

    froots.sort(function(a, b) {
        return 0.5 - Math.random();
    });

    console.log(froots); // ['kiwi', 'grape', 'cherry', 'pear', 'mango',
'banana', 'peach', 'orange', 'tangerine', 'apple', 'papaya']
```

F. make an array from an object's keys

Object.keys()

The **Object.keys()** method returns an array of the object's values. The **team** array has 7 objects, all with the same 3 keys: **fName, lName** and **id**.

19. Save the first object in the array as its own variable and then save the **keys** to a new aray:

```
    const obj = team[0];
    const keysArr = Object.keys(obj);
    console.log(keysArr); // [fName, lName, id]
```

G. make an array from an object's values

Object.values()

The **Object.values()** method returns an array of the object's values.

20. Save the object **values** to a new aray:

```
    const valuesArr = Object.values(obj);
    console.log(valuesArr); // ['Pat', 'Hanson', 7445]
```

21. Use **pop()** to remove the last item, the id:

```
valuesArr.pop();
console.log(valuesArr); // ['Pat', 'Hanson']
```

22. Use **join()** to turn the **valuesArr** array into a string of first and last name:

```
const fullName = valuesArr.join(' ');
console.log('Greetings, ' + fullName + '!');
// Greetings, Pat Hanson
```

- **END: Lesson 06.05**
- **NEXT: Lesson 06.06**



SORTABLE MOVIES DATA

Addding Properties to an Array of Objects

Making Arrays of Objects Sortable by key

in ascending / descending order

In this lesson we will make a web page of movie info, consisting of a grid of divs, each containing a movie poster image and info about the film.

Let's start by checking out the final product:

1. Open **06.06-Sortable-Movies.html** and preview it in the browser.

The movies load in alphabetical order, although leading articles (A, The) are ignorned, so "A Beautiful Mind" starts with "B" and "The Deer Hunter" starts with "D".

From the **Sort by..** menu:

2. Choose **Year**. The movies are reordered from oldest to newest; check **Descending** to reverse the order (newest to oldest).

3. Choose **Length**. The movies are reordered from shortest to longest; check **Descending** to reverse the order (longest to shortest).

4. Choose **Title**. The movies are reordered alphabetically, by title from A-Z; check **Descending** to reverse the order (Z-A).

The JS code consists of four main parts:

- **movies.js**, the movie dataset, which is an array of objects
- a loop that adds some new properties to the objects
- a **sortMovies** function that sorts the movies by key, per the menu choice
- a **renderMovies** function which makes and outputs the movie info

5. Open **movies.js**. It contains an array, **movies**, of objects, each with **name, year** and **mins** (minutes) properties:

```
const movies = [
    { name: "2001: A Space Odyssey", year: 1968, mins: 149 },
```

```
        { name: "Ali", year: 2001, mins: 157 },
        -- ETC. --
        { name: "The Wizard of Oz", year: 1939, mins: 101 },
    ];
```

6. Switch from **FINAL.js** to **START.js** and reload the page.

The movies are gone. We will now code it all again from scratch.

7. Start by getting the **movie-holder** div, for displaying the movies:

```
    const movieHolder = document.querySelector('.movie-holder');
```

adding three properties to the movies objects

- **hrMin**
- We want to display the movie running time in 00:00 format.
- For this we need to convert total minutes to hours:minutes.
- We will add th 00:00 time format to the object as a property, **hrMin**
- **imgFile**
- We need the movie name for concatenating the image file path, but the movie names and file name don't exactly match.
- We will convert the movie names to image file name format.
- We will addd the image file name to the object as a property, **imgFile**.
- **noAThe**
- We will make a property to store the movie name with no leading "A" and "The". This will make it easier to sort by movie name.

We will start by adding the **hrMin** property.

8. Set up a loop to iterate the **movies** array:

```
    for(let i = 0; i < movies.length; i++) {
    }
```

9. Inside the loop, start by simplifying the current movie object:

```
        let movie = movies[i];
```

10. Next, get the **mins** property, and simplifiy it as a variable:

```
        let mins = movie.mins;
```

11. Get the **hours** by dividing **mins** by **60** and rounding down:

```
        let hours = Math.floor(mins / 60);
```

12. Use modulo **%** to get the remainder minutes:

```
        mins = mins % 60;
```

13. Add a leading zero if **mins** is less than 10. This way, we get **2:03**, not **2:3**.

```
        if(mins < 10) mins = '0' + mins;
```

14. Add a key, **hrMin**, to the object. The value is the **hours:mins** format:

```
        movie.hrMin = `${hours}:${mins}`;
```

Now, let's move on to adding the **imgFile** property:

15. Still in the loop, get the name of the movie and make it lowercase:

```
        let imgFile = movie.name.toLowerCase();
```

16. Use **replaceAll()** to replace the spaces in the movie names with hyphens, since the image file names have hyphens instead of spaces between the words:

```
        imgFile = imgFile.replaceAll(' ', '-');
```

17. Remove punctuation from the movie names by replacing commas, apostrophes, colons and periods with nothing.

```
        imgFile = imgFile.replaceAll("'", "");
        imgFile = imgFile.replaceAll(":", "");
```

```
        imgFile = imgFile.replaceAll(",", "");
        imgFile = imgFile.replaceAll(".", "");
```

18. Complete the file name by tacking on '.jpg' at the end:

```
        imgFile += '.jpg';
```

19. Add the **imgFile** property to the object:

```
        movie.imgFile = imgFile;
```

making a property with no leading articles

Now for the part where we make a property for the movie name, sans leading "A" or "The". This is the name that we will sort by.

20. Still in the loop, get the movie's first two and first four characters:

```
        let first2chars = movie.name.slice(0, 2);
        let first4chars = movie.name.slice(0, 4);
```

21. If the first two characters are "A ", make a new property **noAThe**, set equal to the full name, starting at the third character (at index 2) and going to the end:

```
        if(first2chars == "A ") {
                movie.noAThe = movie.name.slice(2);
        }
```

22. Else if the first four characters are "The ", make a new property **noAThe**, set equal to the full name, starting at the fifth character (at index 2) and going to the end:

```
        } else if(first4chars == "The ") {
                movie.noAThe = movie.name.slice(4);
        }
```

23. Else, the movie does not start with "A" or "The", so just set **noAThe** to equal the original name:

```
        } else {
                movie.noAThe = movie.name;
        }
```

24. Add one more line before closing the loop: a call to the **renderMovies()** function. This function runs on page load when the loop ends, and also whenever a choice is made to the **Sort by..** menu.

```
        renderMovies();
    } // end loop
```

25. Log the updated **movies** array. It should have all six properties: the original three: name, year and mins as well as the three new ones: hrMin, imgFile and noAThe:

```
    console.log(movies);
```

making the divs and images dynamically

Now that the movie objects have their new properties, we are ready to output movies to the DOM. Each movie will be in a div containing an image and some text. The same loop used to make the new properties can also be used to output the movie info.

renderMovies() loops through the movies array and makes and outputs the grid of movies:

26. Define the **renderMovies()** function. First thing to do is clear the **movieHolder** of any existing content:

```
        function renderMovies() {
                movieHolder.innerHTML = "";
        }
```

27. Set up a loop to iterate the movies array and simplify the current movie by passing it to a variable:

```
        for(let i = 0; i < movies.length; i++) {
                let movie = movies[i];
        }
```

28. Next in the loop, make a div, give it its class and output it to **movieHolder**:

```
        let divvy = document.createElement('div');
        divvy.className = 'divvy';
        movieHolder.appendChild(divvy);
```

29. Still in the loop, make an image object, set its source to the movie's jpg and output it to the div:

```
        let moviePoster = new Image();
        moviePoster.src = 'images/' + movie.imgFile;
        divvy.appendChild(moviePoster);
```

30. Below the image, output the text info for the movie:

```
        divvy.innerHTML += `${movie.name}<br>${movie.year} -
${movie.hrMin}`;
```

31. Close the loop, close the function and Reload the page. The grid of movies should be back, although the sort feature doesn't work yet. That's next.

sorting movies

Now for the user interactivity part: enabling the user to sort the movies by Title (name), Year and Duration (mins). Here's how that works:

- a **change** to the **select** menu calls the **sortMovies** function
- the function gets the selected **value**, which matches an object key
- the function sorts the movie objects by that **key**
- if the **Duration** checkbox is checked, the sort is reversed
- the function ends by calling the **renderMovies** function, which outputs the freshly sorted movies

32. Get the select menu and have it call the **sortMovies** function:

```
        const sortMenu = document.getElementById('sort-menu');
        sortMenu.addEventListener('change', sortMovies);
```

33. Get the checkbox. When a change occurs (check/uncheck), run an inline function that reverses the order of the movies and calls **renderMovies()** to refresh the display:

```
        const descCB = document.getElementById('desc');
        descCB.addEventListener('change', function() {
                movies.reverse();
                renderMovies();
        });
```

In a function, **this** refers to the object that called the function, which in the case of **sortMovies** is the menu itself. When the function is called, **this.value** is the menu choice, which equals the name of the key to sort by.

34. Define the **sortMovies()** function and get the menu choice, which is the sort key:

```
function sortMovies() {
        let sortKey = this.value;
}
```

In the last lesson, we learned that the sort() method works differently for strings and numbers. Our sort menu choices are:

- Title (name), which is a string
- Year, which is a number
- Duration, which is also a number in minutes
- we will use if-else logic to choose string or number sort.
- for a number sort, **sortKey** can be either **year** or **mins**.

35. If **sortKey == "name"**, do the string sort on the **noAThe** key, which is the name of the movie with no leading "A" or "The":

```
if(sortKey == "name") {
        movies.sort(function(a, b) {
                if(a.noAThe > b.noAThe) {
                        return 1;
                } else {
                        return -1;
                }
        });
}
```

35. Else the key is not "name", so do the number sort. Use the dynamic property access syntax, the square bracket, with **sortKey**, which is either "year" or "mins":

```
} else {
        movies.sort(function(a, b) {
                return a[sortKey] - b[sortKey];
        });
}
```

36. If the Duration checkbox is checked, reverse the sort:

```
if(descCB.checked) movies.reverse();
```

37. Also have the checkbox reverse the order of the movies whenever it is checked or unchecked:

```
descCB.addEventListener('change', function() {
        movies.reverse();
```

```
                renderMovies();
        });
```

38. Still inside sortMovies(), call the renderMovies() function and end the sortMovies() function:

```
           renderMovies();
        } // end sortMovies()
```

39. Reload the page and put the application through its paces. The sort feature should be working.

- **NEXT: Lesson 06.06**
- **Next: Lesson 06.07**



UNIT 07

LESSON 07.01



array callback methods

map(), filter()

Array **callback methods** are methods called on arrays that take a function as their argument. This is in contrast to array methods such as **push()** and **pop()**, which do *not* have functions for arguments. We have actually already been working with array callbacks. The **sort(function(a, b))** for sorting arrays of objects by key.

array.map()

- the **map()** method is called on an array: **array.map()**
- the **map()** method call is set equal to a variable: **const newArr = array.map()**
- the **map()** method takes a callback function: **array.map(function() {})**
- the callback takes the current array item (default value: **e**) as *its* argument
- the callback performs some operation on **e** on each item in the array, one by one (as with a loop)
- the **map()** method returns an array of transformed items

1. Open **01.02-Map-and-Filter.html**, preview it in the browser and check the console. The output is the product of the the map and filter methods.
2. Switch from using **FINAL.js** to **START.js**.
3. Open **07.01-Map-and-Filter-START.js**, which is where we will recreate the **FINAL.js** file from scratch.
4. Declare an array so that we can try out the **map()** method:

```
const fruits = ['apple', 'blueberry', 'cherry'];
```

comparing map() to a for-loop

The **map()** method iterates over an array like a for loop but without the actual loop. So before we run the **map()** method on our fruits array, let's review iterating an array with a for-loop. The code in the loop will make fruit pies and push them into a **pies** array.

5. Declare a new empty array called **pies**.

```

```

```
const fruits = ['apple', 'blueberry', 'cherry'];
const pies = [];
```

6. Set up a for-loop that runs once for each item in the array:

```
const pies = [];
for(let i = 0; i < fruits.length; i++) {
}
```

7. Each time the loop runs, declare a variable **pie** and set the value to the current fruit, as **fruits[i]** followed by the word "pie":

```
for(let i = 0; i < fruits.length; i++) {
  let pie = fruits[i] + ' pie';
}
```

8. Also each time through the loop, push the new pie into the array:

```
const pies = [];
for(let i = 0; i < fruits.length; i++) {
  let pie = fruits[i] + ' pie';
  pies.push(pie);
}
```

9. Below the loop, log **pies** to the console:

```
  pies.push(pie);
}
console.log(pies);
```

Now, for the **map()** version of our "pie maker"; it gets the job done without a loop:

10. Call the **map()** method on the **fruits** array, setting it equal to a new array, **piesArr**, which will receive the return value of the **map()** method:

```
const piesArr = fruits.map();
```

11. Pass a callback function to the **map()** method. The callback takes an argument of its own, **e**, which represents the current fruits array item:

```
const piesArr = fruits.map(function(e) {
});
```

12. Inside the curly braces, concatenate and **return** the pie, all in one line. The **return** pushes the result into the new array:

```
const piesArr = fruits.map(function(e) {
  return e + ' pie';
});
```

13. Log the new array to verify that it worked:

```
console.log(piesArr);
// ['apple pie', 'blueberry pie', 'cherry pie']
```

calling the argument something besides e

The callback function argument is **e** by default, but we could call it anything.

14. Run **map()** again, saving the return value to a new variable and with **fru**, short for fruit, as the callback argument, instead of **e**. It works exactly the same as before:

```
const piesAgain = fruits.map(function(fru) {
  return fruit + ' pie';
});

console.log(piesAgain);
// ['apple pie', 'blueberry pie', 'cherry pie']
```

two arguments: e and i

In addition to the current item, default value **e**, the current array **index** is available, with a default value of **i**.

15. Run **map()** again, saving the return value to a new array again, but this time with the fruit pies numbered. To number them, use the index plus 1. The concatenation is getting longer, so switch to **string interpolation**:

```
const numberedPies = fruits.map(function(e, i) {
  return `${i+1}. ${e} pie`;
});

console.log(numberedPies);
// ['1. apple pie', '2. blueberry pie', '3. cherry pie']
```

array.filter()

- the **filter()** method works like **map()**, except that **filter()** returns an array of fewer items than in the original array
- the items in the new array may not be transformed in any way: it's the same items as in the original array--just fewer of them.
- the **filter()** method uses boolean comparison logic to evaluate the items. Only evaluations that **return true** will result in the time being saved to the new aray.

Let's save to a new array only five-letter fruits. First, we'll do it with a loop:

16. Given a new array called fruitsArr that includes several five-letter items, declare a new, empty array to hold the five-letter fruits:

```
  const fruitsArr = ['apple', 'banana', 'cherry', 'grape', 'lemon',
'lime', 'mango', 'papaya', 'peach', 'orange'];

  const fiveLetterFruits = [];
```

17. Loop the array:

```
  for(let i = 0; i < fruitsArr.length; i++) {
  }
```

18. With each iteration, check if the length of the current string equals 5:

```
  for(let i = 0; i < fruitsArr.length; i++) {
    if(fruitsArr[i].length == 5) {
    }
  }
```

19. If the condition is true, push the fruit into the **fiveLetterFruits** array:

```
  for(let i = 0; i < fruitsArr.length; i++) {
    if(fruitsArr[i].length == 5) {
      fiveLetterFruits.push(fruitsArr[i]);
    }
  }
```

20. Log the **fiveLetterFruits** array:

```
  console.log(fiveLetterFruits);
  // ['apple', 'grape', 'lemon', 'mango', 'peach'];
```

Next, let's make same array using **filter()**, instead of a loop:

21. Call **filter()** on **fruitsArr**, setting the expression equal to a variable which will "catch" the returned items:

```
const fiveChars = fruitsArr.filter(function(fru) {
  if(fru.length == 5) {
    return fru;
});
```

22. Log the result:

```
console.log(fiveChars);
// ['apple', 'grape', 'lemon', 'mango', 'peach'];
```

23. It worked, but we can simplify things. The filter method returns true comparisons, so the if part is not necessary. Just return the comparison itself:

```
const fivers = fruits.filter(function(fru) {
  return fru.length == 5;
});

console.log(fivers);
```

chaining methods: filter() into map()

Suppose we want pies of only five-letter fruits. This requires **filter()** to save the five-letter fruits, and then **map()** to add "pie" to the string. We can run the two methods one right after another, with the second method called diretly on the first. This technique is called **chaining**.

24. Declare an array called **char5Pies** and set it equal to the filter part:

```
const char5Pies = fruitsArr.filter(function(e) {
  return e.length == 5;
})
```

25. Chain the **map()** method onto the end of the **filter()** method:

```
const char5Pies = fruitsArr.filter(function(e) {
  return e.length == 5;
}).map(function(e) {
```

```
    return e + ' pie';
  });
```

26. Log the resulting **char5Pies** array. It should be piess of just five-letter fruits:

```
  console.log(char5Pies)
  // ['apple pie', 'grape pie', 'lemon pie',
  'mangp pie', 'peach pie']
```

three arguments: e, i and a

In addition to the current item **e** and the current index **i**, there is a third argument available, and that is **a**, the array itself.

In this next example, we will use **map()** with all three arguments to make strings of consecutive items joined by a hyphen ('apple-banana', etc.). These will be saved to a new **smoothies** array.

27. Call **map()** on the array, setting it equal to a new array, **smoothies**:

```
  const smoothies = fruitsArr.map();
```

28. Pass in the callback function with all three arguments: **(e, i, a)**:

```
  const smoothies = fruitsArr.map(function(e, i, a) {
  });
```

We need both **a** (array) and **i** (current index), so that we can get not only the current item, but the *next* item, too:

29. Inside the map function, concatenate and return the hyphenated, consecutive words combos:

```
  const smoothies = fruitsArr.map(function(e, i, a) {
    return e + '-' + a[i+1];
  });
```

30. Console log the result:

```
  console.log(smoothies);
  // ['apple-banana', 'banana-kiwi', 'kiwi-mango', 'mango-orange',
'orange-papaya', 'papaya-peach', 'peach-undefined']
```

31. To get rid of the 'peach-undefined', just pop it off the array:

```
  smoothies.pop();
  console.log(smoothies);
  // ['apple-banana', 'banana-kiwi', 'kiwi-mango', 'mango-orange',
'orange-papaya', 'papaya-peach']
```

filter() by object property

Given an array of objects, each a food menu item:

```
  const entrees = [
    { name: 'Chicken with Waffles', vegetarian: false, price: 18, cals:
1200 },
    { name: 'Tofuburger', vegetarian: true, price: 8, cals: 480 },
    { name: 'Porterhouse Steak', vegetarian: false, price: 29, cals: 1180
},
    { name: 'Quinoa Casserole', vegetarian: true, price: 14, cals: 560 },
    { name: 'Lobster', vegetarian: false, price: 28, cals: 750 },
    { name: 'Ribeye Steak', vegetarian: false, price: 32, cals: 1320 },
    { name: 'Banon Cheeseburger', vegetarian: false, price: 14, cals: 1400
},
    { name: 'Shrimp Scampi', vegetarian: false, price: 23, cals: 1060 },
    { name: 'Quinoa Burger Deluxe', vegetarian: true, price: 16, cals: 630
},
    { name: 'Chicken Salad Supreme', vegetarian: false, price: 13, cals:
710},
    { name: 'Salmon Steak', vegetarian: false, price: 22, cals: 680 },
    { name: 'Pork Chop', vegetarian: false, price: 18, cals: 800 },
  ];
```

We will use **filter()** to save to a new array all **non-vegetarian** entrees with a minimum **price** of 15.

31. Call **filter()** with its callback on arrays and set that equal to a new array:

```
  const nonVeg15 = entrees.filter(function(e) {
  });

  console.log(nonVeg15);
```

&& (AND) operator for filtering with two conditions

32. Use the **&&** operator to return non-vegetarian menu items that have a minimum price of $25. The two conditions go together in parentheses:

```
  const nonVeg$25Min = entrees.filter(function(e) {
    return (!e.vegetarian && e.price >= 25);
  });
```

```
console.log(nonVeg$25Min);
```

32. Use the **&&** operator to return non-vegetarian items under 1000 calories:

```
const nonVeg1000CalMax = entrees.filter(function(e) {
  return (!e.vegetarian && e.cals < 1000);
});
console.log(nonVeg1000CalMax);
```

END: Lesson 07.01 NEXT: Lesson 07.02

07.01 Lab Exercises

array.map()

array.filter()

1. Given an array of vegetables, use **map()** to make new array of fresh veggies: ['fresh beet', 'fresh carrot', etc]

```
    const veggies = ['beet', 'carrot', 'celery', 'cucumber', 'broccoli',
'cauliflower', 'lettuce'];

    // YOUR CODE

    // console.log(freshVeggies); // ['fresh beet', 'fresh carrot', 'fresh
celery', 'fresh cucumber', 'fresh broccoli', 'fresh cauliflower', 'fresh
lettuce'];
```

2. Use **filter()**, make a new array containing only the veggies that start with the letter 'c':

```
    // YOUR CODE

    // console.log('C_veggies', C_veggies);
    // ['carrot', 'celery', 'cucumber', 'cauliflower']
```

3. Using filter-into-map chaining, get just the veggies that start with 'c', but with the word 'crunchy' before each veggie:

```
    // YOUR CODE

    // ['crunchy carrot', 'crunchy celery', 'crunchy cucumber', 'crunchy
cauliflower']
```

pluralize words
Using **map**, pluralize each fruit in the provided fruits array:

```
    const fruits = ["apple", "banana", "blueberry", "cherry", "grape",
"kiwi", "lemon", "mango", "orange", "papaya", "peach", "strawberry"]
```

Pluralization rules are as follows:

- if word ends in "y", drop the "y" and add "ies" ("cherry" -> "cherries")
- if word ends in "h" or "o", add "es" ("peach" --> "peaches", "mango" --> "mangoes")

- otherwise, add "s" ("apple" --> "apples")

```
    // YOUR CODE

    // console.log(fruitPl);
```

4. Given two arrays, **furniture** and **woods**, use map to generate a new **woodFurniture** array, having all ten pieces of furniture, each with a random wood types:

```
    const furniture = ["Desk", "Chair", "Bed", "Table", "Sofa", "Card
Table", "Tea Table", "Chest", "Dresser", "Sideboard"];

    const woods = ["Oak", "Walnut", "Mahogany", "Maple"];
```

5. Use map; inside the function, generate a random number in the range of the woodTypes array and use that value to select a random wood type:

```
    // YOUR CODE

    console.log(woodFurn);
```

assigning apartment numbers using map to make a 2D array from a 1D array

An apartment building has four apartments on each of six floors. The units are provided:

```
    const letters = ['A', 'B', 'C', 'D'];
```

Using map, generate all 24 apartment units and save them to a nested array, consisting of 6 items, each an array of 4 items.

Desired output: // [['1A', '1B', '1C', '1D'], ['2A', '2B', '2C', '2D'], ['3A', '3B', '3C', '3D'], ['4A', '4B', '4C', '4D']]

6. Call the map method on the array, saving whatever is returned in the end to a new array, aptNums:

```
    // YOUR CODE
```

7. each iteration of map requires a loop to make the four-item array, which is all the apts for one floor.

```
    // YOUR CODE

    console.log(apts); // [ ['1A', '1B', '1C', '1D'], ['2A', '2B', '2C',
'2D'], ['3A', '3B', '3C', '3D'], ['4A', '4B', '4C', '4D'] ]
```

07.01 Lab Exercise - SOLUTION

array.map()

array.filter()

1. Given an array of vegetables, use **map()** to make new array of fresh veggies: ['fresh beet', 'fresh carrot', etc]

```
    const veggies = ['beet', 'carrot', 'celery', 'cucumber', 'broccoli',
'cauliflower', 'lettuce'];

    const freshVeggies = veggies.map(function(e) {
        return 'fresh ' + e;
    })

    console.log(freshVeggies); // ['fresh beet', 'fresh carrot', 'fresh
celery', 'fresh cucumber', 'fresh broccoli', 'fresh cauliflower', 'fresh
lettuce'];
```

2. Use **filter()**, make a new array containing only the veggies that start with the letter 'c':

```
    const C_veggies = veggies.filter(function(veg) {
       return veg[0] == 'c';
    }

    console.log('C_veggies', C_veggies);
    // ['carrot', 'celery', 'cucumber', 'cauliflower']
```

3. Using filter-into-map chaining, get just the veggies that start with 'c', but with the word 'crunchy' before each veggie:

```
    const cVeg = veggies.filter(v => v[0] == 'c').map(v => 'fresh ' + v);
    console.log('cVeg:', cVeg); // ['crunchy carrot', 'crunchy celery',
'crunchy cucumber', 'crunchy cauliflower']
```

pluralize words

Using **map**, pluralize each fruit in the provided fruits array:

```
    const fruits = ["apple", "banana", "blueberry", "cherry", "grape",
"kiwi", "lemon", "mango", "orange", "papaya", "peach", "strawberry"]
```

Pluralization rules are as follows:

- if word ends in "y", drop the "y" and add "ies" ("cherry" -> "cherries")
- if word ends in "h" or "o", add "es" ("peach" --> "peaches", "mango" --> "mangoes")
- otherwise, add "s" ("apple" --> "apples")

```
    const fruitPl = fruits.map(function(fru) {
        let lastChar = fru[fru.length-1];
        let allButLastChar = fru.slice(0, fru.length-1);
        if(lastChar == "y") {
            return allButLastChar + "ies";
        } else if(lastChar == "h" || lastChar == "o") {
            return fru + "es";
        } else {
            return fru + "s";
        }
    });

    console.log(fruitPl);
```

4. Given two arrays, **furniture** and **woods**, use map to generate a new **woodFurniture** array, having all ten pieces of furniture, each with a random wood types:

```
    const furniture = ["Desk", "Chair", "Bed", "Table", "Sofa", "Card
Table", "Tea Table", "Chest", "Dresser", "Sideboard"];

    const woods = ["Oak", "Walnut", "Mahogany", "Maple"];
```

5. Use map; inside the function, generate a random number in the range of the woodTypes array and use that value to select a random wood type:

```
    const woodFurn = furniture.map(function(e) {
        let r = Math.floor(Math.random() * woods.length);
        return woods[r] + " " + e;
    });

    console.log(woodFurn);
```

assigning apartment numbers using map to make a 2D array from a 1D array

An apartment building has four apartments on each of six floors. The units are provided:

```
    const letters = ['A', 'B', 'C', 'D'];
```

Using map, generate all 24 apartment units and save them to a nested array, consisting of 6 items, each an array of 4 items.

Desired output: // [['1A', '1B', '1C', '1D'], ['2A', '2B', '2C', '2D'], ['3A', '3B', '3C', '3D'], ['4A', '4B', '4C', '4D']]

6. Call the map method on the array, saving whatever is returned in the end to a new array, aptNums:

```
    const aptUnits = unitLetters.letters.map();
```

7. each iteration of map requires a loop to make the four-item array, which is all the apts for one floor.

```
    const apts = letters.map(function(e) {
        let floor = [];
        for(i = 1; i <= 4; i++) {
            floor.push(i + e);
        }
        return floor;
    });

    console.log(apts); // [ ['1A', '1B', '1C', '1D'], ['2A', '2B', '2C',
'2D'], ['3A', '3B', '3C', '3D'], ['4A', '4B', '4C', '4D'] ]
```



Arrow Functions =>

An **arrow function** is a function that uses the **=>** symbol instead of the **function** keyword. Other things to know about arrow functions:

- **arrow functions return *implicitly***, which means that they always return a value--even when the **return** keyword is absent.
- **arrow functions** have concise sytnax:
  - **=>** is 2 characters, **function** is 8 characters
  - if there is only one line of logic, the curly braces **{ }** and **return** keyword may be omitted
  - if there is only one argument, the argument **( )** may be omitted.

Let's start by refactoring a "regular" function definition into an **arrow function**:

1. Define a "regular" function, which, though simple, has a paramter, so expects an input (argument) and also returns a value:

```
function greetUser(user) {
  return `Welcome back, ${user}!`;
}
```

hoising Function definitions are **hoisted**, meaning that they are lifted to the top of their **scope**, regardless of where they occur in the lines of code.

2. Call the function, supplying the expected argument. Since the function returns a value, set the call equal to a variable. This captures the return value; log the variable to see the greeting:

```
function greetUser(user) {
  return `Welcome back, ${user}!`;
}

let greeting = greetUser("Zebra123");
console.log(greeting); // Welcome back, Zebra123!
```

3. To verify that the function definition has been hoisted, call the function above the function definition; it still works:

```
  let greet = greetUser("Hoisted1");
  console.log(greet); // Welcome back, Hoisted1!

  function greetUser(user) {
    return `Welcome back, ${user}!`;
  }

  let greeting = greetUser("Zed1");
  console.log(greeting); // Welcome back, Zed1!
```

converting the function definition into an arrow function

4. Replace the word **function** with **const** or **let**. This turns **greetUser** into a variable declaration, so put an equal sign right after the name of the variable,:

```
const greetUser = (user) {
  return `Welcome back, ${user}!`;
}
```

5. Put the "fat arrow" **=>** to the right of the parentheses:

```
const greetUser = (user) => {
  return `Welcome back, ${user}!`;
}
```

6. Run the page. Now, the first function call--the one above the function--fails. This is because a fat-arrow function, set equal to a variable, is not hoisted. Comment out the first function call, so that it stops throwing the error.

making the fat-arrow function even simpler

7. Since there's only the one line of logic, we can make the fat-arrow function syntax even more concise. Get rid of the curly braces, the *return* keyword and the parentheses around the argument:

```
const greetUser = user => `Welcome back, ${user}!`;

console.log(greetUser('Zed1')); // Welcome back, Zed1!
```

It is even easier to convert a function expression into a fat-arrow function, since it a function expression is already set equal to a variable:

5. Start with a function expression:

```

```

```
const addNums = function(n1, n2) {
  return n1 + n2;
}

console.log(addNums(135, 246)); // 381
```

Convert the function expression to an arrow function:

6. Delete the word **function**:

```
const addNums = (a, b) {
    return a + b;
}
```

7. Add **=>** after the parentheses:

```
const addNums = (a, b) => {
    return a + b;
}
```

8. Since there is only one line of logic, delete **{}** and **return**. We have to leave the parentheses, because there is more than one argument

```
const addNums = (a, b) => a + b;

console.log(addNums(237, 365)); // 602
```

when you cannot simplify the arrow syntax too much
If there is more than one line of logic, as in this next fat-arrow function, you have to keep the **{}** and **return** keyword:

9. Declare a variable, set equal to a fat-arrow function. Have it peform a few lines of code, so that we must keep the **{}** and **return** keyword. However, since there is only one argument, omit the **()** around the argument:

```
const capitalizeWord = word => {
  let firstChar = word[0].toUpperCase()
  let restOfChars = word.slice(1); // isolate the rest of the word
  return firstChar + restOfChars; // concat and return the 2 parts
}

capit = capitalizeWord('elephant')
console.log(capit) // Elephant
```

arrow functions with map() and filter()

The concise syntax of the arrow function makes it well suited for deployment as the callback function of array methods, **map()** and **filter()** We will now do a map-filter chaining example, first as a "regular" function and then with arrow functons. The function specs are as follows:

- the function will take an array of strings and numbers as its argument
- the function will a new array, **nums**, containing just the numbers
- "number-like strings" are to be converted to real numbers and included in **nums**
- if the callback has only one callback argument, its parentheses can be omitted

First, we'll do the map-filter chaining version with the **function** keyword.

9. Declare an array,**mix**, of strings and numbers, including a few "number-like strings":

```
  const mix = ['appple', 3, 21, 'banana', 33, '55', 89, 'cherry', '63',
"77"];
```

10. Declare **nums**, set equal to **mix** with the **map()** method called on it and with its callback function with argument, **e**:

```
  const nums = mix.map(function(e) {
  });
```

11. Pass each item to the **Number()** method and return it:

```
  const nums = mix.map(function(e) {
    return Number(e);
  })
```

- if the item is already a number, passing it to the **Number()** method will have no effect.
- if the item is already a "number-like string", passing it to the **Number()** method return an actual number
- if the item is a string, such as "banana", passing it to the **Number()** method will return **NaN**, which is falsey.
- as we recall, falsey values return false in a boolean context, which means **NaN** will be filtered out by the **filter()** method

12. Chain **filter()** onto the end of **map()** to weed out the **NaN** values produced by the **Number()** method. Just return is **e**, the current item. The **NaN** items will return false and therefore be filtered out:

```
  const nums = mix.map(function(e) {
    return Number(e);
  }).filter(function(e) {
```

```
    return e;
});
```

13. Log **nums** to make sure it worked:

```
console.log(nums); // [3, 21, 33, 55, 89, 63, 77];
```

14. Now, try that again, but with arrow functions for callbacks. Just remove the word **function** and replace it with **=>** on the other side of the parentheses:

```
const numz = mix.map((e) => {
  return Number(e);
}).filter((e) => {
    return e;
});

console.log(numz); // [3, 21, 33, 55, 89, 63, 77];
```

15. Make the code more concise, as this is a hallmark of arrow functions:

- get rid of the curly braces and return keyword
- omit the callback argument parentheses, since there's just the one argument
- it is customary when writing such lean code to also lose the semi-colons
- as a final nod to minimalism, you can even make the array just one character:

```
const n = mix.map(e => Number(e)).filter(e => e);

console.log(n); // [3, 21, 33, 55, 89, 63, 77];
```

END Lesson 07.02
NEXT Lesson 07.03

07.02 Lab Exercises

converting map() & filter() to arrow function versions

This is a repeat of Lab 07.01, except this time, use arrow functions. Where possible, use the most concise syntax by omitting **{}**, the **return** keyword and argument **()**.

1. Given an array of vegetables, use **map()** to make new array of fresh veggies: ['fresh beet', 'fresh carrot', etc]

```
    const veggies = ['beet', 'carrot', 'celery', 'cucumber', 'broccoli',
'cauliflower', 'lettuce'];

    // YOUR CODE

    // console.log(freshVeggies); // ['fresh beet', 'fresh carrot', 'fresh
celery', 'fresh cucumber', 'fresh broccoli', 'fresh cauliflower', 'fresh
lettuce'];
```

2. Use **filter()**, make a new array containing only the veggies that start with the letter 'c':

```
    // YOUR CODE

    // console.log('C_veggies', C_veggies);
    // ['carrot', 'celery', 'cucumber', 'cauliflower']
```

3. Using filter-into-map chaining, get just the veggies that start with 'c', but with the word 'crunchy' before each veggie:

```
    // YOUR CODE

    // ['crunchy carrot', 'crunchy celery', 'crunchy cucumber', 'crunchy
cauliflower']
```

pluralize words
Using **map**, pluralize each fruit in the provided fruits array:

```
    const fruits = ["apple", "banana", "blueberry", "cherry", "grape",
"kiwi", "lemon", "mango", "orange", "papaya", "peach", "strawberry"]
```

Pluralization rules are as follows:

- if word ends in "y", drop the "y" and add "ies" ("cherry" -> "cherries")

- if word ends in "h" or "o", add "es" ("peach" --> "peaches", "mango" --> "mangoes")
- otherwise, add "s" ("apple" --> "apples")

```
    // YOUR CODE

    // console.log(fruitPl);
```

4. Given two arrays, **furniture** and **woods**, use map to generate a new **woodFurniture** array, having all ten pieces of furniture, each with a random wood types:

```
    const furniture = ["Desk", "Chair", "Bed", "Table", "Sofa", "Card
Table", "Tea Table", "Chest", "Dresser", "Sideboard"];

    const woods = ["Oak", "Walnut", "Mahogany", "Maple"];
```

5. Use map; inside the function, generate a random number in the range of the woodTypes array and use that value to select a random wood type:

```
    // YOUR CODE

    console.log(woodFurn);
```

assigning apartment numbers using map to make a 2D array from a 1D array

An apartment building has four apartments on each of six floors. The units are provided:

```
    const letters = ['A', 'B', 'C', 'D'];
```

Using map, generate all 24 apartment units and save them to a nested array, consisting of 6 items, each an array of 4 items.

Desired output: // [['1A', '1B', '1C', '1D'], ['2A', '2B', '2C', '2D'], ['3A', '3B', '3C', '3D'], ['4A', '4B', '4C', '4D']]

6. Call the map method on the array, saving whatever is returned in the end to a new array, aptNums:

```
    // YOUR CODE
```

7. each iteration of map requires a loop to make the four-item array, which is all the apts for one floor.

```
    // YOUR CODE
```

```
    console.log(apts); // [ ['1A', '1B', '1C', '1D'], ['2A', '2B', '2C',
'2D'], ['3A', '3B', '3C', '3D'], ['4A', '4B', '4C', '4D'] ]
```

07.02 Lab Exercises - SOLUTION

converting map() & filter() to arrow function versions

This is a repeat of Lab 07.01, except this time, use arrow functions. Where possible, use the most concise syntax by omitting **{}**, the **return** keyword and argument **()**.

1. Given an array of vegetables, use **map()** to make new array of fresh veggies: ['fresh beet', 'fresh carrot', etc]

```
    const veggies = ['beet', 'carrot', 'celery', 'cucumber', 'broccoli',
'cauliflower', 'lettuce'];

    const freshVegs = veggies.map(e 'fresh ' + e);

    console.log(freshVegs); // ['fresh beet', 'fresh carrot', 'fresh
celery', 'fresh cucumber', 'fresh broccoli', 'fresh cauliflower', 'fresh
lettuce'];
```

2. Use **filter()**, make a new array containing only the veggies that start with the letter 'c':

```
    const Cvegs = veggies.filter(v => v[0] == 'c');

    console.log(Cvegs); // ['carrot', 'celery', 'cucumber', 'cauliflower']
```

3. Using filter-into-map chaining, get just the veggies that start with 'c', but with the word 'crunchy' before each veggie:

```
    const crunchyVegs = veggies.filter(v => v[0] == 'c').map(v => 'fresh '
+ v);
    console.log(crunchyVegs); // ['crunchy carrot', 'crunchy celery',
'crunchy cucumber', 'crunchy cauliflower']
```

pluralize words
Using **map**, pluralize each fruit in the provided fruits array:

```
    const fruits = ["apple", "banana", "blueberry", "cherry", "grape",
"kiwi", "lemon", "mango", "orange", "papaya", "peach", "strawberry"]
```

Pluralization rules are as follows:

- if word ends in "y", drop the "y" and add "ies" ("cherry" -> "cherries")
- if word ends in "h" or "o", add "es" ("peach" --> "peaches", "mango" --> "mangoes")

- otherwise, add "s" ("apple" --> "apples")

```
    const fruitPl = fruits.map(fru {
        let lastChar = fru[fru.length-1];
        let allButLastChar = fru.slice(0, fru.length-1);
        if(lastChar == "y") {
            return allButLastChar + "ies";
        } else if(lastChar == "h" || lastChar == "o") {
            return fru + "es";
        } else {
            return fru + "s";
        }
    });

    console.log(fruitPl);
```

4. Given two arrays, **furniture** and **woods**, use map to generate a new **woodFurniture** array, having all ten pieces of furniture, each with a random wood types:

```
    const furniture = ["Desk", "Chair", "Bed", "Table", "Sofa", "Card
Table", "Tea Table", "Chest", "Dresser", "Sideboard"];

    const woods = ["Oak", "Walnut", "Mahogany", "Maple"];
```

5. Use map; inside the function, generate a random number in the range of the woodTypes array and use that value to select a random wood type:

```
    const woodFurn = furniture.map(e => {
        let r = Math.floor(Math.random() * woods.length);
        return woods[r] + " " + e;
    });

    console.log(woodFurn);
```

assigning apartment numbers using map to make a 2D array from a 1D array

An apartment building has four apartments on each of six floors. The units are provided:

```
    const letters = ['A', 'B', 'C', 'D'];
```

Using map, generate all 24 apartment units and save them to a nested array, consisting of 6 items, each an array of 4 items.

Desired output: // [['1A', '1B', '1C', '1D'], ['2A', '2B', '2C', '2D'], ['3A', '3B', '3C', '3D'], ['4A', '4B', '4C', '4D']]

6. Call the map method on the array, saving whatever is returned in the end to a new array, aptNums:

```
    const aptUnits = unitLetters.letters.map();
```

7. each iteration of map requires a loop to make the four-item array, which is all the apts for one floor.

```
    const apts = letters.map(e => {
        let floor = [];
        for(i = 1; i <= 4; i++) {
            floor.push(i + e);
        }
        return floor;
    });

    console.log(apts); // [ ['1A', '1B', '1C', '1D'], ['2A', '2B', '2C',
'2D'], ['3A', '3B', '3C', '3D'], ['4A', '4B', '4C', '4D'] ]
```



UNIT 07

LESSON 07.03



---

array.forEach()

array.find()

array.findIndex()

The **forEach** method is called on an array.

- Like **map** and **filter**, the **forEach** method passes the items of an array, one item at a time, to a callback function.
- Also like **map** and **filter**, the **forEach** method may be used instead of a loop to perform an operation on each item of an array.
- Unlike **map** and **filter**, **forEach** does not automatically return the result of that procedure.
- With no automatic return value, **forEach** is not subject to the strict rules of returning a new array with **map** ("must return the same number of items") or **filter** ("cannot transform the new items")
- The **forEach** method can be used as an alternative to map-into-filter chaining, since it can transform *and* filter items in an array, all in one go.

In this first example, we will square each number in an array of numbers and console.log the result.

1. Start with an array of numbers, and loop through it, logging each item to the console:

```
const nums = [3, 4, 5, 6, 7, 8, 9];
for(let i = 0; i < nums.length; i++) {
  console.log(i ** 2);
}
```

2. Now, refactor the code. Instead of using a loop, call **forEach()** on the array:

```
  nums.forEach();
```

3. Pass the **forEach** method an anonymous callback function, the argument of which represents the current item in the array. For this example, we will use **n**, representing the current **number** in the **nums** array:

```
  nums.forEach(function(n) {
  });
```

4. Log the square of the number to the console, and check the console.

```
nums.forEach(function(n) {
  console.log(n ** 2); // 9, 16, 25, 36, 49, 64, 81
});
```

5. It works, but better would be to refactor the callback function with arrow syntax:

```
nums.forEach((n) => {
  console.log(n ** 2);
});
```

6. With only one argument and one line of code inside it, the arrow syntax can be even more concise. Remove the argument parentheses and the function curly braces:

```
nums.forEach(n => console.log(n ** 2));
```

forEach instead of map() and filter()

For this next example, let's call the forEach method on an array of strings and produce a new transformed array. This sounds like a job for map, so let's compare how the procedure is different with forEach.

7. Declare a new array of **things** strings, as well as a new empty array, **greens**:

```
  const things = ['Apple', 'Arrow', 'Bean', 'Coconut', 'Lantern', 'Light',
'Monster', 'Zone'];
  const greens = [];
```

8. Call **forEach** on the **things** array, and pass in the callback function, with **e** representing the current **element**:

```
  things.forEach((e) => {

  });
```

9. Add 'Green ' before the item, and push the result into the *greens** array:

```
  things.forEach((e) => {
    greens.push('Green ' + e);
  });
```

10. Log the result. We should get: 'Green Apples', 'Green Arrow', 'Green Bean', etc.

11. Simplify the arrow function syntax by eliminating the callback argument parentheses as well as the function curly braces:

```
things.forEach(e => greens.push('Green ' + e));
```

Iterating over an array to make a new array of transformed items with the same number of items as in the orginal array can also be done with **map()**.

12. Try making the same "greens", but with **map()** instead of **forEach()**. With map, you set the method call equal to the new array which will store the returned a

```
const greenies = things.map((e) => {
  return 'Green ' + e;
})
console.log(greenies);
```

13. Recall that with map, we can make the code ultra-concise if there is only one argument and one line of logic. Try another version, but with no parentheses around the callback argument, **e**, no **return** keyword, and no function curly braces.

```
const g = things.map(e => 'Green ' + e);
console.log(g);
```

As we see, **forEach** does not return a new array automatically like **map()** does.With **forEach** you have to explicitly build the new array, which we do so using **push()** to add new items to the end of the new array.

forEach instead of map-filter chaining
Let's make "green things" again, but this time restricting our product to those things in the original array that have at least six letters. This requires filtering out the shorter words while transformiing the ones that make the cut. This is a job for map-filter or filter-map chaining. We will start with this before doing same with just forEach.

14. Call **filter()** on the **things** array, setting the call equal to the new array being returned:

```
const greenz = things.filter();
```

15. Pass in the callback function that returns when its condition returns true. The condition is items with a length greater than 5:

```
const greenz = things.filter(e => e.length > 5);
```

16. Log the resulting array, which contains only words of at least 6 letters:

```
  console.log(greenz); // 'Coconut', 'Lantern', 'Monster'
```

17. Chain the map method onto the end, to get Green Energy, Green Lantern, etc.:

```
  const greenz = things.filter(e => e.length > 5).map(e => 'Green ' + e);
  console.log(greenz); // 'Green Coconut', 'Green Lantern', 'Green
Monster'
```

18. Now, let's achieve the same result using just forEach. Since forEach does not return a new array automatically, declare a new, empty array as a first step. This is where we will have to push what we make inside the callback function:

```
    const geez = [];
    things.forEach();
```

19. In the forEach callback function, check to see if the current item meets the condition of having a length of at least 6:

```
    things.forEach(e => {
      if(e.length > 5) {
      }
    });
```

20. Inside the if-statement, push the new "green item" into the new array:

```
    things.forEach(e => {
      if(e.length > 5) {
        geez.push('Green ' + e);
      }
    });
```

21. Since the if-statement has just one line of logic, we can omit its curly braces:

```
    things.forEach(e => {
      if(e.length > 5) geez.push('Green ' + e);
    });
```

Using forEach on an array of objects

In this forEach example, we will revisit the Sortable Movies project, where we added new properties to an array of movie objects. In the original project, we added the new properties by of a for loop. We will revisit that approach and then contrast it with **forEach**. The properties we want to add are:

- **hrMin**: an alternative total minutes, where 134 mins is "2h 14m"
- **noAThe**: a version of each name that omits any leading article ("A" or "The")

```
  const movies = [
    { name: "The Wizard of Oz", year: 1939, mins: 101 },
    { name: "Casablanca", year: 1942, mins: 102 },
    { name: "Roman Holiday", year: 1953, mins: 118 },
    { name: "Lawrence of Arabia", year: 1962, mins: 227 },
    { name: "Lilies of the Field", year: 1963, mins: 94 },
    { name: "The Godfather, Part II", year: 1974, mins: 202 },
    { name: "The Shining", year: 1980, mins: 144 },
    { name: "Beverly Hills Cop", year: 1984, mins: 105 },
    { name: "Ghost", year: 1990, mins: 127 },
    { name: "Forrest Gump", year: 1994, mins: 142 },
    { name: "Ali", year: 2001, mins: 157 },
    { name: "Due Date", year: 2010, mins: 95 },
    { name: "Black Panther", year: 2018, mins: 134 },
    { name: "Avengers: Endgame", year: 2019, mins: 181 },
  ];
```

22. First, let's run the for loop version that we recall from the original project:

```
  for(let i = 0; i < movies.length; i++) {

    let mins = movies[i].mins; // get the minute property

    // make a new property in hour-minute format
    movies[i].hrMin = `${Math.floor(mins/60)}h ${mins%60}m`;

    let name = movies[i].name; // get the name property

    // make the noAThe property by checking if the name starts with "A" or
"The"
    // and then making a version of the name that omits the leading
article

    let newName; // newName starts off equal to the original name

    if(name.slice(0,2) == "A ") { // if movie name starts with "A "
      // make the new noAThe property without the "A "
      movies[i].noAThe = name.slice(2);
    } else if(name.slice(0,4) == "The ") { // if movie name starts with
"The "
      // make the new noAThe property without the leading "The "
      movies[i].noAThe = name.slice(4);
```

```
    } else {
      // keep the movie name the same since it does not start with "A " or
"The "
      movies[i].noAThe = name;
    }

  };

  console.log(movies);
```

23. Now, refactor the above code using **forEach**, with no loop and with the current item as **e**, rather than **movies[i]**:

```
  movies.forEach(e => {

      let mins = e.mins;
      e.hrMin = `${Math.floor(mins/60)}h ${mins%60}m`;
      let name = e.name;
      let newName;

      if(name.slice(0,2) == "A ") {
        e.noAThe = name.slice(2);
      } else if(name.slice(0,4) == "The ") {
        e.noAThe = name.slice(4);
      } else {
        e.noAThe = name;
      }

  });

  console.log(movies);
```

find() method

The **find()** method serves a narrowly specific role: it find the first item in an array that meets a condition, and it returns that item, *only*. It does not look for any more items that meet the condition.

- If NO item that satisfies the is found, the **find()** method returns **undefined**.

First, let's emulate the "find" algorithm using a for loop.

24. Given this **digits** array, set up a for loop that iterates over the array:

```
  let digits = [30, 54, 72, 89, 110, 137, 189];

  for(let i = 0; i < digits.length; i++) {

  }
```

25. Each time through the loop, pass the current item to an if-statement that uses the modulus operator to see if the current number, **digits[i]**, divided by 2 yields a remainder of 1:

```
for(let i = 0; i < digits.length; i++) {
  if(digits[i] % 2 == 1) {
  }
}
```

26. If the condition returns true, the number is odd, so log the odd number, and **break** to quit the loop:

```
for(let i = 0; i < digits.length; i++) {
  if(digits[i] % 2 == 1) {
    console.log(digits[i]);
  }
}
```

27. If we find the target, end the loop with a **break** command. This is especially important if there is a long array with lots of numbers yet to check. We are only finding the first odd number, so there is no point in checking the rest of the array:

```
for(let i = 0; i < digits.length; i++) {
  if(digits[i] % 2 == 1) {
    console.log(digits[i]);
    break;
  }
}
```

28. If no odd number was ever found, log **undeefined**. This must be done only *after* the loop ends and all items in the array have been checked:

```
for(let i = 0; i < digits.length; i++) {
  if(digits[i] % 2 == 1) {
    console.log(digits[i]);
    break;
  }
}
console.log(undefined);
```

The above always logs *undefined*--even if an odd number is found. What we need is a way to log *undefined* only if no odd number is found. We can do this by declaring a boolean **odd = false**, which we toggle to true inside the if-statement. The boolean value is then used as a condition for logging **undefined**:

29. Above the loop, declare a boolean:

```
let odd = false;
```

30. Inside the if statement, flip the boolean to **true**. This must come *before* the break line that ends the loop:

```
for(let i = 0; i < digits.length; i++) {
  if(digits[i] % 2 == 1) {
    console.log(digits[i]);
    odd = true;
    break;
  }
}
```

31. Finally, *after* the loop ends, add an if-statement that logs **undefined** if **odd** has stayed false throughout:

```
if(odd == false) {
  console.log(undefined);
}
```

32. Change the array values so that there are no odd numbers, to make sure that the **find()** method returns **undefined**.

33. Refactor the if-statement at the end:

- Use the **!** (not operator) instead of **== false**.
- Omit the curly braces, since the if-statement contains only one line of logics

```
  if(!odd) console.log(undefined);
```

find() method instead of for loop

Now let's use the **find()** method to hunt for the first odd number:

34. Call the **find()** method on the **digits** array. Since the method returns a value, set the call equal to a variable to store the return value:

```
let foundItem = digits.find();
```

35. Add the callback function. Its only code is the condition we are checking. Since it is the callback's only line, omit the function curly braces:

```
let foundItem = digits.find(e => e % 2 == 1);
console.log(foundItem);
```

array callback functions use loops under the hood
As we can see, the **find()** method is much more concise than a loop with if-statement. But it is important to understand the process terms of a loop, because the array callback functions -- **map()**, **filter()**, **forEach**, **find()** -- actually use loops "under the hood".

findIndex()
To return the position of the first instance of a target value, use **findIndex()** rather than **find()**.

33. Use **findIndex()** to find the position of the first odd number in the digits array:

```
let foundIndex = digits.findIndex(e => e % 2 == 1);
console.log(foundIndex);
```

29. Make sure that the **findIndex()** method returns **-1** by changing the **digits** array values so that there are no odd numbers.

END Lesson 07.03
NEXT: Lab 07.03
Lesson 07.04



forEach

1. Given an array of numbers, use forEach to cube the integers while doubling the floats. Log each result to the console.
   HINT: If rounding off the number does not change its value, it's an integer

```
const nums = [4, 3.14, 5, 7.98, 6, 17.75, 7, 55.55];
```

2. Refactor the above by making a new array of the answers, rather than console.logging them. Then log the resulting array.
   HINT: you need to declare a new, empty array above the forEach

```
const numeros = [];
// your code
console.log(numeros);
```

3. Refactor the above using a ternary instead of an if-else.
   HINT: A ternary is one line, so the arrow function syntax can be very concise. In fact, the entire forEach can be written in one line on code.

```
const numz = [];
// your code
console.log(numz);
```

filter-map-chaining vs. forEach

4. Given this array of fruits, use filter-into-map chaining to make jellybeans of fruits with 5 or fewer letters. The expected output is: 'apple jellybean', 'grape jellybean', 'kiwi jellybean', etc.

```
const fruits = ['apple', 'apricot', 'banana', 'blueberry', 'cherry',
'grape', 'kiwi', 'lime', 'lemon', 'mango', 'orange', 'pineapple',
'strawberry'];
```

5. Do another filter-into-map chaining to make rollups of fruits that begin with a consonant (so, no 'apple', 'apricot' or 'orange', which begin with a vowel).

HINT: It is more efficient to check to see if a word does NOT start with one of 5 vowels, as opposed to seeing if it starts with one of 21 consonants.

6. Use **forEach** with conditional logic to make the jellybeans or rollups, subject to the same conditions:

- 5 or fewer letters for jellybeans
- can't start with a vowel for roll-ups.
  HINT: start with two new empty arrays, one for each category of output

find() and findIndex() methods

7. Use the find() method to find the first animal that *starts* with a vowel.

```
    const animals = ['bunny', 'cat', 'dog', 'elephant', 'frog', 'giraffe',
'hedgehog', 'iguana', 'lynx', 'manatee'];
```

8. Use the findIndex() method to find the index of the first animal that *ends* in a vowel.

9. Use the find() method to find the last number that is equal to twice the number before it. In other words, find the "last double".
   HINT: Use all three available callback arguments: **e**, **i** and **a**.

- a[i+1] is the next item. if a[i+1] / 2 equals e, the current item, it is is doubled
- and so, the current item is not the last double,
- BUT if the a[i+1] / 2 does NOT equal e, then the current item IS the "last "double"
- The answer is 32, since that is the last double

```
    const numz = [8, 16, 32, 75, 100, 256];
```

10. Use **findIndex()** to find the **index** of the first number that has an integer for its square root.
    HINT: If the square root of an integer is rounded off, the integer does not change, so, if the square root is rounded off and it is still equal to the square root itself, then the square root is an integer. The target is 16, since that is the first number that has an integer for a square root. The expected answer is the index, so 1 is the correct answer

END Lab 07.03
SEE Lab 07.03 Solution



array.reduce() method

About the **reduce()** method:

- As with **map()** and **filter()**, **reduce()** is called on an array and runs a function on each array item, with the current item as the argument, **e**.
- The **reduce()** method returns ONE value which could be of any datatype (string, number, object, etc.).
- The **reduce()** method works on the principle of an **accumulator** value, which gets updated with each iteration of the array.
- The **accumulator** should be the first argument of the function.
- The **answer** is saved to the variable: **let answer = array.reduce()**

Let's try an example of **reduce()** to add up an array of numbers. The one value returned will be the **sum**.

```
const nums = [89, 95, 99, 95, 19, 95, 39, 79];
```

Next, call **reduce()** on the **nums** array, setting that equal to a new variable, **sum**.

```
const sum = nums.reduce();
```

Pass the function to the **reduce()** method. The function's arguments are the **accumultor**, which we will call **tot** and the current array item as **e**. The **accumulator** and the **answer** do not have to have different names--they could both be called **tot** or **sum**, but we will use different names to keep them distinct:

```
const sum = nums.reduce(function (tot, e) {
});
```

Inside the function, add the current item **e** to the **tot**, and return it. In this way, **tot**, *accumulates*:

```
const sum = nums.reduce(function(tot, e) {
    return tot + e;
});
```

Log the answer, **sum**, to see if it worked.

```
    const sum = nums.reduce(function (tot, e) {
        return tot + e;
    });
    console.log(sum); // 610
```

Let's try another example of **reduce()**. This time, it's an array of strings, but the answer is still a number:

```
    const veggies = ['beet', 'carrot', 'peas', 'celery', 'cucumber',
'broccoli'];
```

Use **reduce()** to add up the total number of chars of all the items ('beet' = 4, etc). Use the accumulator, **sum** again, since we are adding up a number:

```
    const totChars = veggies.reduce(function (sum, e) {
        return sum + e.length;
    });
    console.log(totChars); // beet64688
```

We get a strange result: **beet64688**. What happend is since the first array item, **beet**, is a string, the data type string was assumed for the accumulator, with the result that the item length values were concatenated--not added. To fix this, declare number as the data type. This is done right after the closing curly brace of the function, with the initial value set to zero:

```
    const totChars = veggies.reduce(function (sum, e) {
        return sum + e.length;
    }, 0);
    console.log(totChars); // 36
```

using an array of objects with reduce()

Given an array of movies, which you may recall from lesson 05.05 (Looping Arrays of Objects), we will use **reduce()** to get the average duration in minutes of all the movies:

```
    const movies = [
        { name: "The Wizard of Oz", year: 1939, mins: 101 },
        { name: "Casablanca", year: 1942, mins: 102 },
        { name: "Roman Holiday", year: 1953, mins: 118 },
        { name: "Lawrence of Arabia", year: 1962, mins: 227 },
        { name: "Lilies of the Field", year: 1963, mins: 94 },
        { name: "The Godfather, Part II", year: 1974, mins: 202 },
        { name: "The Shining", year: 1980, mins: 144 },
        { name: "Beverly Hills Cop", year: 1984, mins: 105 },
        { name: "Ghost", year: 1990, mins: 127 },
```

```
    { name: "Forrest Gump", year: 1994, mins: 142 },
    { name: "Ali", year: 2001, mins: 157 },
    { name: "Due Date", year: 2010, mins: 95 },
    { name: "Black Panther", year: 2018, mins: 134 },
    { name: "Avengers: Endgame", year: 2019, mins: 181 },
];
```

Set up the **reduce()** method call, saved to a variable:

```
const avgMovieMins = movies.reduce();
```

Pass the function to reduce, with arguments of **sum** and **e**. We want the average of the items' **mins** property, but to calculate an average, first we need the sum, which we divide by the total number of items:

```
const avgMovieMins = movies.reduce(function(sum, e) {
});
```

Set the accumulator to zero, even though it will be able to figure out to do math without it:

```
const avgMovieMins = movies.reduce(function(sum, e) {
}, 0);
```

Inside the function, add the current movie's running time as **e.mins** to **sum**.

```
const avgMovieMins = movies.reduce(function(sum, e) {
    sum += e.mins;
}, 0);
```

Calculate the average and return it. This we obtain by dividing the current item's mins property, **e.mins**, by the total number of items in the array, **a.length**. This requires us to pass **a**, representing the array, to the function. To do this, we also need to pass in **i**. The calculated average is added to the **sum**. When it is done, we have the total average running time of the movies.

```
const avgMovieMins = movies.reduce(function(sum, e, i, a) {
    return sum += e.mins / a.length;
}, 0);
console.log(avgMovieMins); // 137.78571428571428
console.log(Math.round(avgMovieMins)); // 138
```

reduce() instead of map-into-filter chaining

- **reduce()** returns a single value of any data type--including an array, if you so choose.
- unlike **map()**, **reduce()** is not constrained to returning an arraay of the same number of items as in the original array.
- unlike **filter()**, which cannot transform any of the items in the array it is called on, **reduce()** CAN change the items.
- **reduce()** can--all by itself--make what otherwise would require a combination of **filter()** and **map()** to produce: a filtered array of transformed items.

In this example, if the vegetable starts with "c", make vegetable juice, and save the juices to a new array. We will first implement this with filter AND map, before achiveing the same result with just reduce:

Copy-paste our **veggies** array, so that we have it close at hand, but change the name to **vegetables**:

```
    const vegetables = ['beet', 'carrot', 'peas', 'celery', 'cucumber',
'broccoli'];
```

Call the **filter()** method on the **vegetables** array, setting that equal to a new array, **cVegetables**:

```
    const cVegetables = vegetables.filter();
```

Pass in the function with argument **e** for the current array item:

```
    const cVegetables = vegetables.filter(function(e) {
    });
```

The first character of a string is at index 0, so **e[0]** is the first letter of the current item. If **e[0]** is "c", we want to save **e** to the new array by returning it:

```
    const cVegetables = vegetables.filter(function(e) {
        return e[0] == 'c';
    });
```

Log the result:

```
    console.log('cVegetables:', cVegetables);
    // ['carrot', 'celery', 'cucumber']
```

Copy-paste the filter code, and rename the new array **cVegJuices**:

```
    const cVegJuices = vegetables.filter(function(e) {
        return e[0] == 'c';
```

```
});
```

With **filter()** we have obtained the vegetables that start with "c", namely 'carrot', 'celery' and 'cucumber'. So now, it is time to make the juices. For this. we chain **map()** to the end of **filter()**:

```
const cVegJuices = vegetables.filter(function(e) {
    return e[0] == 'c';
}).map();
```

Pass **map()** its required function, with argument **e**:

```
const cVegJuices = vegetables.filter(function(e) {
    return e[0] == 'c';
}).map(function(e) {
});
```

Concatenate 'juice' onto the current item, **e**, and **return** that:

```
const cVegJuices = vegetables.filter(function(e) {
    return e[0] == 'c';
}).map(function(e) {
    return e + ' juice';
});
```

Log the result. It should be 'carrot juice', 'celery juice' and 'cucumber juice':

```
console.log('cVegJuices:', cVegJuices);
// ['carrot juice', 'celery juice', 'cucumber juice']
```

So that worked just fine, but now to get the same result using only the reduce method.

Call **reduce()** on the **vegetables** array, saving it to a new array, **cJuices**:

```
const cJuices = vegetables.reduce();
```

Pass in the required function with argument **e**:

```
const cJuices = vegetables.reduce(function(e) {
});
```

We need to tell **reduce()** to make an array, so pass a pair of empty square brackets **[]** to the method. This goes *after* the closing curly brace of the function:

```
const cJuices = vegetables.reduce(function(e) {
}, []);
```

Check to see **if** the first letter of the current item is "c", and if it is, add 'juice' to the item and **return** it:

```
const cJuices = vegetables.reduce(function(e) {
    if(e[0] == 'c') {
        return e + ' juice';
    }
}, []);
```

Log the result:

```
console.log('cJuices', cJuices);
// ['carrot juice', 'celery juice', 'cucumber juice']
```

We see that it worked, so we see that with **reduce()**, we can achieve what otherwise would require both map *and* filter. Also, notice that since we were not adding up a total, there was no **accumulator** argument.

END Lesson 07.04
NEXT: Lab 07.04
Lesson 07.05



setTimeout(function()):

setInterval(function()):

Calling a Function on a Delay with a Timer

setTimeout(function()): calling a function once on a timer

The **setTimeout()** method executes a callback function on a time delay.

- the **setTimeout()** method takes two arguments:
  - an inline, anonymous callback function
  - a delay in milliseconds before the function runs
- as a global **window** method, it can optionally be invoked as **window.setTimeout()**
- the **setTimeout()** runs only once

1. Call the **setTimeout()** method:

```
setTimeout();
```

2. Pass in the first argument: the anonymous, inline callback function:

```
setTimeout(function() {
});
```

3. Write some code for the function. We'll just console.log a greeting:

```
setTimeout(function() {
  console.log("Hello World!");
});
```

4. Pass in the second argument: a five-second delay (5000 ms) before the function runs:

```
setTimeout(function() {
    console.log("Hello World!");
}, 5000);
```

5. Refresh the browser. The greeting should appear after five seconds.
6. Run another setTimeout, but with the more concise arrow function syntax. Increase the delay, and change the message:

```
setTimeout(() => console.log("Hello World, again!"), 7500);
```

7. The callback can also call a function. Write a function, and then call it from inside the setTimeout callback. Again, increase the delay and change the message:

```
function greetWorld() {
    console.log("Hello World one more time!");
}

setTimeout(() => greetWorld(), 10000);
```

8. Write another function to call from the setTimeout callback, only this time give the function a parameter:

```
function greetUser(user) {
    console.log("Hello " + user + "!");
}

setTimeout(() => greetUser("Brian"), 12500);
```

9. Try a set timeout with output to the web page. Get the h2 tag where the output is to appear. The **querySelector('h2')** method gets the first h2 element, which is what we want:

```
const h2 = document.querySelector('h2');
```

10. On a 7-second delay, output a random integer from 1-100:

```
setTimeout(() => {
    h2.textContent = Math.ceil(Math.random() * 100);
}, 7000);
```

setInterval(function()): calling a function repeatedly on a timer

The **setInterval()** method executes a callback function repeatedly on a time delay.

- the **setInterval()** method takes two arguments:

    - an inline, anonymous callback function
    - a delay in milliseconds between function calls
- as a global **window** method, it can optionally be invoked as **window.setInterval()**
- the **setInterval()** runs repeatedly
- to stop the **setInterval()** use **clearInterval()**

11. Call the **setInterval()** method, and pass it its callback function:

```
setInterval(function() {
});
```

12. Have the callback output a random integer from 1-100 every 3 seconds:

```
setInterval(function() {
  h2.textContent = Math.ceil(Math.random() * 100);
}, 3000);
```

stopping setInterval() with clearInterval()
To stop the setInterval callback functino from running, use the **clearInterval()** method:

- set the **setInverval()** equal to a variable
- pass the variable to **clearInterval()**
- clearInterval() can be invoked on a counter or by a DOM event, such as a STOP button click

13. Above the **setInterval()**, declare a counter variable, **i**, and inside the callback, increment the counter with **i++**:

```
let i = 0;

setInterval(function() {
  i++;
  h2.textContent = Math.ceil(Math.random() * 100);
}, 3000);
```

14. Convert the callback to an arrow function. We will try to use arrow syntax for all inline anonymous functions, from now on:

```
let i = 0;

setInterval(() => {
  i++;
  h2.textContent = Math.ceil(Math.random() * 100);
}, 3000);
```

15. Wrap the output line in an if-statement that only runs if **i** is less than or equal to 10:

```
setInterval(() => {
  i++;
  if(i <= 10) {
    h2.textContent = Math.ceil(Math.random() * 100);
  }
}, 3000);
```

The if-statement will stop the output, but not the callback, which will keep running and incrementing the counter. To stop the callback function, we need to call the **clearInterval()** method.

16. Set the **setInterval()** method equal to a variable:

```
let intrvl = setInterval(() => {
  i++;
  if(i <= 10) {
    h2.textContent = Math.ceil(Math.random() * 100);
  }
}, 3000);
```

17. Add an **else** part that calls **myInterval()**, passing it the **intrvl** object:

```
let intrvl = setInterval(() => {
  i++;
  if(i <= 10) {
    h2.textContent = Math.ceil(Math.random() * 100);
  } else {
    clearInterval(intrvl);
  }
}, 3000);
```

setInverval() for choosing items from an array at random on a timer

In this next example, we start with an array of 13 animals. The **setInterval()** callback function will run 13 times, choosing an animal at random from the array each time

```
  const animals = ['aardvark', 'bunny', 'cheetah', 'deer', 'elephant',
'frog', 'giraffe', 'hippo', 'iguana', 'jaguar', 'kangaroo', 'lion',
'moose'];
```

18. Set up the **setInterval()** method with a callback that runs every 2 seconds:

```
setInterval(() => {
}, 2000);
```

19. Generate a random integer from 0-12, which is in the range of the 13-item **animals** array:

```
setInterval(() => {
  let r = Math.floor(Math.random() * animals.length);
});
```

20. Get the other h2, the one with id of **animals**:

```
let animalsH2 = document.getElementById('animals');
```

21. Output the animal whose index is the random number:

```
setInterval(() => {
  let r = Math.floor(Math.random() * animals.length);
  animalsH2.textContent = animals[r];
});
```

22. Specify a timer delay of 2 seconds, and check the browser:

```
setInterval(() => {
  let r = Math.floor(Math.random() * animals.length);
  animalsH2.textContent = animals[r];
}, 2000);
```

23. It should work, but the interval has no way of stopping yet. Set the method equal to a variable:

```
let myInterval = setInterval(() => {
  let r = Math.floor(Math.random() * animals.length);
  animalsH2.textContent = animals[r];
}, 2000);
```

24. Add a counter and increment it inside the callback function:

```
let counter = 0;
let myInterval = setInterval(() => {
  counter++;
  let r = Math.floor(Math.random() * animals.length);
```

```
  animalsH2.textContent = animals[r];
}, 2000);
```

25. Add an if-else-statement, so that the interval is cleared after it has run 13 times, wthe length of the **animals** array:

```
let counter = 0;
let myInterval = setInterval(() => {
  counter++;
  if(counter < animals.length) {
    let r = Math.floor(Math.random() * animals.length);
    animalsH2.textContent = animals[r];
  } else {
    clearInterval(myInterval);
  }
}, 2000);
```

avoiding repeats when choosing array items at random

That should work, but better might be to not repeat animals. The most efficent way to avoid repeats is to remove the item from the array after it has been chosen. This is done with the **splice()** method, which takes two arguments: a start index and the number or items to remove:

26. Use the **splice()** method to remove the chosen item:

```
let counter = 0;
let myInterval = setInterval(() => {
  counter++;
  if(counter < animals.length) {
    let r = Math.floor(Math.random() * animals.length);
    animalsH2.textContent = animals[r];
    animals.splice(r, 1);
  } else {
    clearInterval(myInterval);
  }
}, 2000);
```

The animals are being removed, but the process ends prematurely, once the incrmenting counter variable equals the length of the dwindling array. The answer is to pass **animals.length** to the if-statement. Zero is a **falsey** value, so once **animals.length** reaches 0, the **else** part will run, which clears the interval.

26. Remove the counter everywhere, with **animals.length** becoming the new condition to evaluate. In the ouput, switch to **+=**, so that we see all the animals appear. In this way we will know if our algorithm and code are working:

```
let myInterval = setInterval(() => {
  if(animals.length) { // 0 is falsey
```

```
      // if(animals.length > 0) { // also works
      let r = Math.floor(Math.random() * animals.length);
      animalsH2.textContent += animals[r] + " ";
      animals.splice(r, 1);
    } else {
      clearInterval(myInterval);
    }
  }, 2000);
```

27. Refresh the browser. All 13 animals should output, one at a time, in random order.

28. To see how the random number is always within the range of the dwindling array length, include the array length and the random number in the output. Add a
    tag to separate each animal on its own line. The inclusion of html in the string requires us to switch from **textContent** to **innerHTML**:

29. To visualize how the random number is always within the range of the dwindling array length, output the array length as well as the random number in the output for each animal.
    Add a
    tag to separate each animal on its own line.
    The inclusion of html in the string requires switching from **textContent** to **innerHTML**.
    The output is too much for concatenation, so use backticks:

```
  let myInterval = setInterval(() => {
      if(animals.length > 0) {
          let r = Math.floor(Math.random() * animals.length);
          animalsH2.innerHTML += animals[r] + "  - rand num: " + r + "
array len: " + animals.length;
          animals.splice(r, 1);
      } else {
          animalsH2.innerHTML += " done!";
          clearInterval(myInterval);
      }
  }, 2000);
```

END Lesson 08.01
NEXT: Lesson 08.02



UNIT 08

LESSON 08.02



Assigning Dynamic ID's to DOM Elements

Timing User Interaction with DOM Elements

In this lesson:

- We will use JS to dynamically assign id's and classes to DOM elements.
- We will then keep track of -- and time -- the user's hovering over these elements.
- We will then store the resulting data to an array of objects.

We do this to emulate software which records how long various page elements are holding user attention -- or are being ignored. Naturally, a real application would capture more than hovering; there's scrolling and clicking--and of course swiping on mobile devices. But for our purposes here, just tracking hovering will be a great coding exercise that lets us apply what we have just learned about **setInterval** while also giving us insight into how user engagement is actually tracked on the web.

1. Open **08.02-Timing-Using-Engagement.html** and preview the page in the browser. The user interface consists of three paragraphs and a button that says **Report Hover Time**. Notice that one of the paragraphs as an id, one has a class, and one has neither:

```
<p>
    This p tag has no attributes..
</p>

<p id="my-parag">
    This p tag has no class but it does have an id.
</p>

<p class="special-p">
    This p tag already the "special-p" class, but no id.
</p>
```

The idea is, the p tags have an irregular mix of id's and no id's. We will standardize them by giving them consecutively numbered ids.

2. Open the console side-by-side with the page elements.
3. Mouse over the first paragraph and hold the mouse there. Notice in the console that every second of hovering is being logged by a counter.
4. Mouse out (move the cursor away) from the first paragraph, and notice that a summary of the interaction appears in the console.

5. Repeat the mouseover-mouseout behavior on the other two paragraphs, pausing several seconds each time.
6. Click the button. Now, the console reports a summary of all three interactions: you get an array of objects, with each object being a record of an interaction, with properties of **elementID** and **hoverSeconds**:

```
{elementID: 'parag1', hoverSeconds: 3}
{elementID: 'parag2', hoverSeconds: 6}
{elementID: 'parag3', hoverSeconds: 4}
```

7. Now that we have seen the final product in action, switch from using **FINAL.js** to **PROG.js**.
8. We want to assign id's to all the p tags, so bring all three of them into JS at once with **querySelectorAll('p')**. This will make an array of objects:

```
const pTagArr = document.querySelectorAll('p');
```

9. Log the array to see what we have. It's the actual p-tags:

```
console.log('pTagArr:', pTagArr);
```

10. Loop through the array, assigning each item -- each paragraph object -- two listeners: one to call a function on **mouseover**, the other to call a function on **mouseout**. The function names are **onMouseOver** and **onMouseOut**, respectively:

```
for(let i = 0; i < pTagArr.length; i++) {
    pTagArr[i].addEventListener('mouseover', onMouseOver);
    pTagArr[i].addEventListener('mouseout', onMouseOut);
}
```

11. Assign the dynamic **id**, property, which starts with **parag** and concats on **(i+1)**, the loop counter, plus 1:

```
for(let i = 0; i < pTagArr.length; i++) {
    pTagArr[i].addEventListener('mouseover', onMouseOver);
    pTagArr[i].addEventListener('mouseout', onMouseOut);
    pTagArr[i].id = 'parag' + (i+1);
}
```

12. Assign the dynamic **className** property, which assign the existing **my-parag** class to the p tag element. Log the item each time through the loop:

```
    for(let i = 0; i < pTagArr.length; i++) {
        pTagArr[i].addEventListener('mouseover', onMouseOver);
        pTagArr[i].addEventListener('mouseout', onMouseOut);
        pTagArr[i].id = 'parag' + (i+1);
        pTagArr[i].className = 'my-parag';
        console.log(pTagArr[i]);
    }
```

13. Declare a few global variables:

- **myInterval** for stopping the **setInterval** method on mouseout
- **hoverSecs** for timing how many seconds a p tag is hovered over
- **reportArr** an array for storing the hover time data

```
    let myInterval;
    let hoverSecs = 0;
    const reportArr = [];
```

14. Define the **onMouseOver** function. The first thing it does is instantiate a Date() object and call the getSeconds() method and save it to a variable called **startSec**. This returns the current time's seconds property. So if the time is now 1:23:45, **startSec** equals 45.

```
    function onMouseOver() {
        let startSec = new Date().getSeconds();
    }
```

15. We need to start a timer to see how long before the mouseout occurs, so call the **setInterval** method and pass it an arrow function for its callback. Set the whole expression equal to **myInterval**, so that later we can use the variable to clear the interval:

```
    function onMouseOver() {
        let startSec = new Date().getSeconds();
        myInterval = setInterval(() => {
        });
    }
```

16. Inside the callback function, declare a new instance of Date().getSeconds(), and instruct the callback to repeat every 1000 milliseconds:

```
    function onMouseOver() {
        let startSec = new Date().getSeconds();
        myInterval = setInterval(() => {
            let newSec = new Date().getSeconds();
```

```
        }, 1000);
    }
```

17. Every 1 second, the callback function will run, with **newSec** updated each time. To count the mouseover time, set **hoverSecs** equal to **newSec** minus the original **startSec**:

```
    function onMouseOver() {
        let startSec = new Date().getSeconds();
        myInterval = setInterval(() => {
            let newSec = new Date().getSeconds();
            hoverSecs = newSec - startSec;
        }, 1000);
    }
```

18. Every minute the second counter reaches a maximum of 59 before restarting at 0. If this results in a negative **hoverSecs**, add 60 to the total. Log **hoverSecs** so that we see the hover seconds updating every second:

```
    function onMouseOver() {
        let startSec = new Date().getSeconds();
        myInterval = setInterval(() => {
            let newSec = new Date().getSeconds();
            hoverSecs = newSec - startSec;
            if(hoverSecs < 0) totSec += 60;
            console.log(hoverSecs);
        }, 1000);
    }
```

19. Declare the **onMouseOut** function, which runs when the user mouses out (moves away from) a p tag. The first thing to do is stop the counter, which we do by passing the **myInterval** variable to the **clearInterval** method:

```
    function onMouseOut() {
        clearInterval(myInterval);
    }
```

20. Log a message that tells how long the element of this particular id was hovered over:

```
    function onMouseOut() {
        clearInterval(myInterval);
        console.log(`element of id ${this.id} was hovered over for
${totSecs} seconds.`);
    }
```

21. Declare an object called **reportObj**, and assign it two properties:

- **elementID** set equal to **this.id**, the id of the tag
- **hoverSecs** set equal to the hoverSecs variable--the hover time

```
    function onMouseOut() {
        clearInterval(myInterval);
        console.log(`element of id ${this.id} was hovered over for
${totSecs} seconds.`);
        let reportObj = {
            elementID: this.id,
            hoverSecs: hoverSecs,
        }
    }
```

22. Push the object into the **reportArr** array:

```
    function onMouseOut() {
        clearInterval(myInterval);
        console.log(`element of id ${this.id} was hovered over for
${totSecs} seconds.`);
        let reportObj = {
            elementID: this.id,
            hoverSecs: hoverSecs,
        }
        reportArr.push(reportObj);
    }
```

23. Get the button:

```
    const reportBtn = document.querySelector('button');
```

24. Have the button listen for a click on itself and call the **reportHoverTime** function when a click occurs:

```
    reportBtn.addEventListener('click', reportHoverTime);
```

25. Write the **reportHoverTime** function. All it does is log the **reportArr** array to the console:

```
    function reportHoverTime() {
        console.log(reportArr);
    }
```

26. Refresh the browser and split the screen into web page and console.

27. Hover over the various p tags for several seconds each. Hover over the tags again, in a different order for varying amounts of time.

28. Click the **Report Hover Time** button. The object array with summary of hover data should appear in the console.

And with that we know how long the user hovered over each p tag.

END: Lesson 08.02



Blackjack - Pt. 1: DEAL..!

Review of Lesson 05.04: Making a deck of cards with a nested loop

New for Lesson 08.03: Deal Blackjack on a timer with setInterval

Keep score and display the score to the DOM

Detect Blackjack (21) for the Player, the Dealer -- or both

Prompt Player to Hit or Stand

Review the code for making a deck of cards as an array of card objects:

1. Given: Arrays of **kinds** and **suits**. These are the "raw materials" for making a deck of cards:

```
    const kinds = [2, 3, 4, 5, 6, 7, 8, 9, 10, 'Jack', 'Queen', 'King',
'Ace'];
    const suits = ['Diamonds', 'Hearts', 'Spades', 'Clubs'];
    const deck = [];
```

2. Review: Set up a nested for loop that iterates over the **kinds** and **suits** arrays:

```
    for (let i = 0; i < kinds.length; i++) {
        for (let j = 0; j < suits.length; j++) {
```

3. Review: Simplify the current array items by passing them to variables:

```
            let kind = kinds[i];
            let suit = suits[j];
```

4. Review: Concatenate the card name and image file names. The card "Queen of Diamonds" corresponds to the file name "Queen-of-Diamonds.png"

```
            let name = `${kind} of ${suit}`;
            let file = `${kind}-of-${suit}.png`;
```

5. Review: Declare a variable, **valu**, with an inital value of 0. This variable is for storing the numeric value of the card:

```
            let valu = 0;
```

6. Review: Set the **valu** property, based on the **kind** of card. The **length** of the **kind** reveals if it is a face card, since only "Jack", "Queen", "King" are longer than three characters:

```
            if(kind.length > 3)  {
                valu = 10;
            } else if(kind == 'Ace') {
                valu = 11;
            } else {
                valu = kind;
            }
```

Review: Each card is an object having five properties:

- **name**: the name of the card: "Jack of Hearts"
- **file**: the card file name: "Jack-of-Hearts.png"
- **kind**: 2-10, 'Jack', 'Queen', 'King', 'Ace'
- **suit**: 'Diamonds', 'Hearts', 'Spades', 'Clubs'
- **valu**: numeric value; face card = 10, Ace = 11

7. Review: Declare a card object with the five properties, the values of which are the corresponding variables:

```
            let card = { name: name, file: file, kind: kind,
                         suit: suit, valu: valu };
```

8. Review: Push the card object into the deck array:

```
            deck.push(card);
        }
    }
```

9. Review: Shuffle (randomize) the deck:

```
    deck.sort((a,b) => {
        return Math.random() - 0.5;
    });
    console.log('shuffled deck:', deck);
```

10. Review: Make a shoe consisting of 6 decks of cards, using the spread ... operator:

```
    const shoe = [...deck, ...deck, ...deck, ...deck, ...deck, ...deck];
```

11. Review: Shuffle (randomize) the shoe:

```
    shoe.sort((a,b) => {
        return Math.random() - 0.5;
    });
    console.log('shuffled shoe:', shoe);
```

12. Get the DOM elements:

- get the **DEAL** button and assign it an event listener for calling the **deal** function when clicked
- get the **HIT** and **STAND** buttons (they will not be assigned listeners yet)
- get the **h2**, which is for outputting prompts ("HIT or STAND?", etc.)

```
    const dealBtn = document.getElementById('deal-btn');
    dealBtn.addEventListener('click', deal);
    const hitBtn = document.getElementById('hit-btn');
    const standBtn = document.getElementById('stand-btn');
    const outputH2 = document.querySelector('h2');
```

13. Get the divs that hold the player and dealer hands, as well as those divs which display the player and dealer scores:

```
const playerCardsDiv = document.getElementById('player-cards-div');
const dealerCardsDiv = document.getElementById('dealer-cards-div');
const playerScoreDiv = document.getElementById('player-score-div');
const dealerScoreDiv = document.getElementById('dealer-score-div');
```

14. Declare global variables needed for keeping track of the deal:

- arrays for holding the player's and dealer's hands
- variables for keeping score
- **dealCounter** for keeping track of total cards dealt

```
let dealCounter = 0;
let playerHand = [];
let dealerHand = [];
let playerScore = 0;
let dealerScore = 0;
```

Dealing Blackjack

Now that we have our six-deck shoe, all shuffled and ready to go, let's deal a hand of Blackjack. We dealt a hand of poker in an earlier lesson, but this will be different. To better simulate game play, we will use **setInterval** to deal on a 1 second delay between cards.

- the deal consists of player and dealer hands, each of which receives two cards to start.
- the dealer's first card -- the "hole card" -- is dealt face down, so that the player only gets to see one of the dealer's cards to start.

15. Define the deal function:

```
function deal() {
```

16. Since this is a new hand, reset the scores and "clear the table"

- reset the counter variable that keeps track of cards dealt
- reset the player and dealer scores
- empty the divs that display the cards
- clear the text from the output h2
- empty the arrays that store the player and dealer hands

```
    dealCounter = 0;
    playerScore = 0;
    dealerScore = 0;
    playerScoreDiv.innerHTML = 'Player Score: 0';
```

```
        dealerScoreDiv.innerHTML = 'Dealer Shows: 0';
        playerCardsDiv.innerHTML = '';
        dealerCardsDiv.innerHTML = '';
        outputH2.innerHTML = "";
        playerHand = [];
        dealerHand = [];
```

17. Call the **setInterval** method, with its callback function set equal to a variable, **myInterval**, which will be used to clear the interval (stop deal):

```
    let myInterval = setInterval(() => {
```

18. Increment the counter that keeps track of how many card have been dealt

```
        dealCounter++;
```

19. If this is the 4th card being dealt, make this the last card by clearing the interval, which stops the setInterval timer from running again.

```
        if(dealCounter == 4) clearInterval(myInterval);
```

20. Instantiate a new **Image** object to hold the card image:

```
        let pic = new Image();
```

21. Pop a card object off the **shoe** array, and save it as a new **card**:

```
        let card = shoe.pop();
```

22. If this is not the hole card (which is the second card dealt overall), set the image source to the card image file path, ending in the **card.file** property:

```
        if(dealCounter != 2) {
            pic.src = `images/cards350px/${card.file}`;
```

23. **else**, if this *is* the "hole card", deal it face-down by setting its source to the back-of-card image:

```

```

```
        } else {
            pic.src = 'images/cards350px/0-Back-of-Card-Red.png';
        }
```

24. Set up an if-else statement to handle the logic for dealing two cards each to the player and dealer, starting with the player.

- the if condition uses the **% mod** operator to check the remainder when the counter is divided by 2
- if the remainder is 1, this is either the first or third card, both of which go to the player:

```
        if(dealCounter % 2 == 1) {
```

25. Output the card to the player's div:

```
            playerCardsDiv.appendChild(pic);
```

26. Push the card into the player's hand:

```
            playerHand.push(card);
```

27. Increment the player's score:

```
            playerScore += card.valu;
            playerScoreDiv.innerHTML = "Player Score: " + playerScore;
```

28. Add the else part to handle the two cards which are dealt to the dealer:

```
        } else {
```

29. Make the dealer cards a bit smaller to make them appear farther away:

```
            pic.style.width = "105px";
            pic.style.height = "auto";
```

30. Output the card to the dealer's div:

```
            dealerCardsDiv.appendChild(pic);
```

31. Push the card into the dealer's hand:

```
            dealerHand.push(card);
```

32. Update the dealer's score:

```
            dealerScore += card.valu;
        }
```

33. Update **Dealer Shows** once the deal ends. This is not the dealer's score, but rather the value of the dealer's second card--the "non-hole card":

```
        if(dealerHand.length == 2) {
            dealerScoreDiv.innerHTML = "Dealer Shows: " +
dealerHand[1].valu;
```

34. Log the dealer's full hand and secret score to the console:

```
            console.log('dealer hand: ', dealerHand, 'dealer score: ',
dealerScore);
```

35. If no one has blackjack, deactivate the DEAL button so that it cannot be clicked again:

```
            if(dealerScore != 21 && playerScore != 21) {
                dealBtn.disabled = true;
```

36. Also mute the color of the DEAL button so that it looks unclickable:

```
                dealBtn.classList.add('disabled-btn');
```

37. Un-mute the HIT and STAND buttons and set their **disabled** property to **false**. The button appearance is handled by removing and adding classes:

```
                hitBtn.disabled = false;
                standBtn.disabled = false;
            }
```

38. Prompt the player to "HIT or STAND?". For better game play pacing, do the prompt on a 1.5 second delay with **setTimeout**:

```
            setTimeout(() => {
                if(playerScore != 21) {
                    outputH2.innerHTML = "HIT or STAND?";
                    hitBtn.classList.remove('disabled-btn');
                    standBtn.classList.remove('disabled-btn');
                    hitBtn.classList.add('enabled-btn');
                    standBtn.classList.add('enabled-btn');
                }
            }, 1500);
        }
```

39. Check to see if either the player or dealer (or both, or neither) have Blackjack, and make the correct announcement on a 1 second delay. If neither has Blackjack, prompt the player to "HIT or STAND":

```
            setTimeout(() => {
                if(playerScore == 21 && dealerScore == 21) {
                    outputH2.innerHTML = 'Dealer and Player both have
Blackjack!<br>It's a Push!';
                } else if(playerScore == 21) {
                    outputH2.innerHTML = 'BLACKJACK! You win!';
                } else if(dealerScore == 21) {
                    outputH2.innerHTML = 'Dealer has BLACKJACK! You
lose!';
                } else { // no one has Blackjack
                    outputH2.innerHTML = "HIT or STAND?";
                }
            }, 1000);
```

40. Set the **setInterval** timer to **1000**, so that cards are dealt at 1 second intervals:

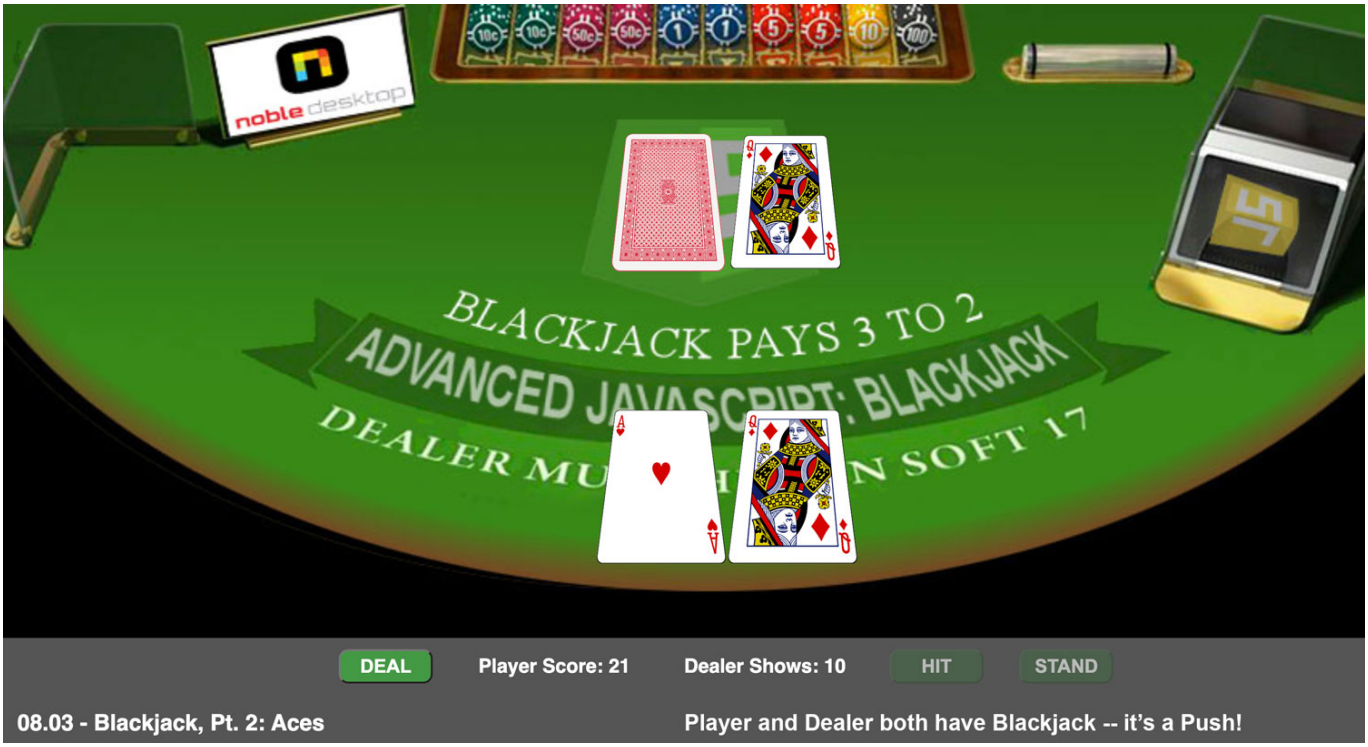
```
        }, 1000);

    } // end deal() function
```

41. Run the game in the browser and click **DEAL** to make sure the cards are being dealt properly and that the scoring is correct.

42. Also check the console to make sure that the dealer hand and full score are appearing.

END: Lesson 08.03
NEXT: Lesson 08.04



Blackjack - Pt. 2: Aces..!

evaluating Aces as worth either 11 or 1

testing the various Blackjack scenarios

The next thing we need to do for our blackjack application is attend to the tricky issue of how to handle Aces, which are worth either 11 or 1, depending on the score. For the purposes of this discussion, an "Ace 11" refers to an Ace whose value is 11, as opposed to 1.

evaluating an Ace: is it worth 11 or 1..?
If an "Ace 11" would bust the player or dealer, its value must be changed to 1:

- an Ace is initially assigned a value of 11 as its **valu** property
- if the player or dealer is dealt an Ace to start, the Ace is worth 11
- but if the player or dealer is dealt another Ace, the new Ace cannot also count as 11, as that would result in a score of 22, which is a bust. So, the second Ace counts as 1, which yields a score of 12.

keeping track of the Ace score separately
In order to set Ace values, we must keep track of Aces dealt. For this we will declare two new global variables, one for the dealer's Ace score, the other for the player's Ace score.

7. Above the **deal()** function, declare these two variables, **pAceScore** (player Ace score) and **dAceScore** (dealer Ace score):

```
    let pAceScore = 0; // player Ace score
    let dAceScore = 0; // dealer Ace score
```

Next, we need to go back into the DEAL function from the last lesson and account for Aces. If anyone has *one* Ace, that Ace counts as 11. Since you cannot bust on the deal, if anyone has *two* Aces, the second Ace is worth 1, for a score of 12.

8. Inside the **deal()** function, review the logic of the if-else block, where cards are dealt and scores are tallied. The numbering in the comments pertains to the steps of the *previous* lesson:

```
        // 24. Set up an if-else statement to handle the logic for dealing
two cards each to player and dealer, starting with the player.
        if(dealCounter % 2 == 1) {
            // 25. Output the card to the player's div
            playerCardsDiv.appendChild(pic);
            // 26. Push the card into the player's hand
            playerHand.push(card);
```

9. Right before "old step 27", check to see if the card is an Ace:

```
            if(card.kind == "Ace") {
            }

            // 27. Increment the player's score
            playerScore += card.valu;
            playerScoreDiv.innerHTML = "Player Score: " + playerScore;
```

10. We need a nested if here. Check to see if the **pAceScore** is 0, which tells us if this is the first Ace the player has been dealt:

```
            if(card.kind == "Ace") {
                if(pAceScore == 0) {
                }
            }
```

11. If **pAceScore** is 0, the player does *not* already have an Ace, so set **pAceScore** to 11:

```
            if(card.kind == "Ace") {
                if(pAceScore == 0) {
                    pAceScore = 11;
```

```
            }
        }
```

12. If the **pAceScore** is *not* 0, the player already *has* an Ace. A hand cannot have two "Ace 11's", so the second Ace counts 1. Right before "old step 27" where the player's score is updated, increase **pAceScore** to 12 *and* set the value of the card to 1. Log **pAceScore** to see if the logic is working:

```
        if(card.kind == "Ace") {
            if(pAceScore == 0) {
                pAceScore = 11;
            } else {
                pAceScore = 12;
                card.valu = 1;
            }
        }
        console.log('pAceScore:', pAceScore);

        // 27. Increment the player's score
        playerScore += card.valu;
        playerScoreDiv.innerHTML = "Player Score: " + playerScore;
```

13. The **else** handles dealer Aces in the same way. Copy the if-else from the previous step and paste it between "old steps" 31 and 32. Change **pAceScore** to **dAceScore**. Log **dAceScore** to see if it worked:

```
        // 28. Add the else part to handle dealers dealt to the dealer
        } else {
            // 29. Make the dealer cards a bit smaller, to make them
 appear farther away
            pic.style.width = "105px";
            pic.style.height = "auto";
            // 30. Output the card to the dealer's div
            dealerCardsDiv.appendChild(pic);
            // 31. Push the card into the dealer's hand
            dealerHand.push(card);

            if(card.kind == "Ace") {
                if(dAceScore == 0) {
                    dAceScore = 11;
                } else {
                    dAceScore = 12;
                    card.valu = 1;
                }
            }
            console.log('dAceScore:', dAceScore);

            // 32. Update the dealer's score
            dealerScore += card.valu;
         }
```

With each clicking of the DEAL button, all global variables need to be reset.

14. Inside the deal function, as part of "old step 16", reset **pAceScore** and **pAceScore** to 0:

```
    // 15. Define the deal function:
    function deal() {
        // 16. Since this is a new hand, reset the scores and "clear the
table"
        pAceScore = 0; // reset player Ace score
        dAceScore = 0; // reset dealer Ace score
```

15. Run the application and deal a few hands. Eventually you should see an Ace. A check of the console will let you know if **pAceScore** and/or **dAceScore** equal 11, as expected.

Conducting a "Two Aces Test"
If we deal enough hands, we'll see an Ace sooner or later. But to get dealt two Aces to see if the score is 12, we will likely have to play dozens of hands. The way around this is to hard-code an Ace object as the dealt card.

16. In the deal function, in "old step 21", comment out the **card = shoe.pop()** line, where a card is dealt from the shoe, and hard-code an Ace:

```
    // 21. Pop a card object off the shoe array and save it as a new card
        // let card = shoe.pop();
        let card = {
            file: "Ace-of-Hearts.png",
            kind: "Ace",
            name: "Ace of Hearts",
            suit: "Hearts",
            valu: 11
        };
```

17. Refresh the browser and deal a hand. Both player and dealer should receive two Ace of Hearts.

18. Check the console. There should be output showing both **pAceScore** and **dAceScore**, which start at 11 and then increase to 12.

19. Now that the **"Two Aces Test"** is done, comment out the hard-coded Ace object, and restore the **let card = shoe.pop()** line, so that the game reverts to normal dealing behavior.

testing Blackjack scenarios
In the previous lesson, we wrote code for announcing Blackjack in the event that either the player or dealer (or both) are dealt 21. As a refresher, have a look at that code now, which comprises "old step 39":

```
        // 39. Check to see if either the player or dealer have Blackjack
        // Announce Blackjack on 1 second delay; if no one has Blackjack,
```

```
        // prompt player to HIT or STAND:
            setTimeout(() => {
                if(playerScore == 21 && dealerScore == 21) {
                     outputH2.innerHTML = 'Dealer and Player both have
Blackjack!<br>It's a Push!';
                } else if(playerScore == 21) {
                    outputH2.innerHTML = 'BLACKJACK! You win!';
                } else if(dealerScore == 21) {
                    outputH2.innerHTML = 'Dealer has BLACKJACK! You
lose!';
                } else { // no one has Blackjack
                    outputH2.innerHTML = "HIT or STAND?";
                }
            }, 1000);
         }
```

To test all these Blackjack scenarios, we need to hard-code Blackjack for the player, the dealer and both.

testing: player and dealer both have Blackjack
First we will test the rarest outcome of all: player and dealer both have Blackjack. We do this by hard-coding Blackjack hands for both.

20. In the **deal()** function, once again comment out **let card = shoe.pop()** line, and replace it with this if-else block that deals a Blackjack hand of Ace-Queen to both the player and the dealer.

- the first two cards are Aces: one each for player and dealer
- the last two cards are Queen: one each for player and dealer

```
        // 21. Pop a card off the shoe array and save it as a new card
        // let card = shoe.pop();
        let card;
        // the first two cards are Aces (one each for player and dealer)
        if(dealCounter < 3) {
            card = {
                file: "Ace-of-Hearts.png",
                kind: "Ace",
                name: "Ace of Hearts",
                suit: "Hearts",
                valu: 11
            };
        // the last two cards are Queens (one each for player and dealer)
        } else {
            card = {
                file: "Queen-of-Diamonds.png",
                kind: "Queen",
                name: "Queen of Diamonds",
                suit: "Diamonds",
                valu: 10
            };
        }
```

21. Test the game by dealing a hand. The player and dealer should both get Blackjack, with the output announcing this.

Dealer Blackjack has to be verified by revealing the hole card. To be able to do that, we need to have saved the hole card Image object to begin with. Then, if we need to prove that the dealer has Blackjack, we can change the hole card source from the back of the card to the front of the card.

22. Declare a variable called **holeCardPic**, but don't give it a value. This will be set equal to an Image object later. Do this right above the deal function:

```
    let holeCardPic;

    // 15. Define the deal function:
    function deal() {
```

23. Inside the setTimeout if-else block that assess blackjack, inside the two scenarios where the dealer has blackjack, call a function by the name of **showHoleCard()**, which we will write next:

```
    setTimeout(() => {
        if(playerScore == 21 && dealerScore == 21) {
            outputH2.innerHTML = "Dealer and Player both have Blackjack!
It's a Push!";
            showHoleCard();
        } else if(playerScore == 21) {
            outputH2.innerHTML = 'BLACKJACK! You win!';
        } else if(dealerScore == 21) {
            outputH2.innerHTML = 'Dealer has BLACKJACK! You lose!';
            showHoleCard();
        } else { // no one has Blackjack
            outputH2.innerHTML = "HIT or STAND?";
        }
    }, 1000);
```

24. Write the **showHoleCard()** function, which reveals the hole card by setting the **holeCardPic** source equal to the **file** property of the first card in the **dealerHand** array. The reveal occurs after a 1 second delay:

```
    function showHoleCard() {
        setTimeout(() => {
            holeCardPic.src = `images/cards350px/${dealerHand[0].file}`;
        }, 1000);
    }
```

25. Run the app again. Both the player and dealer are still dealt blackjack, but this time the hole card is revealed after a 1 second delay.

26. Now to test dealer-only Blackjack, which still must reveal the hole card, but which results in a different message being outputted.

- hard-code the first dealer card object (hole card) to be an Ace
- hard-code the second dealer card object (visible card) to be a Queen
- make the player's cards whatever rando cards get popped off the shoe

```
    // 21. Pop a card object off the shoe array and save it as a new card
        // let card = shoe.pop();
        let card;
        /* Testing dealer-only blackjack */
        // make dealer's hole card an Ace
        if(dealCounter == 2) {
            card = {
                file: "Ace-of-Hearts.png",
                kind: "Ace",
                name: "Ace of Hearts",
                suit: "Hearts",
                valu: 11
            };
        // make dealer's second card, the visible card, a Queen
        } else if(dealCounter == 4) {
           card = {
                file: "Queen-of-Diamonds.png",
                kind: "Queen",
                name: "Queen of Diamonds",
                suit: "Diamonds",
                valu: 10
            };
        // make player's cards whatever gets popped off the shoe
        } else {
            card = shoe.pop();
        }
```

27. Run the app again. The dealer should get Blackjack, with the correct message outputted and the hole-card revealed to prove that the dealer does indeed have Blackjack.

28. Now to test player Blackjack, which does not necessitate revealing the hole card.

- hard-code the first player card to be an Ace
- hard-code the second player card to be a Queen
- make the dealer's cards whatever gets popped off the shoe To do all this, simply change the the if-statement counter values from 2 and 4 to 1 and 3:

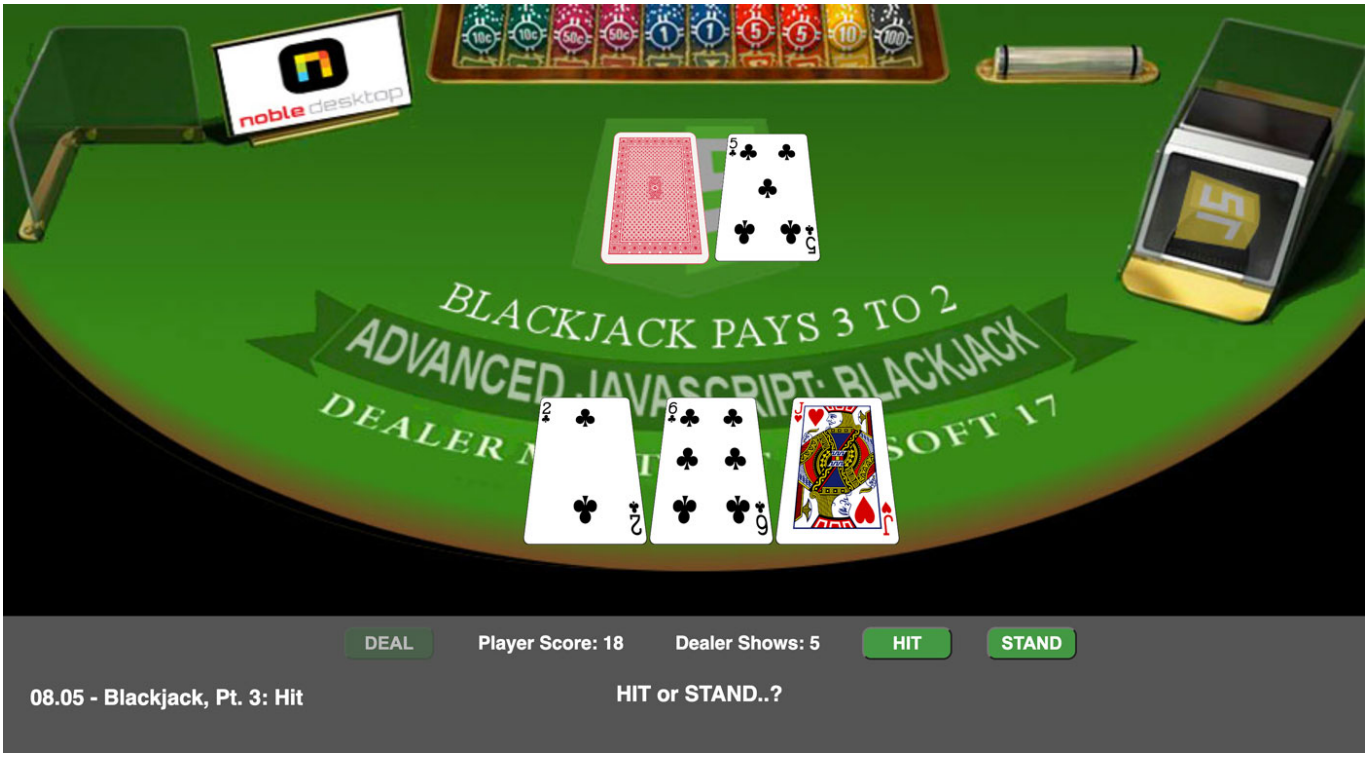
```
        if(dealCounter == 1) {
            -- ETC. --
        if(dealCounter == 3) {
            -- ETC. --
```

29. Test the app to make sure the player is getting blackjack and that the Ace scores are logging as expected.

30. Comment out all Blackjack testing code, and restore the **let card = shoe.pop()** line, so that the dealing goes back to normal.

END: Lesson 08.04
NEXT: Lesson 08.05





Blackjack - Pt. 3: Hit

implementing the HIT button functionality

evaluating Ace value as either 11 or 1

The next thing we need to do for our Blackjack application is handle what happens when the player clicks the HIT button to get another card.

1. We already have the hit button, so give it a listener that calls the **hit()** function when clicked. "Old step 12" is where you add this line:

```
const dealBtn = document.getElementById('deal-btn');
dealBtn.addEventListener('click', deal);

const hitBtn = document.getElementById('hit-btn');
hitBtn.addEventListener('click', hit);
```

HIT ME!

the HIT button calls the hit() function

2. Down at the bottom of the code, after the **showHoleCard()** function, define the **hit()** function:

```
function hit() {

}
```

3. Pop a card off the shoe, and pass it to a local **card** object:

```
function hit() {
    let card = shoe.pop();
}
```

4. The hit card needs to go on the table, so declare a new image object:

```
    let pic = new Image();
```

5. Set the image source to the **file** property of the **card** object:

```
    pic.src = `images/cards350px/${card.file}`;
```

6. Output the pic to the "table" (the player cards div). At this point, you should also run the app to test if the hit card is appearing on the table.

```
    playerCardsDiv.appendChild(pic);
```

handling the "Hit Ace" scenario

We next need to add the hit card to the score. Once again, Aces pose a challenge. Here, an Ace counts as 11 only if it won't bust the player. If an "Ace 11" would put the player over 21, the Ace counts as 1.

7. Check if the Hit card is an Ace:

```
    if(card.kind == "Ace") {
    }
```

If the hit card is an Ace, we need to check to see if an "Ace 11" would bust the player. If so, the Ace counts as 1.

8. Check if the player score is less than 11; if it is, increase **pAceScore** and **playerScore** by 11:

```
        if(card.kind == "Ace") {
            if(playerScore < 11) {
                playerScore += 11;
                pAceScore += 11;
            }
        }
```

9. **else** the player score was at least 11 *before* the hit, so the "Hit Ace" value is 1. Increment **pAceScore** and **playerScore** by 1:

```
        if(card.kind == "Ace") {
            if(playerScore < 11) {
                playerScore += 11; // Ace counts as 11
                pAceScore += 11;
            } else {
                playerScore++; // Ace counts as 1
                pAceScore++;
            }
        }
```

handling existing "Ace 11" scenario

The **"Hit Ace"** scenario has been handled, but what if the hit card is *not* an Ace, but the player nevertheless has an "Ace 11" from the Deal or a prior Hit? In this case, if the hit card would bust the dealer, the existing "Ace 11" must be downgraded to a 1.

10. Check if the hit card has busted the player. This will only be possible if the hit card is not an Ace. This code comes right after the above if-statement:

```
            if(playerScore > 21) {
            }
```

If the player's score is over 21, they bust. But if they have an "Ace 11", it can be changed to an Ace 1, which will "un-bust" them.

11. If the player's score is over 21 (busted), see if the player has an "Ace 11" to "bail them out":

```
            if(playerScore > 21) {
                if(pAceScore >= 11) {
                }
            }
```

If the **pAceScore** is at least 11, the player has an "Ace 11", which can be downgraded to an Ace 1, which "un-busts" the player:

12. Inside the inner if, "un-bust" the player by converting their "Ace 11" to a 1. This is done by subtracting 10 from both the **playerScore** and the **pAceScore**:

```
            if(playerScore > 21) {
                if(pAceScore >= 11) {
                    pAceScore -= 10;
                    playerScore -= 10;
                }
            }
```

Busted!
If the player score exceeds 21 and if the player has no "Ace 11" to "bail them out", they are busted.

13. Add an inner **else** part to announce that the player is busted:

```
            if(playerScore > 21) {
                if(pAceScore >= 11) {
                    pAceScore -= 10;
                    playerScore -= 10;
                } else {
                    outputH2.innerHTML = "Busted! You lose!";
                }
            }
```

Exactly 21
If the player's score is exactly 21, the **stand()** function is called automatically, which ends the player's turn and moves on to assessing the dealer's hand.

14. Add an **else-if** to the outer if block that handles a score of exactly 21 following a hit. Include a call to the **stand()** function, which we will write next:

```
            if(playerScore > 21) {
                if(pAceScore >= 11) {
                    pAceScore -= 10;
                    playerScore -= 10;
                } else {
                    outputH2.innerHTML = "Busted! You lose!";
                }
            } else if(playerScore == 21) {
                outputH2.innerHTML = "You have 21! Dealer's turn.";
                stand();
            }
```

After Hit, Player still has less than 21
If, after hitting, the player still has less than 21, they are prompted again to "HIT or STAND..?"

15. Add an **else** to the that handles what to do if the player still has less than 21 after having hit:

```
            } else if(playerScore == 21) {
                outputH2.innerHTML = "You have 21! Dealer's turn.";
            } else {
                outputH2.innerHTML = "HIT or STAND..?";
            }
```

16. In the nested if, where the dealer has "un-busted", handle a player score of 21 or less than 21:

```
        if(playerScore > 21) {
            if(pAceScore >= 11) {
                pAceScore -= 10;
                playerScore -= 10;

                if(playerScore == 21) {
                    outputH2.innerHTML = "You have 21! Dealer's turn.";
                } else { // player score is less than 21
                    outputH2.innerHTML = "HIT or STAND..?";
                }

            } else {
```

17. Below the big, nested if-else if-else, update the player's score display:

```
            } else {
                outputH2.innerHTML = "HIT or STAND..?";
            }

            playerScoreDiv.innerHTML = "Player Score: " + playerScore;
```

stand() function
The **stand()** function is called when the user clicks the STAND button. If is also called automatically if a Hit achieves a player score of exactly 21.

18. Up in the code where all the buttons are brought in, instruct the STAND button to to listen for a click upon itself and to call the **stand** function when that click occurs:

```
    const standBtn = document.getElementById('stand-btn');
    standBtn.addEventListener('click', stand);
```

minimalist stand() function for testing So that we can test the Hit feature fully, let's start with a minimal implementation of the **stand()** function. Rather than assess the dealer's score with all the tricky

implications for Aces and so forth, we will start by just revealing the hole card, and replacing the "Dealer Shows" with the "Dealer Score".

19. Down at the bottom of the code, below the **hit()** function, define the stand function:

```
    function stand() {
    }
```

20. It's the dealer's turn to play out their hand, so reveal the hole card:

```
    function stand() {
        holeCardPic.src = `images/cards350px/${dealerHand[0].file}`;

    }
```

21. Now that the dealer's score is no longer a secret, replace "Dealer Shows" with "Dealer Score". Also mention that it's the dealer's turn now:

```
    function stand() {
        holeCardPic.src = `images/cards350px/${dealerHand[0].file}`;
        dealerScoreDiv.innerHTML = "Player Score: " + playerScore;
        outputH2.innerHTML = "Dealer's turn..";
    }
```

22. There is plenty more to do with the **stand()** function, and we also have some fine-tuning to do with activating-deactivating the buttons, but test the game now. The **hit()** and **stand()** functions should both be working, as expected.

refactoring for smoother game play with setTimeout
It works, but the **stand()** function runs a bit abrupty. Better would be to spool out the action in phases. We can do this by dividing the code into **setTimeout** callback functions on staggered delays.

23. Refactor **stand()** function to execute its code on staggered delays, with **setTimeout**.

- after 500 milliseconds, we are informed that the player is standing on their score and that it is the dealer's turn now.
- after 2000 milliseconds, we suspense builds as the Hole card is.. teased but not revealed.
- After 3000 milliseconds, the hole card is revealed and the dealer's score is reported, both in the Dealer Score Div and in the output message.

```
    function stand() {

        setTimeout(() => {
            outputH2.innerHTML = "Standing on " + playerScore + ". ..
Dealer's turn..";
```

```
        }, 500);

        setTimeout(() => {
            // 08.05 20. reveal the hole card:
            outputH2.innerHTML = "Hole card is.. ";
        }, 2000);


        setTimeout(() => {
            // 08.05 20. reveal the hole card:
            outputH2.innerHTML += dealerHand[0].name + " .. Dealer has " +
dealerScore;
            holeCardPic.src = `images/cards350px/${dealerHand[0].file}`;
            // 21. replace "Dealer Shows" with "Dealer Score"; also say
it's the dealer's turn:
            dealerScoreDiv.innerHTML = "Dealer Score: " + dealerScore;
        }, 3000);

    } // end stand() function
```

the dealer hits automatically until reaching 17

The dealer must hit until reaching a score of 17 or more. The dealer must hit on "Soft 17", which is a 17 that includes an "Ace 11".

24. Still in the **stand()** function, do another **setTimeout** that checks if the dealer score is less than 17, which indicates that the dealer must hit. Do this on a 4-second delay to keep the timing smooth:

```
        setTimeout(() => {
            if(dealerScore < 17) {
            }
        }, 4000);

    } // end stand() function
```

25. For now, let's just output a message "Dealer Must Hit". We'll handle the actual hit later:

```
        setTimeout(() => {
            if(dealerScore < 17) {
                outputH2.innerHTML = "Dealer Must Hit on " + dealerScore;
            }
        }, 4000);

    } // end stand() function
```

26. As an else-if, check if the dealer score is exactly 17, and if so, if the dealer has "Soft 17", which would be indicated by a **dAceScore** of 11 or more.

- If the dealer *does* have "Soft 17":

- decrease **dealerScore** and **dAceScore** by 10 (downgrade "Ace 11" to 1).
- announce that the "Dealer Must Hit on Soft 17":

```
        setTimeout(() => {
            if(dealerScore < 17) {
                outputH2.innerHTML = "Dealer Must Hit on " + dealerScore;
            } else if(dealerScore == 17) {
                if(dAceScore >= 11) {
                    dAceScore -= 10;
                    dealerScore -= 10;
                    outputH2.innerHTML = "Dealer Must Hit on Soft 17";
                }
            }
        }, 4000);

    } // end stand() function
```

27. **else if** the dealer has exactly 17 (but not "Soft 17"), so announce that the dealer must stand on 17:

```
        setTimeout(() => {
            if(dealerScore < 17) {
                outputH2.innerHTML = "Dealer Must Hit on " + dealerScore;
            } else if(dealerScore == 17) {
                if(dAceScore >= 11) {
                    dAceScore -= 10;
                    dealerScore -= 10;
                    outputH2.innerHTML = "Dealer Must Hit on Soft 17";
                } else {
                    outputH2.innerHTML = "Dealer Stands on " +
dealerScore;
                }
            }
        }, 4000);

    } // end stand() function
```

28. **else** the dealer has less than 17, so announce that the dealer must hit:

```
        setTimeout(() => {
            if(dealerScore < 17) {
                outputH2.innerHTML = "Dealer Must Hit on " + dealerScore;
            } else if(dealerScore == 17) {
                if(dAceScore >= 11) {
                    dAceScore -= 10;
                    dealerScore -= 10;
                    outputH2.innerHTML = "Dealer Must Hit on Soft 17";
                } else {
                    outputH2.innerHTML = "Dealer Stands on " +
```

```
dealerScore;
                }
            } else {
                outputH2.innerHTML = "Dealer Must hit on " + dealerScore;
            }
        }, 4000);

    } // end stand() function
```

29. The nested logic is getting pretty complicated, so now would be a good time to test the application--if you haven't been doing so already.

evaluating winner

We haven't handled giving the dealer more cards yet, but we can still evaluate a winner in one scenario: when the dealer was dealt at least a hard 17 and therefore doesn't get any additional cards.

30. Inside the **else** part where the "Dealer Stands", call the **declareWinner()** function:

```
                } else {
                    outputH2.innerHTML = "Dealer Stands on " +
dealerScore;
                    declareWinner();
                }
```

declareWinner() function

The **declareWinner()** function compares the player and dealer scores and declares the winner. If the score is tied, a "Push" is declared.

31. Define the **declareWinner()** function:

```
    function declareWinner() {
    }
```

32. Inside the function, compare the two scores using if-else if-else logic. Wrap this in a **setTimeout** so that we get a 2-second delay between announcing that the "Dealer Stands" and the declaration of the winner:

```
    function declareWinner() {
        setTimeout(() => {
            if(playerScore > dealerScore) {
                outputH2.innerHTML = "You win! Congrats!";
            } else if(playerScore <> dealerScore) {
                outputH2.innerHTML = "Sorry! You lose!";
            } else { // the scores are equal
                outputH2.innerHTML = "It's a Push..!";
            }
```

```
        }, 2000);
    }
```

33. Run the app. You may have to play a few hands before you see a winner / loser declared. This is because, so far, the dealer can't hit yet, so the **declareWinner()** function only runs when the dealer must stand on the opening deal.

the dealer hits

Last thing to implement is the dealer hitting automatically. We already have some of the logical foundation for this in the **stand()** function where we check if the dealer has less than 17 or 17 exactly (including "Soft 17"). Insidde these parts, we will add calls to a new function, called **hitDealer()**.

34. Call the **hitDealer()** function twice: where the dealer score is less than 17 and also on "Soft 17". The calls go right under the two "Dealer Must Hit" lines:

```
        setTimeout(() => {
            if(dealerScore < 17) {
                outputH2.innerHTML = "Dealer Must Hit on " + dealerScore;
                hitDealer();
            } else if(dealerScore == 17) {
                if(dAceScore >= 11) {
                    dAceScore -= 10;
                    dealerScore -= 10;
                    outputH2.innerHTML = "Dealer Must Hit on Soft 17";
                    hitDealer();
                } else {
                    outputH2.innerHTML = "Dealer Stands on " +
dealerScore;
                }
            } else {
                outputH2.innerHTML = "Dealer Must hit on " + dealerScore;
            }
        }, 4000);

    } // end stand() function
```

END: Lesson 08.05

NEXT: Lesson 08.06

COFFEESHOP CHALKBOARD CHALKBOARD

“TIMELY GREETING” : time: 15 min files: timely-greeting-A.js // A means you need to code from scratch timely-greeting-B.js // B comes with a code “kickstart” timely-greeting-C.js // C means Completed version (JS solution file) timely-greeting-D.html // D means “DOM” (JS in context of a webpage)

A : PROMPT:

Timely Greeting : get the current Date-Time from the user’s computer and output a “Timely Greeting” (Good Morning! Good Afternoon! Good Evening!) based on the current hour, which is an integer from 0-23. Skillset: string variables, Date object, if-else logic

PROCEDURE: Declare a variable called greeting and set it equal to an empty string. Declare a variable called date by instantiating a new Date object. This requires using the Constructor Method and new Keyword. The new keyword creates a new instance (object) of the Class it is called on: var date = new Date(); // date is an object of the Date class Our date has access to all the methods and properties of the Date Class. We can get the current year, month, day, date, hour, minute and second, but all we want is the hour, which we obtain by calling the getHours() method on our date object. The method returns the current hour as an integer from 0-23. We store that return value in a new variable: var hour = date.getHours(); if-else if-else logic: Next, evaluate hour using if-else logic, and set the value of greeting: if hour is less than 12 (noon), the greeting is “Good Morning”; if hour is between 12-18 (noon-6 pm), greeting is “Good Afternoon”; if hour is 18 or greater (6pm or later), greeting = “Good Evening”; log the greeting to the console. extra challenge – Does anyone actually say “Good morning!” at one o’clock in the morning?!? Tweak the if-else logic so that during the wee hours from midnight-3am, the greeting is “Hey, Night Owl!” (Okay, okay – no one says that either!) Beginning at 4am (and going til noon), the greeting is “Good morning!”

Open a file to get started: If you would like to t code this from scratch, based on the above prompt, open: timely-greeting-A.js // A means you need to code from scratch If you would like a kickstart, partially pre-done file, open this one: timely-greeting-B.js // B comes with a code “kickstart” You can always start with A and switch over to B if necessary.

C : The Solution JS File: When you are done or have taken it as far as you can, open: timely-greeting-C.js // C means final version (solution file) To see an example of the code being implemented on a web page, open the .html page in a browser:

D : The HTML Webpage using the JS: timely-greeting-D.html // D means “DOM” (this page shows the JS in context on a webpage)

// make an array of the days of the week var daysOfWeek = ["Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday"]; // get the day of the week as a string by looking it up in the dayOfWeek array

setInterval(() => {

var dateTime = new Date();

var day = dateTime.getDay(); var dayStr = daysOfWeek[day];

var chalkboard = document.getElementById('chalkboard'); var greeting = "Good ";

var hour = dateTime.getHours(); // 0-23 var minu = dateTime.getMinutes(); // 0-59 var secs = dateTime.getSeconds(); // 0-59

if (hour == 0) { greeting = "
Hello, Burner
of Midnight Oil !"; } else if (hour < 4) { greeting = "
Hey, Night Owl!"; } else if (hour < 12) { greeting += "Morning!"; } else if (hour < 17) { greeting += "Afternoon!"; } else { greeting += "Evening!"; }

var amPm = 'am'; if (hour > 12) { amPm = 'pm'; hour -= 12; // reduce mil. time 13 to 1, etc. }

if (hour == 0) { hour = 12; }

if (minu < 10) { minu = '0' + minu; // add a leading zero }

if (secs < 10) { secs = '0' + secs; // add a leading zero }

// on page load output to the chalkboard // the day of the week and the current hour chalkboard.innerHTML = `${dayStr}<br>is Coffee Day! ${greeting}<br>the time is <br>${hour}:${minu}:${secs} ${amPm}`;

}, 1000); // run this every 2 sec

08.07 Coffee Time Clock Hands

Date Object

forEach()

var clock = document.querySelector('.clock'); var sunArmCoffeeCup = document.querySelector('.cup-arm'); var degRotate = -20; var degCounter = 0; sunArmCoffeeCup.style.transform = 'rotate(' + degRotate + 'deg)';

setInterval(() => {

var date = new Date(); var hour = date.getHours(); // 0-23 var mins = date.getMinutes(); // 0-59 var msec = date.getTime(); // milliseconds since 1970 -- a huge number! // The getTime() method returns the number of milliseconds since the ECMAScript epoch // milliseconds since Jan 1, 1970, 00:00:00.000 GMT // milliseconds / 60 = seconds for Rolex-like smooth sweeping second hand

console.log('msec:', msec); // "normalize" time units so that they are all in 0-1 range, with 1 being a // full 360 turn on the clock hand; so when hour = 6, hour for purposes, // of positioning the hour hand on the clock, needs to be normalized to 0.5 // on top of that, hour needs to have the normalized minutes, divided by 12, // added to it // for hour, we need to only have values from 0-12.. so for 12-23, subtract 12 // since we have to do math to arrive at norm values, and since conditions // apply as well, start by just initializing all norms to 0: var hourNorm = 0; var minsNorm = 0; var secsNorm = msec;

// adjust the hour from mil time to am-pm (0-23 to 0-11) if (hour >= 12) { hour -= 12; } // calc normalized hour (0-12 => from 0-1), so divide by 12.. later we will add // hourMin/12 to hourNorm hourNorm = hour / 12 - 0.05; // normalize hour hand and add 'bias'

secsNorm = msec / 1000; // seconds since 1970 -- but as float secsNorm /= 60; // divide that by 60 to get "normalized multiple" (remainder after doing x turns around clock is in 0-1 range)

// add minNorm fraction to hourNorm, so that at, for example, 7:30, // the hour hand is half-way between 7 and 8 minsNorm = mins / 60; // divide mins by 60 to normalize (get 0-1 range)

hourNorm += minsNorm / 6; // divide by 6 results in more accurate hour hand position than dividing by 12

minsNorm = mins / 60; // tweak the min hand

// same for minsNorm? -- min hand should move w sec hand OR just tick every min

var timeUnits = [hourNorm, minsNorm, secsNorm];

// console.log(timeUnits[0] + ':' + timeUnits[1] + ':' + timeUnits[2]);

// run a func on each item in positions arr where the current array item // is timeUnit and arrIndex is the index of same // the one line of code targets the child of the clock div: // the 3 children of the clock div are hr, min and sec hands, so // as the time unit is looped through, that unit is applied as the rotation // turn value of the corresponding clock hand div // 1 turn = 360 degrees timeUnits.forEach((timeunit, arrindex) => { clock.children[arrindex].style.transform = 'rotate(' + timeunit + 'turn)'; });

```
degCounter++; // every 1 min, reset degree counter to 0 if (degCounter >= 2000) { // 40 times per sec x 60 sec = 2400 per min degCounter = 0; }

// rotate sun arm coffee cup -- rotation reg pt is the shoulder if (degCounter < 500 || degCounter > 1500) { degRotate += 0.075; // 1/10 of a degree, so 40x per sec rotates 1/20 deg // so, in 1 sec rotates 4 degs } else { degRotate -= 0.075; }

sunArmCoffeeCup.style.transform = 'rotate(' + degRotate + 'deg)';

}, 40);

/*

*/
```